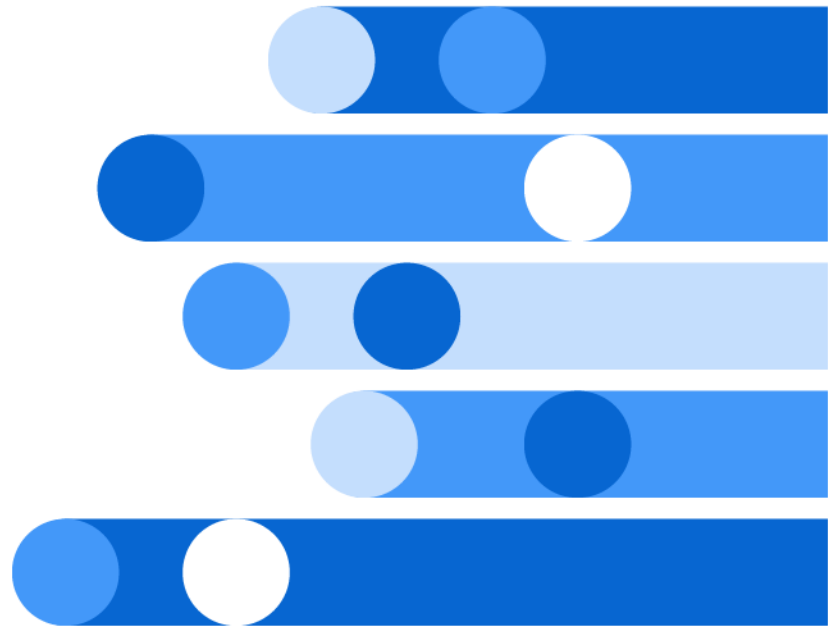




SAS[®] Micro Analytic Service: Programming and Administration Guide

2026.06*



* This document might apply to additional versions of the software. Open this document in [SAS Help Center](#) and click on the version in the banner to see all available versions.



The correct bibliographic citation for this manual is as follows: SAS Institute Inc. 2026. *SAS® Micro Analytic Service: Programming and Administration Guide*. Cary, NC: SAS Institute Inc.

SAS® Micro Analytic Service: Programming and Administration Guide

Copyright © 2026, SAS Institute Inc., Cary, NC, USA

All Rights Reserved. Produced in the United States of America.

For a hard copy book: No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, or otherwise, without the prior written permission of the publisher, SAS Institute Inc.

For a web download or e-book: Your use of this publication shall be governed by the terms established by the vendor at the time you acquire this publication.

The scanning, uploading, and distribution of this book via the Internet or any other means without the permission of the publisher is illegal and punishable by law. Please purchase only authorized electronic editions and do not participate in or encourage electronic piracy of copyrighted materials. Your support of others' rights is appreciated.

U.S. Government License Rights; Restricted Rights: The Software and its documentation is commercial computer software developed at private expense and is provided with RESTRICTED RIGHTS to the United States Government. Use, duplication, or disclosure of the Software by the United States Government is subject to the license terms of this Agreement pursuant to, as applicable, FAR 12.212, DFAR 227.7202-1(a), DFAR 227.7202-3(a), and DFAR 227.7202-4, and, to the extent required under U.S. federal law, the minimum restricted rights as set out in FAR 52.227-19 (DEC 2007). If FAR 52.227-19 is applicable, this provision serves as notice under clause (c) thereof and no other notice is required to be affixed to the Software or documentation. The Government's rights in Software and documentation shall be only those set forth in this Agreement.

SAS Institute Inc., SAS Campus Drive, Cary, NC 27513-2414

June 2026

SAS® and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration.

Other brand and product names are trademarks of their respective companies.

v_037-P1:masag

Contents

<i>About This Book</i>	<i>ix</i>
------------------------------	-----------

PART 1 Understanding SAS Micro Analytic Service 1

Chapter 1 / Introduction to SAS Micro Analytic Service	3
What Is SAS Micro Analytic Service?	3
Chapter 2 / Concepts	5
Overview	5
User or Business Context	6
Module Context	6
Revision	7
Architecture	8
Chapter 3 / Supported Data Source Types	11
Data Sources Supported for Use with SAS Micro Analytic Service	11

PART 2 Deploying and Managing SAS Micro Analytic Service in Kubernetes 13

Chapter 4 / Pre-deployment Considerations	15
Deployment Recommendations	15
Chapter 5 / Deploying SAS Micro Analytic Service in Kubernetes	17
Overview	17
Provisioning Storage	18
Provisioning Python	18
Configuring SAS Micro Analytic Service to Use a Python Distribution	23
Configuring Analytic Store and Python Model Directories	24
Configuring SAS Micro Analytic Service to Use the Archive Feature	24
Configuring SAS Micro Analytic Service Resources	25
Configuring Supplemental SAS Micro Analytic Service Environment Variables ...	26
Configuring SAS Micro Analytic Service to Enable Access to Databricks	26
Understanding Requirements for Using an OpenShift Cluster	26
Chapter 6 / Managing SAS Micro Analytic Service in Kubernetes	29
Graceful Shutdown and Start-Up Operations	29

About Resources for SAS Micro Analytic Service Pods	31
Publishing New Modules to a Deployment with Replicated Pods	31

PART 3 SAS Micro Analytic Service DS2 Extensions Reference 33

Chapter 7 / Datagrid Package	35
About the Datagrid Package	35
Chapter 8 / MASCall Package	37
About the MASCall Package	38
MASCall Methods	38
Dictionary	40
Chapter 9 / MASParallelExec Package	57
About the MASParallelExec Package	57
MASParallelExec Methods	58
Dictionary	59

PART 4 Using SAS Micro Analytic Service with SAS Event Stream Processing 79

Chapter 10 / Publishing to SAS Micro Analytic Service in SAS Event Stream Processing	81
Overview	81
Object Hierarchy	82
XML Example	82
Data Type Mappings	85
Chapter 11 / Generating Derived Events	89
Processing Event Opcodes and Flags	89
Derived Event Suppression and NULL Key Fields	92
Generating Multiple Derived Events from a Single Source Event	92
Multiple Derived Events and NULL Key Fields	94
Chapter 12 / DS2 Programming for SAS Micro Analytic Service	95
Overview	96
DS2 Source Code Prerequisites	96
DS2 Identifiers	97
SAS Micro Analytic Service and SAS Foundation	98
Programming Blocks	99
Restrictions When Working with DS2 and SAS Micro Analytic Service	99
Public and Private Methods and Packages	101
Argument Types Supported in Public Methods	105
About the DS2 init() Method	106
Promoting DS2 Execution Issues to an Execution Failure in SAS	
Event Stream Processing	107
Implicit Data Type Conversions	109

Determining Whether DS2 Code Is Executing in SAS Micro Analytic Service . . .	109
Performing Calls between SAS Micro Analytic Service Modules	110
Using Analytic Store Models	117
Chapter 13 / State Sharing between Modules	121
Overview	121
Shared Vectors	122
Shared Hash Tables	130
Chapter 14 / Best Practices for DS2 Programming in SAS Event Stream Processing	137
Overview	137
Return Results	138
Global Packages versus Local Packages	138
Replacing SCAN (and TRANWRD) with DS2 Code	139
Hash Package	142
Character-to-Numeric Conversions	142
Data Type Conversions	142
Passing Character Values to Methods	143
Performing the Computation Once	143
Moving Invariant Computations Out of Loops	144
Chapter 15 / Python Support in SAS Micro Analytic Service	145
Introduction	145
Example	147
Public and Private Methods	148
Working with Python and SAS Micro Analytic Service	152
Configuring Python for SAS Micro Analytic Service	153
Python Logging	154
Chapter 16 / SAS Micro Analytic Service Logging and Deployment	157
SAS Micro Analytic Service Logging	157
Deployment	162

PART 5 Using SAS Micro Analytic Service with SAS Intelligent Decisioning or SAS Model Manager 163

Chapter 17 / DS2 Programming for SAS Micro Analytic Service	165
Overview	166
DS2 Source Code Prerequisites	166
DS2 Identifiers	167
SAS Micro Analytic Service and SAS Foundation	168
Programming Blocks	169
Restrictions When Working with DS2 and SAS Micro Analytic Service	170
Public and Private Modules and Methods	171
Argument Types Supported in Public Methods	175
About the DS2 init() Method	176
Promoting DS2 Execution Issues to an Execution Failure in SAS Intelligent Decisioning and SAS Model Manager	177
Performing Calls between SAS Micro Analytic Service Modules	178

Performing Parallel Sub-flow Calls in a SAS Micro Analytic Service Module	185
Determining Whether DS2 Code Is Executing in SAS Micro Analytic Service	190
Managing Large DS2 Modules	190
Composite Modules	194
Referencing Modules and Composite Submodules	194
Using Analytic Store Models	195
Chapter 18 / State Sharing between Modules	199
Overview	199
Shared Vectors	200
Shared Hash Tables	208
Chapter 19 / Best Practices for DS2 Programming in SAS Intelligent Decisioning	215
Overview	215
Return Results	216
Global Packages versus Local Packages	216
Replacing SCAN (and TRANWRD) with DS2 Code	217
Hash Package	220
Character-to-Numeric Conversions	220
Passing Character Values to Methods	220
Performing the Computation Once	221
Moving Invariant Computations Out of Loops	221
Chapter 20 / Python Support in SAS Micro Analytic Service	223
Introduction	223
About Creating Python Modules	225
Public and Private Methods	226
Working with Python and SAS Micro Analytic Service	229
Security Considerations for Python Modules	231
Configuring Python for SAS Micro Analytic Service	232
Chapter 21 / Module Execution	237
Executing a Published Decision or Model	237
Synchronous, Asynchronous, and Timed Execution	239
Using Transaction Metadata during Step Execution	242
Chapter 22 / Database Access and Redis Access with DS2	251
Database Connection Configuration Overview	251
Instantiating a SQLSTMT Package	252
Verifying DS2 SQLSTMT Method Return Codes	253
Committing Transactions	254
About Database Connection Retry Functionality	257
Configuring a Connection to a Redis Server	258
Configuring an Authentication Domain	260
Chapter 23 / Administration	263
SAS Micro Analytic Service Administration	263
SAS Micro Analytic Service Configuration	264
SAS Micro Analytic Service Logging	284
DS2 Logging	289
SAS Micro Analytic Service Security and Authorization	292
Secure DS2 HTTP Package Usage	293

Moving Objects by Using the SAS Viya Platform Transfer Service	293
Backup and Restore	295

PART 6 **Appendixes** 297

Appendix 1 / SAS Micro Analytic Service Data Type Mappings	299
SAS Micro Analytic Service Data Type Mappings Table	299
Appendix 2 / Executing Python Modules in DS2 Modules	303
DS2 Interface to Python	303
Sample DS2 Module Operations	306
Configuring Support for a DS2 PyMAS Package	311
About Python and Data Grids	311
Example	313
Appendix 3 / Database Driver Reference	317
About the Connection String Configuration	318
About Using Database Catalogs	319
Amazon Aurora Driver Reference	320
Amazon Redshift Driver Reference	321
DB2 Driver Reference	325
Google BigQuery Driver Reference	328
Microsoft Azure SQL Database Driver Reference	333
Microsoft SQL Server Driver Reference	338
Oracle Driver Reference	342
PostgreSQL and Microsoft Azure PostgreSQL Driver Reference	346
SingleStore Driver Reference	350
Snowflake Driver Reference	353
Additional Database Connection Options	355
Appendix 4 / SAS Data Quality Function Support	359
About SAS Data Quality Functions	359
Appendix 5 / SAS Micro Analytic Service Metrics	361
SAS Micro Analytic Service Grafana Dashboard	361
Appendix 6 / SAS Micro Analytic Service Tuning Guidelines	363
SAS Micro Analytic Service Tuning Guidelines	363
Appendix 7 / SAS Quality Knowledge Base (QKB)	367
Configuring a SAS Quality Knowledge Base (QKB)	367
Appendix 8 / REST Server Error Messages and Resolutions	369
REST Server Error Messages and Resolutions	369
Appendix 9 / SAS Micro Analytic Service Return Codes	377
SAS Micro Analytic Service Return Codes	377

About This Book

Audience

SAS Intelligent Decisioning, SAS Model Manager, and SAS Event Stream Processing include SAS Micro Analytic Service, which enables you to publish SAS analytics, business rules, and user-written modules into operational environments. In addition, a variety of SAS analytics is available to you, and you can author custom logic in DS2 or Python, as well as deploy a combination of the module types that are specified above.

This guide is intended for developers and information technology administrators who use a SAS Event Stream Processing, SAS Intelligent Decisioning, or SAS Model Manager environment. Because aspects of SAS Micro Analytic Service differ according to environment, ensure that you refer to the section that applies to your environment.

Included here is information about how SAS Micro Analytic Service processes transactions and events, as well as tips, best practices, and restrictions on programming DS2 or Python to run in SAS Micro Analytic Service.

Technology administrators can find information about how to configure SAS Micro Analytic Service. Also included is information about how to configure Python and SAS Micro Analytic Service to run Python code (optional).

PART 1

Understanding SAS Micro Analytic Service

Chapter 1	
<i>Introduction to SAS Micro Analytic Service</i>	3
Chapter 2	
<i>Concepts</i>	5
Chapter 3	
<i>Supported Data Source Types</i>	11

Introduction to SAS Micro Analytic Service

<i>What Is SAS Micro Analytic Service?</i>	3
Overview	3
About Using SAS Micro Analytic Service	4

What Is SAS Micro Analytic Service?

Overview

SAS Micro Analytic Service is a stateless, memory-resident, high-performance program execution service. As a SAS platform service, it is not available for individual license, but is included in selected SAS solutions. SAS Micro Analytic Service provides hosting for DS2 and Python programs and supports a “compile-once, execute-many-times” usage pattern. SAS Micro Analytic Service is multi-threaded and can be clustered for scalability and high availability. It can host multiple programs simultaneously, as well as multiple user or business contexts that are isolated from one another.

SAS Micro Analytic Service contains a core engine that is written in C for high performance and, when deployed as part of SAS Event Stream Processing, includes C++ classes that integrate with SAS Event Stream Processing. These capabilities allow both to execute within the same process space for maximum performance. The combination of SAS Event Stream Processing and SAS Micro Analytic Service enables SAS analytics, business logic, and user-written programs to operate on streams of data in motion.

About Using SAS Micro Analytic Service

SAS Micro Analytic Service is integrated with SAS Event Stream Processing and deployed with SAS Intelligent Decisioning and SAS Model Manager. Note the following information about using SAS Micro Analytic Service with these solutions:

- When used in a SAS Intelligent Decisioning environment, SAS Micro Analytic Service is called as a web application with a REST interface. The REST interface (known as the SAS Micro Analytic Score service) provides easy integration with client applications and adds persistence and clustering for scalability and high availability.

SAS Intelligent Decisioning generates DS2 programs that implement user-created rule sets and rule flows. It can combine SAS analytics, such as score code generated by SAS Viya, with business rules in order to form decision logic. SAS Micro Analytic Service is used to compile and execute the generated code. For more information, see [SAS Intelligent Decisioning: User's Guide](#).

- When SAS Model Manager is installed, a publishing destination is created automatically for SAS Micro Analytic Service. Users can publish models to the SAS Micro Analytic Service publishing destination and then score them within the publishing destination. For more information, see [SAS Model Manager: User's Guide](#).
- Users of SAS Model Studio can publish to SAS Micro Analytic Service if SAS Model Manager is also installed.
- Users of SAS Event Stream Processing or SAS Intelligent Decisioning can publish SAS analytics, such as predictive models that were created with a variety of SAS products and analytical procedures. They can also author custom programs using the SAS DS2 or Python programming languages or in the SAS Intelligent Decisioning web application. The custom programs execute inside SAS Event Stream Processing applications. SAS Micro Analytic Service can host multiple programs simultaneously.

For more information about SAS Event Stream Processing, see the product documentation at <http://support.sas.com>.

SAS Micro Analytic Service supports a subset of the DS2 programming language, which includes language features that are suitable for the high-performance execution of transactions.

SAS Micro Analytic Service supports the Python programming language. Python programs that are written for SAS Micro Analytic Service might include custom functions. They can use any third-party Python packages that have been deployed to a local Python environment.

Concepts

<i>Overview</i>	5
<i>User or Business Context</i>	6
<i>Module Context</i>	6
<i>Revision</i>	7
<i>Architecture</i>	8

Overview

DS2 and Python programs that are published to SAS Micro Analytic Service, whether user-written or generated by SAS analytical solutions, are known as *modules*. This term reflects the language-neutral nature of SAS Micro Analytic Service interfaces.

A module is a collection of methods. For DS2, a module represents one DS2 package and its methods. For Python, a module is a collection of Python functions.

Module methods can be used for a wide variety of other purposes, including computing scores, processing data, or making business decisions.

For SAS Event Stream Processing, module methods can be used to process events in a continuous query. The results of such processing create derived events that flow to downstream components in the continuous query. In addition to generating derived events, module methods can influence SAS Event Stream Processing by interrogating and setting event opcodes and flags. Event opcodes and flags are covered in more detail in the DS2 and Python programming chapters that follow.

For SAS Intelligent Decisioning and SAS Model Manager, module methods can be used to automate data-driven decisions. This is accomplished by executing analytical models and business rules against the latest data from online channels, combined with data from operational databases and other data sources.

SAS Micro Analytic Service uses two internal component types to manage the modules that are published to it. These are the module context and the revision. A third component, the user context, provides isolated execution environments that

contain sets of module contexts and revisions. SAS Micro Analytic Service automatically manages user and module contexts for the user.

SAS Micro Analytic Service automatically manages user contexts for SAS Event Stream Processing by maintaining one user context per event stream processing object.

The SAS Micro Analytic Service REST service, used by SAS Intelligent Decisioning and SAS Model Manager, automatically creates and manages one user context.

Before writing modules to deploy to SAS Micro Analytic Service, see the following information:

- For DS2 modules, see the programming guidelines for your environment:
 - SAS Event Stream Processing: [Chapter 12, “DS2 Programming for SAS Micro Analytic Service,” on page 95](#)
 - SAS Intelligent Decisioning and SAS Model Manager: [Chapter 17, “DS2 Programming for SAS Micro Analytic Service,” on page 165](#)
- For Python modules, see the programming guidelines for your environment:
 - SAS Event Stream Processing: [Chapter 15, “Python Support in SAS Micro Analytic Service,” on page 145](#)
 - SAS Intelligent Decisioning and SAS Model Manager: [Chapter 20, “Python Support in SAS Micro Analytic Service,” on page 223](#)

User or Business Context

A *context* is a container for the programs that SAS Micro Analytic Service executes. It is also an isolated execution environment. That is, programs executing in one context are not visible to any other context. Therefore, contexts can be used to provide a separate environment for each user or different business unit, or for any other usage requiring isolation. As noted in the previous section, programs that are hosted by SAS Micro Analytic Service are known as modules. A context is a container of modules.

Because business context and user context are interchangeable terms that describe the two common uses of this single component, this document uses the term *user context* for simplicity.

Module Context

A module represents program code. In the case of DS2, each module represents exactly one DS2 package. If you are unfamiliar with DS2 packages, see

“[Understanding DS2 Methods and Packages](#)” in *SAS DS2 Language Reference*. Every module is owned by exactly one user context.

In the case of Python, each module represents a collection of related Python functions, and each module method represents one of those functions.

SAS Micro Analytic Service supports module revisions and is capable of hosting and executing multiple revisions of a module concurrently. When SAS Micro Analytic Service compiles a DS2 or Python module, it creates a revision of that module. Therefore, a module context is a container of revisions. A module context also houses any compiler warning or error messages that were generated from the latest compilation or compilation attempt.

Note: SAS Micro Analytic Service runs the latest revision of a module by default.

Note: The Micro Analytic Service REST interface supports running only the latest revision of a module.

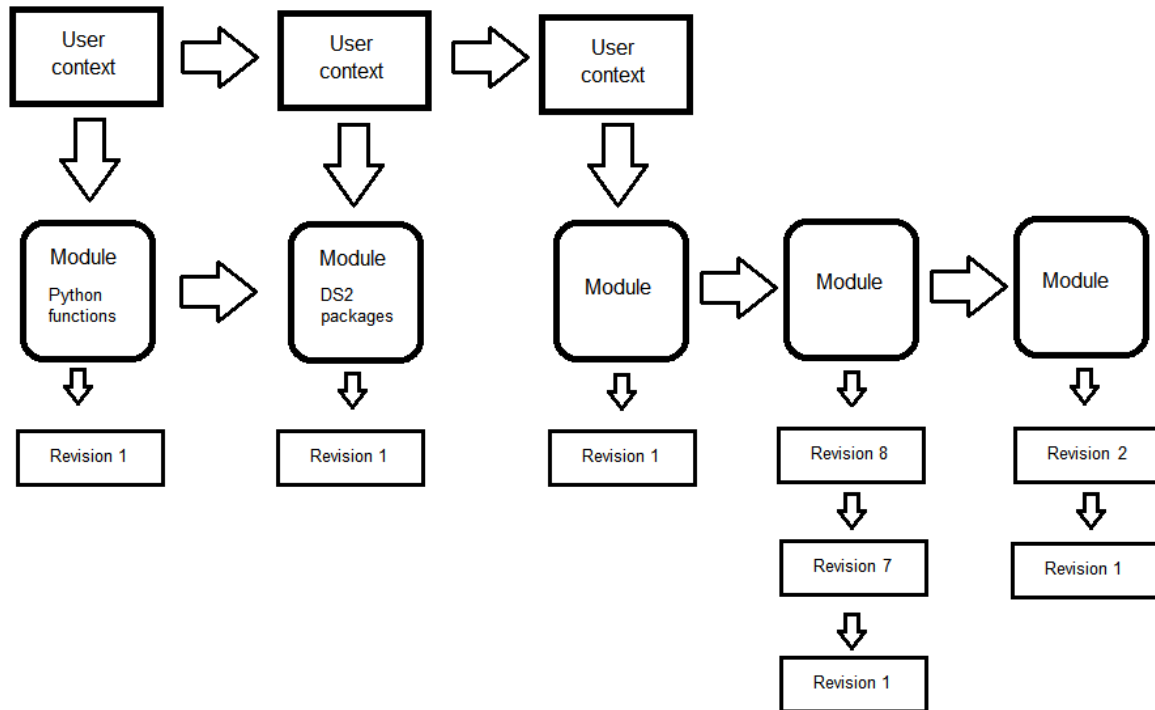
Revision

A revision is a version of a module. Each revision contains source code, an executable code stream (optimized binary executable), and metadata. The metadata describes the methods and method signatures of the module.

Revisions provide several advantages, including the ability to roll back to a previous version of a module.

SAS Micro Analytic Service assigns a revision number to each revision, which is a monotonically increasing integer beginning with 1. A revision is uniquely identified by module name and revision number. When you reference a revision, specifying revision number 0 selects the latest revision.

Note: When modules are published to SAS Micro Analytic Service by SAS Event Stream Processing, only the latest revision is retained.

Figure 2.1 Component Hierarchy

Note: Revisions can be added and deleted, which might result in non-sequential revision numbers. In the figure above, this is illustrated by one of the module's revisions going from Revision 1 to Revision 7.

Architecture

SAS Micro Analytic Service has a layered architecture:

Core Engine

The SAS Micro Analytic Service core engine is written in C and is multi-threaded for high performance.

Note: SAS Event Stream Processing integrates with SAS Micro Analytic Service via a private C interface to the core engine. This interface allows SAS Event Stream Processing to call SAS Micro Analytic Service directly, in-process, for maximum performance. When SAS Micro Analytic Service is deployed with SAS Event Stream Processing, Java and REST layers are omitted.

Java Layer

a thin Java layer communicates with the core engine through the Java Native Interface (JNI). Commands from the REST interface are passed to the core engine through this Java layer.

REST

adds functionality such as persistence and clustering support.

Note: SAS Intelligent Decisioning and SAS Model Manager interface with SAS Micro Analytic Service via REST, and uses all three layers of the architecture.

Supported Data Source Types

<i>Data Sources Supported for Use with SAS Micro Analytic Service</i>	11
---	-----------

Data Sources Supported for Use with SAS Micro Analytic Service

SAS Micro Analytic Service supports a subset of the data sources that the SAS Viya platform supports. Here is the list of supported data sources:

- Amazon Aurora
- DB2
- Google BigQuery
- Microsoft Azure SQL
- Microsoft SQL Server
- Oracle
- PostgreSQL
- SingleStore
- Snowflake

For more information, see [Appendix 3, “Database Driver Reference,”](#) on page 317.

PART 2

Deploying and Managing SAS Micro Analytic Service in Kubernetes

Chapter 4	
<i>Pre-deployment Considerations</i>	15
Chapter 5	
<i>Deploying SAS Micro Analytic Service in Kubernetes</i>	17
Chapter 6	
<i>Managing SAS Micro Analytic Service in Kubernetes</i>	29

Pre-deployment Considerations

<i>Deployment Recommendations</i>	15
---	----

Deployment Recommendations

Here are items for you to consider when you plan to deploy SAS Micro Analytic Service to a production environment:

- To use SAS Micro Analytic Service for transaction processing, SAS strongly recommends that you deploy SAS Micro Analytic Service to its own node pool with dedicated nodes.
- When SAS Micro Analytic Service is deployed in its own node pool, you should configure the node pool with reserve capacity to allow for the addition of new pods. This ensures that there is no downtime when performing rolling updates that are required to apply patches or upgrade software.
- A decision flow that makes calls to an external system can affect performance. SAS recommends that you perform benchmark testing using a decision flow that does not access external resources in order to establish a baseline for benchmark testing.

Deploying SAS Micro Analytic Service in Kubernetes

<i>Overview</i>	17
<i>Provisioning Storage</i>	18
<i>Provisioning Python</i>	18
About Provisioning Python for Use with SAS Micro Analytic Service	18
Building the Docker Image Using a Dockerfile	19
Creating a Temporary Pod to Populate the Persistent Volume with the Python Environment	21
<i>Configuring SAS Micro Analytic Service to Use a Python Distribution</i>	23
<i>Configuring Analytic Store and Python Model Directories</i>	24
<i>Configuring SAS Micro Analytic Service to Use the Archive Feature</i>	24
<i>Configuring SAS Micro Analytic Service Resources</i>	25
<i>Configuring Supplemental SAS Micro Analytic Service Environment Variables</i>	26
<i>Configuring SAS Micro Analytic Service to Enable Access to Databricks</i>	26
<i>Understanding Requirements for Using an OpenShift Cluster</i>	26

Overview

Depending on how you use SAS Micro Analytic Service, certain configuration requirements related to Kubernetes must be completed. This chapter describes those requirements and supplies references for more information.

Note: The configuration information in this chapter is not applicable when using SAS Micro Analytic Service with SAS Event Stream Processing.

Provisioning Storage

SAS Micro Analytic Service requires persistent storage for analytic store models, Python models, and, if the archive feature is enabled, for transaction logs.

For more information, see the following:

- [“Persistent Volumes for Applications” in *System Requirements for the SAS Viya Platform*](#)
- [“Set Up an ASTORES Volume for Python” in *SAS Viya Platform: Integration with External Languages*](#)

Provisioning Python

About Provisioning Python for Use with SAS Micro Analytic Service

If you use modules that are written in Python with SAS Micro Analytic Service, Python must be accessible in your environment via a persistent volume. To accomplish this:

- Write a Dockerfile that will build a Docker image containing a Python distribution.
- Build the Docker image.
- Make the Docker image available from your Docker registry.
- Populate the persistent volume claim (PVC) using the Docker image in a temporary pod.

Note:

- For more information about how to perform these tasks, see the [Kubernetes](#) and [Docker](#) documentation.
 - You can also configure Python by deploying sas-pyconfig (SAS Configurator for Open Source). For more information about that utility and the integration requirements for Python on the SAS Viya platform, see [“Integrating Open Source Tools” in *System Requirements for the SAS Viya Platform*](#).
-

The next sections show sample scripts that provide an example of how to provision Python for use by SAS Micro Analytic Service.

Building the Docker Image Using a Dockerfile

The following example Dockerfile prepares Python 3.8.5 with commonly used Python packages. When you create a Dockerfile, you can add any packages that you require.

Note the following about this Dockerfile example:

- The name of the Dockerfile is `python-volume-creator`.
- Python is installed to `/python` in the Docker image.
- The PVC that was created to store the Python distribution is mounted to the temporary pod as `/target-volume`.
- The final line of the Dockerfile calls a script (`copy-python-to-target.sh`) that copies the Python files from the image to the externally mounted volume.

After creation, the Dockerfile and the referenced copy script are placed in the same directory. Then the `docker build` command is used to create a Docker image.

Here is the sample Dockerfile.

Note: This Dockerfile has been tested using CentOS 8.3.2011.

```
FROM centos:8

RUN dnf -y group install "Development Tools"

RUN dnf -y install curl

#### Build Requirements from RHEL python3 spec file
RUN dnf -y install --enablerepo=powertools\
    bzip2-devel \
    expat-devel \
    gdbm-devel \
    libGL-devel \
    libX11-devel \
    libffi-devel \
    libns12-devel \
    libtirpc-devel \
    ncurses-devel \
    openssl-devel \
    pkgconfig \
    readline-devel \
    sqlite-devel \
    systemtap-sdt-devel \
    tar \
    tcl-devel \
    tix-devel \
    tk-devel \
```

```

xz-devel \
zlib-devel

RUN mkdir -p /python

RUN cd /tmp; \
  curl -O https://www.python.org/ftp/python/3.8.5/Python-3.8.5.tgz; \
  tar -xvzf Python-3.8.5.tgz;

#### configure options from RHEL python3 spec file and then apply
these changes:
#### - add the prefix=/python option, because the whole point of this
Docker container
#### is to build and install python in a specific path
#### - remove enable-shared (non-standard path makes it harder to
load shared libraries)
RUN cd /tmp/Python-3.8.5; \
  ./configure \
    --prefix=/python \
    --enable-ipv6 \
    --with-computed-gotos=no \
    --with-dbmliborder=gdbm:ndbm:bdb \
    --with-system-expat \
    --with-system-ffi \
    --enable-loadable-sqlite-extensions \
    --with-dtrace \
    --with-lto \
    --with-ssl-default-suites=openssl; \
  make install

RUN rm -rf /tmp/Python*

RUN /python/bin/python3.8 -m pip install --upgrade pip

#### install tensorflow before the other packages to ensure the right
versions of numpy and scipy are installed
RUN /python/bin/pip3 install \
  tensorflow

#### these are packages commonly used. Add more if needed
RUN /python/bin/pip3 install \
  graphviz \
  html5lib \
  image \
  matplotlib \
  opencv-python \
  pandas \
  plotly \
  requests \
  sas-dlpy \
  sas-espy \
  sasoptpy \
  saspy==3.3.7 \
  scikit-learn \
  swat \
  tensorflow \

```

```

ws4py

#### VDMML (as of 2020-09-02) expects the python executable to be
"python", not "python3", so we symlink it
RUN ln -s /python/bin/python3.8 /python/bin/python

COPY copy-python-to-target.sh /

```

Here are the contents of the copy-python-to-target.sh script that copies the Python files from the image to the externally mounted volume:

```

#!/bin/bash

chmod 755 /target-volume
cp -vpRd /python/* /target-volume
echo "done."

```

Creating a Temporary Pod to Populate the Persistent Volume with the Python Environment

After the Docker image is prepared, the next step is to make it available from your Docker registry.

When that is complete, you create a .yaml file that you use to create the temporary pod that populates the PVC with the Python environment.

Below is a sample .yaml file that creates a temporary pod. Note the following values that are specified in this file. When you perform this step, use the values that are applicable to your site.

- Docker registry host name: **registry.sas.com/infra-dev-pub/adx/viya4**

This is the private Docker registry from which the python-volume-creator image is accessed.

- PVC name: **open-source-volume-python**

This is the PVC to which the Docker image is copied and from which the Python environment is accessed by SAS Micro Analytic Service.

Here is the sample .yaml file:

```

apiVersion: v1
kind: Secret
metadata:
  name: python-volume-creator
stringData:
  .dockerconfigjson: |
    {"auths":{"registry.sas.com":{"auth":"INSERT_AUTH_STRING_HERE"}}}
type: kubernetes.io/dockerconfigjson
---
apiVersion: v1
kind: Pod
metadata:
  name: viya4-python-volume-creator

```

```

spec:
  imagePullSecrets:
    - name: python-volume-creator
  initContainers:
    - name: python-volume-creator
      image: registry.sas.com/infra-dev-pub/adx/viya4/python-volume-creator:latest
  resources:
    requests:
      memory: "500Mi"
      cpu: "250m"
    limits:
      memory: "2Gi"
      cpu: "1.5"
  command:
    - /copy-python-to-target.sh
  imagePullPolicy: Always
  volumeMounts:
    - name: python-install-target
      mountPath: /target-volume
  containers:
    - name: python-volume-tester
      image: centos:8
      resources:
        requests:
          memory: "500Mi"
          cpu: "250m"
        limits:
          memory: "2Gi"
          cpu: "1.5"
      command:
        - sleep
        - "3600"
      imagePullPolicy: IfNotPresent
      volumeMounts:
        - name: python-install-target
          mountPath: /python
      restartPolicy: Always
  volumes:
    - name: python-install-target
      persistentVolumeClaim:
        claimName: open-source-volume-python

```

To create the pod, you use the following command:

```
kubectl apply -f filename.yaml
```


Configuring SAS Micro Analytic Service to Use a Python Distribution

There are two configuration methods available to use a Python distribution with SAS Micro Analytic Service:

- SAS Micro Analytic Service and the SAS Viya platform can use a customer-prepared environment that consists of a Python installation (and any required packages) that is stored on a Kubernetes persistent volume.

Note:

The Python overlay for `sas-microanalytic-score` uses a persistent volume named `astores-volume`, which is defined in the `astores` overlay. The Python and `astores` overlays must be installed together. That is, if you install the Python overlay, you must also install the `astores` overlay. For more information, see [“Set Up an ASTORES Volume for Python” in SAS Viya Platform: Integration with External Languages](#).

-
- SAS Micro Analytic Service can use a customer-prepared environment that consists of a Python installation (and any required packages) that is stored in a Docker image.

IMPORTANT The SAS CAS Server and Compute Server do not support accessing a Python distribution via a Docker image. Therefore, if you use this method and you require Python for the Compute Server or CAS, a Python distribution must also be available via a Kubernetes persistent volume.

For more information about how to integrate Python with the SAS Viya platform, see [SAS Viya Platform: Integration with External Languages](#).

Note the following additional information about using Python with SAS Micro Analytic Service:

- If you are executing Python modules in DS2 modules and using SAS Cloud Analytic Server (CAS), you must understand and configure the authentication environment variables that are applicable to your site. For more information, see [“Configure External Access to the CAS Server” in SAS Viya Platform: Integration with External Languages](#).
- For configuration information that is associated with using SAS Intelligent Decisioning or SAS Model Manager, see [“Configuring Python for SAS Micro Analytic Service” on page 232](#).

Configuring Analytic Store and Python Model Directories

Access to storage is required in order to store Python models and to publish analytic store models to a SAS Micro Analytic Service publishing destination.

Configuring SAS Micro Analytic Service to use analytic store (ASTORE) and Python model files inside the container requires persistent storage from the cloud provider. A persistent volume claim (PVC) is defined to specify the storage requirements from cloud providers. The storage provided by the cloud is mapped to predefined paths across services that collaborate to handle the files.

For complete configuration information, see [“Set Up an ASTORES Volume for Python” in SAS Viya Platform: Integration with External Languages](#).

Configuring SAS Micro Analytic Service to Use the Archive Feature

If enabled, the SAS Micro Analytic Service archive feature records the inputs and outputs of step execution to a set of rolling log files. To use the archive feature, SAS Micro Analytic Service must be configured with a persistent volume to use as a location in which to store the log files.

The archive feature requires storage with ReadWriteMany access mode for storing transaction logs. A persistent volume claim is defined to specify the storage that is required.

For complete configuration information, see the following:

- The README.md file that is located in the `$deploy/sas-bases/examples/sas-microanalytic-score/archive` directory where you downloaded your files for deployment.
- The `archive.enabled` property in `“sas.microanalyticservice.properties: service”` on page 274.

Note: By default, the archive feature is not enabled.

Configuring SAS Micro Analytic Service Resources

By default, SAS Micro Analytic Service is deployed with the following resource requests and limits:

- Memory:
 - Request: 855Mi
 - Limit: 2Gi
- CPU:
 - Request: 250m
 - Limit: 4000m

Note: *m* represents millicpu.

Kubernetes schedules the SAS Micro Analytic Service deployment in a node that has at least the minimum resources that are specified above.

The appropriate request and limit values of memory and CPU depend on the available resources in the cluster and the workload. Loading a lot of modules requires more memory. Complex decisioning logic and sophisticated scoring require more CPU. The best way to arrive at workable numbers is to perform a sizing exercise.

As modules are loaded, the SAS Micro Analytic Service engine might obtain additional resources. Note that if SAS Micro Analytic Service exceeds the configured memory limit, Kubernetes might terminate the container.

Note: If this occurs, the status of the pod shows as OOMKilled.

To avoid this, increase your SAS Micro Analytic Service memory resource limit values as necessary.

To change the SAS Micro Analytic Service deployment resource request and limit values, use the `resources-transformer.yaml` file in the `$deploy/sas-bases/examples/sas-microanalytic-score/resources` directory to configure different values.

For complete configuration information, see the `README.md` file that is located in the `$deploy/sas-bases/examples/sas-microanalytic-score/resources` directory where you downloaded your files for deployment.

For more information about pod resource limits, see [Kubernetes Documentation](#).

Configuring Supplemental SAS Micro Analytic Service Environment Variables

Some SAS Micro Analytic Service features require environment variables that are not set as part of the deployment process. When an environment variable is required to enable the use of a feature, it is noted in this guide.

To set an environment variable and corresponding value, use the `mas-add-environment-variables` transformer in the `mas-add-environment-variables.yaml` file. It is located in the `$deploy/sas-bases/examples/sas-microanalytic-score/config` directory.

For complete configuration information, see the `README.md` file that is located in the `$deploy/sas-bases/examples/sas-microanalytic-score/config` directory where you downloaded your files for deployment.

Configuring SAS Micro Analytic Service to Enable Access to Databricks

For information about how to configure SAS Micro Analytic Service to access Databricks, see the `README.md` file that is located in the `$deploy/sas-bases/examples/sas-microanalytic-score/databricks` directory where you downloaded your files for deployment.

For information about how to connect to Databricks from SAS Micro Analytic Service Python code, see [“Connecting to Databricks from SAS Micro Analytic Service Python Code” on page 234](#).

Understanding Requirements for Using an OpenShift Cluster

In an OpenShift cluster, SAS Micro Analytic Service runs with standard privileges by default. Security context constraints are required in an OpenShift cluster if the `sas-micro-analytic-score` pod needs to mount an NFS volume. If the Python

environment is made available through an NFS mount, the service account requires NFS volume mounting privileges.

For more information, see the README.md file that is located in the `$deploy/sas-bases/overlays/sas-microanalytic-score/service-account/` directory where you downloaded your files for deployment.

Managing SAS Micro Analytic Service in Kubernetes

<i>Graceful Shutdown and Start-Up Operations</i>	29
Overview	29
Updating SAS Software	30
Applying Configuration Changes	30
Scaling Up SAS Micro Analytic Service Pods	30
<i>About Resources for SAS Micro Analytic Service Pods</i>	31
<i>Publishing New Modules to a Deployment with Replicated Pods</i>	31

Graceful Shutdown and Start-Up Operations

Overview

SAS Micro Analytic Service provides a seamless transition of service when doing any of the following:

- updating SAS software
- applying configuration changes
- scaling up pods

While these operations are being performed, SAS Micro Analytic Service continues to process execution requests without errors. Following the completion of an operation that creates a new pod, a Kubernetes readiness probe to SAS Micro Analytic Service determines when the new pod is ready to service requests. Service requests are never sent to a pod until Kubernetes confirms that the pod is ready.

Updating SAS Software

At times it is necessary to update your SAS software. When the update is in progress, all existing SAS Micro Analytic Service pods remain functional. This ensures that any requests to SAS Micro Analytic Service are serviced during the software update.

When the Kubernetes readiness probe to a newly deployed SAS Micro Analytic Service pod determines that the pod is ready to service requests, the clients are seamlessly switched from the old pod to the new pod. Then, the old pod is terminated.

Applying Configuration Changes

Some SAS Micro Analytic Service configuration properties require that the pod be restarted for updated values to take effect.

To restart the pod, perform a rolling update to apply the changes using the following `kubectl` command:

```
kubectl rollout restart deployment/sas-microanalytic-score
```

While the restart is in progress, the existing pod remains functional. This ensures that requests to SAS Micro Analytic Service are serviced with the existing configuration values before the update.

When the Kubernetes readiness probe to a new SAS Micro Analytic Service pod determines that the pod is ready to service requests, the clients are seamlessly switched from the old pod to the new pod. Then, the old pod is terminated.

Scaling Up SAS Micro Analytic Service Pods

To handle increased load without compromising response time, you can scale up the number of SAS Micro Analytic Service pods. For more information about scaling, see the [Kubernetes documentation](#).

When pods are added via scaling, the Kubernetes readiness probe determines when a newly added pod is ready to service requests.

About Resources for SAS Micro Analytic Service Pods

When SAS Micro Analytic Service starts, it loads all published modules. During the loading process, SAS Micro Analytic Service uses memory to load the modules and utilizes CPU to compile the modules.

In certain circumstances, resources can be exhausted, causing SAS Micro Analytic Service to enter a looping restart cycle. Here are possible reasons:

- SAS Micro Analytic Service can access more memory than is set as a limit for the pod. When Kubernetes discovers this situation, it kills the pod and restarts it.
- If all of the allotted CPU time is used, no CPU cycles remain to service requests. By default, Kubernetes runs a liveness probe every 10 seconds. If the probe times out, Kubernetes restarts the pod.

To avoid or resolve a looping restart cycle, adjust the values in the `resources-transformer.yaml` to ensure that sufficient resources are available for the modules. Then, restart SAS Micro Analytic Service to apply the new values.

For more information about resource limits, see [“Configuring SAS Micro Analytic Service Resources” on page 25](#) and [Kubernetes Resources](#).

Publishing New Modules to a Deployment with Replicated Pods

When a module is published in a pod, it is compiled and available on that instance.

If your deployment includes replica pods, SAS Micro Analytic Service runs a background process that ensures the eventual consistency of the modules running in each pod. Eventual consistency models are commonly used by distributed applications, including those running in a Kubernetes environment.

Because this process works with the readiness probe, requests are not serviced by a pod that is being updated and, therefore, not ready.

In the rare circumstance that an un-updated pod receives a request to run a new module before the pod status is changed to unavailable (to load the new module) you receive a Module Not Found error. In this case, you can retry the call after 10 seconds.

By default, this process checks for pod module consistency every 10 seconds. To modify that value, use the `service.cache.refresh.ratemilliseconds` property in SAS Environment Manager. For more information, see [“sas.microanalyticsservice.system: supplementalProperties”](#) on page 276.

PART 3

SAS Micro Analytic Service DS2 Extensions Reference

Chapter 7	
<i>Datagrid Package</i>	35
Chapter 8	
<i>MASCall Package</i>	37
Chapter 9	
<i>MASParallelExec Package</i>	57

Datagrid Package

<i>About the Datagrid Package</i>	35
---	----

About the Datagrid Package

The datagrid package documentation is located in *SAS DS2 Programmer's Guide* and *SAS DS2 Language Reference*. See the following for more information:

- [“DS2 Datagrid Package Methods, Operators, and Statements” in SAS DS2 Language Reference](#)
- [“Using the Datagrid Package” in SAS DS2 Programmer's Guide](#)

MASCall Package

About the MASCall Package	38
MASCall Methods	38
Dictionary	40
allocParms Method	40
allocRevParms Method	40
asyncCallModule Method	41
callModule Method	42
getDatagrid Method	42
getDouble Method	43
getDoubleArray Method	44
getInt Method	45
getIntArray Method	45
getLong Method	46
getLongArray Method	47
getString Method	47
getStringArray Method	48
isWithinAsyncCall Method	49
releaseParms Method	49
setDatagrid Method	50
setDouble Method	50
setDoubleArray Method	51
setInt Method	52
setIntArray Method	52
setLong Method	53
setLongArray Method	54
setString Method	54
setStringArray Method	55

About the MASCall Package

The MASCall package contains methods that enable separate DS2 modules, published to SAS Micro Analytic Service, to call one another across separate modules. For more information, see [“Performing Calls between SAS Micro Analytic Service Modules” on page 178](#).

MASCall Methods

Category	Language Elements	Description
Execution	asyncCallModule Method (p. 41)	Adds a request to the asynchronous execution queue to execute the external method that is specified in the preceding <code>allocParms()</code> or <code>allocRevParms()</code> method call.
	callModule Method (p. 42)	Executes the external method that is specified in the preceding <code>allocParms()</code> or <code>allocRevParms()</code> method call.
Miscellaneous	isWithinAsyncCall Method (p. 49)	Determines whether the current method is running asynchronously as a result of being scheduled via <code>asyncCallModule()</code> . If the current method is running asynchronously, a call to <code>asyncCallModule()</code> is not supported and returns a nonzero return code. This indicates that the request to execute the external method was rejected.
Resource Management	allocParms Method (p. 40)	Creates a parameter list for the latest revision of the specified method.
	allocRevParms Method (p. 40)	For the specified revision, creates a parameter list for the specified method.
	releaseParms Method (p. 49)	Releases the parameter memory that is used by the MASCall package.
Retrieve Values	getDatagrid Method (p. 42)	For the specified data grid, retrieves the data grid and delivers it to the local variable. This is the data grid that is produced by the external method execution.
	getDouble Method (p. 43)	For the specified parameter, retrieves the double value and delivers it to the local variable. This is the value that is produced by the external method execution.

Category	Language Elements	Description
	getDoubleArray Method (p. 44)	For the specified parameter, retrieves the array of double values and delivers them to the local variable. These are the values that are produced by the external method execution.
	getInt Method (p. 45)	For the specified parameter, retrieves the 32-bit integer value and delivers it to the local variable. This is the value that is produced by the external method execution.
	getIntArray Method (p. 45)	For the specified parameter, retrieves the array of 32-bit integer values and delivers them to the local variable. These are the values that are produced by the external method execution.
	getLong Method (p. 46)	For the specified parameter, retrieves the 64-bit integer value and delivers it to the local variable. This is the value that is produced by the external method execution.
	getLongArray Method (p. 47)	For the specified parameter, retrieves the array of 64-bit integer values and delivers them to the local variable. These are the values that are produced by the external method execution.
	getString Method (p. 47)	For the specified parameter, retrieves the string value and delivers it to the local variable. This is the value that is produced by the external method execution.
	getStringArray Method (p. 48)	For the specified parameter, retrieves the array of string values and delivers them to the local variable. These are the values that are produced by the external method execution.
	setDatagrid Method (p. 50)	For the specified data grid, sets the data grid value to use in the external method execution.
Set Values	setDouble Method (p. 50)	For the specified parameter, sets the double value to use in the external method execution.
	setDoubleArray Method (p. 51)	For the specified parameter, sets the double array values to use in the external method execution.
	setInt Method (p. 52)	For the specified parameter, sets the 32-bit integer value to use in the external method execution.
	setIntArray Method (p. 52)	For the specified parameter, sets the 32-bit integer array values to use in the external method execution.
	setLong Method (p. 53)	For the specified parameter, sets the 64-bit integer value to use in the external method execution.
	setLongArray Method (p. 54)	For the specified parameter, sets the 64-bit integer array values to use in the external method execution.
	setString Method (p. 54)	For the specified parameter, sets the string value to use in the external method execution.

Category	Language Elements	Description
	setStringArray Method (p. 55)	For the specified parameter, sets the string array values to use in the external method execution.

Dictionary

allocParms Method

Creates a parameter list for the latest revision of the specified method.

Category: Resource Management

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

```
rc=package.allocParms(module, method);
```

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

module

specifies the module that contains the method that you want to execute.

method

specifies the method for which you want to obtain a parameter list.

allocRevParms Method

For the specified revision, creates a parameter list for the specified method.

Category: Resource Management

Returned data type:	INTEGER
Note:	This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

```
rc=package.allocRevParms(module, revision, method);
```

Required Arguments

- rc**
specifies the variable in which to place the return code.
- package**
specifies an instance of the MASCall package.
- module**
specifies the module that contains the method that you want to execute.
- revision**
specifies the revision number of the module that you want to use.
- method**
specifies the method for which you want to obtain a parameter list.

asyncCallModule Method

Adds a request to the asynchronous execution queue to execute the external method that is specified in the preceding allocParms() or allocRevParms() method call.

Category:	Execution
Returned data type:	INTEGER
Notes:	This method returns 0 (zero) if it is successful. It returns 1 if the asynchronous task queue is full. Any other nonzero value indicates failure. The asyncCallModule method is not currently supported for use with SAS Event Stream Processing.

Syntax

```
rc=package.asyncCallModule();
```

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

Details

There is a configuration property and certain rules that are associated with the use of the `asyncCallModule` method. For more information, see [“Additional Information about the `asyncCallModule` Method” on page 115](#).

callModule Method

Executes the external method that is specified in the preceding `allocParms()` or `allocRevParms()` method call.

Category: Execution

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

```
rc=package.callModule();
```

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

getDatagrid Method

For the specified data grid, retrieves the data grid and delivers it to the local variable. This is the data grid that is produced by the external method execution.

Category: Retrieve Values

Returned data
type: DATAGRID

Syntax

Form 1: `rc=package.getDatagrid(srcParm, destDatagridInstanceVar);`
 Form 2: `package.getDatagrid(srcParm, destDatagridInstanceVar);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

srcParm

specifies the parameter from which to retrieve a value. The value is the result of executing the external method. You can specify the parameter using either a parameter name or a zero-based index value.

destDatagridInstanceVar

specifies the local variable to receive the specified data grid. The data grid value is the result of executing the external method.

.....
Note: If data exists in the specified data grid, that data is overwritten.

getDouble Method

For the specified parameter, retrieves the double value and delivers it to the local variable. This is the value that is produced by the external method execution.

Category: Retrieve Values

Returned data
type: DECIMAL

Note: This method returns MISSING if the specified argument name is not found.

Syntax

`destVar=package.getDouble(srcParm);`

Required Arguments

destVar

specifies the local variable to receive the specified parameter's value. The parameter's value is the result of executing the external method.

package

specifies an instance of the MASCall package.

srcParm

specifies the parameter from which to retrieve a value. The value is the result of executing the external method. You can specify the parameter using either a parameter name or a zero-based parameter index value.

getDoubleArray Method

For the specified parameter, retrieves the array of double values and delivers them to the local variable. These are the values that are produced by the external method execution.

Category:	Retrieve Values
Returned data type:	DECIMAL ARRAY

Syntax

```
package.getDoubleArray(srcParm, destArrayVar, rc);
```

Required Arguments

package

specifies an instance of the MASCall package.

srcParm

specifies the parameter from which to retrieve a value. The value is the result of executing the external method. You can specify the parameter using either a parameter name or a zero-based parameter index value.

destArrayVar

specifies the local array variable to receive the specified parameter's value. The parameter's value is the result of executing the external method.

rc

specifies the variable in which to place the return code.

getInt Method

For the specified parameter, retrieves the 32-bit integer value and delivers it to the local variable. This is the value that is produced by the external method execution.

Category: Retrieve Values

Returned data type: 32-bit INTEGER

Note: This method returns MISSING if the specified argument name is not found.

Syntax

```
destVar=package.getInt(srcParm);
```

Required Arguments

destVar

specifies the local variable to receive the specified parameter's value. The parameter's value is the result of executing the external method.

package

specifies an instance of the MASCall package.

srcParm

specifies the parameter from which to retrieve a value. The value is the result of executing the external method. You can specify the parameter using either a parameter name or a zero-based parameter index value.

getIntArray Method

For the specified parameter, retrieves the array of 32-bit integer values and delivers them to the local variable. These are the values that are produced by the external method execution.

Category: Retrieve Values

Returned data type: 32-bit INTEGER ARRAY

Syntax

```
package.getIntArray(srcParm, destArrayVar, rc);
```

Required Arguments

package

specifies an instance of the MASCall package.

srcParm

specifies the parameter from which to retrieve a value. The value is the result of executing the external method. You can specify the parameter using either a parameter name or a zero-based parameter index value.

destArrayVar

specifies the local array variable to receive the specified parameter's value. The parameter's value is the result of executing the external method.

rc

specifies the variable in which to place the return code.

getLong Method

For the specified parameter, retrieves the 64-bit integer value and delivers it to the local variable. This is the value that is produced by the external method execution.

Category: Retrieve Values

Returned data type: 64-bit INTEGER

Note: This method returns MISSING if the specified argument name is not found.

Syntax

```
destVar=package.getLong(srcParm);
```

Required Arguments

destVar

specifies the local variable to receive the specified parameter's value. The parameter's value is the result of executing the external method.

package

specifies an instance of the MASCall package.

srcParm

specifies the parameter from which to retrieve a value. The value is the result of executing the external method. You can specify the parameter using either a parameter name or a zero-based parameter index value.

getLongArray Method

For the specified parameter, retrieves the array of 64-bit integer values and delivers them to the local variable. These are the values that are produced by the external method execution.

Category: Retrieve Values
Returned data type: 64-bit INTEGER ARRAY

Syntax

```
package.getLongArray(srcParm, destArrayVar, rc);
```

Required Arguments

package

specifies an instance of the MASCall package.

srcParm

specifies the parameter from which to retrieve a value. The value is the result of executing the external method. You can specify the parameter using either a parameter name or a zero-based parameter index value.

destArrayVar

specifies the local array variable to receive the specified parameter's value. The parameter's value is the result of executing the external method.

rc

specifies the variable in which to place the return code.

getString Method

For the specified parameter, retrieves the string value and delivers it to the local variable. This is the value that is produced by the external method execution.

Category: Retrieve Values
Returned data type: STRING

Note: This method returns MISSING if the specified argument name is not found.

Syntax

```
destVar=package.getString(srcParm);
```

Required Arguments

destVar

specifies the local variable to receive the specified parameter's value. The parameter's value is the result of executing the external method.

package

specifies an instance of the MASCall package.

srcParm

specifies the parameter from which to retrieve a value. The value is the result of executing the external method. You can specify the parameter using either a parameter name or a zero-based parameter index value.

getStringArray Method

For the specified parameter, retrieves the array of string values and delivers them to the local variable. These are the values that are produced by the external method execution.

Category:	Retrieve Values
Returned data type:	32-bit STRING ARRAY

Syntax

```
package.getStringArray(srcParm, destArrayVar, rc);
```

Required Arguments

package

specifies an instance of the MASCall package.

srcParm

specifies the parameter from which to retrieve a value. The value is the result of executing the external method. You can specify the parameter using either a parameter name or a zero-based parameter index value.

destArrayVar

specifies the local array variable to receive the specified parameter's value. The parameter's value is the result of executing the external method.

rc

specifies the variable in which to place the return code.

isWithinAsyncCall Method

Determines whether the current method is running asynchronously as a result of being scheduled via `asyncCallModule()`. If the current method is running asynchronously, a call to `asyncCallModule()` is not supported and returns a nonzero return code. This indicates that the request to execute the external method was rejected.

Category: Miscellaneous

Returned data type: INTEGER

Note: This method returns 1 (true) if the current method is being executed. It returns 0 (false) if the current method is not being executed.

Syntax

```
package.isWithinAsyncCall();
```

Required Argument

package

specifies an instance of the MASCall package.

Example

```
if ( mc.isWithinAsyncCall() ) then do;
```

releaseParms Method

Releases the parameter memory that is used by the MASCall package.

Category: Resource Management

Syntax

```
package.releaseParms();
```

setDatagrid Method

For the specified data grid, sets the data grid value to use in the external method execution.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

```
rc=package.setDatagrid(destParm, srcDatagridInstanceVar);
```

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

destParm

specifies the external method parameter to be set. You can specify the parameter using either a parameter name or a zero-based parameter index value.

srcDatagridInstanceVar

specifies the local data grid to set in the specified external method parameter. When `callModule()` or `asyncCallModule()` is executed, this data grid is used in the execution of the external method.

setDouble Method

For the specified parameter, sets the double value to use in the external method execution.

Category: Set Values

Syntax

```
rc=package.setDouble(destParm, srcValue);
```

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

destParm

specifies the external method parameter to be set. You can specify the parameter using either a parameter name or a zero-based parameter index value.

srcValue

specifies the local value to set in the specified external method parameter. When `callModule()` or `asyncCallModule()` is executed, this value is used in the execution of the external method.

setDoubleArray Method

For the specified parameter, sets the double array values to use in the external method execution.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

```
rc=package.setDoubleArray(destParm, srcValue);
```

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

destParm

specifies the external method parameter to be set. You can specify the parameter using either a parameter name or a zero-based parameter index value.

srcValue

specifies the local value to set in the specified external method parameter. When `callModule()` or `asyncCallModule()` is executed, this value is used in the execution of the external method.

setInt Method

For the specified parameter, sets the 32-bit integer value to use in the external method execution.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

```
rc=package.setInt(destParm, srcValue);
```

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

destParm

specifies the external method parameter to be set. You can specify the parameter using either a parameter name or a zero-based parameter index value.

srcValue

specifies the local value to set in the specified external method parameter. When `callModule()` or `asyncCallModule()` is executed, this value is used in the execution of the external method.

setIntArray Method

For the specified parameter, sets the 32-bit integer array values to use in the external method execution.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

```
rc=package.setIntArray(destParm, srcValue);
```

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

destParm

specifies the external method parameter to be set. You can specify the parameter using either a parameter name or a zero-based parameter index value.

srcValue

specifies the local value to set in the specified external method parameter. When `callModule()` or `asyncCallModule()` is executed, this value is used in the execution of the external method.

setLong Method

For the specified parameter, sets the 64-bit integer value to use in the external method execution.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

```
rc=package.setLong(destParm, srcValue);
```

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

destParm

specifies the external method parameter to be set. You can specify the parameter using either a parameter name or a zero-based parameter index value.

srcValue

specifies the local value to set in the specified external method parameter. When `callModule()` or `asyncCallModule()` is executed, this value is used in the execution of the external method.

setLongArray Method

For the specified parameter, sets the 64-bit integer array values to use in the external method execution.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

```
rc=package.setLongArray(destParm, srcValue);
```

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

destParm

specifies the external method parameter to be set. You can specify the parameter using either a parameter name or a zero-based parameter index value.

srcValue

specifies the local value to set in the specified external method parameter. When `callModule()` or `asyncCallModule()` is executed, this value is used in the execution of the external method.

setString Method

For the specified parameter, sets the string value to use in the external method execution.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

```
rc=package.setString(destParm, srcValue);
```

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

destParm

specifies the external method parameter to be set. You can specify the parameter using either a parameter name or a zero-based parameter index value.

srcValue

specifies the local value to set in the specified external method parameter. When `callModule()` or `asyncCallModule()` is executed, this value is used in the execution of the external method.

setStringArray Method

For the specified parameter, sets the string array values to use in the external method execution.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

```
rc=package.setStringArray(destParm, srcValue);
```

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASCall package.

destParm

specifies the external method parameter to be set. You can specify the parameter using either a parameter name or a zero-based parameter index value.

srcValue

specifies the local value to set in the specified external method parameter. When `callModule()` or `asyncCallModule()` is executed, this value is used in the execution of the external method.

MASParallelExec Package

<i>About the MASParallelExec Package</i>	57
<i>MASParallelExec Methods</i>	58
<i>Dictionary</i>	59
addCall Method	59
call Method	60
callResults Method	60
callStatus Method	61
getBool Method	62
getDatagrid Method	63
getDouble Method	64
getInt Method	65
getLong Method	66
getResults Method	67
getString Method	68
getVarbinary Method	69
setBool Method	70
setDatagrid Method	71
setDouble Method	72
setInt Method	73
setLong Method	74
setString Method	75
setVarbinary Method	76
wait Method	77

About the MASParallelExec Package

The MASParallelExec package contains methods that enable a DS2 module to call module methods concurrently. For more information, see [“Performing Parallel Sub-flow Calls in a SAS Micro Analytic Service Module”](#) on page 185.

IMPORTANT To enable use of the MASParallelExec package, you must set the maximum number of threads to use with parallel execution by assigning a value to the `core.maxpthreads` property. For more information, see [“sas.microanalyticservice.system: supplementalProperties” on page 276](#).

Note: At this time, the MASParallelExec package is not supported for use with SAS Event Stream Processing.

MASParallelExec Methods

Category	Language Elements	Description
Execution	call Method (p. 60)	By default, the set of parallel execution calls are not scheduled to execute until the WAIT call is made. You can use this method to initiate the parallel execution of a call definition prior to the WAIT call.
	wait Method (p. 77)	Wait for parallel execute to complete.
Resource Management	addCall Method (p. 59)	Add a parallel method call to the parallel execution request.
Results	callResults Method (p. 60)	Retrieve an executed parallel call output datagrid parameter.
	callStatus Method (p. 61)	Retrieve a parallel call status.
	getResults Method (p. 67)	Retrieve an executed parallel call output datagrid parameter.
Retrieve Values	getBool Method (p. 62)	Fetch a parallel call Boolean output parameter value.
	getDatagrid Method (p. 63)	Fetch a parallel call datagrid output parameter value.
	getDouble Method (p. 64)	Fetch a parallel call double output parameter value.
	getInt Method (p. 65)	Fetch a parallel call integer output parameter value.
	getLong Method (p. 66)	Fetch a parallel call long output parameter value.
	getString Method (p. 68)	Fetch a parallel call string output parameter value.
	getVarbinary Method (p. 69)	Fetch a parallel call varbinary output parameter value.
Set Values	setBool Method (p. 70)	Assign a parallel call Boolean input parameter value.

Category	Language Elements	Description
	setDatagrid Method (p. 71)	Assign a parallel call datagrid input parameter value.
	setDouble Method (p. 72)	Assign a parallel call double input parameter value.
	setInt Method (p. 73)	Assign a parallel call integer input parameter value.
	setLong Method (p. 74)	Assign a parallel call long input parameter value.
	setString Method (p. 75)	Assign a parallel call string input parameter value.
	setVarbinary Method (p. 76)	Assign a parallel call varbinary input parameter value.

Dictionary

addCall Method

Add a parallel method call to the parallel execution request.

Category: Resource Management

Returned data type: INTEGER

Notes: The methodName must have an in_out package datagrid as its last argument. This method returns 0 (zero) or a positive value if it is successful. The returned value is the unique identifier, within the context of the given package, of the added call definition. A negative value indicates failure. The negative value indicates the cause of failure.

Syntax

```
rc=package.addCall(name, methodName);
```

Required Arguments

rc
specifies the variable in which to place the return code.

package
specifies an instance of the MASParallelExec package.

name

Specifies the name to assign to the parallel method call definition. The name must be unique within the parallel execution instance.

methodName

Specifies the name of the module method to call.

call Method

By default, the set of parallel execution calls are not scheduled to execute until the WAIT call is made. You can use this method to initiate the parallel execution of a call definition prior to the WAIT call.

Category:	Execution
Returned data type:	INTEGER
Note:	This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

Form 1: `rc=package.call(callId);`

Form 2: `rc=package.call(callName);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

callResults Method

Retrieve an executed parallel call output datagrid parameter.

Category:	Results
-----------	---------

Returned data type:	INTEGER
Notes:	This method returns 0 (zero) if it is successful. A nonzero value indicates failure. This method is an alias of the <code>getResults</code> method.

Syntax

- Form 1: `rc=package.callResults(callId, datagrid);`
 Form 2: `rc=package.callResults(callName, datagrid);`

Required Arguments

- rc**
 specifies the variable in which to place the return code.
- package**
 specifies an instance of the MASParallelExec package.
- callId**
 Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via `addCall`.
- callName**
 Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via `addCall`.
- datagrid**
 Specifies the datagrid instance to populate with the requested parallel call output datagrid.

callStatus Method

Retrieve a parallel call status.

Category:	Results
Returned data type:	INTEGER
Note:	This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

- `rc=package.callStatus(callId, value);`
`rc=package.callStatus(callName, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

value

Specifies an integer containing the status of the requested parallel call definition.

The returned status value is one of the following:

Value	Description
<0	Failure
0	Success
1	Timed Out
2	Canceled
3	Queued
4	Unscheduled

getBool Method

Fetch a parallel call Boolean output parameter value.

Category: Retrieve Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

- Form 1: `rc=package.getBool(callId, index, value);`
 Form 2: `rc=package.getBool(callName, index, value);`
 Form 3: `rc=package.getBool(callId, name, value);`
 Form 4: `rc=package.getBool(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the output parameter value to fetch. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the output parameter value to fetch.

value

Specifies a Boolean value in which to assign from the output parameter value.

getDatagrid Method

Fetch a parallel call datagrid output parameter value.

Category: Retrieve Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

- Form 1: `rc=package.getDatagrid(callId, index, value);`
 Form 2: `rc=package.getDatagrid(callName, index, value);`

Form 3: `rc=package.getDatagrid(callId, name, value);`

Form 4: `rc=package.getDatagrid(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the output parameter value to fetch. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the output parameter value to fetch.

value

Specifies a datagrid value in which to assign from the output parameter value.

getDouble Method

Fetch a parallel call double output parameter value.

Category: Retrieve Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

Form 1: `rc=package.getDouble(callId, index, value);`

Form 2: `rc=package.getDouble(callName, index, value);`

Form 3: `rc=package.getDouble(callId, name, value);`

Form 4: `rc=package.getDouble(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the output parameter value to fetch. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the output parameter value to fetch.

value

Specifies a double value in which to assign from the output parameter value.

getInt Method

Fetch a parallel call integer output parameter value.

Category: Retrieve Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

- Form 1: `rc=package.getInt(callId, index, value);`
 Form 2: `rc=package.getInt(callName, index, value);`
 Form 3: `rc=package.getInt(callId, name, value);`
 Form 4: `rc=package.getInt(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the output parameter value to fetch. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the output parameter value to fetch.

value

Specifies an integer value in which to assign from the output parameter value.

getLong Method

Fetch a parallel call long output parameter value.

Category: Retrieve Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

Form 1: `rc=package.getLong(callId, index, value);`

Form 2: `rc=package.getLong(callName, index, value);`

Form 3: `rc=package.getLong(callId, name, value);`

Form 4: `rc=package.getLong(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the output parameter value to fetch. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the output parameter value to fetch.

value

Specifies a long value in which to assign from the output parameter value.

getResults Method

Retrieve an executed parallel call output datagrid parameter.

Category: Results

Returned data type: INTEGER

Notes: This method returns 0 (zero) if it is successful. A nonzero value indicates failure. This method is an alias of the callResults method.

Syntax

Form 1: `rc=package.getResults(callId, datagrid);`

Form 2: `rc=package.getResults(callName, datagrid);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

datagrid

Specifies the datagrid instance to populate with the requested parallel call output datagrid.

getString Method

Fetch a parallel call string output parameter value.

Category: Retrieve Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

Form 1: `rc=package.getString(callId, index, value);`

Form 2: `rc=package.getString(callName, index, value);`

Form 3: `rc=package.getString(callId, name, value);`

Form 4: `rc=package.getString(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the output parameter value to fetch. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the output parameter value to fetch.

value

Specifies a string value in which to assign from the output parameter value.

getVarbinary Method

Fetch a parallel call varbinary output parameter value.

Category: Retrieve Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

- Form 1: `rc=package.getVarbinary(callId, index, value);`
Form 2: `rc=package.getVarbinary(callName, index, value);`
Form 3: `rc=package.getVarbinary(callId, name, value);`
Form 4: `rc=package.getVarbinary(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the output parameter value to fetch. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the output parameter value to fetch.

value

Specifies a varbinary value in which to assign from the output parameter value.

setBool Method

Assign a parallel call Boolean input parameter value.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure.

Syntax

- Form 1: `rc=package.setBool(callId, index, value);`
Form 2: `rc=package.setBool(callName, index, value);`
Form 3: `rc=package.setBool(callId, name, value);`
Form 4: `rc=package.setBool(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the input parameter value to assign. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the input parameter value to assign.

value

Specifies a Boolean value to assign as the input parameter.

setDatagrid Method

Assign a parallel call datagrid input parameter value.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure. The negative value indicates the cause of the failure.

Syntax

- Form 1: `rc=package.setDatagrid(callId, index, value);`
- Form 2: `rc=package.setDatagrid(callName, index, value);`
- Form 3: `rc=package.setDatagrid(callId, name, value);`
- Form 4: `rc=package.setDatagrid(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the input parameter value to assign. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the input parameter value to assign.

value

Specifies a datagrid value to assign as the input parameter.

setDouble Method

Assign a parallel call double input parameter value.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure. The negative value indicates the cause of the failure.

Syntax

Form 1: `rc=package.setDouble(callId, index, value);`

Form 2: `rc=package.setDouble(callName, index, value);`

Form 3: `rc=package.setDouble(callId, name, value);`

Form 4: `rc=package.setDouble(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the input parameter value to assign. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the input parameter value to assign.

value

Specifies a double value to assign as the input parameter.

setInt Method

Assign a parallel call integer input parameter value.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure. The negative value indicates the cause of the failure.

Syntax

- Form 1: `rc=package.setInt(callId, index, value);`
Form 2: `rc=package.setInt(callName, index, value);`
Form 3: `rc=package.setInt(callId, name, value);`
Form 4: `rc=package.setInt(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the input parameter value to assign. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the input parameter value to assign.

value

Specifies an integer value to assign as the input parameter.

setLong Method

Assign a parallel call long input parameter value.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure. The negative value indicates the cause of the failure.

Syntax

- Form 1: `rc=package.setInt(callId, index, value);`
Form 2: `rc=package.setInt(callName, index, value);`
Form 3: `rc=package.setInt(callId, name, value);`
Form 4: `rc=package.setInt(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the input parameter value to assign. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the input parameter value to assign.

value

Specifies a long value to assign as the input parameter.

setString Method

Assign a parallel call string input parameter value.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure. The negative value indicates the cause of the failure.

Syntax

Form 1: `rc=package.setString(callId, index, value);`

Form 2: `rc=package.setString(callName, index, value);`

Form 3: `rc=package.setString(callId, name, value);`

Form 4: `rc=package.setString(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the input parameter value to assign. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the input parameter value to assign.

value

Specifies a string value to assign as the input parameter.

setVarbinary Method

Assign a parallel call varbinary input parameter value.

Category: Set Values

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates failure. The negative value indicates the cause of the failure.

Syntax

- Form 1: `rc=package.setVarbinary(callId, index, value);`
Form 2: `rc=package.setVarbinary(callName, index, value);`
Form 3: `rc=package.setVarbinary(callId, name, value);`
Form 4: `rc=package.setVarbinary(callName, name, value);`

Required Arguments

rc

specifies the variable in which to place the return code.

package

specifies an instance of the MASParallelExec package.

callId

Specifies the unique call identifier to initiate execution. This is the identifier that was returned when the call was created via addCall.

callName

Specifies the unique call name to initiate execution. This is the name that was specified when the call was created via addCall.

index

Specifies an integer value that indicates the position of the input parameter value to assign. Note that the index value is one-based.

name

Specifies a string value that indicates the name of the input parameter value to assign.

value

Specifies a varbinary value to assign as the input parameter.

wait Method

Wait for parallel execute to complete.

Category: Execution

Returned data type: INTEGER

Note: This method returns 0 (zero) if it is successful. A nonzero value indicates that one or more parallel calls did not complete. A negative value indicates that a failure occurred before any of the parallel calls could complete. A value of 1 indicates that the time-out was reached before all defined parallel calls completed.

Syntax

- Form 1: `rc=package.wait();`
 Form 2: `rc=package.wait(dgStatus);`
 Form 3: `rc=package.wait(waitMillis);`
 Form 4: `rc=package.wait(waitMillis,dgStatus);`

Required Arguments

rc
 specifies the variable in which to place the return code.

package
 specifies an instance of the MASParallelExec package.

dgStatus
 Specifies the datagrid package instance to assign with defined parallel call results. The returned datagrid columns are as follows:

Column	Column Name	Column Type	Column Description
1	CallID	Long	A unique identifier, within the context of the containing package, for the call that was returned by addCall.
2	CallName	String	The name of the call passed as <code>name</code> to addCall.
3	Status	Long	The execution completion status: ■ <0 Failure ■ 0 Success ■ 1 Timed out

Column	Column Name	Column Type	Column Description
--------	-------------	-------------	--------------------

■ 2 Canceled

waitMillis

Specifies the number of milliseconds to wait for parallel execution completion. Note the following information:

- If set to zero, the wait method pauses until all of the parallel calls are complete before returning.
- If set to -1, the configured default wait time is used. You can modify the default wait value using the `core.pawaittimemilliseconds` property in [“sas.microanalyticsservice.system: supplementalProperties” on page 276](#).

PART 4

Using SAS Micro Analytic Service with SAS Event Stream Processing

Chapter 10	
<i>Publishing to SAS Micro Analytic Service in SAS Event Stream Processing</i>	81
Chapter 11	
<i>Generating Derived Events</i>	89
Chapter 12	
<i>DS2 Programming for SAS Micro Analytic Service</i>	95
Chapter 13	
<i>State Sharing between Modules</i>	121
Chapter 14	
<i>Best Practices for DS2 Programming in SAS Event Stream Processing</i> .	137
Chapter 15	
<i>Python Support in SAS Micro Analytic Service</i>	145
Chapter 16	
<i>SAS Micro Analytic Service Logging and Deployment</i>	157

Publishing to SAS Micro Analytic Service in SAS Event Stream Processing

<i>Overview</i>	81
<i>Object Hierarchy</i>	82
<i>XML Example</i>	82
<i>Data Type Mappings</i>	85

Overview

SAS Event Stream Processing uses data flow models, known as continuous queries, that define how events are routed among the various windows that make up a continuous query.

Source windows are required for each continuous query. All event streams enter continuous queries by being published or injected into a Source window. Source windows are typically connected to one or more derived windows. Derived windows can detect patterns in the data, transform the data, aggregate the data, analyze the data, or perform computations based on the data. For more information, see the [SAS Event Stream Processing documentation](#).

The derived window in which SAS Micro Analytic Service operates is the Calculate window. Within a continuous query application, Calculate windows can be configured to receive events from one or more source windows. These source events can be processed by modules published to SAS Micro Analytic Service, which can in turn generate zero or more derived events. These derived events can be subscribed to by downstream windows. For more information, see [“Working with SAS Micro Analytic Service Modules” in SAS Event Stream Processing: Using Streaming Analytics](#).

SAS Event Stream Processing Studio can be used to author continuous queries. Continuous queries can include elements that publish modules to SAS Micro Analytic Service, and which identify the methods to use to process specific event streams. Up to one method can be specified for each window that streams events directly into a Calculate window (for example, for each upstream window connected to a Calculate window by an edge).

Object Hierarchy

SAS Event Stream Processing provides an assortment of elements for use in building continuous query applications, enabling a wide variety of business needs to be met. Some of the elements that incorporate SAS Micro Analytic Service functionality are described briefly below. For more information, see the [SAS Event Stream Processing documentation](#).

At a minimum, each event stream processing application consists of a SAS Event Stream Processing Engine (a container of projects), at least one project (a container of continuous queries), and at least one continuous query that contains windows and the edges that connect them.

Each project maintains a SAS Micro Analytic Service environment that is shared among the continuous queries within the project. DS2 and Python modules are published to SAS Micro Analytic Service by including the `<mas-modules>` tag (a sub-element of `<project>`).

SAS Micro Analytic Service operates on the data that is contained in events received by Calculate windows. The Calculate window provides an XML element for mapping source window events to module methods. The XML tag is named `<mas-map>`, which is a sub-element of `<window-calculate>`.

XML Example

Here is an example of an XML continuous query definition. SAS Micro Analytic Service related elements are highlighted.

```
<engine port='55555'>
  <description>
    This example has one source window and one calculate
    window. The calculate window uses DS2 code to calculate
    a value from the source window.
```

```

  The engine element creates the single engine top level container which
  sets up dfESP fundamental services such as licensing and logging.
  This single engine instance wraps one or more projects that wrap
  one or more continuous queries and different types of windows.
```

```

</description>
<projects>
  <project name='trades_proj' pubsub='auto' threads='4'>
    <description>
      This is to create a project. A project specifies a container
      that holds one or more continuous queries and are backed by a
      thread pool of user defined size. You can specify the pubsub
      port and type, number of threads for the project, index type,
      and a tag token data flow model.
    </description>
    <mas-modules>
      <mas-module language="ds2" module="module_1" func-names='compute_volume'>
        <description>
          <![CDATA[This is a SAS Micro Analytic Service module in DS2 ]]>
        </description>
        <code>
          <![CDATA[
            ds2_options sas; /* SAS-style missing value handling */
            package module_1/overwrite=yes;
            method compute_volume(int quantity, double price, in_out int volume);
              volume = quantity * price;
            end;
            endpackage;
          ]]>
        </code>
      </mas-module>
    </mas-modules>
    <contqueries>
      <contquery name='trades_traders_cq' trace='cw1'>
        <description>
          This specifies the continuous query container that holds
          a collection of windows and enables you to specify the
          connectivity between windows. You can turn on tracing
          for a list of windows, and specify the index type for
          windows in the query.
        </description>
        <windows>
          <window-source name='Trades' index='pi_RBTREE'>
            <description>
              This defines a source window. All event streams must
              enter continuous queries by being published or
              injected into a source window.
            </description>
            <schema>
              <fields>
                <field name='tradeID' type='string' key='true'/>
                <field name='security' type='string'/>
                <field name='quantity' type='int32'/>
                <field name='price' type='double'/>
                <field name='traderID' type='int64'/>
                <field name='time' type='string'/>
              </fields>
            </schema>
            <connectors>
              <connector class='fs' name='pub'>
                <properties>

```

```

        <property name='type'>pub</property>
        <property name='fstype'>csv</property>
        <property name='fsname'>input.csv</property>
        <property name='transactional'>true</property>
    </properties>
</connector>
</connectors>
</window-source>
<window-calculate name='cw1' algorithm='MAS'>
    <description>
        This defines a calculate window. The window passes
        all fields of the event as variables to the DS2 program.
    </description>
    <schema>
        <fields>
            <field name='tradeID' type='string' key='true' />
            <field name='security' type='string' />
            <field name='quantity' type='int32' />
            <field name='price' type='double' />
            <field name='traderID' type='int64' />
            <field name='time' type='string' />
            <field name='volume' type='int32' key='true' />
        </fields>
    </schema>
    <mas-map>
        <window-map module="module_1" revision="0" source="Trades"
            function="compute_volume" />
    </mas-map>
</connectors>
    <connector class='fs' name='sub'>
        <properties>
            <property name='type'>sub</property>
            <property name='fstype'>csv</property>
            <property name='fsname'>output.csv</property>
            <property name='snapshot'>true</property>
        </properties>
    </connector>
</connectors>
</window-calculate>
</windows>
<edges>
    <description>
        This fully specifies the continuous query with window
        connectivity, which is a directed graph.
    </description>
    <edge source='Trades' target='cw1' role='data'>
</edges>
</contquery>
</contqueries>
<project-connectors>
    <connector-groups>
        <connector-group name='sub_group'>
            <connector-entry connector='trades_traders_cq/cw1/sub'
                state='running' />
        </connector-group>
        <connector-group name='pub_group'>

```

```

    <connector-entry connector='trades_traders_cq/Trades/pub'
      state='finished' />
  </connector-group>
</connector-groups>
<edges>
  <edge source='sub_group' target='pub_group' />
</edges>
</project-connectors>
</project>
</projects>
</engine>

```

Data Type Mappings

SAS Micro Analytic Service executes within the Calculate windows of continuous query applications. A continuous query can specify that events from one or more upstream windows be processed by SAS Micro Analytic Service when those events are received by a given Calculate window. This specification maps each such input window's events to one of the methods that have been published to SAS Micro Analytic Service.

During continuous query initialization, when a window's events are registered with a method, SAS Micro Analytic Service inspects the window's event schema and automatically maps the event's fields to method input parameters. This is done by matching event field names with parameter names. It is legal for some, none, or all of the names to match. It is also legal for methods to have no input parameters. In that case, no input mapping is done. SAS Micro Analytic Service performs similar matching of the specified method's output parameter names with the Calculate window's event field names. Therefore, at run time, when an event is received from the specified window, method input values are taken from that event, the method is executed, and the results are used to create a derived event. The event flows downstream from the Calculate window to any subscribers.

The following table describes the data type mappings between event field types and DS2 method parameter types, and between event field types and Python function argument types. SAS Micro Analytic Service automatically translates the data types as needed according to the table below. The Event Stream Processing Event Field Type column lists the schema tag of each data type.

Event Stream Processing Event Field Type	Event Stream Processing Type Description	DS2 Method Parameter Type	DS2 Type Description	Python Function Argument Type	Python Type Description
Boolean	8-bit signed	bool	Boolean	bool	Boolean
int32	32-bit signed integer	int	32-bit signed integer	long	Long integer

Event Stream Processing Event Field Type	Event Stream Processing Type Description	DS2 Method Parameter Type	DS2 Type Description	Python Function Argument Type	Python Type Description
int64	64-bit signed integer	BIGINT	64-bit signed integer	long	Long integer
double	IEEE double	double	IEEE double	float	Floating-point real
string	UTF-8 string	CHAR, NCHAR, VARCHAR, NVARCHAR	UTF-8 string	string	Unicode string
money	192-bit fixed decimal	double	IEEE double	float	Floating-point real
date	Date and time, as seconds since January 1, 1970	BIGINT	Seconds since January 1, 1970	long	Seconds since January 1, 1970
stamp	Date and time, as microseconds since January 1, 1970	BIGINT	Microseconds since January 1, 1970	long	Microseconds since January 1, 1970
array(i32)	32-bit signed integer array	int parameter []	32-bit signed integer array	list	List of longs
array(i64)	64-bit signed integer array	bigint parameter []	64-bit signed integer array	list	List of longs
array(dbl)	IEEE double array	double parameter []	IEEE double array	list	List of floating point real
rstring	UTF-8 string	char nchar varchar nvarchar	UTF-8 string	string	Unicode string
blob	Binary large object	binary varbinary	Fixed-length binary data Variable-length binary data	bytes	bytes

Note the following information about data types:

- A string translates to either a single character type or to a varying length string type in DS2, depending on how the DS2 method parameter is declared. Be careful not to pass a multi-character string to a single CHAR argument in DS2, as run-time errors might occur.
- The money type is presented to DS2 or Python as double or float, respectively.
- The SAS Event Stream Processing Engine uses the UNIX epoch for date and time values (January, 1, 1970). The values are presented to DS2 or Python as BIGINT or long, respectively, using the SAS epoch (January 1, 1960). Native DS2 and Python date and time types are not supported for public arguments. Seconds since 1960 is the SAS datetime value, which makes calling SAS date and time functions convenient in DS2. Stamp translates similarly, but with microsecond resolution rather than second resolution.
- For the base data types (string, int32, int64, double, datetime, timestamp, and money), data is stored inline in the event. This allows for fast indexing and serialization.
- For the array(i32), array(i64), array(dbl), blob, and rstring data types, data is not stored in an event, but rather in another location in memory. The event contains a pointer to the actual data. All of these object data types are reference counted at the object level. This allows an object to be referenced in multiple events, which saves memory and the amount of time that it would take to create a new object and copy the data. However, note that these data types cannot be used as key fields for an event.
- When using the field type of blob in Python modules, note the following information:
 - When you use the Source window event schema field type of blob to map a field with a public Python function's input argument, a Python bytes object is created for that input argument.
 - When Python bytes objects are returned as output arguments, they can be mapped to a derived event schema scalar fields of type blob.
 - When a Python function's output argument returns a list of bytes objects, and that list maps to a scalar derived event schema field of type blob, then multiple derived events are created from the output list of bytes returned from the function.

Generating Derived Events

<i>Processing Event Opcodes and Flags</i>	89
Overview	89
Operation Codes and Flags	90
DS2 Opcodes Example	91
<i>Derived Event Suppression and NULL Key Fields</i>	92
<i>Generating Multiple Derived Events from a Single Source Event</i>	92
Overview	92
Unique Keys	93
Setting Opcodes and Flags on Multiple Derived Events	94
<i>Multiple Derived Events and NULL Key Fields</i>	94

Processing Event Opcodes and Flags

Overview

SAS Micro Analytic Service is embedded in the Calculate windows of continuous query applications. One or more upstream windows are connected to a Calculate window by edges. Events *flow* from upstream windows to the Calculate window along these edges.

Modules, which have been published to SAS Micro Analytic Service, process these events. A module is a collection of methods, and each such upstream window can be bound to a specific method. When an event flows from an upstream window to the Calculate window, the method that is associated with that specific upstream window is called. SAS Micro Analytic Service executes the method using input data from the event, and then generates one or more derived events that contain the results of the method execution.

Methods can be used to perform a wide variety of tasks such as scoring, advanced analysis, and real-time decision making.

Operation Codes and Flags

Each event contains an operation code, or opcode, and a set of flags. For more information about these constructs, see [SAS Event Stream Processing: Overview](#). SAS Micro Analytic Service module methods offer you the option of examining a Source window event's opcode and flags and setting the opcode and flags of a derived event.

Note: This practice is recommended only for advanced SAS Event Stream Processing users.

These are the opcodes:

- insert
- update
- delete
- upsert
- safedelelete

One or more flags can be set in a given event, depending on the event opcode and circumstances. Here are the possible flags:

- N — normal
- P — partial update
- R — retention

DS2 and Python module authors can add zero or more of the following special arguments to their DS2 method or Python function signatures:

`_inOpcode`

populated with the Source window event opcode when the module method is called. `_inOpcode` is an input argument and, if included, must appear before any output arguments in the method signature. `_inOpcode` is a string type, and its value must be **insert**, **update**, **delete**, **upsert**, or **safedelelete** when the method is called.

`_outOpcode`

used to either set the opcode of the derived event to emit, or to cause no derived event to be emitted. If `_outOpcode` is omitted from the method signature, SAS Micro Analytic Service transfers the opcode of the Source window event to the derived event. This is the standard behavior under normal circumstances. If `_outOpcode` is included in the method signature and is set to missing, emission of the derived event is skipped. If `_outOpcode` is included and is not set to missing, the value of `_outOpcode` is used to set the opcode of the derived event. The value that is set must be either **insert**, **update**, **delete**, **upsert**, or **safedelelete**. To achieve normal processing when `_outOpcode` is included, the method author must also include `_inOpcode` and set

`_outOpcode=inOpcode`. `_outOpcode` is an output argument. Therefore, it must appear after all input arguments in the method signature.

`_inFlags`

populated with the Source window flags when the module method is called. `_inFlags` is a string type containing one character per Source window event flag that is set. For example, `N` and `NR` are possible values. Reserve space for at least three characters for the `_inFlags` argument. `_inFlags` is an input argument. Therefore, if included, it must appear before any output arguments in the method signature.

`_outFlags`

used to set the flags of the derived event to emit. If `_outFlags` is omitted from the method signature, SAS Micro Analytic Service transfers the flags of the Source window event to the derived event. This is the standard behavior under normal circumstances. If `_outFlags` is included and is set to missing, SAS Micro Analytic Service defaults to standard behavior and copies the Source window event flags to the derived event.

DS2 Opcodes Example

The following simple example illustrates how to conditionally set the derived event opcode.

In this example, if the Source window event's opcode is `delete`, and its quantity is greater than 2000, set the derived event's opcode to `update`, and set the price to 8.50. Otherwise, preserve normal processing by assigning `_inOpcode` to `_outOpcode`, and set the price to 10.00.

```
ds2_options sas;
package module_1/overwrite=yes;
method test_function(vvarchar(16) _inOpcode, int quantity,
  in_out double price, in_out vvarchar _outOpcode);
  if (_inOpcode = 'delete') and (quantity > 2000) then
    do;
      _outOpcode = 'update';
      price = 8.50;
    end;
  else
    do;
      _outOpcode = _inOpcode;
      price = 10.00;
    end;
  end;
end;
endpackage;
```

Derived Event Suppression and NULL Key Fields

In a DS2 method, when output arguments that are mapped to derived event key fields are set to NULL or MISSING, the following rules (in order of precedence) are observed:

- If all key fields are NULL, the derived event is silently suppressed.
- If some, but not all, of the key fields are NULL, a warning is issued indicating that the derived event is being suppressed because of an incomplete key set.
- If the `_outOpcode` pseudo argument is present but NULL, the derived event is silently suppressed.

Generating Multiple Derived Events from a Single Source Event

Overview

The methods that are published to SAS Micro Analytic Service can be used to generate multiple derived events when a single source event is received. This is accomplished by coding an array output argument for each derived event field. The values in the arrays are then used to populate the fields of multiple derived events.

A SAS Micro Analytic Service module method can produce one or more arrays. When such a method is mapped to a SAS Event Stream Processing Source window, and when at least one of the output array names matches a derived event field name, multiple derived events can be generated. The number of elements in the matching array or arrays at run time determines the number of events that are generated. As is the case with scalar outputs, any array output that does not match a derived event field name is ignored.

When you are authoring a method that is capable of generating more than one event, the best practice is to produce parallel arrays of the same size. When you follow this practice, each array element contributes one value to each of the derived events, in order. The arrays can contain different numbers of elements across method calls (for example, they might generate one derived event for one

call, three derived events for another, and so on). However, for a single method call, the best practice is to produce the same number of elements in each array.

If you ignore these best practices, you can still generate multiple events, but keep the following rules in mind:

- The longest array that maps to a derived event field determines the number of events that are generated.
- When the longest array is being determined, trailing missing values are not counted.
- If an array is shorter than the number of events to be generated, missing values are set in the corresponding field of the derived events for which the array has no data.

Scalar output values, if any, are repeated in the corresponding fields of every derived event.

Unique Keys

SAS Event Stream Processing requires that every event have a unique key composed of one or more event fields. Therefore, the source event's key cannot be duplicated in multiple generated derived events. In that case, SAS Event Stream Processing halts processing and returns a duplicate key error. Because of this, the method author must ensure that each derived event has a unique key value. To do that, produce an array containing unique key values and mark the matching event field as a key in the derived event schema. This technique is useful whenever you author your own module.

SAS Micro Analytic Service can execute modules that are generated by SAS analytics products, where the analytic functions do not produce arrays of unique key values. To accommodate such functions, SAS Micro Analytic Service provides a key generation feature. To use this feature, add a key field named `_masRowNum` of type `int32` to the derived event schema, making it part of a composite key. When `_masRowNum` is present, SAS Micro Analytic Service populates the field with the row number of each derived event, in order, starting with 1. This feature ensures that the composite key value is unique across multiple generated derived events.

Here is an example of a derived event schema:

```
ID*:int32, _masRowNum*:int32, symbol:int32,
  quantity:int32, price:double, total:double
```

Note: The module method does not need to produce a SAS Event Stream Processing key value when the meta field `_masRowNum` is present.

Setting Opcodes and Flags on Multiple Derived Events

When a set of derived events is generated from a single source event, you can explicitly set opcodes or flags in the derived events either individually or as a group. To set individual opcodes, use the `_outOpcodeArray` meta argument. Similar to `_outOpcode`, `_outOpcodeArray` can also be used to suppress the generation of any of the individual derived events.

To set flags in the derived events individually, use the `_outFlagsArray` meta argument.

To set the same opcode in all the derived events that are generated from a single source event, use the `_outOpcode` meta argument and omit the `_outOpcodeArray` meta argument. Similarly, to set the same flags in all the derived events that are generated from a single source event, use the `_outFlags` meta argument and omit the `_outFlagsArray` meta argument.

For information about the meta arguments `_outOpcodeArray`, `_outFlagsArray`, `_outOpcode`, and `_outFlags`, see [“Operation Codes and Flags” on page 90](#).

Multiple Derived Events and NULL Key Fields

In a DS2 method that generates multiple derived events from a single input event, when output arguments that are mapped to derived event key fields are set to NULL or MISSING, the following rules (in order of precedence) are observed:

- For any given derived event of a set, if all key fields are NULL, the derived event is silently suppressed.
- For any given derived event of a set, if some of the key fields are NULL, but some are not, a warning is issued indicating that the derived event is being suppressed because of an incomplete key set.
- For any given derived event of a set, if `_outOpcodeArray` is present but the corresponding `_outOpcodeArray` value is NULL or MISSING, the corresponding derived event is suppressed.
- If `_outOpcodeArray` is present and the entire array is NULL or MISSING, the entire derived event set is suppressed.

DS2 Programming for SAS Micro Analytic Service

Overview	96
DS2 Source Code Prerequisites	96
DS2 Identifiers	97
SAS Micro Analytic Service and SAS Foundation	98
Programming Blocks	99
Restrictions When Working with DS2 and SAS Micro Analytic Service	99
Character Restrictions	99
User-Defined Formats	100
About How DS2 Processes Missing and Null Values	100
SYSGET Function Restrictions	101
Public and Private Methods and Packages	101
Overview	101
Public Method Rules	102
Public Method Example	103
Private Method Example	104
Method Overloading	104
Argument Types Supported in Public Methods	105
Overview	105
Supported DS2 Data Types	105
Unsupported DS2 Data Types	106
About the DS2 init() Method	106
Promoting DS2 Execution Issues to an Execution Failure in SAS Event Stream Processing	107
Implicit Data Type Conversions	109
Determining Whether DS2 Code Is Executing in SAS Micro Analytic Service	109
Performing Calls between SAS Micro Analytic Service Modules	110
Overview of Performing Calls between SAS Micro Analytic Service Modules	110
Determining Whether to Use MASCall	111
Using the MASCall Methods	112
Examples	115

Using Analytic Store Models	117
About Analytic Store Models	117
Publishing an Analytic Store Model	117
Calling Analytic Store Models Using DS2	117
Example	118
Accessing ASTORE File System Paths	119
Composite Modules	119

Overview

SAS Micro Analytic Service supports a subset of the DS2 programming language that is suitable for high-performance transaction processing in real time. This chapter covers only that subset. Note that DS2 batch processing is not supported.

The DS2 programming language is thread safe. This helps increase the time to publish DS2 modules and reduces the memory footprint. Note that this does not apply to DS2 run-time behavior. For example, the values of package-scoped variables are thread-specific. In that situation, you would need to use the DS2 package `MASState` for state sharing.

For more information about the DS2 programming language, see [SAS DS2 Language Reference](#).

DS2 Source Code Prerequisites

The DS2 source code submitted to SAS Micro Analytic Service should begin with the following statement, just above the `PACKAGE` statement:

```
"ds2_options sas;"
```

This statement instructs DS2 to use SAS missing value handling and helps ensure that your DS2 program behaves the same as if it were run in SAS Foundation. DS2 source code should end with this statement:

```
"endpackage;"
```

The code cannot contain `DATA` statements, `PROC` statements, or `THREAD` statements. The source code should contain one and only one DS2 package, and this package can contain as many methods as desired.

It is a best practice to include a line feed character at the end of each source code line. This line feed character makes it easier to use compiler warning and error messages that include line numbers.

Note: DS2 supports only a specific style of comment. Comments start with the characters `/*`, and they end with the characters `*/`. All characters between the

starting and ending characters are part of the comment text. When there is ambiguity in determining a token, the compiler always chooses the longest possible sequence of characters that can make up a token. Comments cannot be nested. For more information, see [“Comments” in SAS DS2 Programmer’s Guide](#).

DS2 Identifiers

For DS2 method, package, and argument names, SAS Micro Analytic Service supports regular identifiers and delimited identifiers. When you are using a delimited identifier, any character is allowed, including multi-byte and non-ASCII characters. You must begin and end delimited identifiers with double quotation marks. For more information, see [“DS2 Identifiers” in SAS DS2 Programmer’s Guide](#).

You can specify DS2 delimited identifiers by using any of the following parameters in a SAS Event Stream Processing XML file:

- **func-names**
- **function**
- **field name**

Depending on the parameter that you use, the format requirements differ as follows:

- If you use either the **func-names** parameter or **function** parameter, double quotation marks must be included and the *-novalidate* option must be added to the server’s invocation command. In addition, all of the specified function names must be enclosed in single quotation marks. Here is an example:

```
func-names=' func1, func2, "éfunc3é", func4'
```

Note: You can use double quotation marks if you specify *"* instead of the quotation marks that are part of the delimited identifier. Here is an example:

```
func-names="&quot;éfunc3é&quot; ;".
```

- If you use the **field name** parameter, you must not include the double quotation marks. Here is an example:

```
field name='éfunc3é'
```

SAS Micro Analytic Service and SAS Foundation

Although DS2 is supported by both SAS Foundation and SAS Micro Analytic Service, SAS Micro Analytic Service has a lightweight, high-performance engine that does not support either the full SAS language or PROC statements. Therefore, PROC statements cannot be used. However, here is an effective DS2 authoring and testing mechanism: develop your DS2 packages in SAS Foundation using PROC DS2 and publish those packages to SAS Micro Analytic Service after removing the surrounding PROC DS2 syntax.

Here is an example PROC DS2 step that illustrates the above mechanism:

```
proc ds2;
  package myPackage / inline;
    dcl package logger logr('App.Test');
    method copyArray( char(12) in_array[*], in_out char
out_array[4] );
      out_array := in_array;
      logr.log('i', 'dim(in_array): $s,    dim(out_array): $s',
                dim(in_array),           dim(out_array) );
    end;
  endpackage;
  data _null_;
    method init();
      dcl package logger logr('App.Test');
      dcl int i;
      dcl package myPackage p();
      dcl char(12) inarr[6];
      dcl char(12) outarr[4];
      inarr[1] = 'one';
      inarr[2] = 'two';
      inarr[3] = 'three';
      inarr[4] = 'four';
      inarr[5] = 'five';
      p.copyArray(inarr, outarr);
      do i = 1 to dim(outarr);
        put outarr[i]= ;
      end;
    end;
  enddata;
run;
quit;
```

Here is the log that is returned. Note that in this example, the input array contains six elements but the output array is limited to four.

```
INFO App.Test - dim(in_array): 6,    dim(out_array): 4
INFO App.Program - outarr[1]=one
```

```
INFO App.Program - outarr[2]=two  
INFO App.Program - outarr[3]=three  
INFO App.Program - outarr[4]=four
```

Programming Blocks

Each DS2 module represents exactly one package, and therefore the DS2 PACKAGE statement plays a major role in SAS Micro Analytic Service. A DS2 package contains one or more methods, and methods can contain a wide variety of DS2 language constructs. Package methods work well with rapid transaction processing because they can be called over and over again with little overhead, as transactions flow through the system. By contrast, the DS2 THREAD and TABLE statements are batch-oriented and are not supported.

The following code blocks are supported:

- PACKAGE...ENDPACKAGE
- METHOD...END
- DO...END

The following code blocks are batch-processing oriented and are not supported:

- TABLE...ENDTABLE
- THREAD...ENDTHREAD

Similarly, the following statements are not supported: OUTPUT and SET

- OUTPUT
- SET

Restrictions When Working with DS2 and SAS Micro Analytic Service

Character Restrictions

The following characters are not allowed in module IDs, package names, public method names, and submodule names:

- backslash (\)
- forward slash (/)

- period (.)
- semicolon (;)

User-Defined Formats

SAS Micro Analytic Service does not support the use of SAS sessions. Because a SAS session is required to create and access user-defined formats, SAS Micro Analytic Service does not support the use of user-defined formats.

About How DS2 Processes Missing and Null Values

SAS Micro Analytic Service supports only SAS missing value mode (SAS mode) for the processing of null and missing values. It does not support ANSI mode. Here is some key information about how DS2 processes missing and null values in SAS mode:

- A blank-filled character value is treated as a missing value.
- Null values are interpreted as SAS missing values, which are known values.
- Only DOUBLE or CHAR data types can have missing values. Data types other than DOUBLE or CHAR can have only null values.
- The DS2 NULL function returns a 1 only for a null value. It returns a 0 for any non-null value, including a SAS missing value.
- The DS2 MISSING function checks if a value is a null or missing value and returns a numeric result. If the argument does not contain a null or missing value, SAS returns a value of 0. If the argument contains a null or missing value, SAS returns a value of 1.

For more information, see the following:

- [“How DS2 Processes Nulls and SAS Missing Values” in SAS DS2 Programmer’s Guide](#)
- [“NULL Function” in SAS DS2 Language Reference](#)
- [“MISSING Function” in SAS DS2 Language Reference](#)

The datagrid package processes null values differently than the DS2 package. For more information, see [“About How Datagrid Processes Null and Missing Values” in SAS DS2 Programmer’s Guide](#).

SYSGET Function Restrictions

For security reasons, you cannot use the SYSGET function to retrieve the value of an environment variable from DS2 code that is running in SAS Micro Analytic Service.

Public and Private Methods and Packages

Overview

Public and private packages and methods are SAS Micro Analytic Service concepts, rather than DS2 features.

Public methods are DS2 package methods that can be called by clients that are external to SAS Micro Analytic Service, such as the SAS Event Stream Processing Calculate window. In order to be available, a method can contain only those parameters that map to the following types and return a void result:

Scalar Types	Array Types
binary	bigintArray
bigint	dateTimeArray
dateTime	decimalArray
decimal	integerArray
integer	stringArray
string	
varbinary	

For more information about the DS2 types that correspond to the above types, see [“Argument Types Supported in Public Methods” on page 105](#).

When a public method is registered with SAS Event Stream Processing as an event processor, the method's arguments are automatically mapped to the fields of the given source window and Calculate window events. You register the method either by calling `dfESPproject::registerMethod_MAS()` or by including it in a window-map entry of an XML project definition. For an example, see [“XML Example” on page 82](#).

Note: The method-argument-to-event-field mappings are by name and are case sensitive.

A method that contains other types of parameters or returns a non-void value is a considered *private method* and cannot be called externally. However, these methods can be called by other methods. Private methods are often used as utility or library methods to help solve larger problems.

Public packages are packages that contain public methods that can be called externally.

A public package with no public methods is effectively a *private package*. Though private packages cannot be called externally, they can help to structure and manage code. Private packages can be used to hide internal implementations, share common code between public packages, hold a library of utility code, and so on.

IMPORTANT DS2 package methods to be used for event processing must follow the [“Public Method Rules” on page 102](#).

Public Method Rules

Public methods must conform to the following rules:

- The return type must be void. Rather than using a single return type, public methods can return multiple outputs, where each output argument specifies the `in_out` keyword in the method declaration. Non-void methods are treated as private.
- Arguments that are passed by reference (meaning ones that specify `in_out`) are treated as output only. True update arguments are not supported by public methods. This restriction results in more efficient parameter marshaling and supports all interface layers.
- Input arguments must precede output arguments in the method declaration. It is permissible for a method to have only inputs or only outputs. However, if both are present, all inputs must precede the outputs.
- DS2 packages might not be passed as arguments in public methods. The presence of a DS2 package argument results in the method becoming private.
- The `VARARRAY` statement might not be present in the argument list of a public method. `VARARRAY` is a DS2 statement, not a data type. The presence of `VARARRAY` in a methods argument list causes the method to become private.

- For information about the data types that can be used as public method arguments, see [“Supported DS2 Data Types” on page 175](#).

Public Method Example

The example below illustrates a valid public method. It has a void return type (no RETURNS clause), uses only publicly supported data types, and treats in_out arguments as output only.

```
method quickSortStep (int lowerIndex, int higherIndex, in_out double
numbers[10]);

    dcl int i;
    dcl int j;
    dcl int pivot;
    dcl double temp;

    i = lowerIndex;
    j = higherIndex;

    /* Calculate the pivot number, taking the pivot as the
    * middle index number. */
    pivot = numbers[ceil(lowerIndex+(higherIndex-lowerIndex)/2)];

    /* Divide into two arrays */
    do while (i <= j);
        /**
        * In each iteration, identify a number from the left side
        * that is greater than the pivot value. Also identify a
        * number from the right side that is less than the pivot
        * value. Once the search is done, then exchange both
        * numbers. */
        do while (numbers[i] < pivot);
            i = i+1;
        end;
        do while (numbers[j] > pivot);
            j = j-1;
        end;
        if (i <= j) then do;
            temp = numbers[i];
            numbers[i] = numbers[j];
            numbers[j] = temp;

            /* Move the index to the next position on both
            *sides. */
            i = i+1;
            j = j-1;
        end;
    end;

    /* Call quickSort recursively. */
    if (lowerIndex < j) then do;
        quickSortStep(lowerIndex, j, numbers);
```

```

        end;
        if (i < higherIndex) then do;
            quickSortStep(i, higherIndex, numbers);
        end;
    end;
end;

```

Here is another example of a public method that illustrates the use of the HTTP package calling out to a web service using a POST request and then getting a response.

```

method httpPost( nvarchar(8192) url,
                 nvarchar(67108864) payload,
                 in_out nvarchar respbody,
                 in_out int hstat, in_out int rc );

declare package http h();
rc = h.createPostMethod( url );
if rc ne 0 then goto Exit;
rc = h.setRequestContentType( 'application/json;charset=utf-8' );
if rc ne 0 then goto Exit;
rc = h.setRequestHeader( 'Accept', 'application/json' );
if rc ne 0 then goto Exit;
rc = h.setRequestBodyAsString( payload );
if rc ne 0 then goto Exit;
rc = h.executeMethod();
if rc ne 0 then goto Exit;
hstat = h.getStatusCode();
if hstat lt 400 then h.getResponseBodyAsString( respbody, rc );
else respbody = '';
Exit:
h.delete();
end;

```

Private Method Example

The example below generates a private method in SAS Micro Analytic Service. It has a non-void return type. That is, it has a RETURNS clause in the declaration, which specifies a single integer return value.

```

method isNull(double val) returns int;
    return null(val) OR missing(val);
end;

```

Method Overloading

SAS Micro Analytic Service does not support method overloading. The DS2 programming language does support method overloading for programs running in other environments, but not when running in SAS Micro Analytic Service.

CAUTION

If you publish a DS2 package that contains overloaded methods, run-time errors can occur.

Argument Types Supported in Public Methods

Overview

SAS Micro Analytic Service supports a subset of the DS2 data types for use as public method arguments. Data types in the unsupported list can still be used in the body of a (public or private) DS2 package method, and as arguments to private methods. The lists of publicly supported and unsupported data types are included below.

Note: Any additional types added to the DS2 programming language in future releases should be considered unsupported unless otherwise stated in the SAS Micro Analytic Service documentation.

Supported DS2 Data Types

- BIGINT
- BINARY(n)
- CHAR(n)
- DOUBLE
- INTEGER
- NCHAR(n)
- NVARCHAR(n)
- VARBINARY(n)
- VARCHAR(n)

Note: With the exception of BINARY and VARBINARY, these data types are supported for both scalar values and arrays. BINARY and VARBINARY are supported only for scalar values.

Unsupported DS2 Data Types

- DATE
- DECIMAL(p, s)
- NUMERIC(p, s)
- PACKAGE
- TIME(p)
- TIMESTAMP(p)
- TINYINT

About the DS2 init() Method

When an `init()` method is present in a DS2 package that is published to SAS Micro Analytic Service, be aware of the following information:

- If the top-level package has an `init()` method, SAS Micro Analytic Service automatically executes it in the following circumstances:
 - when the DS2 module is published.
 - if a fatal runtime error is encountered. In this case, SAS Micro Analytic Service destroys and re-creates the DS2 execution context for that DS2 module.
- SAS Micro Analytic Service calls the `init()` method only for the top-level DS2 package.

An application (such as SAS Intelligent Decisioning or SAS Event Stream Processing) publishes DS2 code to SAS Micro Analytic Service via a composite module. A composite module consists of a top-level DS2 package. It includes at least one public method that is available for execution by the client application.

- A composite module can contain submodules that are either DS2 packages or analytic store models (ASTORES). If a DS2 submodule contains `init()` methods and you want those `init()` methods to be called only one time per execution context, ensure that the `init()` method in the top-level package is implemented such that it makes the submodule `init()` method calls.

If you want the `init()` method in a submodule to be called once, but not at publish time, add conditional logic to a method that is called after publishing. Ensure that the condition evaluates to false after the desired `init()` method has been called one time.

- If you publish DS2 modules that contain package-scope variables to SAS Micro Analytic Service, you must be careful as to how you use them. By default, a pool

of worker threads services requests. When SAS Micro Analytic Service is initialized, any free worker can service execution requests. A set once, read many pattern is acceptable in this type of environment.

- SAS Micro Analytic Service expects that the `init()` method has no parameters and expects no fatal errors.
- You must not overload the `init()` method.
- The `init()` method must not have a return value.
- The error promotion feature does not apply to a DS2 `init()` method when it is called automatically at publish time. For more information, see one of the following:
 - For SAS Event Processing, see [“Promoting DS2 Execution Issues to an Execution Failure in SAS Event Stream Processing” on page 107](#).
 - For the SAS Micro Analytic Service REST interface, see [“Promoting DS2 Execution Issues to an Execution Failure in SAS Intelligent Decisioning and SAS Model Manager” on page 177](#).

Promoting DS2 Execution Issues to an Execution Failure in SAS Event Stream Processing

By default, when SAS Micro Analytic Service executes a DS2 method that encounters an error or warning, it is logged but SAS Micro Analytic Service does not return a failure code. Non-detection of a failure code can make it challenging to debug problems. Enabling one of these environment variables can help surface problems during early stages of code development. You can use the following environment variables to control this behavior:

Environment Variable	Description	Default Value
MAS_PROMOTE_DS2_ERROR	When enabled, detects underlying errors that are encountered during the execution of DS2 methods. When these are detected, the SAS Micro Analytic Service execution operation returns a failure	■ In SAS Event Stream Processing, the default value is 0 (Disabled). ■ In SAS Intelligent Decisioning and SAS Model Manager, the default value is 1 (Enabled).

Environment Variable	Description	Default Value
	(nonzero) return code to the client. ■ 0 (Disabled) ■ 1 (Enabled)	
MAS_PROMOTE_DS2_WARNING	When enabled, detects underlying warnings and errors that are encountered during execution of DS2 methods. When these are detected, the SAS Micro Analytic Service execution operation returns a failure (nonzero) return code to the client. ■ 0 (Disabled) ■ 1 (Enabled) Note: Be aware that if you enable the MAS_PROMOTE_DS2_WARNING environment variable, both warnings and errors are promoted. It is not possible to promote warnings only.	The default value is 0 (Disabled) for all products.

For information about how to set environment variables, see the SAS Event Stream Processing documentation.

Note: The error promotion feature does not apply to a DS2 `init()` method when it is called automatically at publish time. For more information, see [“About the DS2 `init\(\)` Method” on page 106](#).

IMPORTANT If one of these environment variables is enabled and the DS2 code encounters a warning or an error, the execution of the code does not stop. It continues with the next statement.

Implicit Data Type Conversions

When the data types in the event schema differ from the data types in the corresponding method arguments, certain implicit data conversions are supported. The following table contains the supported data type conversions. A conversion is supported to and from each type.

Data Type in Schema	Module Method Argument Data Type
64-bit integer	■ 32-bit integer ■ double ■ Boolean
32-bit integer	■ 64-bit integer ■ double ■ Boolean
double	■ 32-bit integer ■ 64-bit integer ■ Boolean

SAS Micro Analytic Service confirms that a 64-bit integer value can successfully be converted to a 32-bit integer value without overflow, but not all conversions are tested. If problems are encountered, including those that are outside the SAS Micro Analytic Service domain, an error occurs. For more information about numerical representation, see the topics in [Numerical Accuracy in SAS Software](#).

Determining Whether DS2 Code Is Executing in SAS Micro Analytic Service

The DS2 function `inmas()` discovers whether SAS Micro Analytic Service is running in the current process and, if so, determines whether the current thread is a member of the SAS Micro Analytic Service worker thread pool. If it is, then the DS2 code is running inside SAS Micro Analytic Service.

The function returns 1 (TRUE) if the DS2 code is executing in SAS Micro Analytic Service, and 0 (FALSE) otherwise.

This can be useful to know when, for example, you have DS2 code that works in various locations, but not in SAS Micro Analytic Service.

Performing Calls between SAS Micro Analytic Service Modules

Overview of Performing Calls between SAS Micro Analytic Service Modules

When a DS2 module references another DS2 package, the DS2 compiler does the following:

- 1 Copies, from the source code repository, the source code of the referenced package into the source code of the module to be published. The code is copied inline.

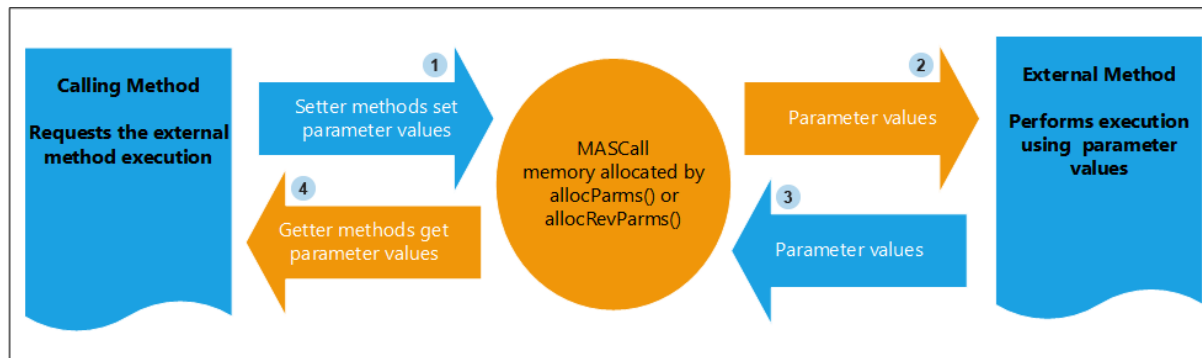
Note: The current DS2 language infrastructure supports source code repositories only.

- 2 Compiles the combined source code into a single executable code stream.

A drawback to this approach is that the DS2 package code is repeated in every module that references it. This results in increased memory usage for every redundant copy of a package and longer compilation times.

However, SAS Micro Analytic Service resolves these issues via the DS2 MASCall package. This package contains methods that enable separate DS2 modules, published to SAS Micro Analytic Service, to call one another across separate module executables.

The following figure shows the details of how the MASCall package works. For details about method usage, see [“MASCall Package” on page 37](#).

Figure 12.1 The MASCall Process

- 1 The calling method uses setter methods to store argument values in MASCall memory.
- 2 The `callModule()` method or `asyncCallModule()` method executes the external method using the argument values that were written to MASCall memory in Step 1.
- 3 The `callModule()` method copies the external method's output parameter values to the MASCall memory.

This step does not apply to the `asyncCallModule()` method. When you use `asyncCallModule()`, any output arguments that are declared in an external module method signature are ignored. Therefore, output parameter values are not copied to MASCall memory.

- 4 The calling method retrieves, from MASCall memory, the output parameter values that contain the results of the execution of the external method.

Determining Whether to Use MASCall

Application Types and Using a Dynamic Call Interface

If your application is organized using composite modules in which the code is static, it is likely that MASCall would not benefit you.

However, if your application is designed such that the module code changes frequently and can benefit from making external calls, consider using MASCall.

It is also possible for your application to use a hybrid approach. This means that some packages are included statically as part of a composite module, and some methods that are located in external modules can be executed by using dynamic calls. The hybrid approach is helpful when your application needs to accommodate multiple revisions of some modules as well as dynamically calling others.

Benefits of Using MASCall

- Because MASCall does not retain the original called module into the caller, you can modify the called module after the caller has been published, as long as the called method has the same signature.

By contrast, when you do not use the MASCall dynamic call interface, a called module is compiled with the caller module. Therefore, if the called module is changed in the system, the caller module still refers to the original called module.

This can be useful if your versioning strategy requires that you change the called module after publishing the caller that is deployed. You can also deploy the new caller with the new module.

- Only a single copy of the called module code exists in the system, even if multiple callers exist. This reduces the memory footprint.
- In addition to a smaller memory footprint and shorter compilation times, this functionality enables a library of modules to be reused by many higher-level modules without penalty.

Items to Consider Before Using MASCall

- Using MASCall increases the complexity of your code.
- Marshaling method arguments can decrease performance.
- If the method to be called exists in a package that is small, rarely changes, or will not be called by many modules, the advantages of cross-module execution might not justify using the MASCall package.
- Not all methods that are called can be externalized. The external method must comply with the following requirements:
 - It must meet SAS Micro Analytic Service public method requirements. For more information, see [“Public Method Rules” on page 102](#).
 - The method parameter data types must be supported by MASCall. For more information, see the getter and setting methods in [“MASCall Package” on page 37](#).
- Compatibility errors are not generated from the compiler. Only run-time errors are generated.

Using the MASCall Methods

For detailed information about MASCall package usage, see [“MASCall Package” on page 37](#).

About the MASCall Methods

Here are the package MASCall methods:

- `allocParms( module_name, method_name )`
This method creates a parameter list for the latest revision of the specified method.
- `allocRevParms( module_name, revision_number, method_name )`
This method is similar to `allocParms`, but it enables you to specify a revision number.

Note: The called module name is case-sensitive on UNIX systems. It is not case-sensitive on Windows systems.

After `allocParms()` or `allocRevParms()` is called, the parameter setter methods operate on the newly created parameter list.

The argument takes a numeric index value or a parameter name. Note that for both the setter and getter methods, the index value is zero-based.

Here are the setter methods:

- Scalar argument setters are of the form:
`return_code = set<type>( arg, value )`
- Array argument setters are of the form:
`return_code = set<type>Array( arg, array_value )`

Executing the External Method

After the input values are set, you can execute the external method by making one of the following calls. Both call the external module method that was previously specified by `allocParms()` or `allocRevParms()`:

- `callModule()`
Use this method to make external calls and wait for them to run to completion.
- `asyncCallModule()`
Use this method to make external module calls without waiting for them to run. This means that any output arguments are ignored. For more information about using this method, including how to enable its use, see [“Additional Information about the `asyncCallModule` Method” on page 115](#).

Note: The `asyncCallModule` method is not currently supported for use with SAS Event Stream Processing.

As is noted above, because the method to execute was previously specified using `allocParms()` or `allocRevParms()`, `callModule()` and `asyncCallModule()` do not accept parameters.

Error Promotion

When executing external methods using `MASCall`, you can promote any errors or warnings that are encountered as a SAS Micro Analytic Service execution error. For more information, see one of the following:

- For SAS Event Processing, see [“Promoting DS2 Execution Issues to an Execution Failure in SAS Event Stream Processing” on page 107](#).
- For the SAS Micro Analytic Service REST interface, see [“Promoting DS2 Execution Issues to an Execution Failure in SAS Intelligent Decisioning and SAS Model Manager” on page 177](#).

Return Codes and Output

For all methods that return a status code, you must check the return code to determine whether the call was successful. For more information about nonzero return codes, see [Appendix 9, “SAS Micro Analytic Service Return Codes,” on page 377](#). A zero return code indicates success. A nonzero return code indicates the following:

- **`callModule()`**: a problem occurred when executing the method specified in the `allocParms` or `allocRevParms` method.
- **`asyncCallModule()`**: a problem occurred when adding the request to a queue for asynchronous processing.

When you are using `asyncCallModule()` to execute an external module’s method, any output arguments that are declared in an external module method signature are ignored.

When you are using `callModule()` to execute an external module’s method, the output values that are returned by the method execution can be retrieved using these `MASCall` getter methods:

- Scalar argument getters are of the form:
`value = get<type>(arg)`
- Array argument getters are of the following form:
`get<type>Array(arg, array_value, rc)`

Note: DS2 passes arrays and output values by reference.

After the output values are retrieved, call `releaseParms()` to release the parameters and other resources that are used for the method. This releases the memory resources that are used for memory execution and prevents memory leaks.

Calling an External Method for Multiple Sequential Transactions

If the specified external method multiple times with no changes to its signature, you do not need to call `releaseParms()` and a subsequent `allocParms()` between each transaction. SAS Micro Analytic Service resets the parameters upon the next call to a setter method. If no setter methods are called before the next call to `callModule()` or `asyncCallModule()`, then the same input arguments are used.

Depending on your code and environment, using this design might increase performance.

Additional Information about the `asyncCallModule` Method

- By default, the `asyncCallModule` feature is disabled. To enable it, use the `core.numasynchthreads` property in SAS Environment Manager to specify the number of asynchronous processing threads to reserve for use by `asyncCallModule`.

For more information, see [“sas.microanalyticsservice.system: core” on page 272](#).

- It is expected that you will design routines and tune the system so that the methods are executed at some point. However, requests that are made via `asyncCallModule` do not time out. Execution is not guaranteed if requests are made faster than they can be serviced. Therefore, if a request takes a long time to run, this could cause a backlog of requests that are waiting to run.

If outstanding `asyncCallModule` requests are queued to run and SAS Micro Analytic Service is terminated, SAS Micro Analytic Service logs a message indicating that the requests will not be processed.

- If the method that is called via `asyncCallModule` fails or encounters errors, the errors are logged.
- Nesting `asyncCallModule` is prohibited. This means that `asyncCallModule` returns a nonzero return code if it is already executing within an asynchronous call.

A return value of 1 from the `isWithinAsyncCall()` method indicates that a call is currently executing within a cross-module call.

Examples

Here are DS2 MASCall package example method calls, where `mc` is the DS2 MASCall instance variable that is created by calling the package constructor, which

takes no arguments. Here are two ways to construct a MASCall package instance named `mc`:

■ Example 1:

```
declare package mascall mc();
```

■ Example 2:

```
declare package mascall mc;
mc = _new_mascall();
```

General example:

```
rc = mc.allocParms( module_name, method_name );
rc = mc.setDouble( 'discount', additionalDiscount );
rc = mc.setString( 'phone', currentPhone );
rc = mc.setString( 'plan', currentPlan );
rc = mc.callModule();
treatments = mc.getString( 'offer' );
mc.releaseParms();
```

The complete set of DS2 package methods follows, where `rc` is the integer return code, and `mc` is the package instance. The argument takes a numeric index value or a parameter name. Note that the index value is zero-based.

Parameter resource management:

```
rc = mc.allocParms( module_name, method_name );
rc = mc.allocRevParms( module_name, revision, method_name );
mc.releaseParms();
```

Scalar argument setters:

```
rc = mc.setString( arg, value );
rc = mc.setLong( arg, value );
rc = mc.setInt( arg, value );
rc = mc.setDouble( arg, value );
```

Array argument setters:

```
rc = mc.setStringArray( arg, string_array );
rc = mc.setLongArray( arg, bigint_array );
rc = mc.setIntArray( arg, integer_array );
rc = mc.setDoubleArray( arg, double_array );
```

Execute or call a module's method:

```
rc = mc.callModule( );
```

Scalar argument getters:

```
string_var = mc.getString( arg );
long_var = mc.getLong( arg );
int_var = mc.getInt( arg );
double_var = mc.getDouble( arg );
```

Array argument getters:

```
mc.getStringArray( arg, string_array, rc );
mc.getLongArray( arg, bigint_array, rc );
mc.getIntArray( arg, integer_array, rc );
mc.getDoubleArray( arg, double_array, rc );
```

Using Analytic Store Models

About Analytic Store Models

An analytic store file, called an ASTORE file, is a system that allows the state of a trained analytical model to be saved in a transportable form. This enables it to subsequently be used to score new data in a variety of environments. Many SAS analytical procedures save the results from the training phase of model development as analytic store models. A key feature of an ASTORE file is that it can be easily transported from one platform to another. When an analytic store is published to SAS Micro Analytic Service, the state of the predictive model is restored and is available for scoring new data.

Publishing an Analytic Store Model

Unlike DS2 and Python modules, analytic store models are not published to SAS Micro Analytic Service as source code. Instead, analytic store models consist of binary code and metadata. Client applications deliver analytic store models to SAS Micro Analytic Service as ASTORE disk files.

Note: For information about calling an analytic store model by a DS2 module, see the next section.

Calling Analytic Store Models Using DS2

If an analytic store model has been registered with SAS Micro Analytic Service, it can be called by a DS2 module.

A DS2 module that calls an analytic store model must include an `init()` method that invokes the score package's `setvars()` and `setkey()` methods.

Note: Failure to set this option can cause the system to stop responding on module deletion or on shutdown.

The `setvars()` method is used by the DS2 score package to map variables to the analytic store model's input and output parameters. The `setkey()` method takes a SHA-1 hexadecimal key as input and uses it to look up the analytic store model.

SAS Micro Analytic Service automatically calls the `init()` method, if present, when a DS2 module is published.

Example

```
ds2_options sas;
package astoretest/overwrite=yes;
dcl package score sc();
dcl double CLAGE;
dcl double CLNO;
dcl double DEBTINC;
dcl double DELINQ;
dcl double NINQ;
dcl double VALUE;
dcl double _P_;
dcl double P__EVENT_0;
dcl double P__EVENT_1;
dcl nchar(32) I__EVENT_;
dcl nchar(4) _WARN_;
varlist allvars [_all_];

method init();
    sc.setvars(allvars);
    sc.setkey(n'EB3D1CA20AA0CB74465D25EEE2290E13692AF750');
end;

method preCode();
    _P_ = 0.999;
end;

method postCode();
end;

method term();
end;

method astoreScore(double inCLAGE, double inCLNO, double inDEBTINC,
                    double inDELINQ, double inNINQ, double inVALUE,
                    in_out double out_P_, in_out double outP__EVENT_0,
                    in_out double outP__EVENT_1, in_out nchar
outI__EVENT_,
                    in_out nchar out_WARN_);
    CLAGE = inCLAGE;
    CLNO = inCLNO;
    DEBTINC = inDEBTINC;
    DELINQ = inDELINQ;
    NINQ = inNINQ;
    VALUE = inVALUE;

    preCode();
    sc.scoreRecord();
    postCode();
```



```

out_P_ = _P_;
outP__EVENT_0 = P__EVENT_0;
outP__EVENT_1 = P__EVENT_1;
outI__EVENT_ = I__EVENT_;
out_WARN_ = _WARN_;
end;

endpackage;

```

Accessing ASTORE File System Paths

In order to publish decisions that use analytic store models to a SAS Micro Analytic Service destination, the `/models/astores/viya` mounted directory path must be accessible.

This directory path points to the ASTORES persistent volume claim (PVC) that is configured during the deployment of the SAS Viya platform.

For more information, see the README.md file that is located in the `$deploy/sas-bases/examples/sas-microanalytic-score/astores` directory where you downloaded your files for deployment, as well as the [SAS Viya Platform: Deployment Guide](#).

Note that users who need to work with analytic store models must have Read and Write access to analytic store directories.

Composite Modules

Composite modules enable DS2 code running in SAS Micro Analytic Service to call one or more analytic store models. A composite module consists of one DS2 module and zero or more analytic store models, which are known as the members of the composite. All members of a composite module are published, replaced, or deleted as a set. For example, if one member of a composite module fails to publish, then all members of the composite fail to publish. Also, any prior revisions of the composite module remain unchanged.

Like other module types, the revisions of a composite module are owned by a module context, have dictionaries, and can be queried for compilation messages, creation date-times, and so on. The members of a composite module can include either a DS2 module or an analytic store model, or both. Other module types, such as Python and C, are not supported as composite members.

Only the methods of the DS2 module of the composite are exposed for client applications to call.

To avoid name collisions with DS2 modules that exist outside a composite module, each composite revision has its own namespace. Therefore, each composite module must be self-contained and without dependencies on any non-member module. Unlike other module types, composite modules can be called only externally (for example, driven by SAS Event Stream Processing events).

In SAS Event Stream Processing, a composite module is defined by specifying a DS2 module that calls an analytic store model or models in the `<mas-module>` tag and by including the new `<module-members>` and `<module-member>` sub-tags of `<mas-module>`. Each `<module-member>` sub-tag should specify the location of an ASTORE file on disk. The following example is an excerpt from an ESP continuous query XML definition:

```
<mas-modules>

  <mas-module language="ds2" module="module_1" func-names='astoreScore'>
    <code-file>/your_ds2_folder/your_astore_caller.ds2</code-file>
    <module-members>
      <module-member member='astore_1'
        SHAkey='EB3D1CA20AA0CB74465D25EEE2290E13692AF750' type='astore'>
        <code-file>/your_astore_model_folder/your_astore_modelcode-file>
        </codefile>
      </module-member>
    </module-members>
  </mas-module>

</mas-modules>
```

For more information, see [SAS Event Stream Processing: Using SAS Event Stream Processing Studio](#).

State Sharing between Modules

Overview	121
Shared Vectors	122
Overview	122
About Supported DS2 Data Types	123
State Vector Types	123
Local State Vector Methods	125
Shared State Vector Methods	125
Setter and Getter Examples	127
Shared Hash Tables	130
Overview	130
About Using Shared Hash Tables in DS2	130
Methods That Operate on the Default Shared Hash Table	132
Default Shared Hash Table Example	133
Methods That Operate on Non-default Shared Hash Tables	134

Overview

SAS Micro Analytic Service provides two ways to share data between modules that are executing within a user context: shared vectors and shared hash tables. *Shared vectors* are collections of data values. *Shared hash tables* are containers of shared vectors; the vectors accessed by using keys.

When it is possible to represent the data, or *state*, that you want to share across modules by a small number of vectors, the vectors can be shared with other modules by name. However, vector lookup by name is a linear search and is therefore inefficient when larger numbers of vectors are present. In such cases, shared hash tables are highly recommended because of their efficiency.

Note: If vectors are not contained in a shared hash table, the number of vectors is limited to 64.

When using shared hash tables, an efficient non-cryptographic hashing function is applied to a key to quickly compute the desired vector's location within the hash table. Shared hash tables also use non-locking synchronization mechanisms to further increase efficiency.

Whether using shared vectors or shared hash tables, DS2 authors can use the MASState package to create, share, retrieve, and delete data. In all cases, a zero return code indicates success. A nonzero return code indicates failure. For more information about non zero return codes, see [Appendix 9, "SAS Micro Analytic Service Return Codes," on page 377](#). If you encounter a nonzero return code, check the applicable log file for more information.

IMPORTANT

- SAS Micro Analytic Service shared state vectors and shared hash tables are available only for DS2 modules. They are not supported for Python modules.
- These features support in-memory state sharing. They are not intended for state-sharing across cluster nodes.

Shared Vectors

Overview

Collections of state data fields that are managed as a unit are referred to as *state vectors*. Here are some key points about state vectors:

- A state vector contains one or more values, which are referred to by vector name and a zero-based index.
- The data values in a state vector can contain the same data types or a mix of data types.
- The number of data elements that is contained in a state vector is limited only by the available memory.
- A state vector is similar to a database record in that it can contain multiple data values of various types. However, it differs from a database record in that data values are positional, rather than organized in named columns.
- You can initialize a shared state vector from a DS2 package method, including the `init()` and constructor methods.
- A shared state vector name must be unique within the current user context.

About Supported DS2 Data Types

State vector values can have any of the following DS2 data types:

BIGINT	DOUBLE	NVARCHAR
BINARY	DOUBLE ARRAY	NVARCHAR ARRAY
CHAR	INTEGER	VARCHAR
CHAR ARRAY	INTEGER ARRAY	VARCHAR ARRAY

In SAS Micro Analytic Service, *binary data* typically refers to binary or character long objects. These can be expressed as pointer and length pairs or as character strings. Because DS2 does not support pointers directly, operations on binary data are typically performed with string manipulation functions. An example would be on blobs that have been Base64 encoded into a string.

The `setReference()` method accepts a bigint reference value (for example, a pointer to a blob or other binary data in memory) and a size (blob size in bytes or length of other binary data).

You can use the `stringAddrLen()` method to obtain the internal memory address of the data buffer and length of a DS2 string. Here is an example:

```
rc=stringAddrLen( myBase64encodeBLOB, addr, len )
```

The first parameter is a string input argument and the second and third arguments are bigint output parameters.

The return code indicates success or failure. When successful, the `addr` and `len` output arguments correspond to the address and the length of the input string. You would use these values as the input to a subsequent `setReference()` call.

The `getReference` method returns a DS2 binary data type. (See [“Scalar Getters Example” on page 128](#), for an example.) The asymmetrical nature of this setter/getter pair is due to limitations with binary processing that exist only on the setter side. With the exception of binary, all other data types are handled symmetrically.

State Vector Types

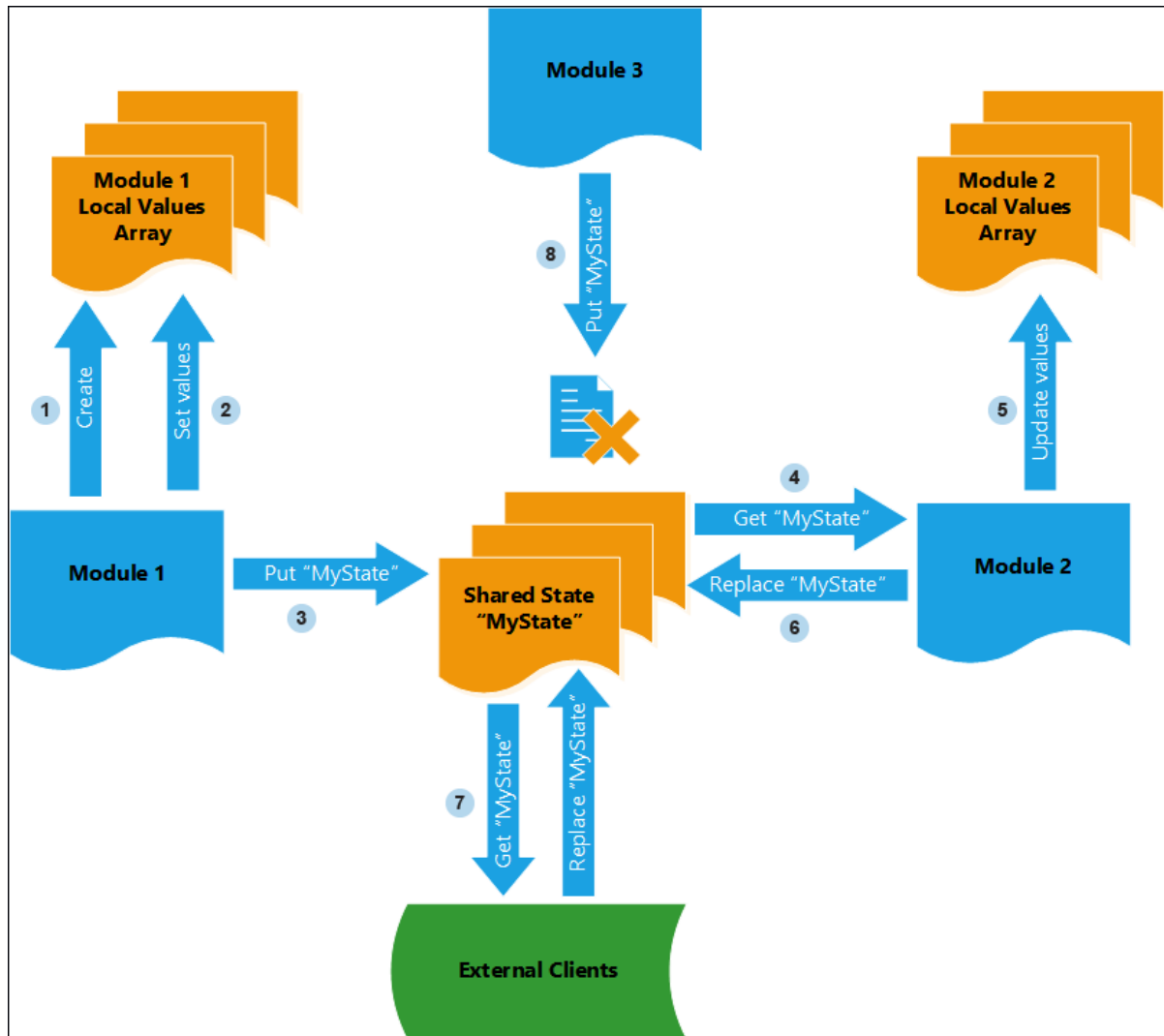
There are two categories of `MASState` package methods— those that operate on local state vectors and those that control state vector sharing.

Setting and retrieving individual values is always performed using local state vectors. When a shared state vector is fetched, a local copy of that vector is created and returned to the caller.

Similarly, when a state vector is shared, a copy of the local vector is created and made centrally available for fetching by other modules and transactions.

Working with local state vectors has the advantage of allowing a set of values to be updated and shared as a unit. This eliminates race conditions that could otherwise occur, and enables consistent and complete state representations.

Figure 13.1 The State Vector Sharing Process



- 1 Module 1 creates a local values array.
- 2 Module 1 sets the values for the array.
- 3 These values are published as a shared state vector and assigned the name MyState. This makes a deep copy of the vector.
- 4 Module 2 retrieves the MyState local vector. This makes a deep copy of the vector.
- 5 Module 2 updates the values and replaces the values in the local values array.
- 6 Module 2 replaces the values in the MyState shared state vector.
- 7 External clients retrieve and replace values for the MyState shared state vector.

- 8 Module 3 attempts to create a shared state vector called `MyState`. This is rejected because a shared state vector with that name already exists.

The `MASState` package includes 28 methods. The following sections contain usage examples for each of these methods.

Note that each example assumes that an instance of the `MASState` package, called `st`, has been created:

```
dcl package masstate st()
```

Local State Vector Methods

The following methods control the creation and deletion of local vectors.

`createVector( name, size )`

This method creates a local state vector with the specified name, and space for the number of values that is indicted by the specified size. The following example creates a local state vector named `MyVector` with a size of 4:

```
rc = st.createVector('MyVector', 4);
```

`deleteVector( name )`

This method deletes the local state vector referenced by name. The following example deletes the local vector created above:

```
rc = st.deleteVector('MyVector');
```

`deleteAllVectors()`

This method deletes all local vectors. The following example deletes all local vectors managed by the current `MASState` package instance:

```
rc = st.deleteAllVectors();
```

Shared State Vector Methods

The following methods control the sharing and unsharing of state vectors with other modules, and across transaction boundaries.

shareVector(name)

This method creates a copy of the named local state vector and makes it accessible to other modules within the current user context. The name passed to `shareVector()` must be unique within the user context. Otherwise, a duplicate name error is returned and the vector is not shared. To update an existing shared state vector, call `replaceSharedVector()`.

```
method setValuesAndShareVector(in_out int rc);

    /* Create local vector */
    rc = st.createVector('MyVector', 4);

    /* Populate it with values*/
    rc = st.setInt('MyVector', 0, 100);
    if (rc ne 0) then return;
    rc = st.setInt('MyVector', 1, 200);
    if (rc ne 0) then return;
    rc = st.setInt('MyVector', 2, 300);
    if (rc ne 0) then return;
    rc = st.setInt('MyVector', 3, 400);
    if (rc ne 0) then return;

    /* Share vector with other modules */
    rc = st.shareVector('MyVector');
end;
```

fetchSharedVector(name)

This method fetches the shared state vector referenced by name and returns a local copy of it. It is used to retrieve stateful data that has been published or updated by other modules. After calling this method, the `MASState` package instance holds a local copy of the shared state vector, which can be referenced by name.

```
method fetchSharedVector(in_out int rc);
    rc = st.fetchSharedVector('MyVector');
end;
```

unshareVector(name)

This method removes sharing for the vector referenced by name. The shared copy of the vector is deleted from the current user context, and modules are no longer able to access it. If no shared vector with the given name exists, this is considered a valid condition and `unshareVector()` does not return an error. The `unshareVector()` method does not affect a local state vector.

```
method unshareVector(in_out int rc);
```



```
rc = st.unshareVector('MyVector');
end;
```

replaceSharedVector(name)

This method creates a copy of the named local state vector and replaces the existing shared state vector of the same name, making the updated data accessible to other modules within the user context. The name that is passed to `replaceSharedVector()` must refer to an existing shared state vector. Otherwise, a **not found** error is returned and the data is not shared.

```
method setNewValuesAndReplaceSharedVector(in_out int rc);

    /* Populate vector */
    rc = st.setInt('MyVector', 0, 111);
    if (rc ne 0) then return;
    rc = st.setInt('MyVector', 1, 222);
    if (rc ne 0) then return;
    rc = st.setInt('MyVector', 2, 333);
    if (rc ne 0) then return;
    rc = st.setInt('MyVector', 3, 444);
    if (rc ne 0) then return;

    /* Share vector with other modules */
    rc = st.replaceSharedVector('MyVector');
end;
```

isVectorShared(name)

This method returns integer 1 (TRUE) if a shared state vector with the given name exists within the current user context. Otherwise, it returns integer 0 (FALSE).

```
method isVectorShared(in_out int result);
    result = st.isVectorShared('MyVector');
end;
```

Setter and Getter Examples

Setter and getter methods are provided for each data type. These methods operate on local vectors only. Individual data items are referenced by local vector name and by the zero-based index of the data value.

The examples in this section illustrate each type-specific setter method. The MASState package guards against errors such as index out of range and invalid data. As a best practice, you should check return codes, and if applicable, return them to the caller.

Scalar Setters Example

```
method testScalarSetters(vvarchar(32) strVal,
                        int intVal,
                        bigint longVal,
                        double dblVal,
                        bigint refAddr,
                        bigint refLen,
                        in_out int rc);

rc = -1;

/* Populate the vector with scalars of each type */
rc = st.setString('AllScalarsVector', 0, strVal);
if (rc ne 0) then return;
rc = st.setInt('AllScalarsVector', 1, intVal);
if (rc ne 0) then return;
rc = st.setLong('AllScalarsVector', 2, longVal);
if (rc ne 0) then return;
rc = st.setDouble('AllScalarsVector', 3, dblVal);
if (rc ne 0) then return;
rc = st.setReference('AllScalarsVector', 4, refAddr, refLen);
if (rc ne 0) then return;
end;
```

Array Setters Example

```
method testArraySetters(vvarchar(32) strVal[3],
                       int intVal[3],
                       bigint longVal[3],
                       double dblVal[3],
                       in_out int rc);

rc = -1;

/* Populate the vector with arrays of each type */
rc = st.setStringArray('AllArraysVector', 0, strVal);
if (rc ne 0) then return;
rc = st.setIntArray('AllArraysVector', 1, intVal);
if (rc ne 0) then return;
rc = st.setLongArray('AllArraysVector', 2, longVal);
if (rc ne 0) then return;
rc = st.setDoubleArray('AllArraysVector', 3, dblVal);
if (rc ne 0) then return;
end;
```

Scalar Getters Example

```
method testScalarGetters(in_out varchar strVal,
                        in_out int intVal,
```

```

        in_out bigint longVal,
        in_out double dblVal,
        in_out varbinary refAddr,
        in_out int rc);

/* Retrieve scalars of each type from the vector */
strVal = st.getString('AllScalarsVector', 0);
if (missing(strVal)) then do;
    rc = -1;
    return;
end;
intVal = st.getInt('AllScalarsVector', 1);
if (missing(intVal)) then do;
    rc = -1;
    return;
end;
longVal = st.getLong('AllScalarsVector', 2);
if (missing(longVal)) then do;
    rc = -1;
    return;
end;
dblVal = st.getDouble('AllScalarsVector', 3);
if (missing(dblVal)) then do;
    rc = -1;
    return;
end;
refAddr = st.getReference('AllScalarsVector', 4);
end;

```

Note that the reference value is returned as a DS2 BINARY type, as indicated in [“Scalar Setters Example” on page 128](#).

Array Getters Example

```

method testArrayGetters(in_out varchar strVal[3],
                        in_out int intVal[3],
                        in_out bigint longVal[3],
                        in_out double dblVal[3],
                        in_out int rc);

/* Retrieve arrays of each type from the vector */
st.getStringArray('AllArraysVector', 0, strVal, rc);
if (rc ne 0) then return;
st.getIntArray('AllArraysVector', 1, intVal, rc);
if (rc ne 0) then return;
st.getLongArray('AllArraysVector', 2, longVal, rc);
if (rc ne 0) then return;
st.getDoubleArray('AllArraysVector', 3, dblVal, rc);
end;

```

Shared Hash Tables

Overview

SAS Micro Analytic Service shared hash tables enable high-performance sharing of in-memory stateful data between modules and across transactions. Shared hash tables consist of key/value pairs, where the keys are strings and the values are state vectors. For more information about state vectors, see the previous section [“Shared Vectors”](#).

Here are some key points about shared hash tables:

- State vectors with different sizes can reside within the same shared hash table.
- Shared hash tables are visible to all modules within the same user context.
- Up to eight hash tables can exist per user context, and each hash table can contain up to 2,147,483,659 state vectors. Each state vector can contain any number of data elements.
- You can initialize a shared state vector from a DS2 package method, including the `init()` and constructor methods.

About Using Shared Hash Tables in DS2

The `MASState` package contains all the methods that are required for DS2 modules to share data across SAS Micro Analytic Service modules and transaction boundaries. These methods include operations on local state vectors and on shared hash tables.

Data can be shared among modules when you do either of the following:

- call methods that create a local state vector, populating it with values, and then putting it in a shared hash table.
- call methods that get an existing vector from a shared hash table (which makes a local copy), modifying its contents, and then replacing the vector in the hash table.

Shared hash tables are accessible by all DS2 modules within a user context.

When you create a new local state vector, you assign it a name. The name must be unique within the hash table in which the vector will be stored. This name is used as follows:

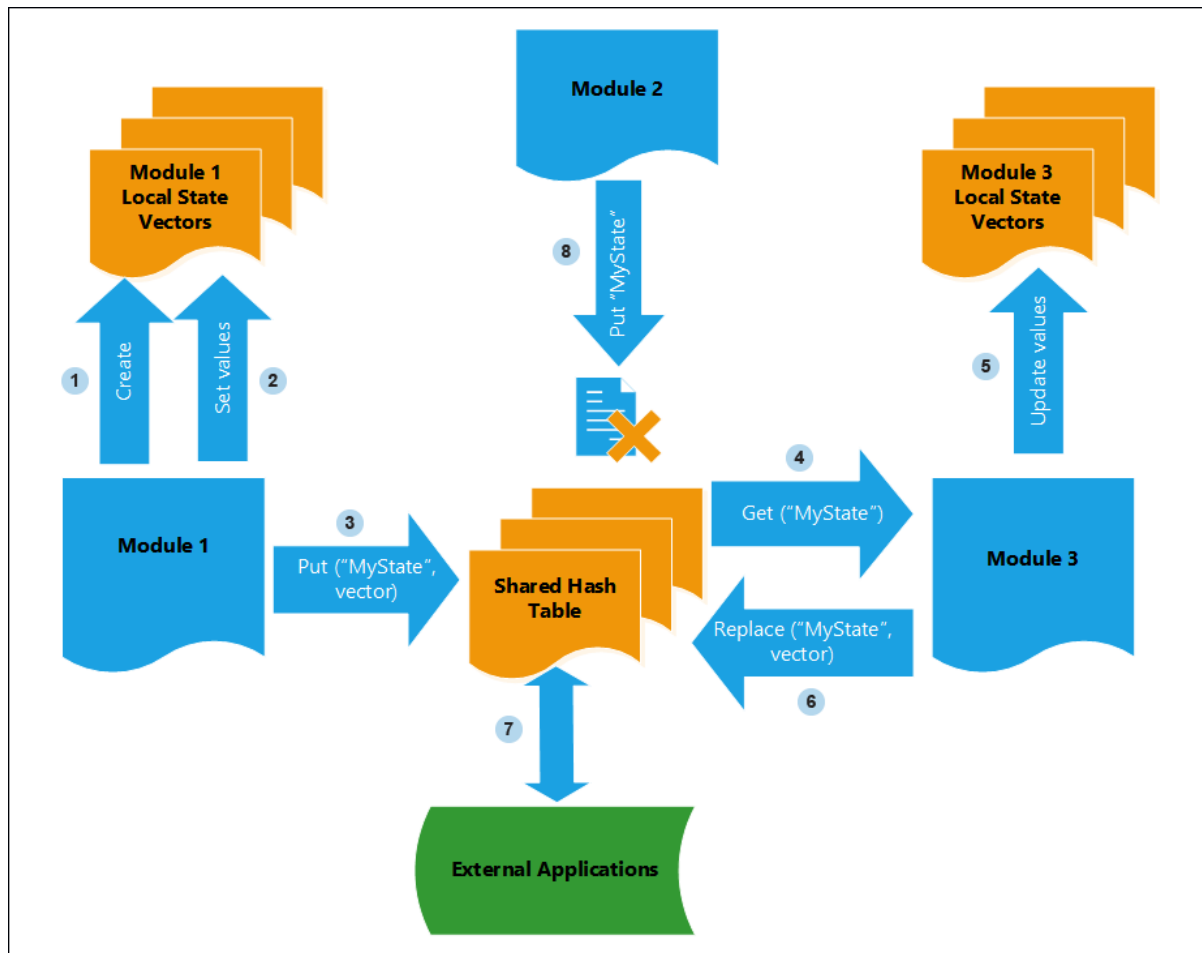
- as a key when subsequently storing the vector in a shared hash table.

That is, the name is used internally as input to a hashing algorithm that quickly computes the hash table location where the vector will be stored.

- when deleting the state vector.
- when storing or retrieving state vector data values.
- when retrieving the vector from a shared hash table.
- when replacing the vector within a shared hash table.

Up to eight shared hash tables can be defined per user context. Hash tables are referenced by index numbers zero through seven, where index zero refers to the default hash table. The default hash table is created automatically when a new user context is created. It is operated on by convenience methods that omit the table index argument. The convenience methods are `clear()`, `isEmpty()`, `size()`, `containsKey()`, `put()`, `get()`, `replace()`, and `remove()`. They are described in “[Methods That Operate on the Default Shared Hash Table](#)” on page 132.

Figure 13.2 The Shared Hash Table Process



- 1 Module 1 creates a local state vector.
- 2 Module 1 sets the values for the local state vector.
- 3 Module 1 puts these values, contained in the MyState vector, into a shared hash table.

- 4 Module 3 gets the MyState vector.
- 5 Module 3 updates the values in its local state vector.
- 6 Module 3 replaces the MyState state vector in the shared hash table.
- 7 External applications access the shared hash table to retrieve and replace the MyState state vector.
- 8 Module 2 attempts to store a state vector called MyState in the shared hash table. This is rejected because a state vector with that name already exists in the table.

Methods That Operate on the Default Shared Hash Table

Method Signature	Description
<code>int clear()</code>	Removes all state vectors from the default hash table. Returns zero if successful, and nonzero otherwise.
<code>int isEmpty()</code>	Returns 1 if the default hash table contains no state vectors, and zero otherwise.
<code>bigint size()</code>	Returns the number of state vectors currently in the default hash table.
<code>int containsKey(key)</code>	Returns 1 if the default hash table contains a state vector with a name matching key, and zero otherwise.
<code>int put(key)</code>	Inserts the state vector with the name indicated by the key into the default shared hash table. Returns zero if successful. Nonzero result codes are returned if a duplicate key exists in the default hash table or if a local state vector with a name matching key does not exist.
<code>int get(key)</code>	Finds a state vector in the default shared hash table with a name matching key. If found, a local copy of the state vector is made, and a zero result code is returned. If not found, a nonzero result code is returned. Note: If a local state vector with a name matching key already exists, and a state vector matching the key is found in the default hash table, then the existing local state vector is overwritten with the data values that are retrieved from the default shared hash table.

Method Signature	Description
<code>int replace(key)</code>	Finds a state vector in the default shared hash table with a name matching key. If found, the state vector in the default hash table is replaced with a copy of the corresponding local state vector and a zero result code is returned. Nonzero result codes are returned if the key is not found in the default hash table, or if a local state vector with a name matching key does not exist.
<code>int Remove(key)</code>	Finds a state vector in the default shared hash table with a name matching key. If found, removes it and returns a zero result code. A nonzero result code is returned if the key does not exist in the default hash table.

Default Shared Hash Table Example

In the following example, method `createAndPutVector()` inserts a new state vector containing two integer values into the default shared hash table. Method `incrementSharedValue()` retrieves a state vector, named `MyVector`, from the default shared hash table, making a local copy. It increments the integer data value within the vector and then replaces the `MyVector` state vector in the default shared hash table.

```

ds2_options sas;
package statepkgtest/overwrite=yes;
dcl package masstate st();

method createAndPutVector(vvarchar(32) key, in_out int rc);
  rc = st.createVector(key, 2);
  rc = st.setInt(key, 0, 100);
  if (rc ne 0) then return;
  rc = st.setInt(key, 1, 200);
  if (rc ne 0) then return;
  rc = st.put(key);
  rc = st.deleteVector(key);
end;

method incrementSharedValue(in_out int rc, in_out int int0Val);
  rc = st.get('MyVector');
  if (rc eq 0) then do;
    int0Val = st.getInt('MyVector', 0);
    int0Val = int0Val + 1;
    rc = st.setInt('MyVector', 0, int0Val);
    rc = st.replace('MyVector');
  end;
end;

```

```
endpackage;
```

Methods That Operate on Non-default Shared Hash Tables

Note: For the methods in the table, the following arguments apply:

- **tableIndex** indicates the hash table (0-7) on which to operate.
- **key** is a string value that uniquely identifies a vector within the hash table.

Method Signature	Description
<code>int hashTblCreate(tableIndex)</code>	Creates a new empty hash table, which can be referenced by the given table index. Returns zero if successful, and nonzero otherwise.
<code>int hashTblDestroy(tableIndex)</code>	Removes all state vectors from the indicated hash table, and then deletes the table. Returns zero if successful, and nonzero otherwise.
<code>int hashTblClear(tableIndex)</code>	Removes all state vectors from the indicated hash table. Returns zero if successful, and nonzero otherwise.
<code>int hashTblIsEmpty(tableIndex)</code>	Returns 1 if the indicated hash table contains no state vectors, and zero otherwise.
<code>bigint hashTblSize(tableIndex)</code>	Returns the number of state vectors currently in the indicated hash table.
<code>int hashTblContainsKey(tableIndex, key)</code>	Returns 1 if the indicated hash table contains a state vector with a name matching key, and zero otherwise.
<code>int hashTblPut(tableIndex, key)</code>	Inserts the state vector into the indicated hash table at the position indicated by key. Returns zero if successful. Nonzero result codes are returned if a duplicate key already exists in the indicated hash table, or if a local state vector with a name matching key does not exist.

Method Signature	Description
<code>int hashTblGet(tableIndex, key)</code>	Finds a state vector in the indicated hash table with a name matching key. If found, a local copy of the state vector is made, and a zero result code is returned. If not found, a nonzero result code is returned. Note: If a local state vector with a name matching key already exists, and a state vector matching the key is found in the indicated hash table, then the existing local state vector is overwritten with the data values that are retrieved from the hash table.
<code>int hashTblReplace(tableIndex, key)</code>	Finds a state vector in the indicated hash table with a name matching key. If found, the state vector in the indicated hash table is replaced with a copy of the corresponding local state vector and a zero result code is returned. Nonzero result codes are returned if the key is not found in the hash table, or if a local state vector with a name matching key does not exist.
<code>int hashTblRemove(tableIndex, key)</code>	Finds a state vector in the indicated hash table with a name matching key and, if found, removes it and returns a zero result code. A nonzero result code is returned if the key does not exist in the hash table.

Best Practices for DS2 Programming in SAS Event Stream Processing

<i>Overview</i>	137
<i>Return Results</i>	138
<i>Global Packages versus Local Packages</i>	138
Overview	138
Example of Optimized Code	138
Example of Poorly Optimized Code	139
<i>Replacing SCAN (and TRANWRD) with DS2 Code</i>	139
<i>Hash Package</i>	142
<i>Character-to-Numeric Conversions</i>	142
<i>Data Type Conversions</i>	142
<i>Passing Character Values to Methods</i>	143
<i>Performing the Computation Once</i>	143
<i>Moving Invariant Computations Out of Loops</i>	144

Overview

This section describes best practices that are recommended when programming in DS2 for any environment. They are not unique to SAS Micro Analytic Service.

Return Results

If a DS2 method, or any method it calls, can result in a status code or failure, always include a method output argument for returning the result to the caller.

Global Packages versus Local Packages

Overview

The scope of a package instance makes a difference. Package instances that are created in the global scope typically are created and deleted (allocated and freed) once and used over and over again. Package instances that are created in a local scope are created and deleted each time the scope is entered and exited. For example, a package instance that is created in a method's scope is created and deleted each time a method is called. The creation and deletion time can be costly for some packages.

The following examples use the hash package. This technique can be used for all packages.

Example of Optimized Code

This example creates a hash package instance that is global, created and deleted with the package instance, and reused between calls to `load_and_clear`.

```
/** FAST **/  
package mypack;  
  dcl double k d;  
  dcl package hash h([k], [d]);  
  
  method load_and_clear();  
    dcl double i;  
    do k = 1 to 100;  
      d = 2*k;  
      h.add();  
    end;  
    h.clear();  
  end;  
endpackage;
```

Example of Poorly Optimized Code

This example creates a hash package instance that is local to the method and created and deleted for each call to `load_and_clear`.

```
/** SLOW **/
package mypack;
  dcl double k d;

  method load_and_clear();
    dcl package hash h([k], [d]);
    dcl double i;
    do k = 1 to 100;
      d = 2*k;
      h.add();
    end;
    h.clear();
  end;
endpackage;
```

Replacing SCAN (and TRANWRD) with DS2 Code

Consider the following code:

```
i = 1;
onerow = TRANWRD(SCAN(full_table, i, '|'), ';;', ';-');
do while (onerow ~= '');
  j = 1;
  elt = scan(onerow, j, ';');
  do while (elt ~= '');
    * processing of each element in the row;
    j = j+1;
    elt = SCAN(onerow, j, ';');
  end;
  i = i+1;
  onerow = TRANWRD(SCAN(full_table, i, '|'), ';;', ';-');
end;
```

You can make the following observations:

- SCAN consumes adjacent delimiters. Therefore, TRANWRD is required to manipulate each row into a form that can be traversed element by element.
- SCAN starts at the front of the string each time. Therefore, the aggregate cost is $O(N^2)$.

- SCAN and TRANWRD require NCHAR or NVARCHAR input. If full_table is declared as a CHAR or VARCHAR input, it must be converted to NVARCHAR, then processed, and then converted back to VARCHAR in order to be captured into the onerow value.

Here is code that replaces this type of loop with a native DS2 solution and that thus avoids these problems by collecting the necessary details into a package:

```
dcl package STRTOK row_iter();
dcl package STRTOK col_iter();
row_iter.load(full_table, '|');
do while (row_iter.hasmore());
  row_iter.getnext(onerow);
  col_iter.load(onerow, ';');
  do while (col_iter.hasmore());
    col_iter.getnext(elt)
    * processing of each element;
  end;
end;
```

The supporting package, STRTOK, is shown below. It can be used to replace SCAN and TRANWRD pairs anywhere in DS2.

```
/** STRTOK package - extract subsequent tokens from a string.
 * So named because it mirrors (in a safe way) what is done by the
original
 * strtok(1) function available in C.
 */
package sasuser.strtok/overwrite=yes;
dcl varchar(32767) _buffer;
dcl int strt blen;
dcl char(1) _delim;

/* Loads the current object with the supplied buffer and delimiter
 * information. This avoids the cost of constructing and destructing
the
 * object, and allows the declaration of a STRTOK outside of the loop
 * in which it is used.
 */
method load(in_out varchar bufinit, char(1) delim);
  _buffer = bufinit .. delim;
  _delim = delim;
  strt = 1;
  blen = length(_buffer);
end;

/* Are there more fields? 1 means there are more fields. 0 means
there are
 * no more fields.
 */
method hasmore() returns integer;
  if (strt >= blen) then return 0;
  return 1;
end;

/* The void-returning GETNEXT method places the next token in the
supplied
 * variable, tok.
```

```

*/
method getnext(in_out varchar tok);
  dcl char(1) c;
  dcl int e;
  tok = '';
  if (hasmore()) then do;
    e = strt;
    c = substr(_buffer,e,1);
    do while (c ~= _delim);
      tok = tok .. c;
      e = e + 1;
      c = substr(_buffer,e,1);
    end;
    strt = e + 1;
  end;
end;

/* The value-returning GETNEXT method returns the next token. This
version is
  * more computationally expensive because it requires an extra copy,
as
  * opposed to the void-returning version, above.
*/
method getnext() returns varchar(32767);
  dcl varchar(32767) tok;
  getnext(tok);
  return tok;
end;

/* Construct a STRTOK object using the parameters as initial values.
*/
method strtok(varchar(32766) bufinit, char(1) delim);
  load(bufinit, delim);
end;

/* Construct a STRTOK object without an initial buffer to be
consumed.
*/
method strtok();
  strt = 0; blen = 0;
end;
endpackage; run;

```

Using STRTOK instead of SCAN and TRANWRD avoids the CHAR to NCHAR conversions and reduces the CPU load due to how STRTOK retains the intermediate state between calls to the getnext() methods. Therefore, it is $O(N)$ instead of $O(N^2)$.

Hash Package

With both the DATA step and DS2, note the size of the key. A recent program carried out many hash lookups with a 356-byte key. Hashing is an $O(1)$ algorithm; the "1" with the hash package is the length of the key. The longer the key, the longer the hash function takes to operate.

```
dcl char(200) k1 k2;
dcl double d1 d2;

/* If k1 and k2 are always smaller than 200, then */
/* size them smaller to reduce the time spent in */
/* the hash function when adding and finding values */
/* in the hash package. */
dcl package hash([k1 k2], [d1 d2]);
```

Character-to-Numeric Conversions

When converting a string to a numeric value, note the encoding of the string. When the string is a single-byte encoding, DS2 translates the value to a TKChar (UCS-2 or UCS-4) for conversion. The longer the string, the longer the time it takes to do the conversion.

```
dcl char(512) s;
dcl nchar(512) ns;
dcl double x;
s = '12.345';
ns = '12.345';

x = s; /* slow */
x = substr(s,1,16); /* faster */
x = substr(ns,1,16); /* even faster, avoids transcoding */
```

Data Type Conversions

When a source or derived event window includes an array of a particular type, and the corresponding argument type in the module method is another type, a data type conversion can occur automatically. This happens only under certain

circumstances. For more information, see [“Implicit Data Type Conversions”](#) on page 109.

Passing Character Values to Methods

In SAS Micro Analytic Service, DS2 method input parameters are passed by value. What this means is that a copy of the value is passed to the method. When passing character parameters, a copy of the parameter is made to ensure that the original value is not modified. Making sure that character data is sized appropriately ensures that less copying occurs.

DS2 method output parameters, which are specified by the `in_out` keyword, are passed by reference. Therefore, no copy is made.

```
method copy_made(vvarchar(256) x);
...
end;

method no_copy(in_out vvarchar x);
...
end;
```

Performing the Computation Once

If a computation is repeated multiple times to compute the same value, you can perform the computation once and save the computed value. For example, the following code block performs the computation, `compute(x)`, four times:

```
if compute(x) > computed_max then computed_max = compute(x);
if compute(x) < computed_min then computed_min = compute(x);
```

If `compute(x)` always computes the same value for a given value of `x`, then the code block can be modified to perform the computation once and save the computed value:

```
computed_x = compute(x);
if computed_x > computed_max then computed_max = computed_x;
if computed_x < computed_min then computed_min = computed_x;
```

Moving Invariant Computations Out of Loops

If a computation inside a loop computes the same value for each iteration, improve performance by moving the computation outside the loop. Compute the value once before the loop begins and use the computed value in the loop. For example, in the following code block, `compute(x)` is evaluated during each iteration of the DO loop:

```
do i = 1 to dim(a);  
  if (compute(x) eq a[i]) then ...;  
end;
```

If `compute(x)` is invariant (meaning that it always computes the same value for each iteration of the loop), then the code block can be modified to perform the computation once outside the loop:

```
computed_x = compute(x);  
do i = 1 to dim(a);  
  if (computed_x eq a[i]) then ...;  
end;
```

Python Support in SAS Micro Analytic Service

Introduction	145
Example	147
Public and Private Methods	148
Overview	148
About Private Methods	149
About Public Methods	149
Return Values	150
Examples: Public and Private Methods	151
Working with Python and SAS Micro Analytic Service	152
Configuring Python for SAS Micro Analytic Service	153
About Using Python with SAS Micro Analytic Service	153
Environment Variables	154
Python Logging	154
Overview	154
Specifying Logging Configuration Using a JSON File	155
Capturing Python Process STDOUT/STDERR Data	156

Introduction

SAS Micro Analytic Service supports modules that are written in the Python programming language. A Python module represents a group of related Python functions.

Input arguments are given in the function's argument list. The objects, variables, and expressions listed in a Python function's return statement are positional with respect to the output variables.

The output variables are listed in the function's "Output:" docstring that is specified in the first statement of the function. Any method that includes the "Output:"

docstring is considered a public method. Otherwise, it is considered a private method. For more information, see the sections later in this chapter.

Input and output argument names live in a single namespace and therefore cannot be the same. This means that update arguments are not supported. This is true for all module types in SAS Micro Analytic Service, even though the Python language does not enforce such a restriction.

Here is an example of a Python public function that can be hosted by SAS Micro Analytic Service.

```
import sys
import math
import pandas as pd
import numpy as np
def nppd(a):
    "Output: ser1"
    npa = np.array([[1,2,3],[4,5,6]])
    ser1 = pd.Series([212, a, -273])
    return ser1.tolist()

def trucks(Eng_Load, Oil_Temp, Eng_RPM):
    "Output: ser1, x, syspath"
    inputs = pd.Series([Eng_Load, Oil_Temp, Eng_RPM])
    b = np.arange(100)
    number = 0
    for index, item in enumerate(inputs):
        number += item + b[index + 7]
    # is it even or odd?
    x = math.fmod(number, 2)
    return nppd(Oil_Temp), x, getsyspath()

def getsyspath():
    "Output: p"
    p = [None] * 50
    # print(sys.path)
    syspaths = sys.path
    i = 0
    for path in syspaths:
        p[i] = path
        i = i + 1
    return p
```

Here is an example of a Python public function that has input arguments a and b, and no output.

```
def calcATimesB(a, b):
    "Output: "
    print ("Function with no output variables.")
    c = a * b
    print ("Result is: ", c, ", but is not returned")
    return None
```

After Python is configured, see [Appendix 2, “Executing Python Modules in DS2 Modules,” on page 303](#) for more information.

Example

The following example illustrates the use of each SAS Event Stream Processing data type as input to and as output from a public Python function.

```
#
#   Name: scalarsTest.py
# Purpose: Test the Python program for scalar types
#
# Inputs (name)           (type)
#   inString              String
#   inBool                Boolean
#   inLong                Long
#   inDouble              Double
#   inTimestamp           Long microseconds since 1960
#   inDatetime            Long seconds since 1960
#   inMoney               Double
#
# Outputs (name)          (type)
#   outString             String
#   outBool               Boolean
#   outLong               Long
#   outDouble             Double
#   outTimestamp          Long microseconds since 1960
#   outDatetime           Long seconds since 1960
#   outMoney              Double
#
# Note: Event stream processing presents the timestamp as
#       long microseconds since 1960 and datetime as long
#       seconds since 1960.

# Import the datetime module to perform datetime operations.
import datetime

def scalarsTest(inString, inBool, inLong, inDouble,
                inTimestamp, inDatetime, inMoney):
    "Output: outString, outBool, outLong, outDouble,
    outTimestamp, outDatetime, outMoney"

    if inString == None:
        outString = None
    else:
        # Convert the casing of the string input.
        outString = inString.swapcase()

    if inBool == None:
        outBool = None
    else:
        # Reverse value of the Boolean.
        outBool = not inBool
```

```

    if inLong == None:
        outLong = None
    else:
        # Add 10 to long.
        outLong = inLong + 10

    if inDouble == None:
        outDouble = None
    else:
        # Add 10.1 to the double.
        outDouble = inDouble + 10.1

    if inTimestamp == None:
        outTimestamp = None
    else:
        # Since this is defined as a stamp in event stream processing
        # schema, this number is long microseconds since 1960.
        # Add one second == 1000000 microseconds.
        outTimestamp = inTimestamp + 1000000

    if inDatetime == None:
        outDatetime = None
    else:
        # Since this is defined as date in the event stream processing
        schema,
        # this number is long seconds since 1960.
        # Add one day.
        outDatetime = inDatetime + (3600 * 24)

    if inMoney == None:
        outMoney = None
    else:
        # Add 25 cents.
        outMoney = inMoney + 0.25

    # Return all of the outputs.
    return outString, outBool, outLong, outDouble, outTimestamp,
        outDatetime, outMoney

```

Public and Private Methods

Overview

SAS Micro Analytic Service enables the use of hosting public and private methods, where a method is a Python function. Note that public and private methods are SAS Micro Analytic Service concepts, and are not Python features.

In general, any method that includes the "Output:" docstring is considered a public method. If a method does not have the "Output:" docstring, then it is considered a private method. However, there are syntax requirements that must be followed for the docstring and the output arguments. For more information, see ["About Public Methods" on page 149](#).

Python modules can be published containing all public methods, or a mixture of public and private methods.

Both public and private methods can call other functions that either exist within the module internally or in external Python packages, including third-party libraries.

SAS Micro Analytic Service supports the use of Python decorators in Python modules. If you use decorators, be aware that the original function that includes decorators requires a docstring with the "Output: <vars>" format.

About Private Methods

Here are details about using a private method:

- A private method can be called internally by other methods (either public or private).
- A private method cannot be called directly (externally).
- Private methods are useful when used as utility functions within a package.

About Public Methods

Here are details about using a public method:

- When there is more than one output argument, the output argument names must be comma separated.
- Line two of the function must begin with a docstring, and the first non-whitespace token must be "Output:".
- All public functions that return more than one output argument must return a tuple containing all of the output arguments.

This can be done by returning all of the arguments separated by commas.

- When returning zero arguments from a public function you are still required to include the "Output:" docstring to indicate a public function. It should simply be "Output:", with no output arguments listed.
- Order does matter. Therefore, the order in the return statement must match the order in the "Output:" line. A best practice is to copy and paste from one to the other.

Return Values

When Python integers are returned, they are converted to `int64_t`. If the Python integer is too large for the `int64_t` type, a run-time error is reported.

Although Python packages might support abstract types that provide an additional layer on top of the built-in scalar types, such as integer and float, they cannot be returned from SAS Micro Analytic Service public functions. Packages that have these abstract types provide functions that can be used to extract the built-in type. Here is an example:

```
return numpy.float(x), numpy.int(y)
```

Note the following information about the return statement for Python functions:

- Functions that return nothing do not require a return statement. Python returns the `NoneType` object. Therefore, an empty return statement (`return`) and returning `None` (`return None`) are equivalent.
- Functions that return a single argument can return `None`. SAS Micro Analytic Service maps the `None` value to the specific Missing value for the expected static type.
- Functions that return multiple values can be formatted with or without parentheses (for example, `return a,b,c` or `return (a,b,c)`).

Note: An attempt to return `None` from a function that expects multiple values is invalid.

When return values are transferred between Python and SAS Micro Analytic Service, the following rules are enforced by SAS Micro Analytic Service:

- A singleton value (non-tuple) is mapped to a tuple with a single value.
- Tuples map to multiple return arguments.
- Arrays and tuples cannot be used interchangeably. The Python tuple type is used to return multiple values. The Python list type is used for SAS Micro Analytic Service arrays.
- The argument count must match.

The following code sample and table show some examples that illustrate the application of these rules.

```
def one(a):
    "output: b"
    return oneVal

def two(a,b)
    "output: a,b"
    return twoVal
```


Python Value Assignment	SAS Micro Analytic Service Value Assignment
<code>oneVal = 1</code>	<code>oneVal = (1)</code> (one integer value)
<code>oneVal = 1,</code>	<code>oneVal = (1)</code> (one integer value)
<code>oneVal = (1,)</code>	<code>oneVal = (1)</code> (one integer value)
<code>oneVal = (1, 2)</code>	An error occurs. It indicates that the expected output variables do not match the actual output variables.
<code>twoVal = 1,2</code>	<code>twoVal = (1,2)</code> (two integer values)
<code>twoVal = (1,2),</code>	An error occurs. It indicates that an unknown tuple type is present.
<code>twoVal = ([1],[2])</code>	<code>twoVal = ([1],[2])</code> (two integer arrays)

Examples: Public and Private Methods

As mentioned in [“Overview”](#), for a method to be public, the output variables must be listed in the function's "Output:" docstring that is specified on the first statement of the function. This is the second line of the method, immediately following the "def" line.

Here are some examples. Note that the fun2 function would be considered private because the docstring does not begin with "Output:".

```
def fun1( a, b ):
    "Output:"
    """ This is a public function,
        but has no output args."""
    return

def fun2( a, b ):
    """This will be private since the docstring doesn't begin with
    Output:
        Output: x, y, z"""
    return a+2, b*4, a/b

def fun3( a, b ):
    '    Output: x, y, z'
```

```

''' multi
    Line
    doc string'''
return a+2, b*4, a/b

def fun4( a, b ):
    ''' Output: x, y, z'''
    """ multi
        Line
        doc string"""
    return a+2, b*4, a/b

def fun5( a, b ):
    '''Output: q, r, s,
        t, u,
        v,
        w'''
    return a+2, b*4, a/b

```

Working with Python and SAS Micro Analytic Service

When you work with Python and SAS Micro Analytic Service, note the following:

- When sharing modules with other modules, you should understand the following logic:
If module A and module B each import module C, updates to module C affect both module A and module B.
- All client requests are passed to SAS Micro Analytic Service with an OAuth access token. This token is not passed to the Python subprocess. Therefore, Python code cannot connect to a SAS service that requires an access token.

Note: This is a SAS Micro Analytic Service limitation on Python modules.

- If you execute the CALL method in SAS Cloud Analytic Services (CAS), your Python function must be stateless and not depend on data from a previous Python function call.
- SAS Micro Analytic Service Python interface usage is prohibited when any of the following situations exist:
 - The application enabled MASHostAccessDeny when initializing SAS Micro Analytic Service.
 - The MASHostAccess=DENY environment variable is set.
 - The application provides MASHostOSID credentials that fail to authenticate.

- ❑ The CAS session is not running under a host operating system identity and it attempts to run an action that uses SAS Micro Analytic Service.
- Maintaining multiple revisions of a Python module is not supported.
- Source code encoding, if provided, must be UTF-8.
- After a Python module has been published, you cannot update it on the fly. If you update a Python module, you must restart SAS Micro Analytic Service or SAS Event Stream Processing (depending on your environment) for the change to take effect.

Configuring Python for SAS Micro Analytic Service

About Using Python with SAS Micro Analytic Service

SAS Micro Analytic Service supports Python 3.x (note that Python 3.9 or later is recommended). Though SAS Micro Analytic Service does not require a specific version of Python, it is possible that Python code that you publish to SAS Micro Analytic Service might have dependencies to a specific version of Python or Python packages.

These are the commonly used packages:

graphviz	html5lib
image	matplotlib
opencv-python	pandas
plotly	requests
saspy==3.3.7	sas-dlpy
sas-espppy	sasoptpy
scikit-learn	swat
tensorflow	ws4py

Note: Installed packages affect the storage footprint. You should install only the packages that you require.

Environment Variables

To use Python with SAS Micro Analytic Service, note the following information about environment variables. Refer to the applicable product documentation, such as SAS Event Stream Processing, for information about how to set environment variables.

Environment Variable	Description
MAS_M2PATH	Specifies the absolute path to the mas2py.py file that is included with SAS Micro Analytic Service. It is used to execute Python code within a Python process that is launched by SAS Micro Analytic Service. This environment variable must be set.
MAS_PYPATH	Specifies the absolute path to the Python executable file. This environment variable must be set.
MAS_PYWAIT	Specifies the amount of time, in milliseconds, that SAS Micro Analytic Service waits for the spawned Python process to reconnect to the SAS Micro Analytic Service server after launching the Python client. The default value, 30000, is equal to 30 seconds. This environment variable is optional. Note: If 0 or a negative value is specified, it is ignored and the default value is used.

Python Logging

Overview

The Python logging facility provides a mechanism for additional instrumentation of executing Python modules.

Your Python code can define loggers using the [logging](#) facility for Python. Output from this logger is routed to the SAS Micro Analytic Service log.

The SAS Micro Analytic Service Python interface logs messages to the SAS Micro Analytic Service log. Internally, SAS Micro Analytic Service uses the following loggers:

- `App.MAS.Python`
- `App.MAS.Python.Trace`
- `App.tk.MAS.Python`

Specifying Logging Configuration Using a JSON File

The `MAS_PYLOG_CONFIG` environment variable enables you to specify logging configuration using a JSON file. For information about the valid keys that you can include in this file, see the Python [Dictionary Schema Details](#).

Here are some examples of Python logging configuration files.

- This configuration file sets the root logger to the DEBUG level:

```
{
  "version": 1,
  "incremental": true,
  "loggers": {
    "": {
      "level": "DEBUG"
    }
  }
}
```

- This configuration file sets the `App.MAS.Python` logger to the INFO level:

```
{
  "version": 1,
  "incremental": true,
  "loggers": {
    "App.MAS.Python": {
      "level": "INFO"
    }
  }
}
```

- This configuration file sets the root logger level to ERROR and the `App.Example.Python` logger to INFO.

It also attaches a file handler to the `App.Example.Python` logger. This means that this logger sends messages to SAS Micro Analytic Service (the root logger in this logging hierarchy) and to a file named `logfile.txt`.

```
{
  "version": 1,
  "disable_existing_loggers": false,
  "loggers": {
```

```

        "": {
            "level": "ERROR",
        },
        "App.Example.Python": {
            "level": "INFO",
            "handlers": ["file"]
        }
    },
    "handlers": {
        "file": {
            "class": "logging.FileHandler",
            "filename": "logfile.txt"
        }
    }
}

```

Capturing Python Process STDOUT/STDERR Data

You can capture the Python process STDOUT/STDERR to a file by using an environment variable. Two formats are available, as follows:

- To capture the data to a file, specify the file name using the MAS_PYOUT_FILE environment variable.
- To capture the data to a file that appends the process ID to the file name as a suffix, specify the file name using the MAS_PYOUT_FILE_ADDPID environment variable. The file is created using this format:

filename.<PID>

The PID value corresponds to the Python process ID. For example, here is a sample file name named mypyout.log in which the Python process ID is 7363:

mypyout.log.7363

By default, the file is overwritten at each Python invocation. If you prepend a plus sign (+) to the file name that you specify, the logger appends data to the log file, instead of overwriting the file. Here are some examples:

- To write the file to the local directory: MAS_PYOUT_FILE=+*mypyout.log*
- To write the file to the /home directory: MAS_PYOUT_FILE_ADDPID=+/home/*mypyout.log*

SAS Micro Analytic Service Logging and Deployment

<i>SAS Micro Analytic Service Logging</i>	157
Overview	157
SAS Micro Analytic Service Core Logging	158
DS2 Logging	159
<i>Deployment</i>	162

SAS Micro Analytic Service Logging

Overview

SAS Event Stream Processing provides a default logging configuration file. That file specifies loggers and appenders in addition to those described in this chapter.

Compiler messages are retrieved and logged by SAS Event Stream Processing when it publishes a module to SAS Micro Analytic Service.

You configure SAS Micro Analytic Service loggers using SAS Environment Manager. For information about how to configure loggers, see [“Specify a Service and Threshold Level for Service Logs” in SAS Viya Platform: Log Monitoring](#).

For more information about the SAS Viya platform and SAS Event Stream Processing logging, see the following documentation:

- [SAS Viya Platform: Log Monitoring](#)
- [“Logging” in SAS Event Stream Processing: Troubleshooting](#)

SAS Micro Analytic Service Core Logging

Logger names that start with the following prefixes apply to the SAS Micro Analytic Service core: Admin, App, Audit, Perf.

Here are some important SAS Micro Analytic Service core loggers:

Name	Events Logged	Default Threshold
App.license App.SQLServices.license	Licensing	ERROR
App.MAS.Python	Python process infrastructure	WARN
App.MAS.Python.Trace	Detailed Python process infrastructure	WARN
App.tk.MAS	SAS Micro Analytic Service	INFO
App.tk.MAS.CodeGen	Compilation messages produced during an attempt to publish Note: When a publish request fails, error information is logged regardless of the App.tk.MAS.CodeGen logger level.	FATAL
App.tk.MAS.CrossModCall	Cross module calling infrastructure	INFO
App.tk.MAS.CrossModCall.DS2	DS2 MASCall package	INFO
App.tk.MAS.Datagrid	Datagrid infrastructure	INFO
App.tk.MAS.Datagrid.DS2	DS2 Datagrid package	INFO
App.tk.MAS.GarbageCollection	Activity related to releasing resources that are associated with data structures that are no longer referenced	INFO
App.tk.MAS.JNI	Java Native Interface activity	INFO
App.tk.MAS.ParallelExec	DS2 MASParallelExec package activity	INFO
App.tk.MAS.Python	SAS Micro Analytic Service Python interface	INFO
App.tk.MAS.Python.DS2	DS2 PyMAS package	INFO

Name	Events Logged	Default Threshold
App.tk.MAS.RTrace	Shared object load activity	INFO
App.tk.MAS.Service	Start-up and shutdown events and SAS Micro Analytic Service version information	INFO
App.tk.MAS.StateSharing	State sharing infrastructure layer	INFO
App.tk.MAS.SysRsrc	Resource configuration activity	INFO
App.tk.MAS.StateSharing.DS2	DS2 MASState package	INFO
App.tk.MAS.Thread	Selection of worker thread activity	INFO
Audit.Table.Connection	Database connection	INFO

Detailed information about operations such as compilation start and finish is logged at the DEBUG level. Warning and error conditions are logged at the WARN or ERROR levels, as appropriate.

When App.tk.MAS.Service logger's level is set to DEBUG or TRACE, you see a message logging event that provides the SAS Micro Analytic Service version number in the log. For information about the SAS Viya platform software version numbers, see [“Software Releases and Versions” in Getting Started with SAS Viya Platform Operations](#).

IMPORTANT

If you change the value for any SAS Micro Analytic Service core loggers, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the `kubectl rollout restart` command to restart the pod without an interruption of service. For more information, see [“Applying Configuration Changes” on page 30](#).

DS2 Logging

Overview

The following loggers are available to diagnose DS2 problems:

- Configuration loggers that track information as the DS2 compiler starts up and provide context for the actual execution of the user's code.

- Run-time loggers that track actual execution.
- SQL services loggers that track information when the SQLSTMT package is used or when creating connections that are used by SQL statements.

TIP SAS recommends that you configure these loggers only when diagnosing a DS2 problem since the additional logging traffic affects performance.

DS2 Configuration Loggers

Logger	Description
App.TableServices.DS2.Config.Options	Shows the options that are supplied to the DS2 compiler.
App.TableServices.DS2.Config.Source	Shows the DS2 source code that is processed by the DS2 compiler.
App.TableServices.DS2.Config.Version	Shows version information for all threaded kernel extensions that are loaded by the DS2 compiler.

For more information, see [“Configuration Loggers”](#) in *SAS DS2 Programmer’s Guide*.

DS2 Run-Time Loggers

Logger	Description
App.TableServices.DS2.Runtime.Calls	Shows a trace of all method calls during execution.
App.TableServices.DS2.Runtime.SQL	Shows all SQL statements that are either prepared by the DS2 compiler, executed by the DS2 compiler, or both.
App.TableServices.DS2.Runtime.Timing	Shows the time that is spent during code compilation and execution. Depending on the level of information that is requested (INFO, DEBUG, TRACE), timing information is provided at various points throughout DS2 execution. Examples are parse time, compilation

Logger	Description
	time, various audit and transformation pass times, as well as the INIT, RUN, and TERM method execution times.
App.TableServices.DS2.Runtime.Put	Records all PUT statement output.
App.TableServices.DS2.Runtime.Log	Records all messages that are sent to the SAS log in a SAS session.

For more information, see [“Run-Time Loggers” in SAS DS2 Programmer’s Guide](#).

DS2 SQL Services Loggers

Logger	Description
App.SQLServices.API	This logger must be set to DEBUG to enable logging of connection and statement activity.
App.SQLServices.API.Environment	When set to TRACE, logs environment handle activity. Note: This is driver-independent activity.
App.SQLServices.API.Connection	When set to TRACE, logs driver-independent connection activity. Note: Prior to connection and after disconnection, the connection handle interoperates with certain API calls (such as setting or getting handle attributes). This logger contains those messages since the driver is not involved.
App.SQLServices.API.DRIVER_NAME.Connection	When set to TRACE, logs driver-specific connection activity. <i>DRIVER_NAME</i> must be the name of the database driver (for example, ORACLE, BASE, etc.) or <i>_ALL_</i> to log all database drivers. Here are some examples: ■ App.SQLServices.API.ORACLE.Connection ■ App.SQLServices.API._ALL_.Connection
App.SQLServices.API.DRIVER_NAME.Statement	When set to TRACE, logs driver-specific statement activity. <i>DRIVER_NAME</i> must be the name of the database driver (for example, ORACLE, BASE, and so on) or <i>_ALL_</i> to log all database drivers. Here are some examples: ■ App.SQLServices.API.BASE.Statement ■ App.SQLServices.API._ALL_.Statement

Logger	Description
<code>App.SQLServices.API.TF</code>	When set at any level, additional trace flag information is included in SQLStmt TRACE log events.
<code>App.SQLServices.API.TF</code> <code>.TIMESTAMP</code>	When set at any level, SQLStmt TRACE log events include a starting timestamp. This is useful to determine how much time was spent in various calls.

Deployment

SAS Micro Analytic Service is deployed automatically when SAS Event Stream Processing is deployed. If necessary, you can add environment customizations to your `.bashrc` file. For example, if you will use Python modules, you must complete the deployment and configuration steps that are described in [“Configuring Python for SAS Micro Analytic Service” on page 153](#).

If you encounter stack overflow errors when you are using the default SAS Micro Analytic Service worker thread stack size of 8 MB, you can override the default value by adding a definition of the `MAStktstacksize` environment variable to your `.bashrc` file. Note the following information when assigning a value to `MAStktstacksize`:

- You can specify a value using a unit of kilobytes or megabytes. The default value unit is kilobytes.
- To specify a value using megabytes, you must include “m” or “M” as a suffix following the number (for example, 10M or 10m).
- To specify a value using kilobytes, you can specify no suffix or add either a “k” or a “K” following the number. This means that any of the following are valid values: 10240, 10240k, 10240K.

Here are examples of different ways to assign a value of 10 MB to the `MAStktstacksize` environment variable:

- `export MAStktstacksize=10240`
- `export MAStktstacksize=10240k`
- `export MAStktstacksize=10240K`
- `export MAStktstacksize=10m`
- `export MAStktstacksize=10M`

PART 5

Using SAS Micro Analytic Service with SAS Intelligent Decisioning or SAS Model Manager

Chapter 17	
<i>DS2 Programming for SAS Micro Analytic Service</i>	165
Chapter 18	
<i>State Sharing between Modules</i>	199
Chapter 19	
<i>Best Practices for DS2 Programming in SAS Intelligent Decisioning</i>	215
Chapter 20	
<i>Python Support in SAS Micro Analytic Service</i>	223
Chapter 21	
<i>Module Execution</i>	237
Chapter 22	
<i>Database Access and Redis Access with DS2</i>	251
Chapter 23	
<i>Administration</i>	263

DS2 Programming for SAS Micro Analytic Service

Overview	166
DS2 Source Code Prerequisites	166
DS2 Identifiers	167
SAS Micro Analytic Service and SAS Foundation	168
Programming Blocks	169
Restrictions When Working with DS2 and SAS Micro Analytic Service	170
Character Restrictions	170
User-Defined Formats	170
About How DS2 Processes Missing and Null Values	170
SYSGET Function Restrictions	171
Public and Private Modules and Methods	171
Overview	171
Public Method Rules	172
Public Method Example	173
Private Method Example	174
Method Overloading	174
Argument Types Supported in Public Methods	175
Overview	175
Supported DS2 Data Types	175
Unsupported DS2 Data Types	176
About the DS2 init() Method	176
Promoting DS2 Execution Issues to an Execution Failure in SAS	
Intelligent Decisioning and SAS Model Manager	177
Performing Calls between SAS Micro Analytic Service Modules	178
Overview of Performing Calls between SAS Micro Analytic Service Modules	178
Determining Whether to Use MASCall	180
Using the MASCall Methods	181
Examples	184
Performing Parallel Sub-flow Calls in a SAS Micro Analytic Service Module	185
Overview	185
Determining Whether to Use MASParallelExec	185

Using the MASParallelExec Methods	186
Examples	188
Determining Whether DS2 Code Is Executing in SAS Micro Analytic Service	190
Managing Large DS2 Modules	190
Overview	190
Asynchronous Module Creation and Update	191
SAS Micro Analytic Service Time-Out Values	191
Loading Modules On-Demand	192
Verifying Minimum Memory	192
Loading Modules from the Repository at Start-Up	193
Understanding Eventual Consistency and Module Availability	193
Large Modules and Virtual Memory Resources	194
Composite Modules	194
Referencing Modules and Composite Submodules	194
Using Analytic Store Models	195
About Analytic Store Models	195
Publishing an Analytic Store Model	195
Calling Analytic Store Models Using DS2	195
Example	196
Accessing ASTORE File System Paths	197
Composite Modules	197

Overview

SAS Micro Analytic Service supports a subset of the DS2 programming language that is suitable for high-performance transaction processing in real time. This chapter covers only that subset. Note that DS2 batch processing is not supported.

For more information about the DS2 programming language, see [SAS DS2 Language Reference](#).

DS2 Source Code Prerequisites

The DS2 source code submitted to SAS Micro Analytic Service should begin with the following statement, just above the PACKAGE statement:

```
"ds2_options sas;"
```

This statement instructs DS2 to use SAS missing value handling and helps ensure that your DS2 program behaves the same as if it were run in SAS Foundation. DS2 source code should end with this statement:

```
"endpackage;"
```


The code cannot contain DATA statements, PROC statements, or THREAD statements. The source code should contain one and only one DS2 package, and this package can contain as many methods as desired.

It is a best practice to include a line feed character at the end of each source code line. This line feed character makes it easier to use compiler warning and error messages that include line numbers.

Note: DS2 supports only a specific style of comment. Comments start with the characters `/*`, and they end with the characters `*/`. All characters between the starting and ending characters are part of the comment text. When there is ambiguity in determining a token, the compiler always chooses the longest possible sequence of characters that can make up a token. Comments cannot be nested. For more information, see [“Comments” in SAS DS2 Programmer’s Guide](#).

DS2 Identifiers

For DS2 method, package, and argument names, SAS Micro Analytic Service supports regular identifiers and delimited identifiers. When using a delimited identifier, any character is allowed, including multi-byte and non-ASCII characters. You must begin and end delimited identifiers with double quotation marks. For complete information, see [“DS2 Identifiers” in SAS DS2 Programmer’s Guide](#).

In the SAS Micro Analytic Service REST service, the names of the output parameters of a step are the same as the `in_out` parameters names of the method. The parameter names of the method can use delimited identifiers. However, they are difficult to use in a JSON payload that uses double quotation marks to specify field names. Therefore, the double quotation marks around the delimited identifiers for output parameters are not included in the StepOutput.

Consider the following example:

A DS2 method returns a parameter called `CustomerAge` that has a value of 45. The JSON output would appear as follows:

```
{
  "name"      : "CustomerAge"
  "value"     : 45
}
```

However, if an embedded space was included for readability purposes, the delimited identifier `"Customer Age"` must be used. Because the double quotation mark (`"`) is a reserved character in JSON, it is represented by the escape characters `\`. This would result in the following JSON representation:

```
{
  "name"      : "\"Customer Age\"",
  "value"     : 45
}
```

Because this is difficult to read, the SAS Micro Analytic Service REST service removes the escape sequence characters. This results in the following JSON output:

```
{
    "name"      :    "Customer Age",
    "value"     "    "45"
```

This is easy to read, and it is compatible with standard JSON tools.

SAS Micro Analytic Service and SAS Foundation

Although DS2 is supported by both SAS Foundation and SAS Micro Analytic Service, SAS Micro Analytic Service has a lightweight, high-performance engine that does not support either the full SAS language or PROC statements. Therefore, PROC statements cannot be used. However, here is an effective DS2 authoring and testing mechanism: develop your DS2 packages in SAS Foundation using PROC DS2 and publish those packages to SAS Micro Analytic Service after removing the surrounding PROC DS2 syntax.

Here is an example PROC DS2 step that illustrates the above mechanism:

```
proc ds2;
  package myPackage / inline;
    dcl package logger logr('App.Test');
    method copyArray( char(12) in_array[*], in_out char
out_array[4] );
      out_array := in_array;
      logr.log('i', 'dim(in_array): $s,    dim(out_array): $s',
                dim(in_array),          dim(out_array) );
    end;
  endpackage;
  data _null_;
    method init();
      dcl package logger logr('App.Test');
      dcl int i;
      dcl package myPackage p();
      dcl char(12) inarr[6];
      dcl char(12) outarr[4];
      inarr[1] = 'one';
      inarr[2] = 'two';
      inarr[3] = 'three';
      inarr[4] = 'four';
      inarr[5] = 'five';
      p.copyArray(inarr, outarr);
      do i = 1 to dim(outarr);
        put outarr[i]= ;
      end;
    end;
  end;
```

```

        enddata;
    run;
quit;

```

Here is the log that is returned. Note that in this example, the input array contains six elements but the output array is limited to four.

```

INFO App.Test - dim(in_array): 6,    dim(out_array): 4
INFO App.Program - outarr[1]=one
INFO App.Program - outarr[2]=two
INFO App.Program - outarr[3]=three
INFO App.Program - outarr[4]=four

```

Programming Blocks

Each DS2 module represents exactly one package, and therefore the DS2 PACKAGE statement plays a major role in SAS Micro Analytic Service. A DS2 package contains one or more methods, and methods can contain a wide variety of DS2 language constructs. Package methods work well with rapid transaction processing because they can be called over and over again with little overhead, as transactions flow through the system. By contrast, the DS2 THREAD and TABLE statements are batch-oriented and are not supported.

The following code blocks are supported:

- PACKAGE...ENDPACKAGE
- METHOD...END
- DO...END

The following code blocks are batch-processing oriented and are not supported:

- TABLE...ENDTABLE
- THREAD...ENDTHREAD

Similarly, the following statements are not supported: OUTPUT and SET

- OUTPUT
- SET

Restrictions When Working with DS2 and SAS Micro Analytic Service

Character Restrictions

The following characters are not allowed in module IDs, package names, public method names, and submodule names:

- backslash (\)
- forward slash (/)
- period (.)
- semicolon (;)

User-Defined Formats

SAS Micro Analytic Service does not support the use of SAS sessions. Because a SAS session is required to create and access user-defined formats, SAS Micro Analytic Service does not support the use of user-defined formats.

About How DS2 Processes Missing and Null Values

SAS Micro Analytic Service supports only SAS missing value mode (SAS mode) for the processing of null and missing values. It does not support ANSI mode. Here is some key information about how DS2 processes missing and null values in SAS mode:

- A blank-filled character value is treated as a missing value.
- Null values are interpreted as SAS missing values, which are known values.
- Only DOUBLE or CHAR data types can have missing values. Data types other than DOUBLE or CHAR can have only null values.
- The DS2 NULL function returns a 1 only for a null value. It returns a 0 for any non-null value, including a SAS missing value.
- The DS2 MISSING function checks if a value is a null or missing value and returns a numeric result. If the argument does not contain a null or missing

value, SAS returns a value of 0. If the argument contains a null or missing value, SAS returns a value of 1.

For more information, see the following:

- [“How DS2 Processes Nulls and SAS Missing Values” in SAS DS2 Programmer’s Guide](#)
- [“NULL Function” in SAS DS2 Language Reference](#)
- [“MISSING Function” in SAS DS2 Language Reference](#)

The datagrid package processes null values differently than the DS2 package. For more information, see [“About How Datagrid Processes Null and Missing Values” in SAS DS2 Programmer’s Guide](#).

SYSGET Function Restrictions

For security reasons, you cannot use the SYSGET function to retrieve the value of an environment variable from DS2 code that is running in SAS Micro Analytic Service.

Public and Private Modules and Methods

Overview

Public and private modules and methods are SAS Micro Analytic Service concepts, rather than DS2 features. A module creator can choose to specify whether a module is public or private.

Public methods are methods of a module that are available for execution using the REST interface. In order to be available, a method can contain only those parameters that map to the following types and return a void result:

Scalar Types	Array Types
binary	bigintArray
bigint	dateTimeArray
datagrid	decimalArray
dateTime	integerArray

Scalar Types	Array Types
decimal	stringArray
integer	
string	
varbinary	

For more information about the DS2 types that correspond to the above types, see [“Argument Types Supported in Public Methods” on page 175](#).

A method that contains other types of parameters or returns a non-void value is a considered a *private method* and cannot be executed using the REST service. However, these methods can be called by other methods. Private methods are often used as utility or library methods to help solve larger problems.

Public modules are modules that contain public methods that can be executed using the REST interface.

A public module with no public methods is effectively a *private module*. Though private modules cannot be called through the REST interface, they can help to structure and manage code. Private modules can be used to hide internal implementations, share common code between public modules, hold a library of utility code, and so on.

Public Method Rules

Public methods must conform to the following rules:

- The return type must be void. Rather than using a single return type, public methods can return multiple outputs, where each output argument specifies the in_out keyword in the method declaration. Non-void methods are treated as private.
- Arguments that are passed by reference (meaning ones that specify in_out) are treated as output only. True update arguments are not supported by public methods. This restriction results in more efficient parameter marshaling and supports all interface layers.
- Input arguments must precede output arguments in the method declaration. It is permissible for a method to have only inputs or only outputs. However, if both are present, all inputs must precede the outputs.
- DS2 packages might not be passed as arguments in public methods. The presence of a DS2 package argument results in the method becoming private.
- The VARARRAY statement might not be present in the argument list of a public method. VARARRAY is a DS2 statement, not a data type. The presence of VARARRAY in a methods argument list causes the method to become private.

- For information about the data types that can be used as public method arguments, see [“Supported DS2 Data Types” on page 175](#).

Public Method Example

The example below illustrates a valid public method. It has a void return type (no RETURNS clause), uses only publicly supported data types, and treats in_out arguments as output only.

```
method quickSortStep (int lowerIndex, int higherIndex, in_out double
numbers[10]);

    dcl int i;
    dcl int j;
    dcl int pivot;
    dcl double temp;

    i = lowerIndex;
    j = higherIndex;

    /* Calculate the pivot number, taking the pivot as the
    * middle index number. */
    pivot = numbers[ceil(lowerIndex+(higherIndex-lowerIndex)/2)];

    /* Divide into two arrays */
    do while (i <= j);
        /**
        * In each iteration, identify a number from the left side
        * that is greater than the pivot value. Also identify a
        * number from the right side that is less than the pivot
        * value. Once the search is done, then exchange both
        * numbers. */
        do while (numbers[i] < pivot);
            i = i+1;
        end;
        do while (numbers[j] > pivot);
            j = j-1;
        end;
        if (i <= j) then do;
            temp = numbers[i];
            numbers[i] = numbers[j];
            numbers[j] = temp;

            /* Move the index to the next position on both
            *sides. */
            i = i+1;
            j = j-1;
        end;
    end;

    /* Call quickSort recursively. */
    if (lowerIndex < j) then do;
        quickSortStep(lowerIndex, j, numbers);
```

```

        end;
        if (i < higherIndex) then do;
            quickSortStep(i, higherIndex, numbers);
        end;
    end;
end;

```

Here is another example of a public method that illustrates the use of the HTTP package calling out to a web service using a POST request and then getting a response.

```

method httpPost( nvarchar(8192) url,
                 nvarchar(67108864) payload,
                 in_out nvarchar respbody,
                 in_out int hstat, in_out int rc );
declare package http h();
rc = h.createPostMethod( url );
if rc ne 0 then goto Exit;
rc = h.setRequestContentType( 'application/json;charset=utf-8' );
if rc ne 0 then goto Exit;
rc = h.setRequestHeader( 'Accept', 'application/json' );
if rc ne 0 then goto Exit;
rc = h.setRequestBodyAsString( payload );
if rc ne 0 then goto Exit;
rc = h.executeMethod();
if rc ne 0 then goto Exit;
hstat = h.getStatusCode();
if hstat lt 400 then h.getResponseBodyAsString( respbody, rc );
else respbody = '';
Exit:
h.delete();
end;

```

Private Method Example

The example below generates a private method in SAS Micro Analytic Service. It has a non-void return type. That is, it has a RETURNS clause in the declaration, which specifies a single integer return value.

```

method isNull(double val) returns int;
    return null(val) OR missing(val);
end;

```

Method Overloading

SAS Micro Analytic Service does not support method overloading. The DS2 programming language does support method overloading for programs running in other environments, but not when running in SAS Micro Analytic Service.

CAUTION

If you publish a DS2 package that contains overloaded methods, run-time errors can occur.

Argument Types Supported in Public Methods

Overview

SAS Micro Analytic Service supports a subset of the DS2 data types for use as public method arguments. Data types in the unsupported list can still be used in the body of a (public or private) DS2 package method, and as arguments to private methods. The lists of publicly supported and unsupported data types are included below.

Note: Any additional types added to the DS2 programming language in future releases should be considered unsupported unless otherwise stated in the SAS Micro Analytic Service documentation.

Supported DS2 Data Types

- BIGINT
- BINARY(n)
- CHAR(n)
- DOUBLE
- INTEGER
- NCHAR(n)
- NVARCHAR(n)
- VARBINARY(n)
- VARCHAR(n)

Note: All of these data types are supported for both scalar values and arrays.

The DS2 package datagrid is supported for scalar values.

Unsupported DS2 Data Types

- DATE
- DECIMAL(p, s)
- NUMERIC(p, s)
- PACKAGE

Note: The DS2 package datagrid is supported for scalar values. All other DS2 packages are not supported.

- TIME(p)
- TIMESTAMP(p)
- TINYINT

About the DS2 init() Method

When an `init()` method is present in a DS2 package that is published to SAS Micro Analytic Service, be aware of the following information:

- If the top-level package has an `init()` method, SAS Micro Analytic Service automatically executes it in the following circumstances:
 - when the DS2 module is published.
 - if a fatal runtime error is encountered. In this case, SAS Micro Analytic Service destroys and re-creates the DS2 execution context for that DS2 module.
- SAS Micro Analytic Service calls the `init()` method only for the top-level DS2 package.

An application (such as SAS Intelligent Decisioning or SAS Event Stream Processing) publishes DS2 code to SAS Micro Analytic Service via a composite module. A composite module consists of a top-level DS2 package. It includes at least one public method that is available for execution by the client application.

- A composite module can contain submodules that are either DS2 packages or analytic store models (ASTORES). If a DS2 submodule contains `init()` methods and you want those `init()` methods to be called only one time per execution context, ensure that the `init()` method in the top-level package is implemented such that it makes the submodule `init()` method calls.

If you want the `init()` method in a submodule to be called once, but not at publish time, add conditional logic to a method that is called after publishing.

Ensure that the condition evaluates to false after the desired `init()` method has been called one time.

- If you publish DS2 modules that contain package-scope variables to SAS Micro Analytic Service, you must be careful as to how you use them. By default, a pool of worker threads services requests. When SAS Micro Analytic Service is initialized, any free worker can service execution requests. A set once, read many pattern is acceptable in this type of environment.
- SAS Micro Analytic Service expects that the `init()` method has no parameters and expects no fatal errors.
- You must not overload the `init()` method.
- The `init()` method must not have a return value.
- The error promotion feature does not apply to a DS2 `init()` method when it is called automatically at publish time. For more information, see one of the following:
 - For SAS Event Processing, see [“Promoting DS2 Execution Issues to an Execution Failure in SAS Event Stream Processing” on page 107](#).
 - For the SAS Micro Analytic Service REST interface, see [“Promoting DS2 Execution Issues to an Execution Failure in SAS Intelligent Decisioning and SAS Model Manager” on page 177](#).

Promoting DS2 Execution Issues to an Execution Failure in SAS Intelligent Decisioning and SAS Model Manager

By default, when SAS Micro Analytic Service executes a DS2 method that encounters an error, it is logged and SAS Micro Analytic Service returns a failure code. This can help surface problems during early stages of code development. You can use the following environment variables to control this behavior:

Environment Variable	Description
MAS_PROMOTE_DS2_ERROR	When enabled, detects underlying errors that are encountered during execution. When these are detected, the SAS Micro Analytic Service execution operation returns a failure code to the client. ■ 0 (Disabled) ■ 1 (Enabled)
MAS_PROMOTE_DS2_WARNING	When enabled, detects underlying warnings and errors that are encountered during execution. When these are detected, the SAS Micro Analytic Service execution operation returns a warning code to the client.

Environment Variable	Description
	■ 0 (Disabled) ■ 1 (Enabled) Note: Be aware that if you enable the MAS_PROMOTE_DS2_WARNINGS environment variable, errors are promoted. It is not possible to promote warnings only.

For information about how to set a SAS Micro Analytic Service environment variable, see [“Configuring Supplemental SAS Micro Analytic Service Environment Variables” on page 26](#).

Note: When you set both of these environment variables to 0 (disabled), and SAS Micro Analytic Service executes a DS2 method that encounters an error or warning, it is logged but SAS Micro Analytic Service does not return a failure code to the client. Non-detection of a failure code can make it challenging to debug problems.

Note: The error promotion feature does not apply to a DS2 `init()` method when it is called automatically at publish time. For more information, see [“About the DS2 `init\(\)` Method” on page 176](#).

IMPORTANT If one of these environment variables is enabled and the DS2 code encounters a warning or an error, the execution of the code does not stop. It continues with the next statement. When the execution is complete, SAS Micro Analytic Service inspects the status and sends the return code in the HTTP response.

Performing Calls between SAS Micro Analytic Service Modules

Overview of Performing Calls between SAS Micro Analytic Service Modules

When a DS2 module references another DS2 package, the DS2 compiler does the following:

- 1 Copies, from the source code repository, the source code of the referenced package into the source code of the module to be published. The code is copied inline.

Note: The current DS2 language infrastructure supports source code repositories only.

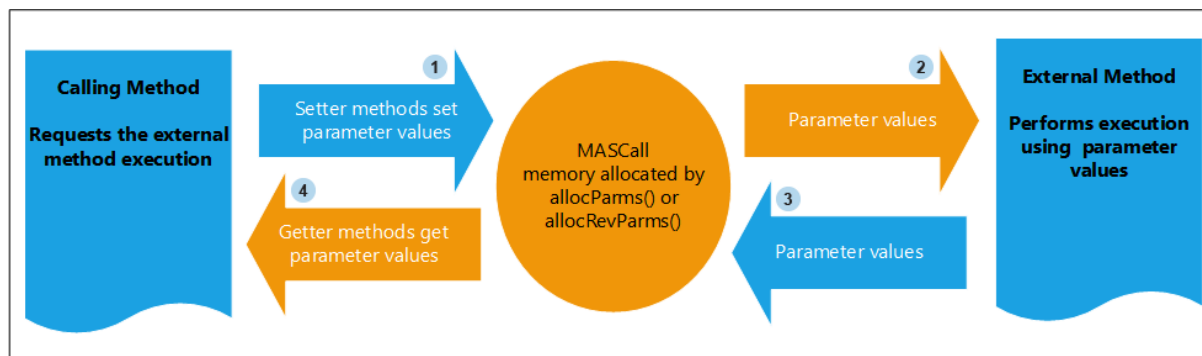
- 2 Compiles the combined source code into a single executable code stream.

A drawback to this approach is that the DS2 package code is repeated in every module that references it. This results in increased memory usage for every redundant copy of a package and longer compilation times.

However, SAS Micro Analytic Service resolves these issues via the DS2 MASCall package. This package contains methods that enable separate DS2 modules, published to SAS Micro Analytic Service, to call one another across separate module executables.

The following figure shows the details of how the MASCall package works. For details about method usage, see [“MASCall Package” on page 37](#).

Figure 17.1 The MASCall Process



- 1 The calling method uses setter methods to store argument values in MASCall memory.
- 2 The callModule() method or asyncCallModule() method executes the external method using the argument values that were written to MASCall memory in Step 1.
- 3 The callModule() method copies the external method's output parameter values to the MASCall memory.

This step does not apply to the asyncCallModule() method. When you use asyncCallModule(), any output arguments that are declared in an external module method signature are ignored. Therefore, output parameter values are not copied to MASCall memory.

- 4 The calling method retrieves, from MASCall memory, the output parameter values that contain the results of the execution of the external method.

Determining Whether to Use MASCall

Application Types and Using a Dynamic Call Interface

If your application is organized using composite modules in which the code is static, it is likely that MASCall would not benefit you.

However, if your application is designed such that the module code changes frequently and can benefit from making external calls, consider using MASCall.

It is also possible for your application to use a hybrid approach. This means that some packages are included statically as part of a composite module, and some methods that are located in external modules can be executed by using dynamic calls. The hybrid approach is helpful when your application needs to accommodate multiple revisions of some modules as well as dynamically calling others.

Benefits of Using MASCall

- Because MASCall does not retain the original called module into the caller, you can modify the called module after the caller has been published, as long as the called method has the same signature.

By contrast, when you do not use the MASCall dynamic call interface, a called module is compiled with the caller module. Therefore, if the called module is changed in the system, the caller module still refers to the original called module.

This can be useful if your versioning strategy requires that you change the called module after publishing the caller that is deployed. You can also deploy the new caller with the new module.

- Only a single copy of the called module code exists in the system, even if multiple callers exist. This reduces the memory footprint.
- In addition to a smaller memory footprint and shorter compilation times, this functionality enables a library of modules to be reused by many higher-level modules without penalty.

Items to Consider Before Using MASCall

- Using MASCall increases the complexity of your code.
- Marshaling method arguments can decrease performance.

- If the method to be called exists in a package that is small, rarely changes, or will not be called by many modules, the advantages of cross-module execution might not justify using the MASCall package.
- Not all methods that are called can be externalized. The external method must comply with the following requirements:
 - It must meet SAS Micro Analytic Service public method requirements. For more information, see [“Public Method Rules” on page 172](#).
 - The method parameter data types must be supported by MASCall. For more information, see the getter and setting methods in [“MASCall Package” on page 37](#).
- Compatibility errors are not generated from the compiler. Only run-time errors are generated.

Using the MASCall Methods

For detailed information about MASCall package usage, see [“MASCall Package” on page 37](#).

About the MASCall Methods

Here are the package MASCall methods:

- `allocParms( module_name, method_name )`
This method creates a parameter list for the latest revision of the specified method.
- `allocRevParms( module_name, revision_number, method_name )`
This method is similar to `allocParms`, but it enables you to specify a revision number.

Note: The called module name is case-sensitive on UNIX systems. It is not case-sensitive on Windows systems.

After `allocParms()` or `allocRevParms()` is called, the parameter setter methods operate on the newly created parameter list.

The argument takes a numeric index value or a parameter name. Note that for both the setter and getter methods, the index value is zero-based.

Here are the setter methods:

- Scalar argument setters are of the form:
`return_code = set<type>( arg, value )`
- Array argument setters are of the form:
`return_code = set<type>Array( arg, array_value )`

Executing the External Method

After the input values are set, you can execute the external method by making one of the following calls. Both call the external module method that was previously specified by `allocParms()` or `allocRevParms()`:

- **`callModule()`**

Use this method to make external calls and wait for them to run to completion.

- **`asyncCallModule()`**

Use this method to make external module calls without waiting for them to run. This means that any output arguments are ignored. For more information about using this method, including how to enable its use, see [“Additional Information about the `asyncCallModule` Method” on page 183](#).

Note: The `asyncCallModule` method is not currently supported for use with SAS Event Stream Processing.

As is noted above, because the method to execute was previously specified using `allocParms()` or `allocRevParms()`, `callModule()` and `asyncCallModule()` do not accept parameters.

Error Promotion

When executing external methods using `MASCall`, you can promote any errors or warnings that are encountered as a SAS Micro Analytic Service execution error. For more information, see one of the following:

- For SAS Event Processing, see [“Promoting DS2 Execution Issues to an Execution Failure in SAS Event Stream Processing” on page 107](#).
- For the SAS Micro Analytic Service REST interface, see [“Promoting DS2 Execution Issues to an Execution Failure in SAS Intelligent Decisioning and SAS Model Manager” on page 177](#).

Return Codes and Output

For all methods that return a status code, you must check the return code to determine whether the call was successful. For more information about nonzero return codes, see [Appendix 9, “SAS Micro Analytic Service Return Codes,” on page 377](#). A zero return code indicates success. A nonzero return code indicates the following:

- **`callModule()`**: a problem occurred when executing the method specified in the `allocParms` or `allocRevParms` method.

- **asyncCallModule():** a problem occurred when adding the request to a queue for asynchronous processing.

When you are using `asyncCallModule()` to execute an external module's method, any output arguments that are declared in an external module method signature are ignored.

When you are using `callModule()` to execute an external module's method, the output values that are returned by the method execution can be retrieved using these MASCall getter methods:

- Scalar argument getters are of the form:
value = `get<type>(arg)`
- Array argument getters are of the following form:
`get<type>Array(arg, array_value, rc)`

Note: DS2 passes arrays and output values by reference.

After the output values are retrieved, call `releaseParms()` to release the parameters and other resources that are used for the method. This releases the memory resources that are used for memory execution and prevents memory leaks.

Calling an External Method for Multiple Sequential Transactions

If the specified external method multiple times with no changes to its signature, you do not need to call `releaseParms()` and a subsequent `allocParms()` between each transaction. SAS Micro Analytic Service resets the parameters upon the next call to a setter method. If no setter methods are called before the next call to `callModule()` or `asyncCallModule()`, then the same input arguments are used.

Depending on your code and environment, using this design might increase performance.

Additional Information about the `asyncCallModule` Method

- By default, the `asyncCallModule` feature is disabled. To enable it, use the `core.numasynchthreads` property in SAS Environment Manager to specify the number of asynchronous processing threads to reserve for use by `asyncCallModule`.

For more information, see [“sas.microanalyticsservice.system: core” on page 272](#).

- It is expected that you will design routines and tune the system so that the methods are executed at some point. However, requests that are made via `asyncCallModule` do not time out. Execution is not guaranteed if requests are

made faster than they can be serviced. Therefore, if a request takes a long time to run, this could cause a backlog of requests that are waiting to run.

If outstanding `asyncCallModule` requests are queued to run and SAS Micro Analytic Service is terminated, SAS Micro Analytic Service logs a message indicating that the requests will not be processed.

- If the method that is called via `asyncCallModule` fails or encounters errors, the errors are logged.
- Nesting `asyncCallModule` is prohibited. This means that `asyncCallModule` returns a nonzero return code if it is already executing within an asynchronous call.

A return value of 1 from the `isWithinAsyncCall()` method indicates that a call is currently executing within a cross-module call.

Examples

Here are DS2 MASCall package example method calls, where `mc` is the DS2 MASCall instance variable that is created by calling the package constructor, which takes no arguments. Here are two ways to construct a MASCall package instance named `mc`:

- Example 1:

```
declare package mascall mc();
```

- Example 2:

```
declare package mascall mc;
mc = _new_ mascall();
```

General example:

```
rc = mc.allocParms( module_name, method_name );
rc = mc.setDouble( 'discount', additionalDiscount );
rc = mc.setString( 'phone', currentPhone );
rc = mc.setString( 'plan', currentPlan );
rc = mc.callModule();
treatments = mc.getString( 'offer' );
mc.releaseParms();
```

The complete set of DS2 package methods follows, where `rc` is the integer return code, and `mc` is the package instance. The argument takes a numeric index value or a parameter name. Note that the index value is zero-based.

Parameter resource management:

```
rc = mc.allocParms( module_name, method_name );
rc = mc.allocRevParms( module_name, revision, method_name );
mc.releaseParms();
```

Scalar argument setters:

```
rc = mc.setString( arg, value );
rc = mc.setLong( arg, value );
rc = mc.setInt( arg, value );
```

```
rc = mc.setDouble( arg, value );
```

Array argument setters:

```
rc = mc.setStringArray( arg, string_array );
rc = mc.setLongArray( arg, bigint_array );
rc = mc.setIntArray( arg, integer_array );
rc = mc.setDoubleArray( arg, double_array );
```

Execute or call a module's method:

```
rc = mc.callModule( );
```

Scalar argument getters:

```
string_var = mc.getString( arg );
long_var = mc.getLong( arg );
int_var = mc.getInt( arg );
double_var = mc.getDouble( arg );
```

Array argument getters:

```
mc.getStringArray( arg, string_array, rc );
mc.getLongArray( arg, bigint_array, rc );
mc.getIntArray( arg, integer_array, rc );
mc.getDoubleArray( arg, double_array, rc );
```

Performing Parallel Sub-flow Calls in a SAS Micro Analytic Service Module

Overview

SAS Micro Analytic Service provides the ability to execute module sub-flows via the [“MASParallelExec Package” on page 57](#).

Determining Whether to Use MASParallelExec

Items to Consider Before Using MASParallelExec

- Using MASParallelExec increases the complexity of your code.
- Not all methods can be called in parallel. The method to call in parallel must comply with the following requirements:

- It must meet SAS Micro Analytic Service public method requirements. For more information, see [“Public Method Rules” on page 172](#).

In addition, it cannot have an argument that is not supported by the MASParallelExec getters and setters.

- It must be defined in the module that constructs the parallel execution request.

- It must have the following as its final argument:

```
in_out package datagrid <varName>
```

- Compatibility errors are not generated from the compiler. Only run-time errors are generated.

Executing Python Modules in DS2 Modules

When a parallel executed call is using a PyMAS Python module, the publish of the Python module should be performed before the parallel execution request.

If the Python module publish occurs in the method that is being executed in parallel, the module is published only once, and the published module is shared among the parallel execution processes during the parallel execution.

Using the MASParallelExec Methods

For detailed information about the MASParallelExec package usage, see [“MASParallelExec Package” on page 57](#).

IMPORTANT To enable use of the MASParallelExec package, you must set the maximum number of threads to use with parallel execution by assigning a value to the `core.maxpethreads` property in SAS Environment Manager. For more information, see [“sas.microanalyticsservice.system: supplementalProperties” on page 276](#).

About the MASParallelExec Methods

Here is the package MASParallelExec method that defines a method to call in parallel:

```
callId = addCall( call_name, method_name )
```

This method creates a parameter list for the specified method.

After the `addCall()` method is called, use the parameter setter methods to set the values to be consumed by the called method. The setter methods take the numeric

call identifier that is returned by the `addCall()` method and a numeric index value or a parameter name. Note that for both the setter and getter methods, the index value is one-based.

The setter methods are of the form:

```
return_code = set<type>( callId, arg, value )
```

Executing the Parallel Methods

After all of the parallel calls to execute have been defined and their input values are set, you can execute the defined methods by calling the `wait()` method.

The `wait()` method schedules all unscheduled call definitions and waits for their completion. To schedule an `addCall()` earlier than the `wait()` method execution, you can use the `call()` method.

The default time-out value is five seconds. You can modify the default wait value using the `core.pewaittimemilliseconds` property in [“sas.microanalyticsservice.system: supplementalProperties” on page 276](#).

Error Promotion

When executing external methods using `MASParallelExec`, you can promote any errors or warnings that are encountered as a SAS Micro Analytic Service execution error. For more information, see [“Promoting DS2 Execution Issues to an Execution Failure in SAS Intelligent Decisioning and SAS Model Manager” on page 177](#).

Return Codes and Output

For all methods that return a status code, you must check the return code to determine whether the call was successful. For more information about nonzero return codes, see [“SAS Micro Analytic Service Return Codes” on page 377](#).

Note the following:

- For all methods, a negative return code indicates that an error occurred.
- Except for the `addCall()` and `wait()` methods, a zero return code indicates success.
 - For the `addCall()` method, a 0 (zero) or positive return code indicate success.
 - For the `wait()` method, a 0 (zero) or 1 (timed out) return code indicate success.

Output values from the parallel executed calls are available only for calls that completed successfully. The `callStatus()` method can be used to retrieve the call completion status. This information is also available in the datagrid that is returned from a successful `wait()` method call. As noted above, be aware that a `wait()` method call that returns success or timed out are both considered successful.

The resources that are used by the MASParallelExec package instance are not released until all parallel activity has completed and the package instance destructor has been called.

Examples

Here are DS2 MASParallelExec package example method calls, where **mpe** is the DS2 MASParallelExec instance variable that is created by calling the package constructor (which takes no arguments). Here are two ways to construct a MASParallelExec package instance named **mpe**:

■ Example 1:

```
declare package masparallelexec mpe();
```

■ Example 2:

```
declare package masparallelexec mpe;
mpe = _new_ masparallelexec();
```

Note: The MASParallelExec instance does not release its resources until all parallel execution has completed and the instance is destroyed. Therefore, it is recommended that the MASParallelPackage instance exist within the method that builds the parallel execution request. Also, be aware that MASParallelExec instances are not reusable.

General example:

```
...
method callExecuteA(varchar(255) Str, in_out package datagrid answer);
...
end;

...
method callExecuteB(double dbl, in_out package datagrid answer);
...
end;

...
method execute(in_out rc);
  dcl int          call01 call02;
  dcl package datagrid callOutput('callOutput')
results('results');
  dcl package logger      logr('Example.General');
  dcl package masparallelexec mpe();

  /* Define the methods to execute in parallel */
  call01 = mpe.addCall('call01', 'callExecuteA');
  if (call01 < 0) then do;
    logr.log('e', 'addCall 1 rc=$s', call01);
    return;
  end;
  rc = mpe.setString(call01, 1, 'String 1 value');
  if (rc) then do;
```

```

    logr.log('e', 'setString 1 rc=$s', rc);
    return;
end;

/* Error checking left off for brevity */
call02 = mpe.addCall('call02', 'callExecuteB');
rc = mpe.setDouble(call02, 1, 3.);

/* Start execution of call02 immediately */
rc = mpe.call(call02);

/* Wait for sub-decisions to complete */
rc = mpe.wait(results);

/* Process parallel call01 output datagrid */
rc = mpe.getResults(call01, callOutput);
end;

```

The complete set of DS2 package methods is as follows, where `rc` is the integer return code, and `mpe` is the package instance. The argument takes a numeric index value or parameter name. Note that the index value is one-based.

Resource Management:

```
callId = mpe.addCall(call_name, method_name);
```

Argument Setters:

```

rc = mpe.setBool(callId, arg, value);
rc = mpe.setDatagrid(callId, arg, value);
rc = mpe.setDouble(callId, arg, value);
rc = mpe.setInt(callId, arg, value);
rc = mpe.setLong(callId, arg, value);
rc = mpe.setString(callId, arg, value);
rc = mpe.setVarstring(callId, arg, value);

```

Execution:

```

rc = mpe.call(callId);
rc = mpe.wait();
rc = mpe.wait(millisecods);
rc = mpe.wait(value);
rc = mpe.wait(millisecods, value);

```

Results:

```

rc = mpe.callStatus(callId, value);
rc = mpe.getResults(callId, value);

```

Argument Getters

```

rc = mpe.getBool(callId, arg, value);
rc = mpe.getDatagrid(callId, arg, value);
rc = mpe.getDouble(callId, arg, value);
rc = mpe.getInt(callId, arg, value);
rc = mpe.getLong(callId, arg, value);
rc = mpe.getString(callId, arg, value);
rc = mpe.getVarstring(callId, arg, value);

```

Determining Whether DS2 Code Is Executing in SAS Micro Analytic Service

The DS2 function `inmas()` discovers whether SAS Micro Analytic Service is running in the current process and, if so, determines whether the current thread is a member of the SAS Micro Analytic Service worker thread pool. If it is, then the DS2 code is running inside SAS Micro Analytic Service.

The function returns 1 (TRUE) if the DS2 code is executing in SAS Micro Analytic Service, and 0 (FALSE) otherwise.

This can be useful to know when, for example, you have DS2 code that works in various locations, but not in SAS Micro Analytic Service.

Managing Large DS2 Modules

Overview

SAS Micro Analytic Service compiles DS2 code and retains that compiled code in memory (known as the *repository*). Thus, SAS Micro Analytic Service can execute code and return responses in near real-time. A large DS2 module requires a lot of time to compile. Subsequently, this affects the time that it takes to load the module from the repository which, in turn, effects server start-up time.

Large modules also require more memory to host in a server. Memory usage depends on the size of the module code as well as the number of core threads. More core threads use more memory.

SAS Micro Analytic Service includes features and options to support and manage large DS2 modules. This enables a system administrator to balance the size and complexity of the modules against system memory, system responsiveness, and availability. This balancing process includes asynchronous calls to create or update modules, as well as time-outs on operations that create or update modules. It can also include disallowing a module to the persistence store if it takes too long to compile or there is insufficient available memory.

The topics in this section describe these features and options.

Asynchronous Module Creation and Update

Calls to the SAS Micro Analytic Service REST service are governed by the time-out value that is configured for the SAS Viya platform web server. If the time to compile the content exceeds that value, a call times out for the client.

The SAS Micro Analytic Service jobs endpoint allows the caller to submit a Create request or an Update request that is longer than the SAS Viya platform web server time-out value. Some benefits include the following:

- The caller can check the status of a job and monitor its progress.
- The caller can delete jobs before they are executed or after completion.
- Outdated jobs are automatically purged. By default, this occurs after 24 hours.

You can modify this value by using the `service.jobs.cleanup.agehours` and `service.jobs.cleanup.frequencyhours` properties in SAS Environment Manager. For more information, see [“sas.microanalyticsservice.system: supplementalProperties” on page 276](#).

- The module ID that is created or updated is logged for all messages that are related to an associated job.

For more information about the jobs API, see the [REST API documentation](#).

SAS Micro Analytic Service Time-Out Values

Compiling DS2 Modules

SAS Micro Analytic Service imposes a time limit for compiling DS2 modules. If the compilation time exceeds this time-out value, the module is not compiled and therefore is not persisted in the repository. Keep in mind that because extremely large modules take a long time to compile, they might not be suitable for deployment in SAS Micro Analytic Service.

This compilation time-out value is configurable. You can modify it by using the `service.timeouts.maxmodulecompiletimemillis` property in SAS Environment Manager.

For more information, see [“sas.microanalyticsservice.system: supplementalProperties” on page 276](#).

Creating and Updating Synchronous Modules

In addition to the previously mentioned asynchronous methods, you can also create or update modules synchronously. To configure a time-out value for these methods, use the `service.timeouts.maxloadwaitnonexecmillis` property in SAS Environment Manager.

For more information, see [“sas.microanalyticsservice.system: supplementalProperties”](#) on page 276.

Loading Modules On-Demand

If a call to access a module is made, and it has not been loaded from the repository, SAS Micro Analytic Service attempts to compile the module. This might occur following a restart if the server is still loading modules from the library or if the module was created or updated on another cluster node.

This also might occur in cases in which a module is compiled because of a query request for detailed information that can be retrieved only from a compiled module, or a request to execute that module.

Both of these situations have a corresponding configurable time-out value that you can modify by using the following configuration items:

- **query operation time-out** : `service.timeouts.maxloadwaitnonexecmillis`
- **execution operation time-out** : `service.timeouts.maxloadwaitexecmillis`

For more information, see [“sas.microanalyticsservice.system: supplementalProperties”](#) on page 276.

Verifying Minimum Memory

If a minimum amount of memory is not available on the server, SAS Micro Analytic Service does not compile a module. In this case, an internal error is returned to the caller. If the low memory condition occurs during the loading of the module from the repository, the messages are logged and the module is not loaded.

The logged message contains the details of available and used memory.

You can modify the minimum value limit by using the `core.minfreememoryfloor` configuration item.

For more information, see [“Configure Default Virtual Memory Resources”](#) in *SAS Viya Platform: Deployment Guide*.

Loading Modules from the Repository at Start-Up

When a cluster node starts, it attempts to load all the modules that are in the repository. These modules are already in the system, so loading them is prioritized over processing any job requests to create or update modules.

Because these modules were validated before they were added to the repository, errors are not expected when they load. In the rare case that a specific module fails to load, possibly because of an environmental factor (for example, a memory issue), that module is not loaded. There are no subsequent attempts to load the module unless the node is restarted.

Understanding Eventual Consistency and Module Availability

Some special considerations apply to clustered deployments with regard to large modules.

In a clustered multi-node system, modules must be compiled on all nodes of the cluster that host SAS Micro Analytic Service. The first node that receives a Create or Update request compiles the module and responds to the caller with any compilation errors. If there are no errors, SAS Micro Analytic Service saves the module code to the repository and loads it to the other nodes for compilation.

Depending on any other modules that are in the process of compiling, some nodes might not be ready to load this module. During this time, a request to execute that module succeeds only if it is serviced by a node in which the module is compiled and loaded. Modules that were previously loaded are available and requests to execute are serviced.

If a module is compiled synchronously (by using the create module or update module API) control returns to the caller as soon as the module is compiled on one node of the cluster. Similarly, if a module is compiled asynchronously, a job status of **completed** is returned as soon as the module is compiled on one node of the cluster. The other nodes continue to compile the module. Eventually, all the nodes compile and load the module. However, before that happens, requests to execute that module might fail. In this case, you need to resubmit the request.

Note: You can manage SAS Micro Analytic Service readiness by using the `service.cache.refresh.readiness.startuponly` and `service.cache.refresh.readiness.disabled` properties in SAS Environment Manager. For more information, see [“sas.microanalyticservice.system: supplementalProperties” on page 276](#).

Large Modules and Virtual Memory Resources

Large modules or large numbers of modules often require a large number of mapped memory requests. This might lead to a situation in which the virtual memory is exceeded, resulting in a memory allocation failure.

If you encounter unexpected memory allocation failures and your system has plenty of free memory, you should increase the value that is assigned to the `vm.max_map_count` property. For more information, see [“Configure Default Virtual Memory Resources” in SAS Viya Platform: Deployment Guide](#).

Composite Modules

A composite module is a module that contain zero or more submodules. Only a top-level module can contain public methods that are available for execution through the REST interface.

Submodules are similar to private modules in the following ways:

- they are not available for execution through the REST interface
- they are available for use only by other submodules and by the top-level module

Unlike private modules, which must have unique names across the system, a submodule name must be unique only for that composite module.

The top-level module of a composite module must contain DS2 source code. Submodules can contain DS2 source code or refer to a file that contains an analytic store model. Note that this is the only way to execute analytic store models.

Referencing Modules and Composite Submodules

A module can reference other DS2 packages or analytic store models.

When a module is compiled, the system is searched to satisfy these dependencies in the following order:

- 1 the submodules in any of the supplied module definition

2 the modules in the repository

The included submodules or modules can contain further dependencies. These are resolved in the same order as specified above.

Using Analytic Store Models

About Analytic Store Models

An analytic store file, called an ASTORE file, is a system that allows the state of a trained analytical model to be saved in a transportable form. This enables it to subsequently be used to score new data in a variety of environments. Many SAS analytical procedures save the results from the training phase of model development as analytic store models. A key feature of an ASTORE file is that it can be easily transported from one platform to another. When an analytic store is published to SAS Micro Analytic Service, the state of the predictive model is restored and is available for scoring new data.

Publishing an Analytic Store Model

Unlike DS2 and Python modules, analytic store models are not published to SAS Micro Analytic Service as source code. Instead, analytic store models consist of binary code and metadata. Client applications deliver analytic store models to SAS Micro Analytic Service as ASTORE disk files.

Note: For information about calling an analytic store model by a DS2 module, see the next section.

Calling Analytic Store Models Using DS2

If an analytic store model has been registered with SAS Micro Analytic Service, it can be called by a DS2 module.

A DS2 module that calls an analytic store model must include an `init()` method that invokes the score package's `setvars()` and `setkey()` methods.

Note: Failure to set this option can cause the system to stop responding on module deletion or on shutdown.

The `setvars()` method is used by the DS2 score package to map variables to the analytic store model's input and output parameters. The `setkey()` method takes a SHA-1 hexadecimal key as input and uses it to look up the analytic store model.

SAS Micro Analytic Service automatically calls the `init()` method, if present, when a DS2 module is published.

Example

```
ds2_options sas;
package astoretest/overwrite=yes;
dcl package score sc();
dcl double CLAGE;
dcl double CLNO;
dcl double DEBTINC;
dcl double DELINQ;
dcl double NINQ;
dcl double VALUE;
dcl double _P_;
dcl double P__EVENT_0;
dcl double P__EVENT_1;
dcl nchar(32) I__EVENT_;
dcl nchar(4) _WARN_;
varlist allvars [_all_];

method init();
  sc.setvars(allvars);
  sc.setkey(n'EB3D1CA20AA0CB74465D25EEE2290E13692AF750');
end;

method preCode();
  _P_ = 0.999;
end;

method postCode();
end;

method term();
end;

method astoreScore(double inCLAGE, double inCLNO, double inDEBTINC,
  double inDELINQ, double inNINQ, double inVALUE,
  in_out double out_P_, in_out double outP__EVENT_0,
  in_out double outP__EVENT_1, in_out nchar
outI__EVENT_,
  in_out nchar out_WARN_);
  CLAGE = inCLAGE;
  CLNO = inCLNO;
  DEBTINC = inDEBTINC;
```

```

DELINQ = inDELINQ;
NINQ = inNINQ;
VALUE = inVALUE;

preCode();
sc.scoreRecord();
postCode();

out_P_ = _P_;
outP__EVENT_0 = P__EVENT_0;
outP__EVENT_1 = P__EVENT_1;
outI__EVENT_ = I__EVENT_;
out_WARN_ = _WARN_;
end;

endpackage;

```

Accessing ASTORE File System Paths

In order to publish decisions that use analytic store models to a SAS Micro Analytic Service destination, the `/models/astores/viya` mounted directory path must be accessible.

This directory path points to the ASTORES persistent volume claim (PVC) that is configured during the deployment of the SAS Viya platform.

For more information, see the README.md file that is located in the `$deploy/sas-bases/examples/sas-microanalytic-score/astores` directory where you downloaded your files for deployment, as well as the [SAS Viya Platform: Deployment Guide](#).

Note that users who need to work with analytic store models must have Read and Write access to analytic store directories.

Composite Modules

Composite modules enable DS2 code running in SAS Micro Analytic Service to call one or more analytic store models. A composite module consists of one DS2 module and zero or more analytic store models, which are known as the members of the composite. All members of a composite module are published, replaced, or deleted as a set. For example, if one member of a composite module fails to publish, then all members of the composite fail to publish. Also, any prior revisions of the composite module remain unchanged.

Like other module types, the revisions of a composite module are owned by a module context, have dictionaries, and can be queried for compilation messages, creation date-times, and so on. The members of a composite module can include either a DS2 module or an analytic store model, or both. Other module types, such as Python and C, are not supported as composite members.

Only the methods of the DS2 module of the composite are exposed for client applications to call.

To avoid name collisions with DS2 modules that exist outside a composite module, each composite revision has its own namespace. Therefore, each composite module must be self-contained and without dependencies on any non-member module. Unlike other module types, composite modules can be called only externally.

State Sharing between Modules

Overview	199
Shared Vectors	200
Overview	200
About Supported DS2 Data Types	201
State Vector Types	201
Local State Vector Methods	203
Shared State Vector Methods	203
Setter and Getter Examples	205
Shared Hash Tables	208
Overview	208
About Using Shared Hash Tables in DS2	208
Methods That Operate on the Default Shared Hash Table	210
Default Shared Hash Table Example	211
Methods That Operate on Non-default Shared Hash Tables	212

Overview

SAS Micro Analytic Service provides two ways to share data between modules that are executing within a user context: shared vectors and shared hash tables. *Shared vectors* are collections of data values. *Shared hash tables* are containers of shared vectors; the vectors accessed by using keys.

When it is possible to represent the data, or *state*, that you want to share across modules by a small number of vectors, the vectors can be shared with other modules by name. However, vector lookup by name is a linear search and is therefore inefficient when larger numbers of vectors are present. In such cases, shared hash tables are highly recommended because of their efficiency.

Note: If vectors are not contained in a shared hash table, the number of vectors is limited to 64.

When using shared hash tables, an efficient non-cryptographic hashing function is applied to a key to quickly compute the desired vector's location within the hash table. Shared hash tables also use non-locking synchronization mechanisms to further increase efficiency.

Whether using shared vectors or shared hash tables, DS2 authors can use the `MASState` package to create, share, retrieve, and delete data. In all cases, a zero return code indicates success. A nonzero return code indicates failure. For more information about non zero return codes, see [Appendix 9, "SAS Micro Analytic Service Return Codes," on page 377](#). If you encounter a nonzero return code, check the applicable log file for more information.

IMPORTANT

- SAS Micro Analytic Service shared state vectors and shared hash tables are available only for DS2 modules. They are not supported for Python modules.
- These features support in-memory state sharing. They are not intended for state-sharing across cluster nodes.

Shared Vectors

Overview

Collections of state data fields that are managed as a unit are referred to as *state vectors*. Here are some key points about state vectors:

- A state vector contains one or more values, which are referred to by vector name and a zero-based index.
- The data values in a state vector can contain the same data types or a mix of data types.
- The number of data elements that is contained in a state vector is limited only by the available memory.
- A state vector is similar to a database record in that it can contain multiple data values of various types. However, it differs from a database record in that data values are positional, rather than organized in named columns.
- You can initialize a shared state vector from a DS2 package method, including the `init()` and constructor methods.
- A shared state vector name must be unique within the current user context.

About Supported DS2 Data Types

State vector values can have any of the following DS2 data types:

BIGINT	DOUBLE	NVARCHAR
BINARY	DOUBLE ARRAY	NVARCHAR ARRAY
CHAR	INTEGER	VARCHAR
CHAR ARRAY	INTEGER ARRAY	VARCHAR ARRAY

In SAS Micro Analytic Service, *binary data* typically refers to binary or character long objects. These can be expressed as pointer and length pairs or as character strings. Because DS2 does not support pointers directly, operations on binary data are typically performed with string manipulation functions. An example would be on blobs that have been Base64 encoded into a string.

The `setReference()` method accepts a bigint reference value (for example, a pointer to a blob or other binary data in memory) and a size (blob size in bytes or length of other binary data).

You can use the `stringAddrLen()` method to obtain the internal memory address of the data buffer and length of a DS2 string. Here is an example:

```
rc=stringAddrLen( myBase64encodeBLOB, addr, len )
```

The first parameter is a string input argument and the second and third arguments are bigint output parameters.

The return code indicates success or failure. When successful, the `addr` and `len` output arguments correspond to the address and the length of the input string. You would use these values as the input to a subsequent `setReference()` call.

The `getReference` method returns a DS2 binary data type. (See [“Scalar Getters Example” on page 206](#), for an example.) The asymmetrical nature of this setter/getter pair is due to limitations with binary processing that exist only on the setter side. With the exception of binary, all other data types are handled symmetrically.

State Vector Types

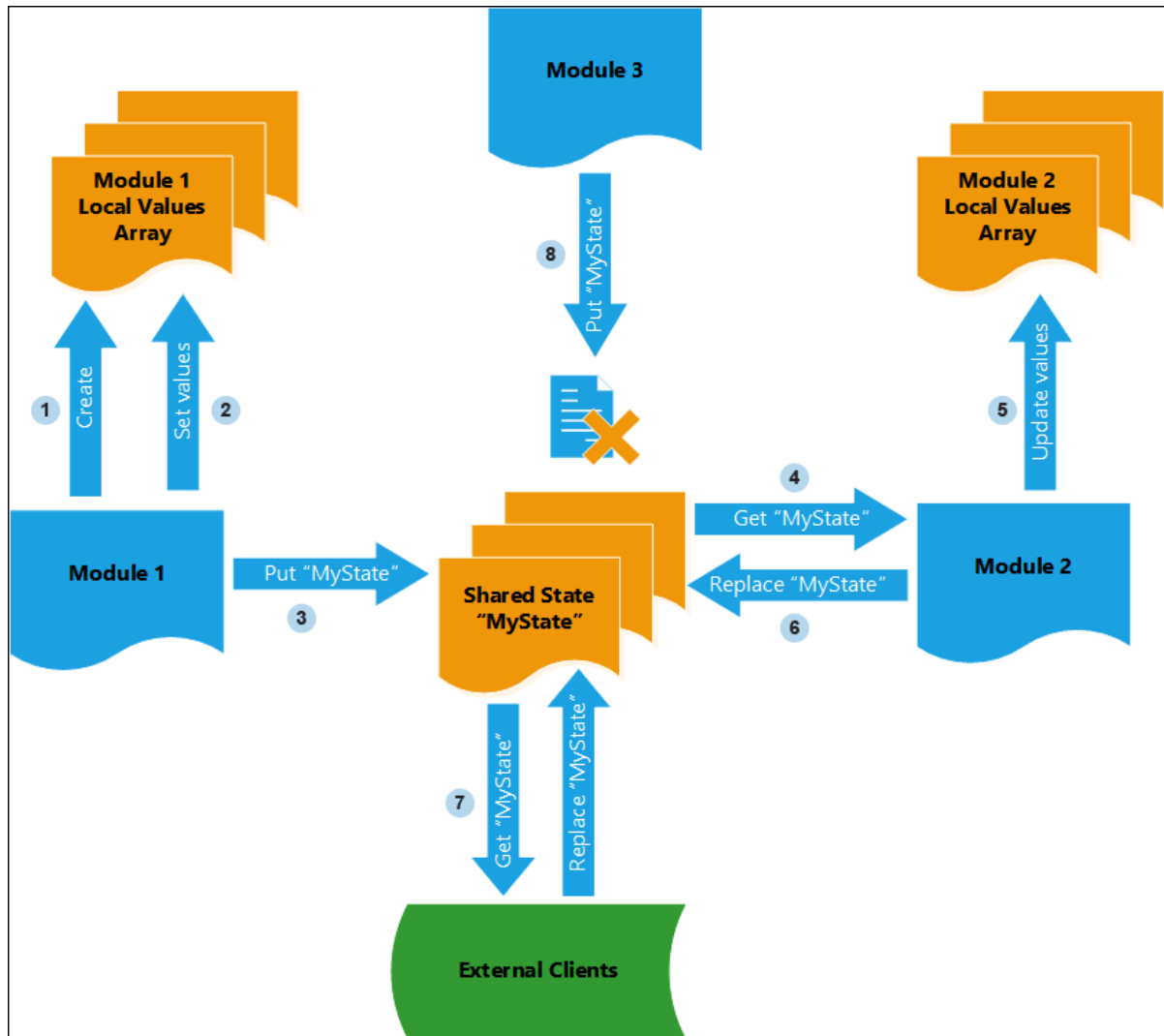
There are two categories of `MASState` package methods— those that operate on local state vectors and those that control state vector sharing.

Setting and retrieving individual values is always performed using local state vectors. When a shared state vector is fetched, a local copy of that vector is created and returned to the caller.

Similarly, when a state vector is shared, a copy of the local vector is created and made centrally available for fetching by other modules and transactions.

Working with local state vectors has the advantage of allowing a set of values to be updated and shared as a unit. This eliminates race conditions that could otherwise occur, and enables consistent and complete state representations.

Figure 18.1 The State Vector Sharing Process



- 1 Module 1 creates a local values array.
- 2 Module 1 sets the values for the array.
- 3 These values are published as a shared state vector and assigned the name MyState. This makes a deep copy of the vector.
- 4 Module 2 retrieves the MyState local vector. This makes a deep copy of the vector.
- 5 Module 2 updates the values and replaces the values in the local values array.
- 6 Module 2 replaces the values in the MyState shared state vector.
- 7 External clients retrieve and replace values for the MyState shared state vector.

- 8 Module 3 attempts to create a shared state vector called `MyState`. This is rejected because a shared state vector with that name already exists.

The `MASState` package includes 28 methods. The following sections contain usage examples for each of these methods.

Note that each example assumes that an instance of the `MASState` package, called `st`, has been created:

```
dcl package masstate st()
```

Local State Vector Methods

The following methods control the creation and deletion of local vectors.

`createVector( name, size )`

This method creates a local state vector with the specified name, and space for the number of values that is indicated by the specified size. The following example creates a local state vector named `MyVector` with a size of 4:

```
rc = st.createVector('MyVector', 4);
```

`deleteVector( name )`

This method deletes the local state vector referenced by name. The following example deletes the local vector created above:

```
rc = st.deleteVector('MyVector');
```

`deleteAllVectors()`

This method deletes all local vectors. The following example deletes all local vectors managed by the current `MASState` package instance:

```
rc = st.deleteAllVectors();
```

Shared State Vector Methods

The following methods control the sharing and unsharing of state vectors with other modules, and across transaction boundaries.

shareVector(name)

This method creates a copy of the named local state vector and makes it accessible to other modules within the current user context. The name passed to `shareVector()` must be unique within the user context. Otherwise, a duplicate name error is returned and the vector is not shared. To update an existing shared state vector, call `replaceSharedVector()`.

```
method setValuesAndShareVector(in_out int rc);

    /* Create local vector */
    rc = st.createVector('MyVector', 4);

    /* Populate it with values*/
    rc = st.setInt('MyVector', 0, 100);
    if (rc ne 0) then return;
    rc = st.setInt('MyVector', 1, 200);
    if (rc ne 0) then return;
    rc = st.setInt('MyVector', 2, 300);
    if (rc ne 0) then return;
    rc = st.setInt('MyVector', 3, 400);
    if (rc ne 0) then return;

    /* Share vector with other modules */
    rc = st.shareVector('MyVector');
end;
```

fetchSharedVector(name)

This method fetches the shared state vector referenced by name and returns a local copy of it. It is used to retrieve stateful data that has been published or updated by other modules. After calling this method, the `MASState` package instance holds a local copy of the shared state vector, which can be referenced by name.

```
method fetchSharedVector(in_out int rc);
    rc = st.fetchSharedVector('MyVector');
end;
```

unshareVector(name)

This method removes sharing for the vector referenced by name. The shared copy of the vector is deleted from the current user context, and modules are no longer able to access it. If no shared vector with the given name exists, this is considered a valid condition and `unshareVector()` does not return an error. The `unshareVector()` method does not affect a local state vector.

```
method unshareVector(in_out int rc);
```

```

        rc = st.unshareVector('MyVector');
    end;

```

replaceSharedVector(name)

This method creates a copy of the named local state vector and replaces the existing shared state vector of the same name, making the updated data accessible to other modules within the user context. The name that is passed to `replaceSharedVector()` must refer to an existing shared state vector. Otherwise, a **not found** error is returned and the data is not shared.

```

method setNewValuesAndReplaceSharedVector(in_out int rc);

    /* Populate vector */
    rc = st.setInt('MyVector', 0, 111);
    if (rc ne 0) then return;
    rc = st.setInt('MyVector', 1, 222);
    if (rc ne 0) then return;
    rc = st.setInt('MyVector', 2, 333);
    if (rc ne 0) then return;
    rc = st.setInt('MyVector', 3, 444);
    if (rc ne 0) then return;

    /* Share vector with other modules */
    rc = st.replaceSharedVector('MyVector');
end;

```

isVectorShared(name)

This method returns integer 1 (TRUE) if a shared state vector with the given name exists within the current user context. Otherwise, it returns integer 0 (FALSE).

```

method isVectorShared(in_out int result);
    result = st.isVectorShared('MyVector');
end;

```

Setter and Getter Examples

Setter and getter methods are provided for each data type. These methods operate on local vectors only. Individual data items are referenced by local vector name and by the zero-based index of the data value.

The examples in this section illustrate each type-specific setter method. The MASState package guards against errors such as index out of range and invalid data. As a best practice, you should check return codes, and if applicable, return them to the caller.

Scalar Setters Example

```
method testScalarSetters(vvarchar(32) strVal,
                        int intVal,
                        bigint longVal,
                        double dblVal,
                        bigint refAddr,
                        bigint refLen,
                        in_out int rc);

rc = -1;

/* Populate the vector with scalars of each type */
rc = st.setString('AllScalarsVector', 0, strVal);
if (rc ne 0) then return;
rc = st.setInt('AllScalarsVector', 1, intVal);
if (rc ne 0) then return;
rc = st.setLong('AllScalarsVector', 2, longVal);
if (rc ne 0) then return;
rc = st.setDouble('AllScalarsVector', 3, dblVal);
if (rc ne 0) then return;
rc = st.setReference('AllScalarsVector', 4, refAddr, refLen);
if (rc ne 0) then return;
end;
```

Array Setters Example

```
method testArraySetters(vvarchar(32) strVal[3],
                       int intVal[3],
                       bigint longVal[3],
                       double dblVal[3],
                       in_out int rc);

rc = -1;

/* Populate the vector with arrays of each type */
rc = st.setStringArray('AllArraysVector', 0, strVal);
if (rc ne 0) then return;
rc = st.setIntArray('AllArraysVector', 1, intVal);
if (rc ne 0) then return;
rc = st.setLongArray('AllArraysVector', 2, longVal);
if (rc ne 0) then return;
rc = st.setDoubleArray('AllArraysVector', 3, dblVal);
if (rc ne 0) then return;
end;
```

Scalar Getters Example

```
method testScalarGetters(in_out varchar strVal,
                        in_out int intVal,
```



```

        in_out bigint longVal,
        in_out double dblVal,
        in_out varbinary refAddr,
        in_out int rc);

/* Retrieve scalars of each type from the vector */
strVal = st.getString('AllScalarsVector', 0);
if (missing(strVal)) then do;
    rc = -1;
    return;
end;
intVal = st.getInt('AllScalarsVector', 1);
if (missing(intVal)) then do;
    rc = -1;
    return;
end;
longVal = st.getLong('AllScalarsVector', 2);
if (missing(longVal)) then do;
    rc = -1;
    return;
end;
dblVal = st.getDouble('AllScalarsVector', 3);
if (missing(dblVal)) then do;
    rc = -1;
    return;
end;
refAddr = st.getReference('AllScalarsVector', 4);
end;

```

Note that the reference value is returned as a DS2 BINARY type, as indicated in [“Scalar Setters Example” on page 206](#).

Array Getters Example

```

method testArrayGetters(in_out varchar strVal[3],
                        in_out int intVal[3],
                        in_out bigint longVal[3],
                        in_out double dblVal[3],
                        in_out int rc);

/* Retrieve arrays of each type from the vector */
st.getStringArray('AllArraysVector', 0, strVal, rc);
if (rc ne 0) then return;
st.getIntArray('AllArraysVector', 1, intVal, rc);
if (rc ne 0) then return;
st.getLongArray('AllArraysVector', 2, longVal, rc);
if (rc ne 0) then return;
st.getDoubleArray('AllArraysVector', 3, dblVal, rc);
end;

```

Shared Hash Tables

Overview

SAS Micro Analytic Service shared hash tables enable high-performance sharing of in-memory stateful data between modules and across transactions. Shared hash tables consist of key/value pairs, where the keys are strings and the values are state vectors. For more information about state vectors, see the previous section [“Shared Vectors”](#).

Here are some key points about shared hash tables:

- State vectors with different sizes can reside within the same shared hash table.
- Shared hash tables are visible to all modules within the same user context.
- Up to eight hash tables can exist per user context, and each hash table can contain up to 2,147,483,659 state vectors. Each state vector can contain any number of data elements.
- You can initialize a shared state vector from a DS2 package method, including the `init()` and constructor methods.

About Using Shared Hash Tables in DS2

The `MASState` package contains all the methods that are required for DS2 modules to share data across SAS Micro Analytic Service modules and transaction boundaries. These methods include operations on local state vectors and on shared hash tables.

Data can be shared among modules when you do either of the following:

- call methods that create a local state vector, populating it with values, and then putting it in a shared hash table.
- call methods that get an existing vector from a shared hash table (which makes a local copy), modifying its contents, and then replacing the vector in the hash table.

Shared hash tables are accessible by all DS2 modules within a user context.

When you create a new local state vector, you assign it a name. The name must be unique within the hash table in which the vector will be stored. This name is used as follows:

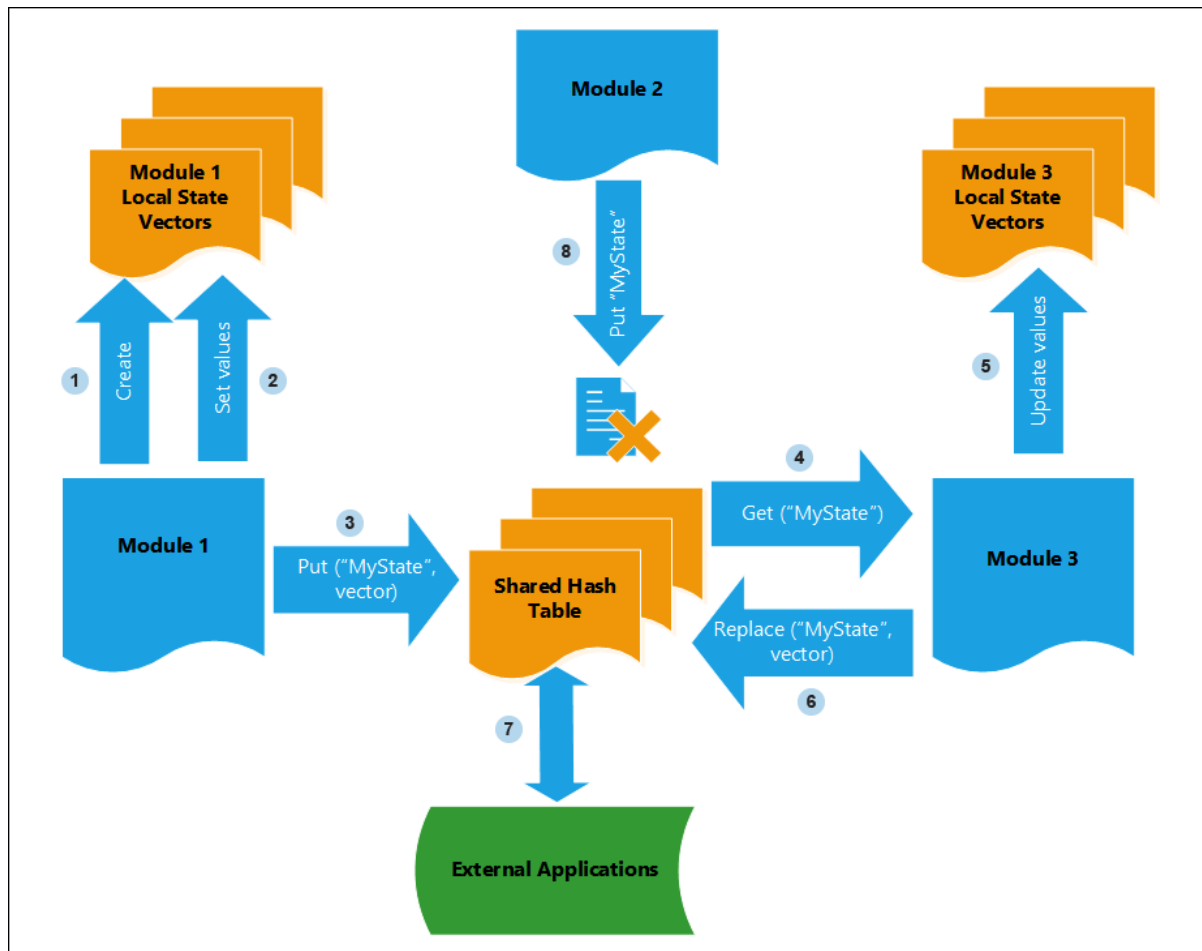
- as a key when subsequently storing the vector in a shared hash table.

That is, the name is used internally as input to a hashing algorithm that quickly computes the hash table location where the vector will be stored.

- when deleting the state vector.
- when storing or retrieving state vector data values.
- when retrieving the vector from a shared hash table.
- when replacing the vector within a shared hash table.

Up to eight shared hash tables can be defined per user context. Hash tables are referenced by index numbers zero through seven, where index zero refers to the default hash table. The default hash table is created automatically when a new user context is created. It is operated on by convenience methods that omit the table index argument. The convenience methods are `clear()`, `isEmpty()`, `size()`, `containsKey()`, `put()`, `get()`, `replace()`, and `remove()`. They are described in [“Methods That Operate on the Default Shared Hash Table”](#) on page 210.

Figure 18.2 *The Shared Hash Table Process*



- 1 Module 1 creates a local state vector.
- 2 Module 1 sets the values for the local state vector.
- 3 Module 1 puts these values, contained in the MyState vector, into a shared hash table.

- 4 Module 3 gets the MyState vector.
- 5 Module 3 updates the values in its local state vector.
- 6 Module 3 replaces the MyState state vector in the shared hash table.
- 7 External applications access the shared hash table to retrieve and replace the MyState state vector.
- 8 Module 2 attempts to store a state vector called MyState in the shared hash table. This is rejected because a state vector with that name already exists in the table.

Methods That Operate on the Default Shared Hash Table

Method Signature	Description
<code>int clear()</code>	Removes all state vectors from the default hash table. Returns zero if successful, and nonzero otherwise.
<code>int isEmpty()</code>	Returns 1 if the default hash table contains no state vectors, and zero otherwise.
<code>bigint size()</code>	Returns the number of state vectors currently in the default hash table.
<code>int containsKey(key)</code>	Returns 1 if the default hash table contains a state vector with a name matching key, and zero otherwise.
<code>int put(key)</code>	Inserts the state vector with the name indicated by the key into the default shared hash table. Returns zero if successful. Nonzero result codes are returned if a duplicate key exists in the default hash table or if a local state vector with a name matching key does not exist.
<code>int get(key)</code>	Finds a state vector in the default shared hash table with a name matching key. If found, a local copy of the state vector is made, and a zero result code is returned. If not found, a nonzero result code is returned. Note: If a local state vector with a name matching key already exists, and a state vector matching the key is found in the default hash table, then the existing local state vector is overwritten with the data values that are retrieved from the default shared hash table.

Method Signature	Description
<code>int replace(key)</code>	Finds a state vector in the default shared hash table with a name matching key. If found, the state vector in the default hash table is replaced with a copy of the corresponding local state vector and a zero result code is returned. Nonzero result codes are returned if the key is not found in the default hash table, or if a local state vector with a name matching key does not exist.
<code>int Remove(key)</code>	Finds a state vector in the default shared hash table with a name matching key. If found, removes it and returns a zero result code. A nonzero result code is returned if the key does not exist in the default hash table.

Default Shared Hash Table Example

In the following example, method `createAndPutVector()` inserts a new state vector containing two integer values into the default shared hash table. Method `incrementSharedValue()` retrieves a state vector, named `MyVector`, from the default shared hash table, making a local copy. It increments the integer data value within the vector and then replaces the `MyVector` state vector in the default shared hash table.

```

ds2_options sas;
package statepkgtest/overwrite=yes;
dcl package masstate st();

method createAndPutVector(vvarchar(32) key, in_out int rc);
  rc = st.createVector(key, 2);
  rc = st.setInt(key, 0, 100);
  if (rc ne 0) then return;
  rc = st.setInt(key, 1, 200);
  if (rc ne 0) then return;
  rc = st.put(key);
  rc = st.deleteVector(key);
end;

method incrementSharedValue(in_out int rc, in_out int int0Val);
  rc = st.get('MyVector');
  if (rc eq 0) then do;
    int0Val = st.getInt('MyVector', 0);
    int0Val = int0Val + 1;
    rc = st.setInt('MyVector', 0, int0Val);
    rc = st.replace('MyVector');
  end;
end;

```

```
endpackage;
```

Methods That Operate on Non-default Shared Hash Tables

Note: For the methods in the table, the following arguments apply:

- **tableIndex** indicates the hash table (0-7) on which to operate.
- **key** is a string value that uniquely identifies a vector within the hash table.

Method Signature	Description
<code>int hashTblCreate(tableIndex)</code>	Creates a new empty hash table, which can be referenced by the given table index. Returns zero if successful, and nonzero otherwise.
<code>int hashTblDestroy(tableIndex)</code>	Removes all state vectors from the indicated hash table, and then deletes the table. Returns zero if successful, and nonzero otherwise.
<code>int hashTblClear(tableIndex)</code>	Removes all state vectors from the indicated hash table. Returns zero if successful, and nonzero otherwise.
<code>int hashTblIsEmpty(tableIndex)</code>	Returns 1 if the indicated hash table contains no state vectors, and zero otherwise.
<code>bigint hashTblSize(tableIndex)</code>	Returns the number of state vectors currently in the indicated hash table.
<code>int hashTblContainsKey(tableIndex, key)</code>	Returns 1 if the indicated hash table contains a state vector with a name matching key, and zero otherwise.
<code>int hashTblPut(tableIndex, key)</code>	Inserts the state vector into the indicated hash table at the position indicated by key. Returns zero if successful. Nonzero result codes are returned if a duplicate key already exists in the indicated hash table, or if a local state vector with a name matching key does not exist.

Method Signature	Description
<code>int hashTblGet(tableIndex, key)</code>	Finds a state vector in the indicated hash table with a name matching key. If found, a local copy of the state vector is made, and a zero result code is returned. If not found, a nonzero result code is returned. Note: If a local state vector with a name matching key already exists, and a state vector matching the key is found in the indicated hash table, then the existing local state vector is overwritten with the data values that are retrieved from the hash table.
<code>int hashTblReplace(tableIndex, key)</code>	Finds a state vector in the indicated hash table with a name matching key. If found, the state vector in the indicated hash table is replaced with a copy of the corresponding local state vector and a zero result code is returned. Nonzero result codes are returned if the key is not found in the hash table, or if a local state vector with a name matching key does not exist.
<code>int hashTblRemove(tableIndex, key)</code>	Finds a state vector in the indicated hash table with a name matching key and, if found, removes it and returns a zero result code. A nonzero result code is returned if the key does not exist in the hash table.

Best Practices for DS2 Programming in SAS Intelligent Decisioning

<i>Overview</i>	215
<i>Return Results</i>	216
<i>Global Packages versus Local Packages</i>	216
Overview	216
Example of Optimized Code	216
Example of Poorly Optimized Code	217
<i>Replacing SCAN (and TRANWRD) with DS2 Code</i>	217
<i>Hash Package</i>	220
<i>Character-to-Numeric Conversions</i>	220
<i>Passing Character Values to Methods</i>	220
<i>Performing the Computation Once</i>	221
<i>Moving Invariant Computations Out of Loops</i>	221

Overview

This section describes best practices that are recommended when programming in DS2 for any environment. They are not unique to SAS Micro Analytic Service.

Return Results

If a DS2 method, or any method it calls, can result in a status code or failure, always include a method output argument for returning the result to the caller.

Global Packages versus Local Packages

Overview

The scope of a package instance makes a difference. Package instances that are created in the global scope typically are created and deleted (allocated and freed) once and used over and over again. Package instances that are created in a local scope are created and deleted each time the scope is entered and exited. For example, a package instance that is created in a method's scope is created and deleted each time a method is called. The creation and deletion time can be costly for some packages.

The following examples use the hash package. This technique can be used for all packages.

Example of Optimized Code

This example creates a hash package instance that is global, created and deleted with the package instance, and reused between calls to `load_and_clear`.

```
/** FAST **/  
package mypack;  
  dcl double k d;  
  dcl package hash h([k], [d]);  
  
  method load_and_clear();  
    dcl double i;  
    do k = 1 to 100;  
      d = 2*k;  
      h.add();  
    end;  
    h.clear();  
  end;  
endpackage;
```

Example of Poorly Optimized Code

This example creates a hash package instance that is local to the method and created and deleted for each call to `load_and_clear`.

```
/** SLOW **/
package mypack;
  dcl double k d;

  method load_and_clear();
    dcl package hash h([k], [d]);
    dcl double i;
    do k = 1 to 100;
      d = 2*k;
      h.add();
    end;
    h.clear();
  end;
endpackage;
```

Replacing SCAN (and TRANWRD) with DS2 Code

Consider the following code:

```
i = 1;
onerow = TRANWRD(SCAN(full_table, i, '|'), ';;', ';-;');
do while (onerow ~= '');
  j = 1;
  elt = scan(onerow, j, ';');
  do while (elt ~= '');
    * processing of each element in the row;
    j = j+1;
    elt = SCAN(onerow, j, ';');
  end;
  i = i+1;
  onerow = TRANWRD(SCAN(full_table, i, '|'), ';;', ';-;');
end;
```

You can make the following observations:

- SCAN consumes adjacent delimiters. Therefore, TRANWRD is required to manipulate each row into a form that can be traversed element by element.
- SCAN starts at the front of the string each time. Therefore, the aggregate cost is $O(N^2)$.

- SCAN and TRANWRD require NCHAR or NVARCHAR input. If full_table is declared as a CHAR or VARCHAR input, it must be converted to NVARCHAR, then processed, and then converted back to VARCHAR in order to be captured into the onerow value.

Here is code that replaces this type of loop with a native DS2 solution and that thus avoids these problems by collecting the necessary details into a package:

```
dcl package STRTOK row_iter();
dcl package STRTOK col_iter();
row_iter.load(full_table, '|');
do while (row_iter.hasmore());
  row_iter.getnext(onerow);
  col_iter.load(onerow, ';');
  do while (col_iter.hasmore());
    col_iter.getnext(elt)
    * processing of each element;
  end;
end;
```

The supporting package, STRTOK, is shown below. It can be used to replace SCAN and TRANWRD pairs anywhere in DS2.

```
/** STRTOK package - extract subsequent tokens from a string.
 * So named because it mirrors (in a safe way) what is done by the
original
 * strtok(1) function available in C.
 */
package sasuser.strtok/overwrite=yes;
dcl varchar(32767) _buffer;
dcl int strt blen;
dcl char(1) _delim;

/* Loads the current object with the supplied buffer and delimiter
 * information. This avoids the cost of constructing and destructing
the
 * object, and allows the declaration of a STRTOK outside of the loop
 * in which it is used.
 */
method load(in_out varchar bufinit, char(1) delim);
  _buffer = bufinit .. delim;
  _delim = delim;
  strt = 1;
  blen = length(_buffer);
end;

/* Are there more fields? 1 means there are more fields. 0 means
there are
 * no more fields.
 */
method hasmore() returns integer;
  if (strt >= blen) then return 0;
  return 1;
end;

/* The void-returning GETNEXT method places the next token in the
supplied
 * variable, tok.
```

```

*/
method getnext(in_out varchar tok);
  dcl char(1) c;
  dcl int e;
  tok = '';
  if (hasmore()) then do;
    e = strt;
    c = substr(_buffer,e,1);
    do while (c ~= _delim);
      tok = tok .. c;
      e = e + 1;
      c = substr(_buffer,e,1);
    end;
    strt = e + 1;
  end;
end;

/* The value-returning GETNEXT method returns the next token. This
version is
  * more computationally expensive because it requires an extra copy,
as
  * opposed to the void-returning version, above.
*/
method getnext() returns varchar(32767);
  dcl varchar(32767) tok;
  getnext(tok);
  return tok;
end;

/* Construct a STRTOK object using the parameters as initial values.
*/
method strtok(vvarchar(32766) bufinit, char(1) delim);
  load(bufinit, delim);
end;

/* Construct a STRTOK object without an initial buffer to be
consumed.
*/
method strtok();
  strt = 0; blen = 0;
end;
endpackage; run;

```

Using STRTOK instead of SCAN and TRANWRD avoids the CHAR to NCHAR conversions and reduces the CPU load due to how STRTOK retains the intermediate state between calls to the getnext() methods. Therefore, it is $O(N)$ instead of $O(N^2)$.

Hash Package

With both the DATA step and DS2, note the size of the key. A recent program carried out many hash lookups with a 356-byte key. Hashing is an $O(1)$ algorithm; the "1" with the hash package is the length of the key. The longer the key, the longer the hash function takes to operate.

```
dcl char(200) k1 k2;
dcl double d1 d2;

/* If k1 and k2 are always smaller than 200, then */
/* size them smaller to reduce the time spent in */
/* the hash function when adding and finding values */
/* in the hash package. */
dcl package hash([k1 k2], [d1 d2]);
```

Character-to-Numeric Conversions

When converting a string to a numeric value, note the encoding of the string. When the string is a single-byte encoding, DS2 translates the value to a TKChar (UCS-2 or UCS-4) for conversion. The longer the string, the longer the time it takes to do the conversion.

```
dcl char(512) s;
dcl nchar(512) ns;
dcl double x;
s = '12.345';
ns = '12.345';

x = s; /* slow */
x = substr(s,1,16); /* faster */
x = substr(ns,1,16); /* even faster, avoids transcoding */
```

Passing Character Values to Methods

In SAS Micro Analytic Service, DS2 method input parameters are passed by value. What this means is that a copy of the value is passed to the method. When passing character parameters, a copy of the parameter is made to ensure that the original

value is not modified. Making sure that character data is sized appropriately ensures that less copying occurs.

DS2 method output parameters, which are specified by the `in_out` keyword, are passed by reference. Therefore, no copy is made.

```
method copy_made (varchar(256) x);
...
end;

method no_copy (in_out varchar x);
...
end;
```

Performing the Computation Once

If a computation is repeated multiple times to compute the same value, you can perform the computation once and save the computed value. For example, the following code block performs the computation, `compute(x)`, four times:

```
if compute(x) > computed_max then computed_max = compute(x);
if compute(x) < computed_min then computed_min = compute(x);
```

If `compute(x)` always computes the same value for a given value of `x`, then the code block can be modified to perform the computation once and save the computed value:

```
computed_x = compute(x);
if computed_x > computed_max then computed_max = computed_x;
if computed_x < computed_min then computed_min = computed_x;
```

Moving Invariant Computations Out of Loops

If a computation inside a loop computes the same value for each iteration, improve performance by moving the computation outside the loop. Compute the value once before the loop begins and use the computed value in the loop. For example, in the following code block, `compute(x)` is evaluated during each iteration of the DO loop:

```
do i = 1 to dim(a);
  if (compute(x) eq a[i]) then ...;
end;
```

If `compute(x)` is invariant (meaning that it always computes the same value for each iteration of the loop), then the code block can be modified to perform the computation once outside the loop:

```
computed_x = compute(x);  
do i = 1 to dim(a);  
    if (computed_x eq a[i]) then ...;  
end;
```


Python Support in SAS Micro Analytic Service

Introduction	223
About Creating Python Modules	225
Public and Private Methods	226
Overview	226
About Private Methods	226
About Public Methods	226
Return Values	227
Examples: Public and Private Methods	228
Working with Python and SAS Micro Analytic Service	229
Security Considerations for Python Modules	231
Configuring Python for SAS Micro Analytic Service	232
About Using Python with SAS Micro Analytic Service	232
Environment Variables	233
CAS Authentication	233
Enabling Python Code to Run on a Locked Down SAS Compute Server	234
Connecting to Databricks from SAS Micro Analytic Service Python Code	234

Introduction

SAS Micro Analytic Service supports modules that are written in the Python programming language. A Python module represents a group of related Python functions.

Input arguments are given in the function's argument list. The objects, variables, and expressions listed in a Python function's return statement are positional with respect to the output variables.

The output variables are listed in the function's "Output:" docstring that is specified in the first statement of the function. Any method that includes the "Output:"

docstring is considered a public method. Otherwise, it is considered a private method. For more information, see the sections later in this chapter.

Input and output argument names live in a single namespace and therefore cannot be the same. This means that update arguments are not supported. This is true for all module types in SAS Micro Analytic Service, even though the Python language does not enforce such a restriction.

Here is an example of a Python public function that can be hosted by SAS Micro Analytic Service.

```
import sys
import math
import pandas as pd
import numpy as np
def nppd(a):
    "Output: ser1"
    npa = np.array([[1,2,3],[4,5,6]])
    ser1 = pd.Series([212, a, -273])
    return ser1.tolist()

def trucks(Eng_Load, Oil_Temp, Eng_RPM):
    "Output: ser1, x, syspath"
    inputs = pd.Series([Eng_Load, Oil_Temp, Eng_RPM])
    b = np.arange(100)
    number = 0
    for index, item in enumerate(inputs):
        number += item + b[index + 7]
    # is it even or odd?
    x = math.fmod(number, 2)
    return nppd(Oil_Temp), x, getsyspath()

def getsyspath():
    "Output: p"
    p = [None] * 50
    # print(sys.path)
    syspaths = sys.path
    i = 0
    for path in syspaths:
        p[i] = path
        i = i + 1
    return p
```

Here is an example of a Python public function that has input arguments a and b, and no output.

```
def calcATimesB(a, b):
    "Output: "
    print ("Function with no output variables.")
    c = a * b
    print ("Result is: ", c, ", but is not returned")
    return None
```

After Python is configured, see [Appendix 2, “Executing Python Modules in DS2 Modules,” on page 303](#) for more information.

About Creating Python Modules

There are two ways that you can create Python modules in SAS Micro Analytic Service:

- Create a DS2 module that uses the PyMAS extension to publish the module.

This is useful when you intend to use a Python module from other DS2 modules. For information, see [Appendix 2, “Executing Python Modules in DS2 Modules,” on page 303](#).

- Directly create a Python module using the REST interface without using any DS2 code.

In this case, the `type` field of the module definition JSON is set to `text/x-python` and the source contains the Python code. This is useful when you need SAS Micro Analytic Service to execute only Python code.

See the [REST API documentation](#) for details about how to create the module definition JSON representation.

Both approaches support the same data types and require that the Python code comply with the same rules and restrictions. Because the Python execution framework handles both approaches, it is possible to have a collection of modules that use a mix of each.

Note the following information about the two Python approaches:

- Here are key differences that you should be aware of:
 - When you use the PyMAS package, the Python code is compiled during the execution of the module. It is important to check any error codes that are generated from the publish call before you execute the code.
 - When you use Python modules directly, the Python code is compiled (using the SAS Micro Analytic Service REST service) when the module is created or updated.
- Python modules are stand-alone—submodules are not supported and inter-module dependencies are not handled.
- Python modules can use other Python modules that have been published to SAS Micro Analytic Service. A DS2 PyMAS package instance can use the `useModule` method to specify a previously published module instead of publishing a new revision of the Python module.

Public and Private Methods

Overview

SAS Micro Analytic Service enables the use of hosting public and private methods, where a method is a Python function. Note that public and private methods are SAS Micro Analytic Service concepts, and are not Python features.

In general, any method that includes the "Output:" docstring is considered a public method. If a method does not have the "Output:" docstring, then it is considered a private method. However, there are syntax requirements that must be followed for the docstring and the output arguments. For more information, see ["About Public Methods" on page 226](#).

Python modules can be published containing all public methods, or a mixture of public and private methods.

Both public and private methods can call other functions that either exist within the module internally or in external Python packages, including third-party libraries.

SAS Micro Analytic Service supports the use of Python decorators in Python modules. If you use decorators, be aware that the original function that includes decorators requires a docstring with the "Output: <vars>" format.

About Private Methods

Here are details about using a private method:

- A private method can be called internally by other methods (either public or private).
- A private method cannot be called directly (externally).
- Private methods are useful when used as utility functions within a package.

About Public Methods

Here are details about using a public method:

- When there is more than one output argument, the output argument names must be comma separated.

- Line two of the function must begin with a docstring, and the first non-whitespace token must be "Output:".
- All public functions that return more than one output argument must return a tuple containing all of the output arguments.
This can be done by returning all of the arguments separated by commas.
- When returning zero arguments from a public function you are still required to include the "Output:" docstring to indicate a public function. It should simply be "Output:", with no output arguments listed.
- Order does matter. Therefore, the order in the return statement must match the order in the "Output:" line. A best practice is to copy and paste from one to the other.

Return Values

When Python integers are returned, they are converted to `int64_t`. If the Python integer is too large for the `int64_t` type, a run-time error is reported.

Although Python packages might support abstract types that provide an additional layer on top of the built-in scalar types, such as integer and float, they cannot be returned from SAS Micro Analytic Service public functions. Packages that have these abstract types provide functions that can be used to extract the built-in type. Here is an example:

```
return numpy.float(x), numpy.int(y)
```

Note the following information about the return statement for Python functions:

- Functions that return nothing do not require a return statement. Python returns the `NoneType` object. Therefore, an empty return statement (`return`) and returning `None` (`return None`) are equivalent.
- Functions that return a single argument can return `None`. SAS Micro Analytic Service maps the `None` value to the specific Missing value for the expected static type.
- Functions that return multiple values can be formatted with or without parentheses (for example, `return a,b,c` or `return (a,b,c)`).

Note: An attempt to return `None` from a function that expects multiple values is invalid.

When return values are transferred between Python and SAS Micro Analytic Service, the following rules are enforced by SAS Micro Analytic Service:

- A singleton value (non-tuple) is mapped to a tuple with a single value.
- Tuples map to multiple return arguments.
- Arrays and tuples cannot be used interchangeably. The Python tuple type is used to return multiple values. The Python list type is used for SAS Micro Analytic Service arrays.

- The argument count must match.

The following code sample and table show some examples that illustrate the application of these rules.

```
def one(a):
    "output: b"
    return oneVal

def two(a,b)
    "output: a,b"
    return twoVal
```

Python Value Assignment	SAS Micro Analytic Service Value Assignment
oneVal = 1	oneVal = (1) (one integer value)
oneVal = 1,	oneVal = (1) (one integer value)
oneVal = (1,)	oneVal = (1) (one integer value)
oneVal = (1, 2)	An error occurs. It indicates that the expected output variables do not match the actual output variables.
twoVal = 1,2	twoVal = (1,2) (two integer values)
twoVal = (1,2),	An error occurs. It indicates that an unknown tuple type is present.
twoVal = ([1],[2])	twoVal = ([1],[2]) (two integer arrays)

Examples: Public and Private Methods

As mentioned in [“Overview”](#), for a method to be public, the output variables must be listed in the function’s “Output:” docstring that is specified on the first statement of the function. This is the second line of the method, immediately following the “def” line.

Here are some examples. Note that the fun2 function would be considered private because the docstring does not begin with “Output:”.

```
def fun1( a, b ):
    "Output:"
```

```

    """ This is a public function,
        but has no output args."""
    return

def fun2( a, b ):
    """This will be private since the docstring doesn't begin with
Output:
    Output: x, y, z"""
    return a+2, b*4, a/b

def fun3( a, b ):
    '    Output: x, y, z'
    ''' multi
        Line
        doc string'''
    return a+2, b*4, a/b

def fun4( a, b ):
    ''' Output: x, y, z'''
    """ multi
        Line
        doc string"""
    return a+2, b*4, a/b

def fun5( a, b ):
    '''Output: q, r, s,
        t, u,
        v,
        w'''
    return a+2, b*4, a/b

```

Working with Python and SAS Micro Analytic Service

When you work with Python and SAS Micro Analytic Service, note the following:

- When sharing modules across other modules, you should understand the following logic:
If module A and module B each import module C, updates to module C affect both module A and module B.
- All client requests are passed to SAS Micro Analytic Service with an OAuth access token. This token is not passed to the Python subprocess. Therefore, Python code cannot connect to a SAS service that requires an access token.

Note: This is a SAS Micro Analytic Service limitation on Python modules.

- If you execute the CALL method in SAS Cloud Analytic Services (CAS), your Python function must be stateless and not depend on data from a previous Python function call.
- SAS Micro Analytic Service Python interface usage is prohibited when any of the following situations exist:
 - The application enabled MASHostAccessDeny when initializing SAS Micro Analytic Service.
 - The MASHostAccess=DENY environment variable is set.
 - The application provides MASHostOSID credentials that fail to authenticate.
 - The CAS session is not running under a host operating system identity and it attempts to run an action that uses SAS Micro Analytic Service.
- Maintaining multiple revisions of a Python module is not supported. A subsequent publish request on the same Python module context replaces the existing revision. This means that the revision number always remains at 1.

For example, consider the following two Python modules:

test.py:

```
def execute(a,b):
    "Output: c"
    c = (a+b)
    return c
```

```
def score(x):
    "Output: y"
    return x * 0.5;
```

test2.py:

```
def execute(a,b,c):
    "Output: d"
    d = (a+b+c)
    return d;
```

If you publish test.py and then publish test2.py to the same module context, the first execute function is replaced by the second one. The new execute function has an additional input argument (c) and a different output argument (d). Also, the new revision no longer includes the score function.

- Source code encoding, if provided, must be UTF-8.
- SAS Micro Analytic Service applications can specify a host operating system account to use as the owner of Python subprocesses that are launched when using SAS Micro Analytic Service modules. For more information, see the next section, [“Security Considerations for Python Modules”](#).

Security Considerations for Python Modules

A Python module is compiled and run in its own process. For security reasons, you might need to limit the access of Python processes to the file system or other resources of the host server. You can accomplish this by using the SAS Viya platform external credentials functionality. The account associated with the credentials that you specify should have a scope that is valid for running Python processes.

It is important to understand that the account that you configure to run Python processes is not a SAS Viya platform account. Rather, it is a host server account that you specify using the external credentials functionality in SAS Environment Manager. For more information, see [“External Credentials: How To” in SAS Viya Platform: External Credentials](#).

For SAS Micro Analytic Service to launch Python processes using this account, you use SAS Environment Manager to configure the following properties:

- **core.mashostuser** : specifies the user ID that is associated with the account
- **core.mashostdomain** : specifies the Authentication domain that is associated with the account

You must re-start SAS Micro Analytic Service to enable the use of the specified credentials.

For more information about configuring these properties, see [“sas.microanalyticsservice.system: supplementalProperties” on page 276](#).

If you do not configure these properties, Python processes are run under the account of the SAS Micro Analytic Service process.

Configuring Python for SAS Micro Analytic Service

About Using Python with SAS Micro Analytic Service

SAS Micro Analytic Service supports Python 3.x (note that Python 3.9 or later is recommended). Though SAS Micro Analytic Service does not require a specific version of Python, it is possible that Python code that you publish to SAS Micro Analytic Service might have dependencies to a specific version of Python or Python packages.

These are the commonly used packages:

graphviz	matplotlib
image	pandas
opencv-python	requests
plotly	sas-dlpy
sas-esppy	sasoptpy
scikit-learn	swat
tensorflow	ws4py
html5lib	

Note: Installed packages affect the storage footprint. You should install only the packages that you require.

Configuring Python with the SAS Viya platform is part of your deployment tasks. For more information, see [“Configuring SAS Micro Analytic Service to Use a Python Distribution” on page 23](#), as well as the [SAS Viya Platform: Deployment Guide](#).

Environment Variables

The following SAS Micro Analytic Service environment variables are set as part of the deployment process when configuring Python for the SAS Viya platform:

Environment Variable	Description
MAS_M2PATH	Specifies the absolute path to the mas2py.py file that is included with SAS Micro Analytic Service. It is used to execute Python code within a Python process that is launched by SAS Micro Analytic Service. This environment variable must be set.
MAS_PYPATH	Specifies the absolute path to the Python executable file. This environment variable must be set.
MAS_PYWAIT	Specifies the amount of time, in milliseconds, that SAS Micro Analytic Service waits for the spawned Python process to reconnect to the SAS Micro Analytic Service server after launching the Python client. The default value, 30000, is equal to 30 seconds. To change the wait time, use the core.maspywaitmilliseconds property in SAS Environment Manager. For more information, see “sas.microanalyticsservice.system: supplementalProperties” on page 276 .

Note: For security reasons, you cannot access the value of an environment variable from Python code that includes the string `_token` (using any letter casing) in its name.

CAS Authentication

To run Python, one of the following is required on the CAS Server for user authentication:

- The user who is logged in must be a member of the CASHostAccountRequired group.
- The CASALLHOSTACCOUNTS environment variable must be set.

For more information, see [“Configure External Access to the CAS Server” in SAS Viya Platform: Integration with External Languages](#).

Enabling Python Code to Run on a Locked Down SAS Compute Server

The SAS Micro Analytic Service Python interface usage is prohibited by default when a SAS session is in lockdown mode. For more information about lockdown mode, see [“Lock Down the SAS Compute Server” in SAS Viya Platform: Programming Run-Time Servers](#).

To enable access, you can specify certain arguments to the SAS Viya platform LOCKDOWN statement as follows:

- To enable Python access, specify the ENABLE_AMS argument and set the argument value to *PYTHON*. Here is an example:

```
lockdown enable_ams=PYTHON;
```

- To enable DS2 PyMAS package access, you must also set the *PYTHON_EMBED* argument value. Here is an example:

```
lockdown enable_ams=PYTHON ENABLE_AMS=PYTHON_EMBED;
```

For more information about the LOCKDOWN statement, see [“LOCKDOWN Statement” in SAS Viya Platform: Programming Run-Time Servers](#).

Connecting to Databricks from SAS Micro Analytic Service Python Code

IMPORTANT Before you can connect to Databricks, you must configure SAS Micro Analytic Service to enable access to Databricks. For more information, see the README.md file that is located in the `$deploy/sas-bases/examples/sas-microanalytic-score/databricks` directory where you downloaded your files for deployment.

To use Databricks from Python code in SAS Micro Analytic Service, you need the following Databricks credentials:

- `server_hostname`: Specify your Databricks workspace URL (for example, `your-workspace.cloud.databricks.com`)
- `http_path`: Specify the HTTP path of your SQL warehouse (for example, `/sql/1.0/warehouses/your-warehouse-id`)
- `access_token`: Specify the personal access token for authentication

The following Python code demonstrates how to connect to Databricks from within your SAS Micro Analytic Service Python code. Include this code in your Python step code or Python model that is executed by SAS Micro Analytic Service:

```
from databricks import sql

# Replace these values with your actual Databricks credentials
connection = sql.connect(
    server_hostname="your-workspace.cloud.databricks.com",
    http_path="/sql/1.0/warehouses/your-warehouse-id",
    access_token="your-access-token"
)

cursor = connection.cursor()
cursor.execute("SELECT * FROM your_table LIMIT 10")
result = cursor.fetchall()
cursor.close()
connection.close()
```

Note: Replace the placeholder values (*your-workspace.cloud.databricks.com*, */sql/1.0/warehouses/your-warehouse-id*, *your-access-token*, and *your_table*) with your actual Databricks configuration details.

Module Execution

<i>Executing a Published Decision or Model</i>	237
POST Request URL	237
Request Body	238
Response Body	238
Additional Information	238
<i>Synchronous, Asynchronous, and Timed Execution</i>	239
Synchronous Execution	239
Timed Asynchronous Execution	239
Canceling Execution	240
Time-out Settings	240
<i>Using Transaction Metadata during Step Execution</i>	242
Overview	242
Reserved Metadata String Names	243
Including Code Execution Timestamp Information in JSON Output	244
Using HTTP Headers as Metadata	244
Using the masmetadata Python Module to Retrieve Metadata	245
DS2 Function to Retrieve Metadata	246
Including Metadata in Logging Events	247
Examples	247

Executing a Published Decision or Model

POST Request URL

To execute a published model or decision, you submit a POST request using the following URL:

```
POST https://www.example.com/microanalyticScore/modules/{moduleId}/
steps/{stepId}
```

Request Body

You can format the input data using an array format or an object format.

Here is a sample request body using an array format:

```
{ "inputs": [
  { "name": "age", "value" :30},
  { "name": "job", "value": "Manager" }
]
```

Here is a sample request body using an object format:

```
{ "data":
  { "age":30, "job":"Manager"},
}
```

Response Body

The format of the response body matches the format of the corresponding request body.

Here is a sample response body using an array format:

```
'I_BAD', 'P_BAD0', 'P_BAD1', 'EM_CLASSIFICATION', 'EM_PROBABILITY', 'EM_EVENTPROBABILITY'

{ "outputs": [
  { "name": " EM_CLASSIFICATION", "value" : 1},
  { "name": " EM_PROBABILITY", "value": 0.7},
  { "name": "EM_EVENTPROBABILITY", "value": 0.3}
]
```

Here is a sample response body using an object format:

```
'I_BAD', 'P_BAD0', 'P_BAD1', 'EM_CLASSIFICATION', 'EM_PROBABILITY', 'EM_EVENTPROBABILITY'

{ "data":
  { "EM_CLASSIFICATION":1, "EM_PROBABILITY":0.7, "EM_EVENTPROBABILITY":0.3}
}
```

Additional Information

For more information, including details about parameters, the request body, responses, and examples see [Execute a step](#).

Synchronous, Asynchronous, and Timed Execution

A SAS Micro Analytic Service module contains one or more steps. Each step is executed by passing input parameter values to the server.

Synchronous Execution

By default, the execution of a module is processed synchronously. This means that the client makes a SAS Micro Analytic Service REST call, and the call returns when the execution is complete. Upon successful execution, the **outputs** variable in the reply contains the output of the execution, and the **executionState** variable is assigned the value **completed**.

Timed Asynchronous Execution

Depending on your requirements, it might be beneficial for SAS Micro Analytic Service to execute a module asynchronously. When you use asynchronous execution, the input parameter values are passed to the server and the execution occurs in a separate thread. Any errors in execution are logged on the server. If logging is set at the TRACE level, the result of the execution is also logged.

To use asynchronous execution, when you specify the POST request to execute a module, include the **waitTime** query parameter with an assigned value (in milliseconds). Here is an example of a request that includes the **waitTime** query parameter:

```
POST https://www.example.com/microanalyticScore/modules/{moduleId}/steps/{stepId}?  
waitTime=6000
```

When you include the **waitTime** query parameter, the **executionState** variable with an assigned value is included in the reply. Here are the possible values:

- **completed**: the execution completed within the specified **waitTime** value. The result is returned.
- **timedOut**: the execution did not complete within the specified **waitTime** value. The **outputs** variable in the reply is empty.
- **submitted**: the assigned **waitTime** query parameter value is 0 (zero). The **outputs** variable in the reply is empty.

TIP Assigning `waitTime=0` is useful when a call does not generate output (for example, when executing a step that fires an event).

Note: When an execution times out, the control returns to the caller, but this does not cancel the underlying execution. For information about canceling execution, see the next section, “Canceling Execution”.

Note: It is possible for a SAS Micro Analytic Service request to fail when it uses asynchronous execution that calls back to another SAS Viya service in the same deployment.

Canceling Execution

When all the SAS Micro Analytic Service threads are executing long running requests (which might be caused by a network or system issue), SAS Micro Analytic Service is unable to process new requests. To resolve this type of situation, SAS Micro Analytic Service attempts to cancel a thread that has not completed within 30 seconds.

The amount of time that a thread can run before a cancel request is sent is managed using the `executiontimeout` property in SAS Environment Manager. For more information about changing the default value of 30 seconds, see [“sas.microanalyticsservice.system: core” on page 272](#).

Note that SAS Micro Analytic Service sends a request to stop the execution. The executing decision must process that request in order to cancel the request.

If the `waitTime` query parameter is set for an asynchronous execution request, here is how that value and the `cancelttime` values are managed:

- If the `waitTime` value is less than the `executiontimeout` value, `waitTime` overrides `executiontimeout`.
- If the `waitTime` value is greater than the `executiontimeout` value, `executiontimeout` is honored.

Time-out Settings

There are multiple configuration settings that manage time-out values for SAS Micro Analytic Service functionality. They are listed in the following table.

Configuration Option	Description	Additional Information
service.timeouts.maxmodulecompiletimemillis	A property that applies a time-out value when compiling DS2 modules. This applies to both asynchronous and synchronous compilation requests.	■ “Compiling DS2 Modules” on page 191 ■ “sas.microanalyticsservice.system: supplementalProperties” on page 276
service.timeouts.maxloadwaitnonexecmillis	A property that applies a time-out value for methods that create or update synchronous modules. It also applies when a module makes a query request to obtain detailed information from a compiled module.	■ “Creating and Updating Synchronous Modules” on page 192 ■ “Loading Modules On-Demand” on page 192 ■ “sas.microanalyticsservice.system: supplementalProperties” on page 276
service.timeouts.maxloadwaitexecmillis	A property that applies a time-out value when a module requests to execute a module.	■ “Loading Modules On-Demand” on page 192 ■ “sas.microanalyticsservice.system: supplementalProperties” on page 276
executiontimeout	A property that sets a time value after which a request is sent to cancel an execution.	■ “Canceling Execution” on page 240 ■ “sas.microanalyticsservice.system: core” on page 272
waitTime	A query parameter that submits an	“Timed Asynchronous Execution” (p. 239)

Configuration Option	Description	Additional Information
	execution request asynchronously with a time-out value.	

Using Transaction Metadata during Step Execution

Overview

SAS Micro Analytic Service supports the use of transaction metadata. Transaction metadata is used to annotate step execution and does not affect computation. The inclusion of transaction metadata in the input is optional.

To use transaction metadata, the client calls the server with the metadata as part of the JSON payload. In addition, certain HTTP headers can be supplied as metadata. You can use any non-reserved name-value pairs as metadata. For a list of reserved names, see [“Reserved Metadata String Names”](#).

Metadata is available to the DS2 module code via the mdcGetVal function. See [“DS2 Function to Retrieve Metadata”](#) for more information.

Logging events that are generated by transaction execution include these metadata values in the log messages:

Tag Name	Log Property ID
client_id	clientId
module_id	moduleId
step_id	stepId
transaction_id	transactionId

See [“Example: Logging Transaction Execution Metadata”](#) on page 247 for input string, output string, and log samples.

Reserved Metadata String Names

The following metadata string names each have a special purpose and are reserved for use by SAS Micro Analytic Service.

If you specify metadata with a reserved name that is to be supplied by SAS Micro Analytic Service in a request, SAS Micro Analytic Service overwrites your metadata specification with the value that corresponds to the reserved metadata field. For example, SAS Micro Analytic Service always generates a value for timestamp at the end of a transaction.

If you include a reserved metadata string name in a request, a warning message is logged.

Name	Description
client_id	The client identifier that signifies the source of execution request. This tag is included in log messages only if it is specified in the JSON payload.
module_id	The module identifier. This tag is automatically added by the SAS Micro Analytic Service REST interface. It is always included in messages that are logged during the execution of a step. Therefore, do not add this tag to the JSON payload.
step_id	The step identifier. This tag is automatically added by the SAS Micro Analytic Service REST interface. It is always included in messages that are logged during the execution of a step. Therefore, do not add this tag to the JSON payload.
transaction_id	The transaction identifier. This tag is included in log messages only if it is specified in the JSON payload.
elapsed_nanos	The duration of time (in nanoseconds) that elapsed during the execution of the module code. This value does not include the time that is spent parsing the JSON payload, creating the JSON output, or sending and receiving the payload over HTTP. For more information, see “Including Code Execution Timestamp Information in JSON Output”.
sas_custom_headers	The JSON serialized string of SAS custom headers. It is used to pass headers from the REST API. For more information, see “Using HTTP Headers as Metadata”.
timestamp	An ISO 8601 timestamp value that indicates when the request was received. Note that the timestamp value is recorded after the JSON payload is parsed.

Name	Description
	For more information, see “Including Code Execution Timestamp Information in JSON Output” .

Including Code Execution Timestamp Information in JSON Output

You can use a configuration setting to enable the inclusion of information that estimates the amount of time it took for SAS Micro Analytic Service to execute module code. When enabled, here are the tags that appear in the JSON output:

- **timestamp:** An ISO 8601 timestamp value that indicates when the request was received by SAS Micro Analytic Service.

Note: The timestamp value is recorded after the JSON payload is parsed.

- **elapsed_nanos:** The duration of time (in nanoseconds) that elapsed during the execution of the module code.

This value does not include the time that is spent parsing the JSON payload, creating the JSON output, or sending and receiving the payload over HTTP.

To include these tags in the JSON output, you must add the following system property to the Micro Analytic Score service **jvm** configuration instance in SAS Environment Manager:

Name: `java_option_sas_mas_profile_execution`

Value: `-Dsas.mas.profile.execution=true`

For more information, see [“jvm” on page 264](#). After you add the property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the `kubectl rollout restart` command to restart the pod without an interruption of service. For more information, see [“Applying Configuration Changes” on page 30](#).

Note: This timestamp information is not included in the log output.

Using HTTP Headers as Metadata

SAS Micro Analytic Service can include custom HTTP header values of a REST request as metadata. The header data is retrieved as key-value pairs in JSON format.

To enable this functionality, you must do the following:

- Set the `service.passheadersenabled` property in SAS Environment Manager to **True**. For more information, see [“sas.microanalyticservice.system: supplementalProperties” on page 276](#). After you add the property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the `kubectl rollout restart` command to restart the pod without an interruption of service. For more information, see [“Applying Configuration Changes” on page 30](#).
- Prefix each header field that you want use as metadata with the following string:
sas-custom-

When these conditions exist, the corresponding header data is added as metadata to the request. You can retrieve these values by using the `mdcGetVal` DS2 function. For more information, see [“DS2 Function to Retrieve Metadata”](#).

Note: Do not include header data that could expose sensitive information.

Using the masmetadata Python Module to Retrieve Metadata

To retrieve a metadata value with SAS Micro Analytic Service module code, use the `masmetadata` module `getMetadata` function to retrieve the Python dictionary that contains the metadata.

Note:

- Metadata support requires Python 3.7 or higher.
 - The name argument that accesses the value is case-sensitive.
 - To access HTTP headers via Python, you must apply the requirements that are defined in [“Using HTTP Headers as Metadata”](#).
-

Here is a Python example:

```
import masmetadata
...
metadata = masmetadata.getMetadata()

try:
    value = metadata[name]
except KeyError:
    value = None
```

Here is a DS2 example:

```
package header_echo;
  dcl package pymas py;
  dcl int revision;
```

```

method execute(in_out int rc);
  dcl varchar(32767) headers;
  if null(py) then do;
    py = _new_ pydas();
    rc = py.useModule('headerpymodule_2', 1);
    if rc then do;
      rc = py.appendSrcLine('import logging');
      rc = py.appendSrcLine('import masmetadata');
      rc = py.appendSrcLine('logger =
logging.getLogger("root")');
      rc =
py.appendSrcLine('logger.setLevel(logging.DEBUG)');
      rc = py.appendSrcLine('def echoHeaders():');
      rc = py.appendSrcLine('    \nOutput:\n');
      rc = py.appendSrcLine('    metadata =
masmetadata.getMetadata()');
      rc = py.appendSrcLine('    try:');
      rc = py.appendSrcLine('        name =
"sas_custom_headers"');
      rc = py.appendSrcLine('        headers =
metadata[name]');
      rc = py.appendSrcLine('    except KeyError:');
      rc = py.appendSrcLine('        logger.debug("Key: %s
does not exist", name)');
      rc = py.appendSrcLine('        headers = None');
      rc = py.appendSrcLine('    logger.info(headers)');
      pycode = py.getSource();
      revision = py.publish(pycode, 'headerpymodule_2');
      if revision lt 1 then do;
        rc = -1;
        return;
      end;
    end;
  end;
  rc = py.useMethod('echoHeaders');
  if rc then return;
end;
rc = py.execute();
end;
endpackage;

```

DS2 Function to Retrieve Metadata

To retrieve a metadata value from within SAS Micro Analytic Service DS2 module code, you can use the DS2 `mdcGetVal` function as follows:

```
value=mdcGetVal(name);
```

Note: The `name` input argument to the `mdcGetVal` function is case-sensitive.

This metadata is available to cross-module executions that are performed by using the `MASCall` package `callModule` and `asyncCallModule` methods. For more information, see [“MASCall Package” on page 37](#).

Including Metadata in Logging Events

Logging events that are generated by transaction execution include these metadata values in the log messages.

Tag Name	Log Property ID
client_id	clientId
module_id	moduleId
step_id	stepId
transaction_id	transactionId

For more information about logging, see [“SAS Micro Analytic Service Logging” on page 284](#).

Examples

Example: Logging Transaction Execution Metadata

Here is a sample DS2 module:

```
package vmasd25;
dcl package logger logr('App.Test');
method m1( varchar(256) vc1, varchar(256) vc2, in_out varchar vc3,
in_out varchar vc4 );
vc3 = MDCGetVal( vc1 );
vc4 = MDCGetVal( vc2 );
logr.log( 'd', 'vmasd25, m1, vc1=$s, vc3=$s, vc2=$s, vc4=$s', vc1,
vc3, vc2, vc4 );
end;
endpackage;
```

Here is a corresponding sample JSON input string:

```
{
  "inputs": [
    {
      "name": "vc1",
      "value": "client_id"
    },
    {
      "name": "vc2",
```

```

        "value": "transaction_id"
    }
],
"metadata": {
    "client_id": "client1w23444",
    "transaction_id": "12345555"
}
}

```

Following execution, this is the corresponding JSON output string:

```

{
  "links" : [ ],
  "version" : 2,
  "moduleId" : "vmasd25",
  "stepId" : "m1",
  "executionState" : "completed",
  "metadata" : {
    "transaction_id" : "12345555",
    "module_id" : "vmasd25",
    "step_id" : "m1",
    "client_id" : "client1w23444",
    "timestamp" : "2021-09-03T14:23:05.110Z",
    "elapsed_nanos" : 1431529
  },
  "outputs" : [
    {
      "name" : "vc3",
      "value" : "client1w23444"
    },
    {
      "name" : "vc4",
      "value" : "12345555"
    }
  ]
}

```

Here is the log output that is generated by SAS Micro Analytic Service:

```

{
  "level": "debug",
  "message": "vmasd25, m1, vc1=client_id, vc3=client1w23444, vc2=transaction_id, vc4=12345555",
  "metadata": {
    "moduleId": "vmasd25",
    "stepId": "m1",
    "transactionId": "12345555",
    "clientId": "client1w23444"
  },
  "properties": {
    "_lineNumber_": "395",
    "_sourceFile_": "d2logger.c",
    "pod": "sas-microanalytic-score-5cc9588cf9-dldxq",
    "logger": "App.tk.test",
    "processId": "55",
    "thread": "00000010"
  },
}

```

```

    "source": "sas-microanalytic-score",
    "timeStamp": "2021-09-03T14:22:06.115Z",
    "version": 1
  }

```

Note: The timestamp that appears in the log output represents the datetime value when the log output was generated. It is not the timestamp value that appears in the JSON output.

Example: Logging Custom HTTP Header Metadata

Here is an example that shows sas-custom HTTP headers. The following shows a POST request with the custom header entries highlighted.

```

POST http://localhost:8080/echo/ HTTP/1.1
Accept-Encoding: gzip, deflate, br
Content-Type: application/json; charset=UTF-8
Authorization: bearer
sas-custom-test-3: 3
sas-custom-test-1: one
sas-custom-test-2: "two"
Content-Length: 15
Host: localhost:8080
Connection: Keep-Alive
User-Agent: Apache-HttpClient/4.5.2 (Java/17.0.5)

{
  "data": {}
}

```

Here is an example of DS2 code that retrieves the sas-custom headers:

```

package header_echo;
  dcl package py as py;
  dcl int revision;
  method execute(in_out int rc);
    dcl varchar(32767) headers;
    if null(py) then do;
      py = _new_ py as py;
      rc = py.useModule('headerpy module', 1);
      if rc then do;
        rc = py.appendSrcLine('import logging');
        rc = py.appendSrcLine('logger =
logging.getLogger(\"root\")');
        rc =
py.appendSrcLine('logger.setLevel(logging.DEBUG)');
        rc = py.appendSrcLine('def echoHeaders(headers):');
        rc = py.appendSrcLine('    \"Output:\");
        rc = py.appendSrcLine('    logger.info(headers)');
        pycode = py.getSource();
        revision = py.publish(pycode, 'headerpy module');
        if revision lt 1 then do;
          rc = -1;

```

```

        return;
    end;
end;
rc = py.useMethod('echoHeaders');
if rc then return;
end;
headers = MDCGetVal('sas_custom_headers');
rc = py.setString('headers', headers);
rc = py.execute();
end;
endpackage;
Collapse

```

Here is the log output, in structured format, that is generated by SAS Micro Analytic Score:

```

2024-05-02T13:47:55,882 [00000008] INFO root    header_echo execute -
{
  "sas-custom-test-3": "3",
  "sas-custom-test-1": "one",
  "sas-custom-test-2": "\"two\""
}

```

Here is the log output, in flat format, that is generated by SAS Micro Analytic Service:

```

2024-05-02T13:47:55,882 [00000008] INFO root    header_echo execute -
{"sas-custom-test-3": "3", "sas-custom-test-1": "one", "sas-custom-
test-2": "\"two\""}

```

Database Access and Redis Access with DS2

<i>Database Connection Configuration Overview</i>	251
<i>Instantiating a SQLSTMT Package</i>	252
<i>Verifying DS2 SQLSTMT Method Return Codes</i>	253
<i>Committing Transactions</i>	254
About Options for Committing Transactions	254
Example	255
<i>About Database Connection Retry Functionality</i>	257
<i>Configuring a Connection to a Redis Server</i>	258
Overview	258
Connect to a Redis Server	259
<i>Configuring an Authentication Domain</i>	260

Database Connection Configuration Overview

SAS Micro Analytic Service includes drivers for the databases that are listed in [Database Driver Reference](#). To configure a connection to any of those database types:

- 1 Enter the connection string for your database in SAS Environment Manager. For information, see the `connectionstring` property in [“sas.microanalyticsservice.properties: core”](#) on page 271.

Note: To configure a connection to a Microsoft Azure SQL database using an authentication domain, you must also configure the SAS Authentication Domain

property, which is located in “[sas.microanalyticservice.properties: core](#)” on page 271.

For information about how to configure an authentication domain, see “[Configuring an Authentication Domain](#)” on page 260.

After installing and configuring the drivers, you must restart the SAS Micro Analytic Service pod to apply the changes. You can use the `kubectl rollout restart` command to restart the pod without an interruption of service. For information, see “[Applying Configuration Changes](#)” on page 30.

Note: Passwords in a connection string can be encoded values that are produced by PROC PWENCODE. The SAS002 (alias SASENC), SAS003, SAS004, and SAS005 encoding methods are supported. For more information, see “[PWENCODE Procedure](#)” in *Base SAS Procedures Guide*.

- 2 Configure transaction options, if necessary. For information, see “[Committing Transactions](#)” on page 254.
- 3 Access the database by instantiating an SQLSTMT package. For information, see “[Instantiating a SQLSTMT Package](#)” on page 252.

Note: If you want to specify the maximum number of connections for SAS Micro Analytic Service or verify whether a SAS Micro Analytic Service connection is valid, see “[sas.microanalyticservice.properties: connectionpool](#)” on page 270 for information about how to configure these items.

IMPORTANT Database connections that are established when SAS Micro Analytic Service starts are intended to be long-lived. To avoid service disruptions, you should configure your database connections to enable keep-alive functionality. This is commonly accomplished by using connection string attributes such as `keepAlive=1` or `KeepConnectionActive=1`. See your database documentation for information about the supported keep-alive options.

Instantiating a SQLSTMT Package

You should not instantiate a DS2 SQLSTMT package when declaring it as a package-scoped member. Instead, you should conditionally instantiate it within the method that is using it. Here is an example:

```
package tSQLStmt;
  dcl package sqlstmt stmt;
  method getid( in_out int rc, in_out varchar custId );
```

```

if null(stmt) then do;
    stmt = _new_sqlstmt();
    rc = stmt.prepare('select customerId from customer');
    if (rc = 1) then do;
        stmt.delete();
        stmt = null;
        return;
    end;
end;
rc = stmt.execute();
if (rc) then do;
    if (rc = 1) then do;
        stmt.delete();
        stmt = null;
        /* other error recovery code if needed. */
    end;
    /* other error recovery code if needed. */
    return;
end;
rc = stmt.fetch();
if (rc) then do;
    if (rc = 1) then do;
        stmt.delete();
        stmt = null;
        /* other error recovery code if needed. */
    end;
    /* other error recovery code if needed. */
    return;
end;
custId = stmt.getvarchar(1);
end;
endpackage;

```

Verifying DS2 SQLSTMT Method Return Codes

As a best practice, you should verify SQLSTMT return codes from methods such as PREPARE, EXECUTE, FETCH, and so on. If a method returns a failure code of 1, then you must re-instantiate the package instance. For more information about SAS Micro Analytic Service database connection properties, see [“sas.microanalyticsservice.properties: connectionpool” on page 270](#).

Committing Transactions

About Options for Committing Transactions

By default, in a DS2 SQLSTMT package, any outstanding changes to the database are committed at the end of each SQL statement execute method. This feature is known as *autocommit*.

However, you can turn off autocommit in SAS Micro Analytic Service when using data sources that support transactions. When autocommit is off, multiple SQL statements can be executed within a single transaction. This means that SAS Micro Analytic Service commits database changes at the end of the public DS2 method execution.

If the DS2 method ends abnormally, SAS Micro Analytic Service rolls back all uncommitted changes. Uncommitted changes can include either of the following:

- all SQL statements since the start of the DS2 method
- all SQL statements since the last COMMIT statement

For more information, see “[COMMIT Statement](#)” in *SAS FedSQL Language Reference* and “[ROLLBACK Statement](#)” in *SAS FedSQL Language Reference*.

Note:

- When autocommit is off, it is not necessary to execute the FedSQL BEGIN statement.
 - The scope of transactions is limited within the public DS2 method that is being executed by SAS Micro Analytic Service.
-

To disable autocommit, the following configuration items are required:

- The underlying database driver must support transactions.

IMPORTANT This might require additional configuration for your database. For example, when you connect to a Microsoft SQL Server database, you must include `ENABLE_MARS=YES` in your connection string. This enables support for Multiple Active Result Sets (MARS). For more information, see “[Disabling Transaction Autocommit](#)” on page 342.

Refer to your database documentation for more information.

- The autocommit property must be set to *False* in SAS Environment Manager. For more information, see “[sas.microanalyticservice.properties](#)” on page 268.

- The number of database connections must be equal to the number of SAS Micro Analytic Service worker threads. This one-to-one mapping ensures that each thread uses the same connection.

To accomplish this, in SAS Environment Manager, confirm that the `maxconnections` property is set to the default value of 0 (zero). This sets the number of database connections equal to the number of SAS Micro Analytic Service worker threads. For more information, see [“sas.microanalyticservice.properties: connectionpool” on page 270](#).

- The connection string must be specified via the `connectionstring` property, not as the second argument of the `SQLSTMT` package constructor.

Example

The following example code is intended for use with `autocommit` disabled.

It splits the total inventory quantity between two stores: StoreA and StoreB. If an error occurs during the execution of either insert, any changes that were made are rolled back.

Here is the connection string that is used to connect to the database in the example:

```
driver=fedsql;conopts=(driver=oracle;catalog=ora12;uid=scott;pwd=tiger;
path=test.a.b.com:1521/exadat12c)
```

Here is the SQL code that is used to create the table in the example:

```
sscr8tbl = _NEW_ sqlstmt( 'execute ( create table ora_dstest1.vmassst2
                           ( store varchar2(20), quantity integer )
                           ) by ora12' );
```

Here is the sample code:

```
package vmassst2;
  dcl package logger logr('App.Test');
  dcl package sqlstmt ss;
  dcl package SQLStmt srollback;

  method insert2(int quantity, in_out int rc);
    dcl int half;
    if null(srollback) then do;
      srollback = _NEW_ sqlstmt('rollback');
      if null(srollback) then do;
        logr.log('e', 'SQLStmt ROLLBACK instantiation failed.');
```

1

```
        rc = -1;
        goto Exit;
      end;
    end;
  end;
  if null(ss) then do;
    ss = _NEW_ sqlstmt();
    rc = ss.prepare('insert into ora_dstest1.vmassst2 values (?,?)');
    if (rc = 1) then do;
      logr.log('e', 'setVarchar failed. rc=$s', rc);
```

```

        ss = null;
        goto tRollback;
    end;
end;
rc = ss.setVarchar(1, 'StoreA');
if (rc) then do;
    logr.log('e', 'setVarchar failed. rc=$s', rc);
    goto tRollback;
end;
half = quantity / 2;
rc = ss.setInt(2, half);
if (rc) then do;
    logr.log('e', 'setInt failed. rc=$s', rc);
    goto tRollback;
end;
rc = ss.execute();
if (rc) then do;
    logr.log('e', 'execute failed. rc=$s', rc);
    goto tRollback;
end;
rc = ss.setVarchar(1, 'StoreB');
if (rc) then do;
    logr.log('e', 'setVarchar failed. rc=$s', rc);
    goto tRollback;
end;
rc = ss.setInt(2, quantity - half);
if (rc) then do;
    logr.log('e', 'setInt failed. rc=$s', rc);
    goto tRollback;
end;
rc = ss.execute();
if (rc) then do;
    logr.log('e', 'execute failed. rc=$s', rc);

tRollback:
    logr.log('d', 'Rolling back uncommitted work...');
    srollback.execute();
end;
Exit:
end; /* End of insert2 method */
endpackage;

```

- 1 This segment of code creates an sqlstmt package instance in which the rollback statement is prepared.
- 2 There is no COMMIT statement here because the intention is to commit the first insert (for Store A) only if the second insert (for Store B) is successful. Otherwise, a rollback occurs.

If a COMMIT statement existed at this location in the code, the operation would be the same as though autocommit were enabled.
- 3 If a failure occurs, it causes a jump to this tRollback label in which srollback.execute() executes a rollback statement.

- 4 When autocommit is disabled, the execution of a public DS2 method includes an automatic implicit commit at the end of the method. If SAS Micro Analytic Service encounters an error while attempting to execute the method, an implicit rollback is executed instead of a commit.

About Database Connection Retry Functionality

Note: When connection issues cause a DS2 SQLSTMT package constructor or prepare method to fail, SAS Micro Analytic Service runs tests to determine whether the issue is due to a bad connection. For connections to Google BigQuery, Oracle, and PostgreSQL databases, this check is performed automatically. For all other database types, SAS Micro Analytic Service can use the `connectioncheckstatement` property (described in this section) to perform the check.

When SAS Micro Analytic Service is notified that a SQLSTMT package instance constructor or prepare method has failed, it can use a SQL statement that you specify to test the health of the connection. This statement is only prepared; it is not executed. For more information, see [“sas.microanalyticsservice.properties: connectionpool” on page 270](#).

The statement must reference the database of interest. For example, in the sample SQL statement `select * from mytable`, the table called `mytable` exists in the database.

Note: When using the `connectioncheckstatement` property to test the health of the connection, you must ensure the statement that you specify causes the FedSQL driver to attempt to use the database connection when preparing it.

When the `connectioncheckstatement` property is configured and a SQLSTMT package constructor or prepare method fails, SAS Micro Analytic Service enters connection retry mode and, by default, attempts to connect to the database 10 times, waiting 5 seconds between each retry attempt. When a reconnection attempt succeeds, any new SQLSTMT package instance constructor or prepare method calls use the new connection.

If necessary, you can modify the assigned reconnection values using the `dbconnretries` and `dbconnretryintervalseconds` properties in SAS Environment Manager. Note that you must restart the SAS Micro Analytic Service pod to apply the new values. You can use the `kubectl rollout restart` command to restart the pod without an interruption of service. For more information, see [“sas.microanalyticsservice.system: core” on page 272](#).

When using the connection retry functionality, be aware of the following additional information:

- SAS Micro Analytic Service does not enter connection retry mode unless it is notified of a failure in a SQLSTMT constructor or prepare method. Therefore, it is important for your SQL code to check for failures and re-instantiate the SQLSTMT package instance when you believe that a failure might be caused by a bad connection.
- If client connections fail because of a database issue that is subsequently resolved, SAS Micro Analytic Service still might need to re-establish client connections that are affected.
- If the connection retry thread uses all of the configured number of retries and the connection is not re-established, SAS Micro Analytic Service does not attempt any additional connection retries. This means that while the client connections are bad, any attempts to re-instantiate SQLSTMT objects fail.

Configuring a Connection to a Redis Server

Overview

When using SAS Micro Analytic Service as an execution service, your code might require access to data that is stored on a Redis server.

Depending on your requirements, you can obtain access to a Redis server by using properties that are configured in SAS Environment Manager or you can use package [DS2 RedisCli Package Methods](#), such as `CONNECT`, `AUTH`, and `SELECT`.

Here is some information to be aware of when configuring a connection to a Redis server:

- SAS Environment Manager enables you to configure multiple sets of Redis configuration properties. Each set, known as a *resource configuration*, is assigned a name that you can reference independently in your DS2 code. The name can be specified as the first argument of the `CONNECT` and `CLUSTERCONNECT` methods. This is useful if you need to connect to different Redis servers, each with its own set of unique property values such as host, logical database, and so on.
- If you want the RedisCli package to establish a connection using a default resource configuration, use SAS Environment Manager to create a resource configuration that is named `DefaultRedisResourceCfg`. For more information, see [“Resource Configuration” in SAS DS2 Language Reference](#).
- SAS Intelligent Decisioning stores advanced lists in Redis. The Redis instance for use with SAS Intelligent Decisioning must be named `sid-lists`. For more information, see [“Configuring Support for Advanced Lists” in SAS Intelligent Decisioning: Administrator’s Guide](#).

- The Cluster and Transport Layer Security (TLS) Boolean properties are available as Redis System Resource Configuration properties along with SAS_REDIS_CLUSTER and SAS_REDIS_TLS environment variables. By default, the Boolean values of the Cluster and TLS are `false`. When connecting to a Redis Cluster, the cluster property must be set to `true`. Similarly, when connecting to the server that requires TLS, the TLS property must be set to `true`.
- SAS Cloud Analytic Services (CAS) and SAS Compute session initialization code does not currently retrieve system resource configurations from SAS Configuration server (Consul) and current applications running in those environments are not creating them. Therefore, applications that are using the DS2 RedisCli package in those server environments can specify Redis connection properties as follows:
 - The `CONNECT` and `CLUSTERCONNECT` methods accept host and port arguments.
 - The SAS_REDIS_* environment variables can be used to set default values (such as host, port, user credentials, cluster, and TLS-related values). For information about the environment variables, see Step 4 in “[Order of Operation to Establish a Connection](#)” in *SAS DS2 Language Reference*.

Connect to a Redis Server

To connect by using properties that you configure in SAS Environment Manager, complete the following steps:

- 1 If one does not already exist, create an authentication domain of type password in which to store the Redis credentials. For more information, see [SAS Viya Platform: External Credentials](#).
- 2 Use the SAS Viya CLI to add client credentials to the authentication domain. These credentials enable the SAS Micro Analytic Service to access the Redis server. Note that to perform this operation, you must be a SAS Viya administrator.

Here is the command that you use:

```
sas-viya credentials clients create --domain-id domain-name --identity-id identity --user user-name --password password
```

In this command, you specify the following:

- `identity-id`: `sas.microanalyticScore`
- `user`: the user name that SAS Micro Analytic Service uses to access the Redis server
- `password`: the password that SAS Micro Analytic Service uses to access the Redis server

Here is an example:

```
sas-viya credentials clients create --domain-id mydomain --identity-id sas.microanalyticScore --user user1 --password mypassword
```

IMPORTANT SAS Environment Manager does not support creating client credentials. For more information about the SAS Viya CLI, see “Credentials Domain: How to (CLI)” in *SAS Viya Platform: External Credentials*.

If you have not downloaded the CLI and the credentials plug-in, see “Get the CLI and Its Plug-Ins” in *SAS Viya Platform: Using the Command-Line Interface* for more information.

- 3 Create a resource configuration that contains the applicable Redis properties. For more information, see “[sas.common.redis.servers](#)” on page 265.

Configuring an Authentication Domain

Note: This is currently supported only for a Microsoft Azure SQL database.

To connect to a database by using an authentication domain, complete the following steps:

- 1 If one does not already exist, create an authentication domain of type password in which to store the credentials. For more information, see *SAS Viya Platform: External Credentials*.
- 2 Use the SAS Viya CLI to add client credentials to the authentication domain. These credentials enable the SAS Micro Analytic Service to access the database. Note that to perform this operation, you must be a SAS Viya administrator.

Here is the command that you use:

```
sas-viya credentials clients create --domain-id domain-name --identity-id identity --user user-name --password password
```

In this command, you specify the following:

- `identity-id`: `sas.microanalyticScore`
- `user`: the user name that SAS Micro Analytic Service uses to access the database
- `password`: the password that SAS Micro Analytic Service uses to access the database

Here is an example:

```
sas-viya credentials clients create --domain-id mydomain --identity-id sas.microanalyticScore --user user1 --password mypassword
```

Note:

- You must assign the domain-id value to the SAS Authentication Domain property in SAS Environment Manager. For more information, see [“sas.microanalyticservice.properties: core” on page 271](#).
- When you configure the connection string, use the AUTHDOMAIN option (instead of the UID and PW options). For more information, see [“Microsoft Azure SQL Database Driver Reference” on page 333](#).

IMPORTANT SAS Environment Manager does not support creating client credentials. For more information about the SAS Viya CLI, see [“Credentials Domain: How to \(CLI\)” in SAS Viya Platform: External Credentials](#).

If you have not downloaded the CLI and the credentials plug-in, see [“Get the CLI and Its Plug-Ins” in SAS Viya Platform: Using the Command-Line Interface](#) for more information.

Administration

<i>SAS Micro Analytic Service Administration</i>	263
<i>SAS Micro Analytic Service Configuration</i>	264
Overview	264
jvm	264
logging.level	265
sas.common.redis.servers	265
sas.microanalytic.service.properties and sas.microanalytic.service.system	267
<i>SAS Micro Analytic Service Logging</i>	284
Overview	284
Loggers and Logging Levels	284
<i>DS2 Logging</i>	289
Overview	289
DS2 Configuration Loggers	290
DS2 Run-Time Loggers	290
DS2 SQL Services Loggers	291
<i>SAS Micro Analytic Service Security and Authorization</i>	292
<i>Secure DS2 HTTP Package Usage</i>	293
<i>Moving Objects by Using the SAS Viya Platform Transfer Service</i>	293
<i>Backup and Restore</i>	295

SAS Micro Analytic Service Administration

SAS Micro Analytic Service is implemented as a SAS Viya platform microservice. Most of the administrative tasks and capabilities are described in [SAS Viya Platform: Overview](#).

Because SAS Micro Analytic Service is implemented using both Java and SAS threaded kernel technology, there are some areas in which it differs from a standard SAS Viya platform microservice.

The items that are implemented using SAS threaded kernel technology are referred to as the Micro Analytic Service core component. The items that are implemented using Java are referred to as the REST service.

SAS Micro Analytic Service Configuration

Overview

You use SAS Environment Manager to configure SAS Micro Analytic Service Score service configuration property settings. SAS Environment Manager is a web application for managing a SAS Viya platform environment. It includes a dashboard view, which provides a quick overall look of your environment's health and status, as well as detailed views that enable you to examine and manage your environment in detail. For more information, see [SAS Environment Manager: User's Guide](#).

In SAS Environment Manager, the SAS Micro Analytic Service Score service properties are organized by configuration instances. Here are the key configuration instances:

- **jvm**
- **logging.level**
- **sas.common.redis.servers**
- **sas.microanalyticsservice.properties**
- **sas.microanalyticsservice.system**

Here are descriptions of the properties that are contained in each configuration instance.

jvm

The **jvm** configuration instance contains the properties to configure the Java virtual machine to run SAS Micro Analytic Service.

IMPORTANT If you change the value for any of these properties, you must restart the SAS Micro Analytic Service pod to apply the changes. You can use the `kubectl rollout restart` command to restart the pod without an interruption of service. For more information, see [“Applying Configuration Changes” on page 30](#).

Property	Type	Default Value	Description
java_option_xmx	String	-Xmx1024m Note: This represents 1GB.	The Java maximum heap space size. Note: You might need to increase this value depending on the number and size of the modules.
java_option_xss	String	-Xss512k Note: This represents 512K.	The Java thread stack size. Note: The assigned value should not be less than 512K.
java_option_xms	String	-Xms1024m Note: This represents 1GB.	The Java minimum heap space size. By default, this property does not appear SAS Environment Manager. You can add it by using the Add property option in the jvm section. Note: You might need to increase this value depending on the number and size of the modules. It is recommended that you do not assign a value smaller than 1GB.

logging.level

There are multiple logging.level configuration instances, each that sets the logging level for a specific area of functionality. For more information, see [“Logging Levels” on page 287](#).

If you need to configure a logging level for a service that is not already associated with a logging level, you can use SAS Environment Manager to create a new configuration instance of logging.level for that service. For more information, see [“Create Configuration Instances” in SAS Environment Manager: User’s Guide](#).

sas.common.redis.servers

The **sas.common.redis.servers** configuration instance contains the properties needed for SAS Micro Analytic Service to access a Redis server. For more information, see [“Configuring a Connection to a Redis Server” on page 258](#).

Property	Type	Default Value	Description
Name	String	Not applicable	The name that identifies a set of Redis configuration properties that are defined for SAS Micro Analytic Service. A Name value is required. IMPORTANT Do not include blank spaces or any of the following characters in the name: / % # \
Command Timeout	Integer	1500ms	The maximum amount of time, in milliseconds, that a command is allowed to run.
Connect Timeout	Integer	1500ms	The maximum amount of time, in milliseconds, that a connection is allowed to be established.
Host	String	Not applicable	The Redis server host name or IP address. A Host value is required.
Logical Database	Integer	0	The logical database to which to connect. Redis databases are specified using a zero-based numeric index.
Port	Integer	6379	The Redis server port number. Specify a value between 1 and 65535.
Redis Cluster	Boolean	False	Specifies whether the target Redis server is running in cluster mode. ■ When connecting to a standalone Redis server, use the default value (False). ■ When connecting to a Redis Cluster, set the value to True.
SAS Authentication Domain	String	Not applicable	The authentication domain in which the Redis credentials are stored.
Server Name Indication	String	Not applicable	Specifies the host name of the Redis server.

Property	Type	Default Value	Description
TLS	Boolean	False	Specifies whether to enable Transport Layer Security (TLS) for securing data transmission.

sas.microanalyticsservice.properties and sas.microanalyticsservice.system

Overview

Most properties for the **sas.microanalyticsservice.properties** and **sas.microanalyticsservice.system** configuration instances are configured in *configuration maps*. Note that some of the configuration maps appear in both configuration instances.

Here are configuration maps by configuration instance:

Configuration Instance	Configuration Map	Notes
sas.microanalyticsservice.properties	connectionpool	Properties to configure database connection options for SAS Micro Analytic Service. See “sas.microanalyticsservice.properties: connectionpool” on page 270.
	core	Database connection string property that affects the SAS Micro Analytic Service core component. See “sas.microanalyticsservice.properties: core” on page 271.
	historyharvester	Properties that are associated with the extraction and publishing of the history record tasks map. See “sas.microanalyticsservice.properties: historyharvester” on page 274.

Configuration Instance	Configuration Map	Notes
	service	Properties that are associated with archive logs and score metrics. See “sas.microanalyticsservice.properties: service” on page 274.
	supplementalProperties	This configuration instance is not currently used. Do not assign properties to it.
sas.microanalyticsservice.system	asynchronousexecution	Properties that configure asynchronous execution. See “sas.microanalyticsservice.system: asynchronousexecution ” on page 269.
	core	Properties that affect the SAS Micro Analytic Service core component. See “sas.microanalyticsservice.system: core” on page 272.
	historyscheduler	Property that configures the number of threads available for scheduling the extraction and publishing jobs. See “sas.microanalyticsservice.system: historyscheduler” on page 274.
	service	Properties that configure the REST service component. See “sas.microanalyticsservice.system: service ” on page 276.
	supplementalProperties	Properties that configure various settings. By default, these settings do not appear in SAS Environment Manager. You must add them if you require them. See “sas.microanalyticsservice.system: supplementalProperties” on page 276.

sas.microanalyticsservice.properties

The **sas.microanalyticsservice.properties** configuration instance contains a property that controls the automatic commitment of database transactions.

Property	Type	Default Value	Description
autocommit	Boolean	true	Specifies whether SQL statements are committed automatically or multiple SQL statements are collected and executed within a single transaction. ■ When set to True, each SQL statement is automatically committed to the database. ■ When set to False, transactions are committed at the end of a method or at the time of a COMMIT statement. If a DS2 method ends abnormally, all uncommitted changes are rolled back. For more information and configuration requirements, see “Committing Transactions” on page 254. IMPORTANT If you change the value for this property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the <code>kubectl rollout restart</code> command to restart the pod without an interruption of service. For more information, see “Applying Configuration Changes” on page 30.

sas.microanalyticsservice.system: asynchronousexecution

This configuration group contains properties that configure asynchronous execution.

Property	Type	Default Value	Description
corepoolsize	Integer	5	The number of core worker threads for asynchronous requests.
maxpoolsize	Integer	10	The number of maximum worker threads for asynchronous requests.
queuecapacity	Integer	100	The maximum number of asynchronous requests that can be queued for execution.

sas.microanalyticsservice.properties: connectionpool

This configuration map contains the properties to configure database connection options for SAS Micro Analytic Service.

IMPORTANT If you change the value for any of these properties, you must restart the SAS Micro Analytic Service pod to apply the changes. You can use the `kubectl rollout restart` command to restart the pod without an interruption of service. For more information, see [“Applying Configuration Changes” on page 30](#).

Property	Type	Default Value	Description
maxconnections	Integer	0	The maximum number of database connections. If this property is set to less than 1 or more than the number of SAS Micro Analytic Service worker threads, SAS Micro Analytic Service creates as many connections as there are core threads.
connectioncheckstatement	String	Not applicable	Note: When a DS2 SQLSTMT package instance fails to construct, SAS Micro Analytic Service runs tests to determine whether the issue is due to a bad connection. For connections to Google BigQuery, Oracle, and PostgreSQL databases, this check is performed automatically. For all other database types, SAS Micro Analytic Service can use the <code>connectioncheckstatement</code> property to perform the check. A SQL statement that is used to verify whether the connection is valid. When a DS2 SQLSTMT package instance construction fails and SAS Micro Analytic Service is not already attempting to re-establish connections, this statement is prepared in an attempt to check the status of the connection. If the preparation of the statement determines that the connection is bad, SAS Micro Analytic Service attempts to re-establish the connections using the value that is assigned to the <code>connectionstring</code> property.

Property	Type	Default Value	Description
			If the preparation of the statement determines that the connection is good, no further action is taken. Note: When using the <code>connectioncheckstatement</code> property to test the health of the connection, you must ensure the statement that you specify causes the FedSQL driver to attempt to use the database connection when preparing it.

sas.microanalyticsservice.properties: core

This configuration map contains a property that affects the Micro Analytic Service core component.

IMPORTANT If you change the value for this property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the `kubectl rollout restart` command to restart the pod without an interruption of service. For more information, see [“Applying Configuration Changes” on page 30](#).

Note that an interruption of service might occur if you are updating a bad connection string that is required by the DS2 SQLSTMT package. Until the new pod with the updated connection string is ready, a method’s use of the SQLSTMT package instance encounters a failure when it attempts an operation that requires the connection to be valid.

Property	Type	Default Value	Description
connectionstring	String	Not applicable	The FedSQL connection string used to connect to a database. For information about connection string format, see “About the Connection String Configuration” on page 318.
SAS Authentication Domain	String	Not applicable	The authentication domain in which the credentials for the microanalyticScore service client identity are stored. This is required only if you have configured an authentication domain. For more information, see “Configuring an Authentication Domain” on page 260.

Property	Type	Default Value	Description
			Note: You can specify only one authentication domain. If your configuration is specified for the FedSQL driver, the credentials are passed to all nested third-party drivers that are included in the connection string.

sas.microanalyticservice.system: core

This configuration map contains properties that affect the Micro Analytic Service core component.

IMPORTANT If you change the value for any of these properties, you must restart the SAS Micro Analytic Service pod to apply the changes. You can use the `kubectl rollout restart` command to restart the pod without an interruption of service. For more information, see [“Applying Configuration Changes” on page 30](#).

Property	Type	Default Value	Description
executiontimeout	Integer	30000	The time duration, in milliseconds, after which a request is sent to cancel an execution. For more information, see “Canceling Execution” on page 240 . Note: If 0 or a negative value is specified, it is ignored and the default value is used.
dbconnretries	Integer	10	The number of times that the system attempts to connect to a database.
dbconnretryintervalseconds	Integer	5	The amount of time, in seconds, that the system waits before retrying to connect to a database.
ds2maxrecompilecount	Integer	0	If there is an error because of failing database connections, the maximum number of times that the system tries to recompile DS2 code before ejecting the module.
gcintervalseconds	Integer	60	The time, in seconds, between garbage collection runs.

Property	Type	Default Value	Description
graceperiodseconds	Integer	10	The amount of time to wait, in seconds, before cleaning up the assets that are associated with a deleted revision. Note: A request to execute a deleted revision, even during the grace period, is rejected with an error message that indicates the revision has been deleted.
nativebuffersize	Integer	512	The size of the buffer, in bytes, to exchange data between the REST service and SAS Micro Analytic Service core.
numasynchthreads	Integer	0	Specifies the number of asynchronous threads in the SAS Micro Analytic Service core that are available for use by the <code>asyncCallModule</code> method. The <code>asyncCallModule</code> threads are retrieved from the pool of SAS Micro Analytic Service worker threads, thus reducing the number of available SAS Micro Analytic Service worker threads. The value that you assign must be between 0 and one less than the value that is assigned to the <code>numthreads</code> property.
numthreads	Integer	4	Specifies the number of threads in the SAS Micro Analytic Service core. The amount of memory that is used by SAS Micro Analytic Service core is determined by the number of threads. Note the following information when setting a value: ■ The compilation time for modules is proportional to the number of threads. ■ You can increase or decrease the value, depending on the availability of system memory. ■ Setting the value to 0 causes the SAS Micro Analytic Service core to use all possible cores on the system. For a system in which SAS Micro Analytic Service is the primary application, this is recommended.

sas.microanalyticsservice.properties: historyharvester

This configuration map contains the properties that are associated with the extraction and publishing of the history record tasks map.

Property	Type	Default Value	Description
maxrecordsinmessage	Integer	100	The maximum number of history records that are included in a message.
ratemilliseconds	Integer	5000	The number of milliseconds to wait between extracting history records.

sas.microanalyticsservice.system: historyscheduler

This configuration map contains a property that configures the number of threads that are available for scheduling the extraction and publishing jobs.

Property	Type	Default Value	Description
poolsize	Integer	5	The number of threads available for scheduling the extraction and publishing jobs.

sas.microanalyticsservice.properties: service

This configuration map contains the properties that configure the REST service component.

Property	Type	Default Value	Description
archive.enabled	Boolean	false	If this property is set to True, the inputs and outputs of step execution are recorded. The following data fields are recorded: ■ StepInput JSON ■ StepOutput JSON

Property	Type	Default Value	Description
			■ Elapsed Nanoseconds ■ End Time (UTC ISO-8601 format) The archive logs are saved in <code>/opt/sas/viya/config/var/log/microanalyticsservice/default/archive/provider/pod</code>. Note: This feature requires a Kubernetes persistentVolumeClaim. For more information, see “Configuring SAS Micro Analytic Service to Use the Archive Feature” on page 24. IMPORTANT If you change the value for this property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the <code>kubectl rollout restart</code> command to restart the pod without an interruption of service. For more information, see “Applying Configuration Changes” on page 30.
metrics.scorevariables	String	Not applicable	Enables you to define the variable name that is used to collect score metrics. To define the variable name, click Add property, and then in the Name field, enter the <i>module</i> and <i>step</i> in this format: <i>module.step</i> Together, they define the score variable name. In the Value field, enter the name of the variable that is returning the score value in the step output parameters. You can configure only one score variable per <i>module.step</i> combination. You can repeat this process to configure a variable name for additional <i>module.step</i> combinations. The metrics that are associated with the defined score variables are returned when either of the following REST calls are issued: ■ <code>http://server/microanalyticScore/commons/metrics/mas.module.score</code> ■ <code>http://server/microanalyticScore/commons/prometheus</code>

sas.microanalyticsservice.system: service

This configuration map contains the properties that configure the REST service component.

Property	Type	Default Value	Description
defaultmasuserctxname	String	defaultMASUserCtxName	The default value of the user context name.
moduleidgeneration			A sub-category containing two properties that control how the module ID is determined. The properties are <code>factorytype</code> and <code>forcelowercase</code> .
factorytype	One of the following: ■ GUID ■ ModuleName ■ ModuleNameOverridePackage ■ PackageName	ModuleNameOverridePackage	This value determines how the module ID is created for a new module. The module name is the one that is passed to the system during module creation.
forcelowercase	Boolean	true	If this property is set to True, the module ID returned is always lowercase.
sasmode	Boolean	true	If this property is set to True, the DS2 code is compiled with <code>ds2_options sas</code> .

sas.microanalyticsservice.system: supplementalProperties

This configuration map contains optional properties that, by default, do not appear in SAS Environment Manager.

For information about creating additional property configuration instances, see [SAS Environment Manager: User's Guide](#).

Property	Type	Default Value	Description
core.mashostuser	String	Not applicable	The user ID for Python processes to use to access the host server. If not specified, Python processes are run under the account of the SAS Micro Analytic Service process. For more information, see “External Credentials: How To” in SAS Viya Platform: External Credentials. IMPORTANT If you change the value for this property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the <code>kubectl rollout restart</code> command to restart the pod without an interruption of service. For more information, see “Applying Configuration Changes” on page 30.
core.mashostdomain	String	Not applicable	The Authentication domain to use to authenticate the user ID that is specified for core.mashostuser. If not specified, the Python processes are run under the account of the SAS Micro Analytic Service process. For more information, see “External Credentials: How To” in SAS Viya Platform: External Credentials. IMPORTANT If you change the value for this property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the <code>kubectl rollout restart</code> command to restart the pod without an interruption of service. For more information, see “Applying Configuration Changes” on page 30.
core.maspywaitmilliseconds	Integer	30000	After launching the Python client, SAS Micro Analytic Service must wait for the spawned Python process to reconnect to the SAS Micro Analytic Service server. This property sets the amount of time. The default value in milliseconds is equal to 30 seconds. Note: If 0 or a negative value is assigned, it is ignored and the default value is used.

Property	Type	Default Value	Description
			IMPORTANT If you change the value for this property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the <code>kubectl rollout restart</code> command to restart the pod without an interruption of service. For more information, see “Applying Configuration Changes” on page 30.
core.maxpethreads	Integer	0	For use with the MASParallelExec package. To enable parallel execution, you must specify the maximum number of threads to use with parallel execution. This value does not overlap with the assigned numThreads value or numAsyncThreads value. This is a separate set of available threads. For more information about parallel execution, see “MASParallelExec Package” on page 57. IMPORTANT If you change the value for this property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the <code>kubectl rollout restart</code> command to restart the pod without an interruption of service. For more information, see “Applying Configuration Changes” on page 30.
core.minfreememoryfloor	Integer	500000000	The minimum amount of memory, in bytes, that is allocated to compile a module. The default value is equal to 500MB. IMPORTANT If you change the value for this property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the <code>kubectl rollout restart</code> command to restart the pod without an interruption of service. For more information, see “Applying Configuration Changes” on page 30.
core.pewaittimemilliseconds	Integer	5000	For use with the MASParallelExec package “wait Method” method. Specifies the default number of milliseconds to wait for parallel execution completion.

Property	Type	Default Value	Description
			If set to zero, the wait method pauses until all of the parallel calls are complete before returning. For more information about parallel execution, see “MASParallelExec Package” on page 57. IMPORTANT If you change the value for this property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the <code>kubectl rollout restart</code> command to restart the pod without an interruption of service. For more information, see “Applying Configuration Changes” on page 30.
core.profilesamplefrequency	Integer	2000	Used in conjunction with profile logging. Specifies the sample rate to log the execution time of the module code. IMPORTANT If you change the value for this property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the <code>kubectl rollout restart</code> command to restart the pod without an interruption of service. For more information, see “Applying Configuration Changes” on page 30.
core.tktstacksizebytes	Integer	8192	Sets stack size, in kilobytes, for worker threads in SAS Micro Analytic Service core. The default value is sufficient for most purposes. You might need to increase the value to compile large DS2 packages that contain numerous (for example, more than 5,000) IF-THEN/ELSE statements. IMPORTANT If you change the value for this property, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the <code>kubectl rollout restart</code> command to restart the pod without an interruption of service. For more information, see “Applying Configuration Changes” on page 30.
service.alwayscheckdatabase onexecute	Boolean	False	Specifies whether the execute call checks the database for the latest copy of the module, before every invocation.

Property	Type	Default Value	Description
			This is relevant only for a clustered deployment that contains multiple SAS Micro Analytic Service nodes. Note the following information when setting a value: ■ The default value (False) results in efficient execution. ■ Set the value to True if modules are updated frequently. It is important to execute the latest copy in a clustered deployment.
service.cache.refresh.ratemillise- seconds	Integer	10000	Specifies the rate, in milliseconds, at which the database is monitored for changes, in microseconds. The default value is equal to 10 seconds. This is relevant only for a clustered deployment that contains multiple SAS Micro Analytic Service nodes. Note the following information when setting a value: ■ For a system to which few changes are anticipated, this parameter can be set to a very large value, such as 30 minutes or more. An execution-only system is an example of a system with infrequent updates. ■ For a system to which many updates are anticipated, this parameter can be set to a lower value, such as 5 seconds.
service.cache.refresh.readiness.disabled	Boolean	False	Does one of the following: ■ When set to False, SAS Micro Analytic Service manages the readiness value. ■ When set to True, SAS Micro Analytic Service does not manage the readiness value. This is useful when there are a lot of modules in the repository and you want the service to be available while modules are in the process of loading. For more information, see “Understanding Eventual Consistency and Module Availability” on page 193.
service.cache.refresh.readiness.startuponly	Boolean	True	Does one of the following:

Property	Type	Default Value	Description
			- When set to True, the SAS Micro Analytic Service readiness probe signals not ready when modules are being loaded from the database during the start-up process. When all modules are loaded, the probe value is set to ready and the service is fully accessible. Subsequent module publishes to this SAS Micro Analytic Service instance do not set the instance to not ready. This means that it is possible to receive an HTTP 503 Service Unavailable error when attempting to score a module that is still being compiled or loaded after it is published or imported. - When set to False, the SAS Micro Analytic Service readiness probe signals not ready anytime that a module is being compiled or loaded after it is published or imported. For more information, see “Understanding Eventual Consistency and Module Availability” on page 193.
service.jobs.cleanup.agehours	Integer	24	Specifies the age, in hours, a job must be in order to be considered for removal. SAS Micro Analytic Service determines the age by using the last modified date value. For more information, see “Asynchronous Module Creation and Update” on page 191.
service.jobs.cleanup.frequencyhours	Integer	24	Specifies the frequency, in hours, that SAS Micro Analytic Service reviews modules for potential removal. For more information, see “Asynchronous Module Creation and Update” on page 191.
service.jobs.requeueonshutdown	Boolean	True	When a SAS Micro Analytic Service pod shuts down and a job is currently running, this property specifies whether the active job is re-queued to be processed either by another SAS Micro Analytic Service instance or when the pod restarts. When set to True, SAS Micro Analytic Service logs a message when the pod

Property	Type	Default Value	Description
			shuts down and the incomplete job is re-queued for execution. ■ When set to False, SAS Micro Analytic Service logs a message when the pod shuts down and the incomplete job is not re-queued for execution.
service.generatemitmissingitems	Boolean	True	Does one of the following: ■ When set to True, SAS Micro Analytic Service includes all parameters in the output of a step, even when the value of a parameter is null. This is useful when integrating with external systems to ensure that a parameter is not inadvertently missed. ■ When set to False, SAS Micro Analytic Service does not include parameters that are assigned a value of null in the output of a step. This is useful when the step output contains numerous null values. It reduces the size of the generated JSON content.
service.passheadersenabled	Boolean	False	Enables the use of HTTP headers as metadata. For more information, see “Using HTTP Headers as Metadata” on page 244.
service.requiremissingitems	Boolean	True	Does one of the following: ■ When set to True, SAS Micro Analytic Service requires that all parameters be included in the request payload, even when the value of a parameter is null. This is useful when integrating with external systems to ensure that a parameter is not inadvertently missed. ■ When set to False, SAS Micro Analytic Service accepts a request payload that does not contain all the parameters that are required by the step. SAS Micro Analytic Service interprets the value of missing parameters as null. This is useful when the step input contains numerous null values. It reduces the size of the JSON content

Property	Type	Default Value	Description
			that is passed to SAS Micro Analytic Service.
service.removetrailingunderscoresfrominputs	Boolean	False	SAS Micro Analytic Service requires separate parameters for input and output. However, DS2 does not allow the same parameter to be listed twice for a method. To ensure that this does not happen, SAS Intelligent Decisioning generates code in which input variables are named with a trailing underscore (_). You can use this property to manage this behavior. This property specifies whether the trailing underscore is retained (False) or discarded (True) from the input variable names in the following situations: ■ when returning the list of parameters for a method through the REST interface ■ when accepting input for the execute call You might need to set this to True if your site uses a repository of data items with which all applications must comply.
service.timeouts.maxloadwaitexecmillis	Integer	10000	Specifies the time-out value that is used when loading a module into memory. If this value is exceeded, the load fails. The value is in milliseconds. The default value is equal to 10 seconds.
service.timeouts.maxloadwaitnonexecmillis	Integer	120000	Specifies the time-out value that is used when you query module metadata or publish content by submitting POST requests directly to the <code>/modules</code> endpoint. The value is in milliseconds. The default value is equal to 2 minutes. TIP To publish large models or decisions, it is recommended that you create an asynchronous job by posting a request to the <code>/jobs</code> endpoint.
service.timeouts.maxmodulecompiletimemillis	Integer	600000	Specifies the time-out value when compiling DS2 modules. This time-out value applies to both asynchronous and synchronous compilation requests.

Property	Type	Default Value	Description
			The value is in milliseconds. The default value is equal to 10 minutes.

SAS Micro Analytic Service Logging

Overview

As a service running on Kubernetes, SAS Micro Analytic Service logs are streamed to stdout, rather than being stored in log files. To view the log files, make sure that your sas-microanalytic-score pod exists.

For more information about how to view log files, see [SAS Viya Platform: Log Monitoring](#).

Note: SAS Viya Monitoring for Kubernetes provides logging and monitoring tools that are designed specifically for the SAS Viya platform deployment. For more information, [SAS Viya Platform: Deploying SAS Viya Monitoring for Kubernetes](#)

Loggers and Logging Levels

Overview

You configure SAS Micro Analytic Service loggers using SAS Environment Manager. For information about how to configure loggers, see “Specify a Service and Threshold Level for Service Logs” in [SAS Viya Platform: Log Monitoring](#).

For information about Python module logging, see “Python Loggers” on page 288.

SAS Micro Analytic Service Core Logging

Logger names that start with the following prefixes apply to the SAS Micro Analytic Service core: Admin, App, Audit, Perf.

Here are some important SAS Micro Analytic Service core loggers:

Name	Events Logged	Default Threshold
App.license App.SQLServices.license	Licensing	ERROR
App.MAS.Python	Python process infrastructure	WARN
App.MAS.Python.Trace	Detailed Python process infrastructure	WARN
App.tk.MAS	SAS Micro Analytic Service	INFO
App.tk.MAS.CodeGen	Compilation messages produced during an attempt to publish Note: When a publish request fails, error information is logged regardless of the App.tk.MAS.CodeGen logger level.	FATAL
App.tk.MAS.CrossModCall	Cross module calling infrastructure	INFO
App.tk.MAS.CrossModCall.DS2	DS2 MASCall package	INFO
App.tk.MAS.Datagrid	Datagrid infrastructure	INFO
App.tk.MAS.Datagrid.DS2	DS2 Datagrid package	INFO
App.tk.MAS.GarbageCollection	Activity related to releasing resources that are associated with data structures that are no longer referenced	INFO
App.tk.MAS.JNI	Java Native Interface activity	INFO
App.tk.MAS.ParallelExec	DS2 MASParallelExec package activity	INFO
App.tk.MAS.Python	SAS Micro Analytic Service Python interface	INFO
App.tk.MAS.Python.DS2	DS2 PyMAS package	INFO
App.tk.MAS.RTrace	Shared object load activity	INFO
App.tk.MAS.Service	Start-up and shutdown events and SAS Micro Analytic Service version information	INFO
App.tk.MAS.StateSharing	State sharing infrastructure layer	INFO
App.tk.MAS.SysRsrc	Resource configuration activity	INFO

Name	Events Logged	Default Threshold
App.tk.MAS.StateSharing.DS2	DS2 MASState package	INFO
App.tk.MAS.Thread	Selection of worker thread activity	INFO
Audit.Table.Connection	Database connection	INFO

Detailed information about operations such as compilation start and finish is logged at the DEBUG level. Warning and error conditions are logged at the WARN or ERROR levels, as appropriate.

When App.tk.MAS.Service logger's level is set to DEBUG or TRACE, you see a message logging event that provides the SAS Micro Analytic Service version number in the log. For information about the SAS Viya platform software version numbers, see [“Software Releases and Versions” in Getting Started with SAS Viya Platform Operations](#).

IMPORTANT

If you change the value for any SAS Micro Analytic Service core loggers, you must restart the SAS Micro Analytic Service pod to apply the change. You can use the `kubectl rollout restart` command to restart the pod without an interruption of service. For more information, see [“Applying Configuration Changes” on page 30](#).

Micro Analytic Service REST Service

SAS Micro Analytic Service REST Service Loggers

Loggers exist that log information about memory usage and profiling. Here are some important SAS Micro Analytic Service REST service loggers:

Logger	Description
MAS_CONFIG	Logs information related to configuration processing. This includes the generation of logging configurations.
MAS_CONTROLLER	Logs all REST calls to SAS Micro Analytic Service.
MAS_JOB_LOGGER	Logs information that is related to job operations.

Logger	Description
MAS_MEM.CODE	Logs memory usage by SAS Micro Analytic Service core when compiling module code.
MAS_MEM.NATIVE	Logs memory usage by SAS Micro Analytic Service core when executing module code.
MAS_MODULE_LOGGER	Logs information that is related to module operations.
MAS_MONITOR	Logs SAS Micro Analytic Service core status on a periodic basis. This includes logging memory usage and the number of loaded modules.
MAS_PROFILE	Logs sampled timings for compilation and execution of module code.

Run-Time Server Logs

For information about the loggers for run-time servers and services that you can use with SAS Micro Analytic Service, see the applicable corresponding documentation. Here is a list of common SAS run-time servers and their documentation:

- SAS Cloud Analytic Services (CAS): [“CAS Server Loggers” in SAS Viya Platform: Log Monitoring](#)
- SAS Compute Server:
 - [“SAS Compute: Server, Service, and Contexts” in SAS Viya Platform: Programming Run-Time Servers](#)
 - [“Server Configuration Instances” in SAS Viya Platform: Programming Run-Time Servers](#)

Logging Levels

The logging levels FATAL, ERROR, WARN, INFO, DEBUG, and TRACE are supported. Consider the following when you are setting logging levels:

- Normal operations, such as start-up and shutdown, are logged at the INFO level. Detailed information is logged at the DEBUG and TRACE level.
- For normal operations, it is recommended that you enable either the ERROR level or the WARN level.
- More verbose and frequent information, such as memory and profile logging, is logged only at the TRACE level.

- Enabling DEBUG and TRACE typically affects performance. Therefore, it is recommended that the DEBUG and TRACE levels are used only during system sizing, performance tuning, or when troubleshooting issues.

Python Loggers

When Python code is used by a module, SAS Micro Analytic Service automatically enables the logging facility for Python module to incorporate logging in published Python modules. Python logging events are returned based on the logging levels that are set for SAS Micro Analytic Service loggers.

For example, assume the following custom Python code:

```
import logging
...
def myMethod(name):
    logger = logging.getLogger("MyMethodLogger")
    ...
    logger.debug("myMethod received name: %s", name)
```

To enable logging for this custom Python code logger, follow [“Specify a Service and Threshold Level for Service Logs” in SAS Viya Platform: Log Monitoring](#) to add a new logging level configuration for the Micro Analytic Score service. Here is an example of the SAS Environment Manager New logging.level Configuration window enabling this sample logger at the DEBUG level:

New logging.level Configuration

Services:

Services to which this configuration instance applies. 'Global' indicates the configuration instance applies to all services.

level: * ▼

Logger level.

name: *

Logger name.

The logging levels that are configured for SAS Micro Analytic Service are used to configure the logging facility for Python. The logging levels are mapped from the traditional logging threshold levels to the levels that are supported by the logging facility for Python. This mapping is shown in the following table:

Traditional Logging Level	Python Logging Level
TRACE	DEBUG
DEBUG	DEBUG
INFO	INFO
WARN	WARNING
ERROR	ERROR
FATAL	CRITICAL

A SAS Micro Analytic Service custom handler object forwards Python LogRecord information to SAS Micro Analytic Service. SAS Micro Analytic Service uses the LogRecord information that it receives to format the logging data into the configured logging output format.

Note the following information:

- If the components that enable SAS Micro Analytic Service Python support start before the Python Logging Facility configuration file is created, a warning is logged. If the creation of the configuration file fails, an error is logged and a warning is logged for each worker thread.
- Because the SAS Micro Analytic Service core logging output configuration defines the output format, the logging facility for Python formatter and filter objects are not used.

DS2 Logging

Overview

The following loggers are available to diagnose DS2 problems:

- Configuration loggers that track information as the DS2 compiler starts up and provide context for the actual execution of the user's code.
- Run-time loggers that track actual execution.
- SQL services loggers that track information when the SQLSTMT package is used or when creating connections that are used by SQL statements.

TIP SAS recommends that you configure these loggers only when diagnosing a DS2 problem since the additional logging traffic affects performance.

DS2 Configuration Loggers

Logger	Description
App.TableServices.DS2.Config.Options	Shows the options that are supplied to the DS2 compiler.
App.TableServices.DS2.Config.Source	Shows the DS2 source code that is processed by the DS2 compiler.
App.TableServices.DS2.Config.Version	Shows version information for all threaded kernel extensions that are loaded by the DS2 compiler.

For more information, see [“Configuration Loggers” in SAS DS2 Programmer’s Guide](#).

DS2 Run-Time Loggers

Logger	Description
App.TableServices.DS2.Runtime.Calls	Shows a trace of all method calls during execution.
App.TableServices.DS2.Runtime.SQL	Shows all SQL statements that are either prepared by the DS2 compiler, executed by the DS2 compiler, or both.
App.TableServices.DS2.Runtime.Timing	Shows the time that is spent during code compilation and execution. Depending on the level of information that is requested (INFO, DEBUG, TRACE), timing information is provided at various points throughout DS2 execution. Examples are parse time, compilation time, various audit and transformation pass times, as well as the INIT, RUN, and TERM method execution times.

Logger	Description
<code>App.TableServices.DS2.Runtime.Put</code>	Records all PUT statement output.
<code>App.TableServices.DS2.Runtime.Log</code>	Records all messages that are sent to the SAS log in a SAS session.

For more information, see [“Run-Time Loggers” in SAS DS2 Programmer’s Guide](#).

DS2 SQL Services Loggers

Logger	Description
<code>App.SQLServices.API</code>	This logger must be set to DEBUG to enable logging of connection and statement activity.
<code>App.SQLServices.API.Environment</code>	When set to TRACE, logs environment handle activity. Note: This is driver-independent activity.
<code>App.SQLServices.API.Connection</code>	When set to TRACE, logs driver-independent connection activity. Note: Prior to connection and after disconnection, the connection handle interoperates with certain API calls (such as setting or getting handle attributes). This logger contains those messages since the driver is not involved.
<code>App.SQLServices.API.DRIVER_NAME.Connection</code>	When set to TRACE, logs driver-specific connection activity. <i>DRIVER_NAME</i> must be the name of the database driver (for example, ORACLE, BASE, etc.) or <code>_ALL_</code> to log all database drivers. Here are some examples: ■ <code>App.SQLServices.API.ORACLE.Connection</code> ■ <code>App.SQLServices.API._ALL_.Connection</code>
<code>App.SQLServices.API.DRIVER_NAME.Statement</code>	When set to TRACE, logs driver-specific statement activity. <i>DRIVER_NAME</i> must be the name of the database driver (for example, ORACLE, BASE, and so on) or <code>_ALL_</code> to log all database drivers. Here are some examples: ■ <code>App.SQLServices.API.BASE.Statement</code> ■ <code>App.SQLServices.API._ALL_.Statement</code>

Logger	Description
App.SQLServices.API.TF	When set at any level, additional trace flag information is included in SQLStmt TRACE log events.
App.SQLServices.API.TF .TIMESTAMP	When set at any level, SQLStmt TRACE log events include a starting timestamp. This is useful to determine how much time was spent in various calls.

SAS Micro Analytic Service Security and Authorization

Access to the REST API endpoints in SAS Micro Analytic Service is determined by authorization rules. For information about how to modify authorization rules, see [SAS Viya Platform: General Authorization](#).

Upon installation, authorization rules are created for each SAS Micro Analytic Service endpoint. For each rule, permissions are granted to all authenticated users for specific operations, such as CREATE, READ, UPDATE, and DELETE.

It is recommended that your system administrator perform the following tasks to control access to the SAS Micro Analytic Service, according to your site requirements:

- create new groups of users or update existing groups
- modify the **Principal** value to assign the applicable group
- modify the **Setting** value (if necessary)

For example, the following settings show the default Create and Delete permissions for the SAS Micro Analytic Service:

Object URI	Principal	Setting	Permissions
/microanalyticScore/modules/*	Authenticated Users	Grant	Create, Delete

You can isolate Create and Delete access to the service by creating a new group and assigning that group as the **Principal** value. For example, the following settings show that a new group, called MAS Administrators, is the only group with Create and Delete permissions for the SAS Micro Analytic Service:

Object URI	Principal	Setting	Permissions
/microanalyticScore/modules/*	MAS Administrators	Grant	Create, Delete

For information about creating new user groups, see [SAS Viya Platform: Identity Management](#).

Secure DS2 HTTP Package Usage

The DS2 HTTP package supports HTTP and HTTPS endpoints. If the default list of trusted CA certificates does not enable access to all of the secure endpoints that you want to reach, see [“How To” in SAS Viya Platform Encryption: Data in Motion](#) for information about how to add variables such as SSLCALISTLOC.

When an HTTP endpoint requires client authentication, it responds to the client with its list of supported authentication mechanisms. The DS2 HTTP package currently supports two of the three most common authentication mechanisms—Basic and Negotiate. It does not support the Digest mechanism. Because Basic authentication does not provide any credential confidentiality, it should be used only when the data is being encrypted through TLS.

The DS2 HTTP package supports certain security-related methods (for example, setOAuthToken, addSASOAuthToken, setUsername, setPassword, setProxyURL, setProxyUsername, and setProxyPassword). For more information, see [“DS2 Hash and Hash Iterator Package Attributes, Methods, Operators, and Statements” in SAS DS2 Language Reference](#).

For DS2 code that is executed by SAS Micro Analytic Service, you can use the addSASOAuthToken method to add the OAuth access token to the request header as a Bearer value of an Authorization field. For more information, see [“ADDSASOAUTHTOKEN Method” in SAS DS2 Language Reference](#).

The Negotiate mechanism supports Kerberos and, when it is used on Windows, NTLM is also supported. For more information, see [“Using the HTTP Package” in SAS DS2 Programmer’s Guide](#).

Moving Objects by Using the SAS Viya Platform Transfer Service

You can use the SAS Viya platform transfer service command-line interface (CLI) to move SAS Micro Analytic Service content from one environment to another. To use

the CLI, you must be logged in to the SAS Viya platform at the command line. To use the transfer commands, you must be logged in to the source and target environments with an account that has administrator privileges. For more information about the CLI and the transfer service, see [SAS Viya Platform: Using the Command-Line Interface](#).

This section contains an example that shows how to transfer modules from a source environment to a target environment. The commands in this example use the named profiles “Source” and “Target”. For more information about how to use a named profile, see “Use Your Profile to Sign In” in [SAS Viya Platform: Using the Command-Line Interface](#).

To move content using the transfer service:

- 1 Create a JSON file of type `ExportRequest` that includes the IDs of the objects to be exported from the source environment. For example, this sample file contains module IDs `exec_module` and `decision_08c8c3b3_bdb8_4be5_967`. The name of the file is `export.json`.

```
{
  "name": "myTransfer",
  "items": [
    "/microanalyticScore/modules/exec_module",
    "/microanalyticScore/modules/decision_08c8c3b3_bdb8_4be5_967"
  ]
}
```

- 2 Create the package file in the source environment:

```
sas-viya --profile Source transfer export --request @export.json
```

Note: Be sure to note the transfer package ID that is returned by this command.

- 3 Download the package file from source environment using the transfer package ID obtained in step 2. The following command downloads the package file to the `TransferPackage.json` file:

```
sas-viya --profile Source transfer download --id transfer-package-ID --file
TransferPackage.json
```

- 4 Upload the package file to the target environment:

```
sas-viya --profile Target transfer upload --file TransferPackage.json
```

Note: Be sure to note the transfer package ID.

- 5 Create an import request to promote the transfer package to the target environment using the transfer package ID obtained in step 4:

```
sas-viya --profile Target transfer import --id transfer-package-ID
```

- 6 (Optional) You can use the following REST API request to confirm that the transferred modules are available:

```
GET service_endpoint/microanalyticScore/modules
```

Note: `service_endpoint` represents the service endpoint in the “Target” profile (for example, `http://localhost`).

Backup and Restore

The operational state of SAS Micro Analytic Service is persisted via an automated backup and restore process. Here is information about how key data is backed up and restored:

- The SAS Micro Analytic Service configuration is backed up and restored by the SAS Viya platform.
- The metadata for modules is persisted in the SAS Infrastructure Data Server and is backed up and restored by the SAS Viya platform.
- Analytic Store (ASTORE) files and other resources that are accessed via a persistent volume claim (PVC) are restored by the SAS Model Repository service Python restore script (`RestoreScript.py`).

For information about the SAS Viya platform backup and restore functionality, see [SAS Viya Platform: Backup and Restore](#).

For information about the SAS Model Repository service Python restore script (`RestoreScript.py`), see “Full-System Migration between SAS Viya Environments” in [SAS Model Manager: Administrator’s Guide](#).

IMPORTANT If you use the SAS Micro Analytic Service archive feature, the input and output transaction logs are stored on a persistent volume. SAS does not include these logs in its backup and restore process. You are responsible for the backup and restoration of these log files.

PART 6

Appendixes

Appendix 1	
SAS Micro Analytic Service Data Type Mappings	299
Appendix 2	
Executing Python Modules in DS2 Modules	303
Appendix 3	
Database Driver Reference	317
Appendix 4	
SAS Data Quality Function Support	359
Appendix 5	
SAS Micro Analytic Service Metrics	361
Appendix 6	
SAS Micro Analytic Service Tuning Guidelines	363
Appendix 7	
SAS Quality Knowledge Base (QKB)	367
Appendix 8	
REST Server Error Messages and Resolutions	369
Appendix 9	
SAS Micro Analytic Service Return Codes	377

Appendix 1

SAS Micro Analytic Service Data Type Mappings

SAS Micro Analytic Service Data Type Mappings Table 299

SAS Micro Analytic Service Data Type Mappings Table

The following table lists the core SAS Micro Analytic Service data types and the data type to which each is mapped for DS2, Python, JSON, and REST.

SAS Micro Analytic Service Data Types	Description	DS2	Python	JSON ¹	REST
BOOLEAN	Boolean (true/false)	Not supported ²	Boolean	boolean	boolean
BOOLEAN ARRAY	Array of Boolean	Not supported ²	list of Booleans	array	booleanArray
DATE	Date without timezone	Not supported	date	string ³	date
DATE ARRAY	Array of date without timezone	Not supported	list of dates	array	dateArray

SAS Micro Analytic Service Data Types	Description	DS2	Python	JSON ¹	REST
DATE-TIME	Timestamp without timezone	Not supported	datetime	string ³	dateTime
DATE-TIME ARRAY	Array of timestamp	Not supported	list of datetimes	array	dateTimeArray
DOUBLE	64-bit floating point	double	float	number ⁴	decimal
DOUBLE ARRAY	Array of 64-bit floating point	double[]	list of floats	array	decimalArray
INTEGER	32-bit integer	int	int, long ⁵	number	integer
INTEGER ARRAY	Array of 32-bit integer	int[]	list of ints / longs ⁵	array	integerArray
LONG	64-bit integer	bigint	int, long ⁵	number	bigint
LONG ARRAY	Array of 64-bit integer	bigint[]	list of ints / longs ⁵	array	bigintArray
REFERENCE	Binary	binary varbinary	bytes	string ⁶	binary ⁷
REFERENCE ARRAY	Array of binary	binary[] varbinary[]	list of bytes objects	array ⁶	binaryArray ⁷
STRING	Character string	varchar / nvarchar / char / nchar	string	string	string
STRING ARRAY	Array of strings	array of varchar / nvarchar / char / nchar	list of strings	array	stringArray
TIME	Time without date or timezone	Not supported	time	string ³	time
TIME ARRAY	Array of time without date or timezone	Not supported	list of times	array	timeArray
DATAGRID	Datagrid data type	datagrid package	dictionary / dataframe ⁸	array ⁹	datagrid

¹Missing values are represented as null.

² DS2 does not have a Boolean data type. An integer value of 0 (zero) means false and any other integer value means true.

³ Datetime, date, and time values are represented as ISO 8601 encoded strings in JSON.

⁴This is a double-precision 64-bit format IEEE-754 standard value. It includes "Infinity", "-Infinity", and "NaN".

⁵ Python integer data types are always returned to SAS Micro Analytic Service as long integers (64-bit).

⁶ If you are using the SAS Micro Analytic Service REST service, the binary data type is represented as Base64 encoded strings. Otherwise, the binary data type is processed as a blob.

⁷ Note the following information about using blobs with SAS Micro Analytic Service:

- SAS recommends that the size of a blob be under 100MB. The size must not exceed 2GB.
- You might need to set the Java maximum heap space size to greater than 1GB. You set this using the SAS Micro Analytic Service Score service `java_option_xmx` property. For information, see ["jvm" on page 264](#).

⁸ By default, data grids are represented as Python dictionaries. If the Pandas library is available and DataFrames are enabled, data grids are represented as DataFrames in Python.

⁹ See ["DS2 Datagrid Package Methods, Operators, and Statements" in SAS DS2 Language Reference](#) for more information.

Appendix 2

Executing Python Modules in DS2 Modules

<i>DS2 Interface to Python</i>	303
Overview	303
About Using a PyMAS Package	304
About Assigning Return Data Types with Getter Methods	305
<i>Sample DS2 Module Operations</i>	306
<i>Configuring Support for a DS2 PyMAS Package</i>	311
<i>About Python and Data Grids</i>	311
<i>Example</i>	313
Overview	313
Model Using Raw Python Code	313
Model Using Python Code with the PyMAS Package	314

DS2 Interface to Python

Overview

DS2 modules, running in SAS Micro Analytic Service, can publish and execute Python modules. Note that Python must be available for SAS Micro Analytic Service to publish and execute Python modules.

DS2 packages that execute outside SAS Micro Analytic Service, such as those within a PROC DS2 program block, can also publish and execute Python modules by using interfaces that are provided via the SAS Micro Analytic Service DS2 PyMAS package.

Note: Python code that is supplied to the PyMAS publish method is not compiled when the DS2 package is published to SAS Micro Analytic Service. Instead, the Python code is compiled when the DS2 module code is executing the PyMAS publish method. The useModule method can be used to specify a Python module that has been previously published. This enables multiple PyMAS package instances to share the same Python module.

About Using a PyMAS Package

Here are some examples of how a PyMAS package might be used:

- In SAS Intelligent Decisioning, you have DS2 code that uses a PyMAS package that is executed using the microanalyticScore microservice.
- In SAS Model Manager, you have DS2 code that uses a PyMAS package that is executed using the Compute Server or the CAS Server.
- In SAS Data and AI Studio, you use PROC DS2, and the PROC uses a PyMAS package that is executed using the Compute Server.

Each PyMAS package instance represents exactly one Python module revision. You can create as many instances as you require, allowing multiple modules to be used.

As is the case when calling any package from DS2, it is recommended that you always check return codes where available, and return any error codes by using an output argument from your DS2 method. A zero return code indicates success. Unless otherwise noted, a nonzero return code indicates failure. For information about nonzero return codes, see [Appendix 9, “SAS Micro Analytic Service Return Codes,” on page 377](#). If you encounter a nonzero return code, check the applicable log file for more information. A zero return code indicates success. A nonzero return code indicates failure.

IMPORTANT When a DS2 module is executing in SAS Micro Analytic Service, you must conditionally initialize a package-scoped PyMAS package variable and publish the Python module once. Here is an example:

```
package pyscore;
  dcl package py as py;
  dcl package logger logr('App.MyLogger');
  dcl varchar(4096) character set utf8 pycode;
  dcl int revision;
  method score( double a, double b,
               in_out int rc,
               in_out double c,
               in_out double d );
    if null(py) then do;
      py = _new_ py as py;
      rc = py.useModule('mypymodule', 1);
      if rc then do;
        rc = py.appendSrcLine('# The first Python function:');
```

```

        rc = py.appendSrcLine('def domath1(w, x):');
        rc = py.appendSrcLine('    "Output: y, z"');
        rc = py.appendSrcLine('    y = w * x');
        rc = py.appendSrcLine('    z = w / x');
        rc = py.appendSrcLine('    return y, z');
        pycode = py.getSource();
        revision = py.publish(pycode, 'mypymodule');
        if revision lt 1 then do;
            logr.log( 'e', 'py.publish() failed. ');
            rc = -1;
            return;
        end; /* End of if revision less than 1 */
    end; /* End of if useModule failed */
    rc = py.useMethod('domath1');
    if rc then return;
end; /* End of if null(py) */
rc = py.setDouble('w', a);
rc = py.setDouble( 'x', b );
rc = py.execute();
c = py.getDouble( 'y' );
d = py.getDouble( 'z' );
end;
endpackage;

```

About Assigning Return Data Types with Getter Methods

The PyMAS package getter functions allow the retrieval of returned values from Python code. When you use a getter function, the type must match the type of the corresponding Python function output argument. If the Python code returns values of a type that does not match the getter method, an error occurs.

Here is an example that shows this type mismatch.

```

dcl package pymas py;
dcl timestamp date_value;

py = _new_ pymas();

/* Python method returns a long */
py.appendSrcLine('def get_date():');
py.appendSrcLine('    "Output: date"');
py.appendSrcLine('    date = 0');
py.appendSrcLine('    return date');

py.publish(py.getSource(), 'pymas_module');
py.useMethod('get_date');
py.execute();

/* ERROR: The Python variable 'date' in this sample is an integer, not
a timestamp */
date_value = py.getDateTime('date');

```

Running this block of code would result in the following error in the logs:

ERROR App.TableServices.DS2.Runtime.Log - Line 21: In method 'getDateTime': found long parameter 'date'.

Sample DS2 Module Operations

Here are some operations that a DS2 module would typically perform.

Calling `publish()` compiles your Python module and sets it as the module that is represented by this PyMAS instance. Subsequent PyMAS function calls, such as setting values and executing methods, operate on this module. The Python code is passed as a string in the first argument. Pass the name that you want to give to your new Python module in the second argument. `publish()` returns the revision number that SAS Micro Analytic Service assigned to your new module. You could use this revision number later to execute or delete a specific revision of your module. If you do not specify a revision number, the latest revision is assumed. If your Python code fails to publish (because of syntax errors, for example), then -1 is returned for the revision number.

```
revision = py.publish(pgm, moduleName);
```

Rather than publishing a Python module from DS2, you might need to specify a previously published Python module. In this case, you can call `useModule()` instead of `publish()`. If a module was already associated with your PyMAS instance before calling `useModule()`, then `useModule()` disassociates the current module from the instance before making the specified module current.

```
rc = py.useModule(moduleName, revision);
```

Before calling Python, you must tell the PyMAS instance which method to execute. This is accomplished by calling `useMethod()`. In addition to specifying the method (Python function) to call, `useMethod()` also validates that the method exists within the current module, prepares the PyMAS instance to receive the input values for the specific method arguments, and prepares to return any output values from the method execution.

```
rc = py.useMethod(methodName);
```

Call the type-specific setter methods to set input values before executing the method. Because these setters store arguments by name, they can be called in any order, and they insert the values in the correct positions:

```
py.setDouble('airflow', sensor_maf);
```

Since the DS2 package instance represents a single revision, the `execute()` method needs no arguments.

```
rc = py.execute();
```

After execution, call getters to retrieve the results.

```
score = py.getDouble('credit_score');
```

Scalar argument setters are of the form:

```
return_code = set<type>(name, value)
```

Scalar argument getters are of the form:

```
value = get<type>(name)
```

Array argument setters are of the form:

```
rc = set<type>Array(name, array-value)
```

Array argument getters are of the following form.

Note: DS2 passes arrays and output values by reference.

```
get<type>Array(name, array-value, rc)
```

The example below assumes that you have declared your package as `py`. The character string variables `python_source_code` and `my_module_name` contain the Python source code and the name that will be associated with the published module.

```
dcl package pymas py;
dcl int rc;
dcl bigint result;
py = _new_ pymas();
rc = py.publish(python_source_code, my_module_name);
rc = py.useMethod('func1');
py.setString('inString', 'A string');

py.execute()

bigintVar = py.getLong('outLongVar');
```

The complete set of DS2 package methods follows, where `rc` is the integer return code, and `py` is the package instance.

Methods for Python module management and execution:

```
rc = py.appendSrcLine( python_src_line );
python_source_code = py.getSource();
rc = py.publish(python_source_code, 'module_name');
rc = py.remove();
rc = py.isLoaded(); // returns true if Python is available and false
otherwise
revision = py.getRevisionNumber();
rc = py.setTimeZone(time_zone_identifier);
rc = py.execute();
```

Scalar argument setters:

```
rc = py.setBool(argument_name, value);
rc = py.setDatagrid(argument_name, datagrid);
rc = py.setDate(argument_name, value);
rc = py.setDateTime(argument_name, value);
rc = py.setDouble(argument_name, value);
rc = py.setInt(argument_name, value);
rc = py.setLong(argument_name, value);
rc = py.setTime(argument_name, value);
rc = py.setString(argument_name, value);
```

Scalar argument getters:

```

int_value      = py.getBool(argument_name);
rc             = py.getDataGrid(argument_name, datagrid_byref);
date_value     = py.getDate(argument_name);
date_time_value = py.getDateTime(argument_name);
double_value   = py.getDouble(argument_name);
int_value      = py.getInt(argument_name);
long_value     = py.getLong(argument_name);
string_value   = py.getString(argument_name);
time_value     = py.getTime(argument_name);

```

Array argument setters:

```

rc = py.setBoolArray(argument_name, integer_array);
rc = py.setDateArray(argument_name, date_array);
rc = py.setDateTimeArray(argument_name, date_time_array);
rc = py.setDoubleArray(argument_name, double_array);
rc = py.setIntArray(argument_name, integer_array);
rc = py.setLongArray(argument_name, bigint_array);
rc = py.setStringArray(argument_name, string_array);
rc = py.setTimeArray(argument_name, time_array);

```

Array argument getters:

```

py.getBoolArray(argument_name, integer_array, rc);
py.getDateArray(argument_name, date_array, rc);
py.getDateTimeArray(argument_name, date_time_array, rc);
py.getDoubleArray(argument_name, double_array, rc);
py.getIntArray(argument_name, integer_array, rc);
py.getLongArray(argument_name, bigint_array, rc);
py.getStringArray(argument_name, string_array, rc);
py.getTimeArray(argument_name, time_array, rc);

```

Note the following important information about Python and SAS Micro Analytic Service:

- Source code encoding, if provided, must be UTF-8.
- The PyMAS package method getters do not support converting the value that is returned from Python to the type named in the getter method. The value of the output argument that is returned from Python must be consistent with the PyMAS getter method that is called. For example, consider the `getDouble` method example:

```
myintvar = py.getDouble('pyoutarg1');
```

The `pyoutarg1` output argument that is returned from Python function must be of type double, but the assignment statement can coerce the value to an integer.

Although Python packages might support abstract types that provide an additional layer on top of the built-in scalar types, such as integer and float, they cannot be returned from SAS Micro Analytic Service public functions. Packages that have these abstract types provide functions that can be used to extract the built-in type. Here is an example:

```
return numpy.float(x), numpy.int(y)
```

- Python modules can use other Python modules that are published to SAS Micro Analytic Service. DS2 PyMAS package instances can use the `useModule`

method to specify a previously published module, instead of publishing a revised Python module.

- Your Python functions must be stateless and not depend on data from a previous Python function call.

When using PROC DS2 in a SAS session to create a PyMAS package instance, you cannot provide the Python program as one quoted literal string. The reason is that the SAS tokenizer strips out the embedded line-ending characters, causing indentation problems in the Python code. In this situation, a PyMAS package's `appendSrcLine()` and `getSource()` methods can be used. The methods produce a DS2 character variable containing the lines of code concatenated together with embedded linefeed characters separating the lines of Python code. Once you have added each line of your Python code to the PyMAS package instance using the `appendSrcLine()` method, you can use the `getSource()` method to retrieve the complete program into a DS2 character variable. The variable can then be provided as the first input argument to the PyMAS `publish()` method. An error is logged when you use the PyMAS `useModule` method to specify a module that has not been published. The following PROC DS2 example encounters the error on the first execution of the `score` method. The `score` method checks the return code and publishes the module. Subsequent executions of the `score` method do not generate the error.

```
%macro chkrc; if rc then put rc=; %mend;
%macro addln( line ); rc = pm.appendSrcLine( &line ); %chkrc; %mend;

/* Input data for the test.*/
data tstinput; a = 8; b = 4; output; a = 10; b = 2; output;
run;

proc ds2;
  ds2_options sas;
  package mypkg;
    dcl package pymas pm;
    dcl package logger logr('App.MyLogger');
    dcl varchar(67108864) character set utf8 pycode;
    dcl int revision;

  method usefunc( varchar(256) pyfuncname, in_out int rc );
    rc = pm.useMethod( pyfuncname );
    if rc then logr.log( 'E', 'pm.useMethod() failed.' );
  end;

  method score( double a, double b,
                in_out int rc,
                in_out double c, in_out double d );
    if null(pm) then do;
      pm = _new_ pymas();
      rc = pm.useModule( 'mypymodule', 1 );
      if rc then do;
        %addln( '# The first Python function:' )
        %addln( 'def domath1(a, b):' )
        %addln( '    "Output: c, d"' )
        %addln( '    c = a * b' )
        %addln( '    d = a / b' )
        %addln( '    return c, d' )
```

```

        %addln( ' ' )
        %addln( '# Here is the second function:' )
        %addln( 'def domath2(a, b):' )
        %addln( '    "Output: c, d"' )
        %addln( '    c,d = domath1( a, b )' )
        %addln( '    return c, d' )
        if rc then do;
            logr.log( 'E', 'pm.appendSrcLine() failed.' );
            pm = null;
            return;
        end;
        pycode = pm.getSource();
        revision = pm.publish( pycode, 'mypymodule' );
        if ( revision < 1 ) then do;
            logr.log( 'E', 'pm.publish() failed.' );
            rc = -1;
            pm = null;
            return;
        end;
        rc = pm.useMethod( 'domath1' );
        if rc then do;
            logr.log( 'E', 'pm.useMethod() failed.' );
            return;
        end;
        rc = pm.setDouble( 'a', a );    if rc then return;
        rc = pm.setDouble( 'b', b );    if rc then return;
        rc = pm.execute();              if rc then return;
        c = pm.getDouble( 'c' );
        d = pm.getDouble( 'd' );
    end;
endpackage;

data _null_;
    dcl package logger logr('App.MyLogger');
    dcl package mypkg t();
    dcl int rc;
    dcl double a b c d;

    method run();
        rc = 0;
        c = d = .;
        set tstinput;
        t.score( a, b, rc, c, d );
        if rc then do;
            logr.log( 'E', 'rc=$s', rc );
            stop;
        end;
        logr.log( 'I', 'Results: a=$s    b=$s    c=$s    d=$s',
            a, b, c, d );
    end;

    method term();
        if not rc then do;
            t.usefunc( 'domath2', rc );

```



```

        if rc then do;
            logr.log( 'E', 'rc=$s', rc );
            return;
        end;
        a = 6; b = 3;
        t.score( a, b, rc, c, d );
        if rc then
            logr.log( 'E', 'rc=$s', rc );
        else
            logr.log( 'I', 'Results: a=$s    b=$s    c=$s    d=
$s',
                                a, b, c, d );
        end;
    end;
enddata;
run;
quit;

```

Configuring Support for a DS2 PyMAS Package

To enable support of a PyMAS package in environments that execute DS2 packages, Python must be installed and configured for the SAS Viya platform in each location that SAS Micro Analytic Service requires it, such as the SAS Compute Server or a CAS Server.

Configuring Python with the SAS Viya platform is a task that is performed during the deployment process. For more information, see the following:

- For SAS Intelligent Decisioning and SAS Model Manager, see [“Configuring Python for SAS Micro Analytic Service” on page 232](#).
- For SAS Event Stream Processing, see [SAS Event Stream Processing: Using the Python Interface](#) and [“Configuring Python for SAS Micro Analytic Service” on page 153](#).

About Python and Data Grids

Currently, data grids are supported as input variables to Python modules. If the Pandas library is available, Python works with a data grid as a DataFrame. Otherwise, a data grid is treated as a Python dictionary.

The following examples show a data grid in JSON format and then how the same data grid appears as a Python DataFrame and as a Python dictionary.

- Here is the data grid in JSON format:

```
[
  {
    "metadata": [
      {"S": "string"},
      {"I": "int"},
      {"L": "long"},
      {"D": "decimal"},
      {"B": "boolean"}
    ]
  },
  {
    "data": [
      ["ONE", 1, 10, 1.1, true],
      ["TWO", 2, 20, 2.2, null],
      ["THREE", 3, 30, 3.3, true]
    ]
  }
]
```

- Here is the data grid represented as a Python DataFrame:

	S	I	L	D	B
0	ONE	1	10	1.1	true
1	TWO	2	20	2.2	null
2	THREE	3	30	3.3	true

- Here is the data grid represented as a Python dictionary:

```
[
  {
    'metadata': {
      'S': 'object',
      'I': 'int32',
      'L': 'int64',
      'D': 'float64',
      'B': 'boolean'
    },
    'data': {
      'S': ['ONE', 'TWO', 'THREE'],
      'I': [1, 2, 3],
      'L': [10, 20, 30],
      'D': [1.1, 2.2, 3.3],
      'B': [true, null, true]
    }
  }
]
```

Example

Overview

SAS Micro Analytic Service accepts Python code in two formats:

- **Raw Python code** : This form is useful when your entire solution consists of Python code. Raw Python code is accepted by SAS Event Stream Processing, the FCMP action set, and the SAS Micro Analytic Service REST service.
- **Python code injected into the PyMAS package** : This form is required to run in SAS servers. It is accepted by PROC DS2 in SAS servers, SAS Cloud Analytic Services (CAS), and the SAS Micro Analytic Service REST service. This form enables you to create a solution by integrating Python, DS2, and ASTORE code.

Note: In both cases, the Python code must follow certain rules to be usable in SAS Micro Analytic Service. This is discussed in [“Public and Private Methods” on page 226](#).

Both SAS Intelligent Decisioning and SAS Model Manager accept the raw Python form and then generate the second form. However, you can create your own custom DS2 programs that use the PyMAS package to execute Python code. The sample code in [“Model Using Python Code with the PyMAS Package”](#) shows the steps that are required to use the PyMAS package to execute the Python code that is shown in [“Model Using Raw Python Code”](#).

Model Using Raw Python Code

Here is the sample raw Python code.

```
# python score code for hmeq logistic.
# import any libraries that are needed.

import math

# We require a function definition with a list of the input variables.
# We require the quoted "Output..." statement to list the output
variables.

def scoreModel (CLAGE,DEBTINC,NINQ,VALUE,LOAN,MORTDUE,YOJ) :
    "Output: p_bad1, i_bad"

    # model computation
```

```

_LP0 = 1.76603584532612
_LP0 += (0.00512706888144) * CLAGE
_LP0 += (-0.04250082715182) * DEBTINC
_LP0 += (-0.19020384025922) * NINQ
_LP0 += (-3.8130517290792E-6) * VALUE
_LP0 += (0.00002251208501) * LOAN
_LP0 += (5.5951450389568E-6) * MORTDUE
_LP0 += (0.00595101042911) * YOJ
if (_LP0 < 0):
    _LP0 = math.exp(_LP0)
    _P0 = _LP0 / (1 + _LP0)
else:
    _P0 = 1 / (1 + math.exp(-_LP0))

# create the output variables

p_bad1 = 1.0 - _P0
if p_bad1 > 0.5:
    i_bad="1"
else:
    i_bad="0"

# return the output variables

return( p_bad1, i_bad )

```

Model Using Python Code with the PyMAS Package

Here is the same Python code injected into the PyMAS package.

```

/* DS2 python package required */
package pythonScore / overwrite=yes;

/* create instance of PYMAS object */
dcl package pymas pm;

/* create a logging destination. The SAS configuration will determine where this goes */
dcl package logger logr('App.MyLogger');

/* create internal variables for handling score code and result codes */
dcl varchar(32767) character set utf8 pypgm;
dcl int resultCode revision;

/* required method for score functions */
method score(double CLAGE, double DEBTINC, double NINQ, double VALUE, double LOAN,
double MORTDUE, double YOJ, in_out double resultCode, in_out double P_BAD1,
in_out double I_BAD);
    resultCode = revision = 0;

/* Create python object if it does not exist */

```

```

if null(pm) then do;
  pm = _new_ pymas();

  /* test model object in case it was already loaded into memory */
  resultCode = pm.useModule('mymodel', 1);
  if resultCode then do;

    /* load python program into the DS2 object */
    resultCode = pm.appendSrcLine('# python score code for hmeq logistic ');
    resultCode = pm.appendSrcLine('import math ');
    resultCode = pm.appendSrcLine('');
    resultCode = pm.appendSrcLine('def scoreModel(CLAGE,DEBTINC,NINQ,VALUE,LOAN,');
    resultCode = pm.appendSrcLine('          MORTDUE,YOJ):');
    resultCode = pm.appendSrcLine('    "Output: p_bad1, i_bad" ');
    resultCode = pm.appendSrcLine('    ');
    resultCode = pm.appendSrcLine('    _LP0 = 1.76603584532612 ');
    resultCode = pm.appendSrcLine('    _LP0 += (0.00512706888144) * CLAGE');
    resultCode = pm.appendSrcLine('    _LP0 += (-0.04250082715182) * DEBTINC');
    resultCode = pm.appendSrcLine('    _LP0 += (-0.19020384025922) * NINQ');
    resultCode = pm.appendSrcLine('    _LP0 += (-3.8130517290792E-6) * VALUE');
    resultCode = pm.appendSrcLine('    _LP0 += (0.00002251208501) * LOAN');
    resultCode = pm.appendSrcLine('    _LP0 += (5.5951450389568E-6) * MORTDUE');
    resultCode = pm.appendSrcLine('    _LP0 += (0.00595101042911) * YOJ');
    resultCode = pm.appendSrcLine('');
    resultCode = pm.appendSrcLine('    if (_LP0 < 0):');
    resultCode = pm.appendSrcLine('        _LP0 = math.exp(_LP0)');
    resultCode = pm.appendSrcLine('        _P0 = _LP0 / (1 + _LP0)');
    resultCode = pm.appendSrcLine('    else:');
    resultCode = pm.appendSrcLine('        _P0 = 1 / (1 + math.exp(-_LP0))');
    resultCode = pm.appendSrcLine('');
    resultCode = pm.appendSrcLine('    p_bad1 = 1.0 - _P0');
    resultCode = pm.appendSrcLine('    if p_bad1 > 0.5:');
    resultCode = pm.appendSrcLine('        i_bad="1"');
    resultCode = pm.appendSrcLine('    else:');
    resultCode = pm.appendSrcLine('        i_bad="0"');
    resultCode = pm.appendSrcLine('    ');
    resultCode = pm.appendSrcLine('    return( p_bad1, i_bad )');

    /* push score code to python interpreter */
    revision = pm.publish(pm.getSource(), 'mymodel');

    /* test return code and log failure */
    if ( revision < 1 ) then do;
      logr.log( 'e', 'py.publish() failed.' );
      resultCode = -1;
      return;
    end;
  end;
end;

/* test method and log failure */
resultCode = pm.useMethod('scoreModel');
if resultCode then do;
  logr.log('E', 'useMethod() failed. resultCode=$s', resultCode);
  return;
end;

```

```

/* set input var values for scoring */
resultCode = pm.setDouble('CLAGE', CLAGE);
resultCode = pm.setDouble('DEBTINC', DEBTINC);
resultCode = pm.setDouble('NINQ', NINQ);
resultCode = pm.setDouble('VALUE', VALUE);
resultCode = pm.setDouble('LOAN', LOAN);
resultCode = pm.setDouble('MORTDUE', MORTDUE);
resultCode = pm.setDouble('YOJ', YOJ);

/* run model scoring */
resultCode = pm.execute();

/* test return code */
if (resultCode) then put 'Error: pm.execute failed.  resultCode=' resultCode;

/* get output values */
else do;
    p_bad1 = pm.getDouble('p_bad1');
    i_bad = pm.getDouble('i_bad');
end;

end;

endpackage;

```

Appendix 3

Database Driver Reference

<i>About the Connection String Configuration</i>	318
<i>About Using Database Catalogs</i>	319
<i>Amazon Aurora Driver Reference</i>	320
Connection String	320
Connection Options	321
<i>Amazon Redshift Driver Reference</i>	321
Connection String	321
Connection Options	321
<i>DB2 Driver Reference</i>	325
Prerequisites	325
Connection String	325
Connection Options	325
<i>Google BigQuery Driver Reference</i>	328
Connection String	328
Connection Options	329
<i>Microsoft Azure SQL Database Driver Reference</i>	333
Connection String	333
Connection Options	334
Disabling Transaction Autocommit	338
<i>Microsoft SQL Server Driver Reference</i>	338
Overview	338
Standard Connection String	339
DSN-Based Connection String	339
Connection Options	340
Disabling Transaction Autocommit	342
<i>Oracle Driver Reference</i>	342
Connection String	342
Connection Options	343
Creating a Materialized View (USE_CACHED_CATALOG)	345
<i>PostgreSQL and Microsoft Azure PostgreSQL Driver Reference</i>	346
Connection String	346
Connection Options	347

SingleStore Driver Reference	350
Connection String	350
Connection Options	351
Snowflake Driver Reference	353
Connection String	353
Connection Options	353
Additional Database Connection Options	355

About the Connection String Configuration

Connection strings are used to specify database connection information such as host, port, driver, database, catalog, schema, credentials, and options. The DS2 SQLSTMT package uses the FedSQL dialect. Therefore, the connection string must begin with `DRIVER=SQL` with the FedSQL language driver as the managing driver. Then, one or more target driver connection strings are specified within a `CONOPTS=()` option. Here is the format:

```
DRIVER=SQL; CONOPTS=((database-connection string1); (database-connection-string2);...)
```

Here is an example:

```
DRIVER=SQL; CONOPTS=((DRIVER=ORACLE; CATALOG=oracle1; UID=MyUID; PWD=MyPW;
PATH=server.example.com:1521/myoradatabase);
(DRIVER=snowflake; UID=MyUID; PWD=MyPWD; SERVER='MyServer.snowflakecomputing.com';
SCHEMA=MYSHEMA))
```

You can also use a nested `CONOPTS` statement if you need to specify a third-party driver. Here is an example:

```
DRIVER=SQL; CONOPTS=(DRIVER=MSSQLSVR; CATALOG=(acat=MyCatalog);
ENABLE_MARS=YES; DM_UNICODE=UTF-16;
CONOPTS=(driver=/opt/microsoft/msodbcsql17/lib64/
libmsodbcsql-17.10.so.2.1; DATABASE=MyDB;
AUTHENTICATION=ActiveDirectoryServicePrincipal; UID=MyUID; PWD=MyPW;
SERVER=MyServer.sas.com; PORT=1433; SSLLIBNAME=/usr/lib64/
libssl.so.1.1; CRYPTOLIBNAME=/usr/lib64/libcrypto.so.1.1));
```

You can provide multiple `AUTHDOMAIN=` parameters within the connection string to connect to each of the databases successfully. Here is an example:

```
DRIVER=SQL;
CONOPTS=((DRIVER=SSTORE;authdomain=singletest3;HOST=singletest02;
PORT=3306;DB=test;CATALOG=X);
(driver=postgres;catalog=pg_catalog;authdomain=test3;server=postgres12
.hes.sashq-d.openstack.sas.com;port=5432;db=elevator);
(DRIVER=MSSQLSVR; CONOPTS=(driver=SAS ACCESS to MS SQL
Server;db=edmdev;authdomain=auth1;Host=pde720x06.na.sas.com;PORT=1433))
```


To learn more about configuring authentication domains, see [“Configuring an Authentication Domain” on page 260](#).

IMPORTANT Make sure that all credentials are valid and all AUTHDOMAINS are defined correctly. If any one credential is not valid or any one AUTHDOMAIN is not defined properly, all the database connections fail.

Note: Although these sample connection strings might appear on multiple lines, you should always enter a connection string as a single line.

You enter a connection string value using the SAS Micro Analytic Service Score service `“sas.microanalyticsservice.properties: core”` configuration property.

The FedSQL driver requires that all of the target driver connections specified in the connection string connect successfully, even if they are not referenced by SQL statements.

Passwords in a connection string can be encoded values that are produced by PROC PWENCODE. The SAS002 (alias SASENC), SAS003, SAS004, and SAS005 encoding methods are supported. For more information, see [“PWENCODE Procedure” in Base SAS Procedures Guide](#).

About Using Database Catalogs

Databases retain a structure that contains data stored in a database. Data is contained in tables, tables are grouped into schemas, and schemas are grouped into catalogs. Catalog and schema names can be used in SQL statements to qualify table references. A catalog is a named collection of logically related schemas. The catalog is the first-level (top) grouping mechanism in a data organization hierarchy that qualifies schemas. At least one schema is required for each catalog.

If your database supports native catalogs, you can use catalog mapping to avoid duplication errors. Certain SAS drivers provide a connection option, `CATALOG=`, that facilitates catalog name mapping. Using `CATALOG=`, you can specify an arbitrary identifier for a SQL catalog that groups logically related schemas.

For databases that do not support native catalogs, any identifier is valid. For databases such as SQL Server that do support native catalogs, `CATALOG=` is not required. Using the `CATALOG` option is also useful in cases where multiple connections in the connection string connect to data sources that share schema names. The connection defaults to `CATALOG=*` unless you specify a logical name for the catalog and map it to the native catalog name in the database. For example, to map the logical catalog `mycat` to the native catalog that is named `newusers`, use the following command: `CATALOG=(mycat=newusers) ;`.

If a catalog name is not specified, then the native catalog name is surfaced to the users. Catalog name maps can be used only with FedSQL. They are not valid with native SQL.

If you are using a heterogeneous FedSQL connection (a FedSQL connection string that specifies multiple connection strings) and two or more of those connection strings specify connections to multi-catalog data sources, then you might need to use a catalog name map to resolve conflicts between those two multi-catalog sources. If you encounter this situation, you might see this type of error during the connection:

ERROR: Duplicate catalog name, <catalog name>, encountered in connection string, DSN or file DSN

This means that the same <catalog name> exists in more than one data source. A mapping is required for one of the connections to map the conflicting name to an arbitrary name that can be used to refer to the database.

Amazon Aurora Driver Reference

Connection String

A SAS Micro Analytic Service deployment includes a driver that can connect to an Amazon Aurora database. To establish a connection, you must specify a valid connection string.

Here is a sample Amazon Aurora connection string:

```
DRIVER=SQL; CONOPTS=(DRIVER=POSTGRES; CATALOG=catalog-identifier; USER=user-name;
PWD=password; SERVER=server-name; PORT=port-number; DB=database-name)
```

Note the following information:

- The DS2 SQLSTMT package supports the FedSQL dialect. Therefore, the connection string must begin with DRIVER=SQL with the FedSQL language driver as the managing driver. For more information, see [“About the Connection String Configuration” on page 318](#).
- Although this sample connection string might appear on multiple lines, you should always enter a connection string as a single line.
- You specify the connection string in SAS Environment Manager using the connectionstring property. For information, see [“sas.microanalyticservice.properties: core” on page 271](#).

Connection Options

For Amazon Aurora driver connection options, see [“Connection Options” on page 347](#).

Amazon Redshift Driver Reference

Connection String

A SAS Micro Analytic Service deployment includes a driver that you can use to connect to an Amazon Redshift database. To connect, you must specify a valid connection string.

Here is a sample Amazon Redshift connection string:

```
DRIVER=SQL; CONOPTS=(DRIVER=REDSHIFT; UID=MyUID; PWD=MyPW;  
SERVER=MyHost; PORT=3306; DB=MyDB; CATALOG=MyCatalog)
```

Note the following information:

- The DS2 SQLSTMT package supports the FedSQL dialect. Therefore, the connection string must begin with DRIVER=SQL with the FedSQL language driver as the managing driver. For more information, see [“About the Connection String Configuration” on page 318](#).
- Although this sample connection string might appear on multiple lines, you should always enter a connection string as a single line.
- You specify the connection string in SAS Environment Manager using the connectionstring property. For information, see [“sas.microanalyticsservice.properties: core” on page 271](#).

Connection Options

Here are the Amazon Redshift connection options:

Note: The [“Additional Database Connection Options”](#) also apply for this database type.

Option	Description
ALLOW_UNQUOTED_NAMES	<code>ALLOW_UNQUOTED_NAMES=YES NO;</code> Specifies whether to enclose table and column names in quotation marks. ■ YES: Specifies that table and column names are not quoted. This allows tables and columns to be created in the default lowercase. ■ NO: Specifies that tables and columns are quoted. Default: NO
CATALOG	<code>CATALOG=catalog-identifier;</code> Specifies an arbitrary identifier for an SQL catalog, which groups schemas that are logically related. You must specify a catalog. For more information, see “About Using Database Catalogs” on page 319.
CLIENT_ENCODING	<code>CLIENT_ENCODING=encoding-value;</code> Specifies the character set encoding value for the client data, if it is different from the session encoding.
DB	<code>DB=database-name;</code> Specifies the name of the Redshift database. Enclose the database name in single quotation marks if it contains spaces or non-alphanumeric characters.
DRIVER	<code>DRIVER=REDSHIFT;</code> Specifies the name of the data source driver. You must specify a driver.
DSN	<code>DSN=data-source-identifier;</code> Specifies a configured Redshift Data Source Name (DSN) to which you want to connect. Use this option if you have existing Redshift ODBC data sources that are configured on your client. This method requires additional setup. This extra setup is done through the <code>odbc.ini</code> file or a similarly named configuration file. Therefore, it is recommended that you use this connection method only if you have existing, functioning data sources that have been specified.
MAX_ACTIVE_STATEMENTS	<code>MAX_ACTIVE_STATEMENTS=integer-value;</code> Specifies the default number of active statements in a connection handle. Default: 1
MAX_BINARY_LEN	<code>MAX_BINARY_LEN=value;</code> Specifies a value, in bytes, that limits the length of long binary fields (LONG VARBINARY). Unlike other databases, Redshift does not have a size limit for long binary fields. Default: 134200000

Option	Description
MAX_CHAR_LEN	<code>MAX_CHAR_LEN=value;</code> Specifies a value, in characters, that limits the length of character fields (CHAR and VARCHAR). Default: 65535
MAX_TEXT_LEN	<code>MAX_TEXT_LEN=value;</code> Specifies a value, in characters, that limits the length of long character fields (LONG VARCHAR). Default: 65535
max_t	<code>MAXVARCHARSIZE=value;</code> Specifies a value, in bytes, that limits the length of long binary fields (LONG VARBINARY). Unlike other databases, Redshift does not have a size limit for long binary fields. Default: 134200000
NUMBYTESPERCHAR	<code>NUMBYTESPERCHAR=maximum-client-bytes;</code> Specifies the maximum number of bytes per single character in the database client encoding, which matches the SAS encoding. Maximum-client-bytes is the multiplying factor to use to adjust column lengths for STRING columns. When a SAS UTF-8 encoding is used, the length of STRING variables is four times the length of the default character length, or four times the length that is specified in the <code>MAX_CHAR_LEN=</code> connection option. Therefore, when <code>MAX_CHAR_LEN=8</code> is specified, a length of 32 bytes of storage is used. To actually get 8 bytes of storage, specify <code>MAX_CHAR_LEN=8</code> and <code>NUMBYTESPERCHAR=1</code>.
PORT	<code>PORT=port_number;</code> Specifies the port number that is used to connect to the specified Redshift server. Default: 5432
PWD	<code>PWD=password;</code> Specifies the password associated with the user ID. Enclose password in single quotation marks if it contains spaces or non-alphanumeric characters. You can also specify <code>PASSWORD=</code> with the <code>PWD=</code>, <code>PASS=</code>, and <code>PW=</code> aliases.
SCHEMA	<code>SCHEMA=value;</code> Specifies the default schema for the connection. If not specified, the schema, or list of schemas, is determined based on the value of the schema search path that is defined on the database server.
SERVER	<code>SERVER='server-name';</code>

Option	Description
	Specifies the server name or IP address of the Redshift server to which you want to connect. Enclose the server name in single quotation marks if the name contains spaces or non-alphanumeric characters.
SSLMODE	SSLMODE=allow disable prefer require verify-ca verify-full; Determines whether or with what priority a secure SSL TCP/IP connection is negotiated with the Redshift server. ■ allow: Specifies that encryption is not required, but encryption is accepted if the DBMS server requires it. ■ disable: Specifies that encryption is not used when connecting to the database. ■ prefer: Specifies that encryption is not required, but encryption should be used if it is available. ■ require: Specifies that encryption is required when connecting to the database ■ verify-ca: Specifies that encryption is required, and an authentication certificate might be required. Whether an authentication certificate is required depends on your database policy. ■ verify-full: Specifies that encryption and an authentication certificate are required. Default: prefer Note: Values are case sensitive. The first letter for each SSLMODE value must be specified in lowercase.
STRIP_BLANKS	STRIP_BLANKS=YES NO; Specifies whether to strip blanks from character fields.
USE_DECLARE_FETCH	USE_DECLARE_FETCH=YES NO; Specifies whether the Redshift attribute UseDeclareFetch=1 is enabled when connecting to the Redshift database. ■ YES: Correlates to UseDeclareFetch=1. Specifies that the result rows from the command cursor are returned to the application one at a time, as they are received from the server. ■ NO: Correlates to UseDeclareFetch=0. Specifies that the entire result set from the cursor is pushed to client memory before the first fetch is returned. This can be unworkable for commands that return a large number of rows. DEFAULT: YES
UID	USER=user-ID; Specifies the Redshift user name (also called the userID) that you use to connect to your database.

For more information, see [“SAS/ACCESS Interface to Amazon Redshift” in SAS/ACCESS for Relational Databases: Reference](#).

DB2 Driver Reference

Prerequisites

For SAS Micro Analytic Service to access a DB2 database, customizations must be performed by the Kubernetes administrator when deploying SAS Micro Analytic Service. Most notably, a customer-provided DB2 client environment that is accessible from a persistent volume is required.

For complete configuration information, see the README.md file that is located in the `$deploy/sas-bases/examples/sas-microanalytic-score/db2-config` directory where you downloaded your files for deployment.

Connection String

Here is a sample DB2 connection string:

```
DRIVER=sql; CONOPTS=(DRIVER=DB2; CATALOG=MyCatalog; UID=MyUID; PWD=MyPW;  
CONOPTS=(HOSTNAME=MyHost; DATABASE=myDB; PROTOCOL=tcip; SERVICENAME=50000;  
AUTHENTICATION=SERVER))
```

Note the following information:

- The DS2 SQLSTMT package supports the FedSQL dialect. Therefore, the connection string must begin with DRIVER=SQL with the FedSQL language driver as the managing driver. For more information, see [“About the Connection String Configuration” on page 318](#).
- Although this sample connection string might appear on multiple lines, you should always enter a connection string as a single line.
- You specify the connection string in SAS Environment Manager using the connectionstring property. For information, see [“sas.microanalyticsservice.properties: core” on page 271](#).

Connection Options

Here are the DB2 connection options:

Argument	Description
AUTHENTICATION	<code>AUTHENTICATION=SERVER CLIENT SERVER_ENCRYPT KERBEROS GSSPLUGIN DATA_ENCRYPT;</code> Specifies the type of authentication that is used to validate user credentials. ■ SERVER: The server performs the authentication. ■ CLIENT: The client side performs the authentication before the connection is made. ■ SERVER_ENCRYPT: The credentials are encrypted on the client side before being sent to the server for authentication. ■ KERBEROS: Kerberos is used for authentication. ■ GSSPLUGIN: A Generic Security Services (GSS) plug-in handles authentication between the client and server. ■ DATA_ENCRYPT: In addition to encrypting the credentials, the data that is exchanged between the client and server is also authenticated.
CATALOG	<code>CATALOG=catalog-identifier;</code> Specifies an arbitrary identifier for a SQL catalog, which groups schemas that are logically related. You must specify a catalog. For more information, see “About Using Database Catalogs” on page 319.
CLIENT_ENCODING	<code>CLIENT_ENCODING=encoding-value;</code> Specifies the character set encoding value for the client data, if it is different from the session encoding. For DB2, specify the encoding of the DB2CODEPAGE to the DB2 driver. When using this option, you must also first set the DB2CODEPAGE environment variable on the client. Note the following information: ■ Changing the value of DB2CODEPAGE affects all applications that run on that system. After your request is complete, you should reset the DB2CODEPAGE environment variable to its usual value, which was derived when the database was created. ■ Setting the DB2CODEPAGE value or the CLIENT_ENCODING value incorrectly can cause unpredictable results.
CONOPTS	<code>CONOPTS=ODBC-compliant connection-string;</code> Specifies ODBC-compliant connection options using ODBC-style syntax. The options are passed to the driver for processing.
DATABASE	<code>DATABASE=database-name;</code> Specifies the name of the DB2 database. Enclose the database name in single quotation marks if it contains spaces or non-alphanumeric characters.
DEFAULT_CURSOR_TYPE	<code>DEFAULT_CURSOR_TYPE=DYNAMIC FORWARD_ONLY KEYSET STATIC;</code> Specifies the cursor type to use for statements.

Argument	Description
	By default, either the application-supplied or the DB2 client internal default cursor type is used. Specify a cursor type with this option if you know that a particular cursor type works more efficiently.
DRIVER	DRIVER=DB2; Specifies the name of the data source driver.
HOSTNAME	HOSTNAME=host-name; Specifies the host name or IP address of the DB2 server.
NUMBYTESPERCHAR	NUMBYTESPERCHAR=maximum-client-bytes; Specifies the maximum number of bytes per single character in the database client encoding, which matches the SAS encoding. Maximum-client-bytes is the multiplying factor to use to adjust column lengths for STRING columns. When a SAS UTF-8 encoding is used, the length of STRING variables is four times the length of the default character length, or four times the length that is specified for the MAX_CHAR_LEN option. Therefore, when MAX_CHAR_LEN=8 is specified, a length of 32 bytes of storage is used. To actually get 8 bytes of storage, specify MAX_CHAR_LEN=8 and NUMBYTESPERCHAR=1.
INVALID_DATE	INVALID_DATE=DEFAULT ERROR SILENT WARN; By default, the driver does not validate the year portion of DATE or TIMESTAMP data to ensure they fall within the SAS date ranges. If INVALID_DATE is set to any other value than DEFAULT, then the driver scans all DATE and TIMESTAMP column data to validate the year values. If an invalid date is found, then the driver first sets the invalid DATE or TIMESTAMP value to NULL, and then either returns no message (SILENT), returns a warning (WARN) for each invalid value, or returns an error (ERROR) for the entire row.
PROTOCOL	PROTOCOL=protocol; Specifies the communication protocol. For remote connections, this is typically TCPIP.
PWD	PWD=password; Specifies the password associated with the user ID. Enclose password in single quotation marks if it contains spaces or non-alphanumeric characters.
SERVICENAME	SERVICENAME=service-name; Specifies the service name that maps to the port number on which the DB2 server is listening.
STRIP_BLANKS	STRIP_BLANKS=YES NO;

Argument	Description
	Specifies whether to strip blanks from character fields.
UID	UID= <i>user-id</i> ; Specifies the DB2 user name that you use to connect to your database.

Google BigQuery Driver Reference

Connection String

A SAS Micro Analytic Service deployment includes a driver that can connect to a Google BigQuery database. To establish a connection to a Google BigQuery database, you must specify a valid connection string.

Here is a sample Google BigQuery connection string:

```
DRIVER=SQL; CONOPTS=(DRIVER=BIGQUERY; PROJECT='project1';
CLIENT_ID="919191919191-abababa5ab96so8v9o298np4tg0la1md.apps.googleusercontent.com";
CLIENT_SECRET=O8XKYBEDfZdX_1BAbABab9AB; REFRESH_TOKEN="1//0dJj ...
13-7cOoDxCr0p9u6k3_jl_i-Y31hdZ4FOumSO0"; SCHEMA=Schema1; BULKLOAD=Yes)
```

Note the following information:

- The DS2 SQLSTMT package supports the FedSQL dialect. Therefore, the connection string must begin with DRIVER=SQL with the FedSQL language driver as the managing driver. For more information, see [“About the Connection String Configuration” on page 318](#).
- Although this sample connection string might appear on multiple lines, you should always enter a connection string as a single line.
- You specify the connection string in SAS Environment Manager using the connectionstring property. For information, see [“sas.microanalyticservice.properties: core” on page 271](#).

IMPORTANT To connect to a Google BigQuery database, you must use OAuth for authentication. For more information, see [“Utility to Obtain an Authentication Token” in SAS/ACCESS for Relational Databases: Reference](#).

IMPORTANT Using SAS Micro Analytic Service, you can perform only Query and Insert operations to a Google BigQuery database. Update and Delete operations are not supported.

Connection Options

Here are the Google BigQuery connection options:

Note: The ["Additional Database Connection Options"](#) also apply for this database type.

Option	Description
ALLOW_LARGE_RESULTS	<code>ALLOW_LARGE_RESULTS=ALWAYS MUST OFF;</code> ■ ALWAYS: Specifies that all query results are written to a temporary storage table, including query results that are smaller than the maximum response size. ■ MUST: Specifies that query results that are larger than the maximum response size are written to a temporary stored table. ■ OFF: Specifies that query results that are larger than the maximum response size are allowed to fail and return an error in the SAS log, and thus the query fails. Default: OFF When ALWAYS or MUST is assigned, you can use the <code>LARGE_RESULTS_DATASET=</code> and <code>LARGE_RESULTS_EXPIRATION=</code> options to control the data set ID and expiration date of the temporary storage table. By default, the data set ID is <code>_sasbq_temp_tables</code> and the table is allowed to exist for 24 hours.
BULKLOAD	<code>BULKLOAD=YES NO;</code> When set to YES, inserts row data using the Google BigQuery bulk-loading interfaces. Default: NO
BULKOPTS	<code>BULKOPTS=(bulk-load-option=value [...bulk-load-option=value]);</code> Specifies one or more bulk load table options, separated by spaces. Bulk load option-value pairs are separated by spaces. You must use <code>BULKLOAD=YES</code> in order to use <code>BULKOPTS=</code>. Here are the <code>BULKOPTS=</code> options: ■ <code>BL_DELETE_DATA_FILE= YES NO</code> When set to YES, the data files that are generated for the load process are deleted after the job completes. Default: YES ■ <code>BL_DEFAULT_DIR='path-to-directory'</code> Specifies the temporary directory to which the data files are written as part of the bulk load process. Default: SAS utility directory

Option	Description
	■ <code>BL_DELIMITER='single-ascii-character octal-representation'</code> Specifies the character to separate column data values in the generated data file. You can use <code>BL_DELIMITER=</code> to specify another delimiter character if your data contains the default delimiter character. Default: <code>0x07 (\007)</code>
CATALOG	<code>CATALOG=catalog-name-map;</code> For a multi-catalog DBMS such as Google BigQuery, the CATALOG option is needed only in a heterogeneous FedSQL connection string to resolve catalog collisions. For more information, see “About Using Database Catalogs” on page 319. By default, Google BigQuery connects with <code>catalog=*</code>. This includes the list of projects that are available for the user as well as <code>bigquery-public-data</code>.
CLIENT_ID	<code>CLIENT_ID="ID-value";</code> Specifies an OAuth client ID. <code>CLIENT_ID=</code> is used with the <code>REFRESH_TOKEN=</code> and <code>CLIENT_SECRET=</code> options. Encoded values are supported.
CLIENT_SECRET	<code>CLIENT_SECRET=authentication-secret;</code> Specifies the OAuth client secret. Encoded values are supported.
CRED_PATH	<code>CRED_PATH='path-and-filename';</code> Specifies the location of a credential file that enables authentication to Google Cloud Platform. For more information about the credential file, see the Getting Started information in the Google Cloud documentation. Note the following information: ■ You must specify the value for <code>CRED_PATH=</code> within single quotation marks (<code>'</code>). Do not use double quotation marks (<code>"</code>) for this option. ■ Encoded values are supported. ■ You can use the <code>GOOGLE_APPLICATION_CREDENTIALS</code> environment variable to specify credentials instead of <code>CRED_PATH=</code>.
DRIVER	<code>DRIVER=BIGQUERY;</code> Specifies the name of the data source driver. You must specify the driver.
KMS_KEY	<code>KMS_KEY='string-representation-of-key';</code> Specifies the string that represents the key in the Google Cloud Key Management Service that should be used for encryption.
LARGE_RESULTS_DATASET	<code>LARGE_RESULTS_DATASET='Google-BigQuery-data-set-ID';</code> Specifies a data set ID for query results. If the specified data set ID does not exist, it is created. Default: <code>_sasbq_temp_tables</code>

Option	Description
LARGE_RESULTS_EXPIRATION_TIME	<code>LARGE_RESULTS_EXPIRATION_TIME=milliseconds;</code> Specifies the amount of time, in milliseconds, before the large result table is deleted. Range: 1 (does not expire); 3600000–2147483647 milliseconds. Default: 86400000 (24 hours)
LOCATION	<code>LOCATION='region-name';</code> Specifies the Google BigQuery data set location if it is different from the default for the project. This can be a specific region name or a multi-region name.
MAX_BINARY_LEN	<code>MAX_BINARY_LEN=value;</code> Specifies the maximum length in bytes of a binary column. The value that you specify must be greater than 0. Default: 2000 bytes
MAX_CHAR_LEN	<code>MAX_CHAR_LEN=value;</code> Specifies the maximum number of characters to allocate for Google BigQuery string columns. Use this option when you work with string types in Google BigQuery. By default, in Google BigQuery strings do not have a fixed length. However, SAS requires a specified length for these values. Specify a length that is long enough to contain all of the string data that you are working with to avoid truncation of string values. Note: Use <code>NUMBYTESPERCHAR=</code> with <code>MAX_CHAR_LEN=</code> when the client encoding is UTF-8. Default: 16,384 characters
NUMBYTESPERCHAR	<code>NUMBYTESPERCHAR=maximum-client-bytes;</code> Specifies the maximum number of bytes per single character in the database client encoding, which matches the SAS encoding. Maximum-client-bytes is the multiplying factor to use to adjust column lengths for <code>STRING</code> columns. In Google BigQuery, when a SAS UTF-8 encoding is used, the length of <code>STRING</code> variables are four times the length of the default character length, or four times the length that is specified in the <code>MAX_CHAR_LEN=</code> option. Therefore, when <code>MAX_CHAR_LEN=8</code> is specified, a length of 32 bytes of storage is used. To actually get 8 bytes of storage, specify <code>MAX_CHAR_LEN=8</code> and <code>NUMBYTESPERCHAR=1</code>.
PROJECT	<code>PROJECT="project-ID";</code> Specifies the project ID for a Google Cloud Platform project.
PROXY	<code>PROXY="URL";</code> Specifies the URL of a proxy server that is used to connect to Google BigQuery.

Option	Description
QUALIFIER	QUALIFIER='project-ID' ; Specifies the current project for the connection. Though the PROJECT= option is used as the connected project, the project that is specified by the QUALIFIER= option is used when generating SQL queries and qualifying table names. This is useful when you want to query data from the Google BigQuery public project, bigquery-public-data.
READMODE	READMODE=STANDARD STORAGE Specifies the method to use when moving data from Google BigQuery. STANDARD uses the default SAS/ACCESS method. STORAGE uses the BigQuery Storage Read API. Default: STANDARD Note: The STORAGE option typically processes data faster than the STANDARD option.
REFRESH_TOKEN	REFRESH_TOKEN="value" ; Specifies a refresh token for connecting to Google BigQuery. Encoded values are supported. For more information, see "Utility to Obtain an Authentication Token" in SAS/ACCESS for Relational Databases: Reference.
SCANSTRINGCOLUMNS	SCANSTRINGCOLUMNS=NO YES ; Specifies whether to determine the maximum length of VARCHAR columns in a database table or a query. ■ YES: Specifies to scan all VARCHAR columns in a database table to determine the actual maximum length of the columns before loading data. If the VARCHAR columns have been created with the maximum VARCHAR precision, then using this option can reduce the size of the resulting table and can accelerate the loading process. This can result in a performance improvement for large tables. A table is considered to be large if it contains a large number of rows, a large number of VARCHAR columns, or both. ■ NO: Specifies that VARCHAR columns are not scanned before loading data. Default: NO Note: Use SCANSTRINGCOLUMNS= or MAX_CHAR_LEN= (and NUMBYTESPERCHAR=).
SCHEMA	SCHEMA=value; Specifies a Google BigQuery data set name. A fully qualified BigQuery table name has the form: project.dataset.table. Note: A schema value is required in all connections.
STRIP_BLANKS	STRIP_BLANKS= YES NO;

Option	Description
	Specifies whether to remove trailing blanks from fixed-length CHARACTER buffers. By default, CHAR buffers are blank padded to the column size. Using <code>STRIP_BLANKS=YES</code> removes those trailing blanks from the buffers and updates the returned length accordingly.
<code>USE_INFORMATION_SCHEMA</code>	<code>USE_INFORMATION_SCHEMA=YES NO;</code> Specifies whether to retrieve a list of schemas and tables or views. ■ YES: Specifies that SAS/ACCESS to Google BigQuery uses <code>INFORMATION_SCHEMA</code> to get metadata for the list of schemas and tables or views, if the names contain a pattern. Names with patterns can be searched using the wildcard characters <code>%</code> or <code>_</code>. If no pattern is requested, SAS/ACCESS uses other API calls to get object lists. ■ NO: Disables <code>INFORMATION_SCHEMA</code> to improve performance. Specify <code>USE_INFORMATION_SCHEMA=NO</code> if you are not using patterns to query schema or table names. Default: YES

`PROJECT=` and `SCHEMA=` are required connection options. Authentication through one of the following options is also required:

- The `GOOGLE_APPLICATION_CREDENTIALS` environment variable.
- The `CRED_PATH=` option.
- The `CLIENT_ID=`, `CLIENT_SECRET=`, and `REFRESH_TOKEN=` options.

For more information, see [“SAS/ACCESS Interface to Google BigQuery” in SAS/ACCESS for Relational Databases: Reference](#).

Microsoft Azure SQL Database Driver Reference

Connection String

A SAS Micro Analytic Service deployment includes drivers that can connect to a Microsoft Azure SQL database. To establish a connection to a Microsoft Azure SQL database, you must specify a valid connection string.

Here are some sample Microsoft Azure SQL connection strings that specify a user name and password for authentication:

```
DRIVER=SQL; CONOPTS=(DRIVER=MSSQLSVR; ODBC_DSN=MyDSN; UID=MyUID; PWD=MyPW)
```

```
DRIVER=SQL; CONOPTS=(DRIVER=MSSQLSVR; CONOPTS=(DRIVER=SAS ACCESS to MS SQL Server;  
DB=MyDB; UID=MyUID; PWD=MyPW; HOST=MyHost.sas.com; PORT=1433; ENABLE_MARS=Yes))
```

Here is a sample Microsoft Azure SQL connection string that uses a service principal for authentication:

```
DRIVER=SQL; CONOPTS=(DRIVER=MSSQLSVR; CONOPTS=(DRIVER=SAS ACCESS to MS  
SQL Server; DB=MyDB; AuthenticationMethod=36; LOGINID=MyClientID;  
PASSWORD=MyPW; HOST=MyHost.sas.com; PORT=1433; SSLLIBNAME=/usr/lib64/  
libssl.so.1.1;  
CRYPTOLIBNAME=/usr/lib64/libcrypto.so.1.1;  
TRUSTSTORE=/opt/sas/viya/home/SASSecurityCertificateFramework/tls/  
certs/ca-bundle.pem; ValidateServerCertificate=1))
```

Here is a sample Microsoft Azure SQL connection string that uses an authentication domain for authentication:

```
DRIVER=SQL; AUTHDOMAIN=MyAuthDomain; CONOPTS=(DRIVER=MSSQLSVR;  
ENABLE_MARS=Yes; CONOPTS=(DRIVER=SAS ACCESS to MS SQLServer; DB=MyDB;  
HOST=MyHost.sas.com; PORT=1433))
```

Note the following information:

- Using an authentication domain requires addition configuration. For more information, see [“Configuring an Authentication Domain” on page 260](#).
- The DS2 SQLSTMT package supports the FedSQL dialect. Therefore, the connection string must begin with DRIVER=SQL with the FedSQL language driver as the managing driver. For more information, see [“About the Connection String Configuration” on page 318](#).
- If your connection string specifies more than one database connection, and the credentials for each database are the same, you can include the AUTHDOMAIN option at the level of the FedSQL driver. The FedSQL driver passes the credentials for each database connection that is defined under it. (The example connection string above that uses an authentication domain shows this configuration.)
- Although this sample connection string might appear on multiple lines, you should always enter a connection string as a single line.
- You specify the connection string in SAS Environment Manager using the connectionstring property. For information, see [“sas.microanalyticsservice.properties: core” on page 271](#).

Connection Options

Here are the Microsoft Azure SQL connection options:

Option	Description
APP_NAME	APP_NAME=application-name;

Option	Description
	Specifies the name of the application that is connecting to the Microsoft Azure SQL Server data source. If you specify a space in the name, enclose the name in single or double quotation marks. If you do not need to specify an application name, specify <code>APP_NAME=""</code> (empty string). Default: SAS
AUTHENTICATIONMETHOD	<code>AUTHENTICATIONMETHOD=method;</code> Specifies the method that the driver uses to authenticate the user when a connection is established. If the specified authentication method is not supported by the database server, the connection fails and the driver generates an error. Note: To use a service principal for authentication, this must be set to 37 (Active Directory Manged Identity). The driver authenticates using a managed identity when accessing Azure resources. All communications between the service are encrypted using TLS/SSL.
AUTHDOMAIN	<code>AUTHDOMAIN=auth-domain-name;</code> Specifies an authentication domain to use for authentication.
AUTHSCOPE	<code>AUTHSCOPE="scope";</code> Specifies the configured resource for authenticating and authorizing access to Microsoft Azure. Use this option to override the default Microsoft Azure authentication method. Enclose the scope value in single or double quotation marks. Default: <code>https://database.windows.net/user_impersonation</code>
CATALOG	<code>CATALOG= * catalog-name-map;</code> For a multi-catalog DBMS, the CATALOG option is needed only in a heterogeneous FedSQL connection string to resolve catalog collisions. For more information, see “About Using Database Catalogs” on page 319. Default: *
CONOPTS	<code>CONOPTS=(ODBC-compliant-connection-string);</code> Specifies ODBC-compliant connection options using ODBC-style syntax. The options are passed to an underlying driver for processing.
CRYPTOLIBNAME	<code>CRYPTOLIBNAME=path-to-library-file;</code> Specifies the absolute path to the OpenSSL library file that contains the cryptographic library to be used by the connection.
DB	<code>DB=database-name;</code>

Option	Description
	Specifies the name of the Azure SQL database. If the database name contains spaces or nonalphanumeric characters, you must enclose it in quotation marks.
DEFAULT_CURSOR_TYPE	DEFAULT_CURSOR_TYPE=FORWARD_ONLY KEYSET DYNAMIC STATIC; Specifies the cursor type to use for statements. By default, the driver default cursor type is used. Specify a cursor type with this option if you know that a particular cursor type works more efficiently.
DM_UNICODE	DM_UNICODE= YES NO; Specifies which Unicode setting (UTF-8, UTF-16) the ODBC Driver Manager uses. By default, the driver assumes that the driver manager uses UTF-8 for Unicode ODBC API calls. If you use a different driver manager (such as unixODBC) that uses a different Unicode encoding, then specify it with this option.
DRIVER	DRIVER=MSSQLSVR SAS ACCESS to MS SQL Server; Specifies the name of the data source driver. You must specify the driver.
ENABLE_MARS	ENABLE_MARS=YES; Specifies support for Multiple Active Result Sets (MARS). For more information, see “Disabling Transaction Autocommit” on page 338.
ENABLEQUOTED IDENTIFIERS	EnableQuotedIdentifiers=1 0; Specifies whether the driver allows the use of quoted identifiers. If set to 1 (enabled), the database enforces ANSI rules regarding quotation marks. Double quotation marks can be used only for identifiers, such as column and table names.
HOST	HOST=host-name; Specifies the host name of the database server.
INVALID_DATE	INVALID_DATE=WARN ERROR SILENT DEFAULT; By default, the TKTS ODBC driver does not validate the year portion of DATE or TIMESTAMP data to ensure they fall within the SAS date ranges. If INVALID_DATE is set to any other value than DEFAULT, then the driver scans all DATE and TIMESTAMP column data to validate the year values. If an invalid date is found, then the driver first sets the invalid DATE or TIMESTAMP value to NULL, and then either returns no message (SILENT), returns a warning (WARN) for each invalid value, or returns an error (ERROR) for the entire row.
LOGONID	LOGONID=client-ID; Specifies the Microsoft Entra ID (or Azure Active Directory) account to use to sign in to the Azure resource.
NUMBYTESPERCHAR	NUMBYTESPERCHAR=value;

Option	Description
NUMBYTESPERCHAR	Specifies a nonzero positive integer to use as the number of bytes per character for character data. The underlying driver returns a specific column size value for each character column. If a NUMBYTESPERCHAR value is specified, then this column size is divided by the NUMBYTESPERCHAR value and the result is used as the column size. This is useful for data sources that are byte-based and have columns that contain variable-length encodings, such as UTF-8. If those UTF-8 columns contained only single-byte UTF-8 characters, then using NUMBYTESPERCHAR=1 can reduce the overall memory needed to read a particular table.
ODBC_DSN	ODBC_DSN=<i>data-source-name</i>; Specifies a configured ODBC Data Source Name (DSN) to which you want to connect. Use this option if you have existing Microsoft SQL Server ODBC data sources that are configured on your client. This method requires additional setup. This extra setup is done through the <code>odbc.ini</code> file or a similarly named configuration file.
PORT	PORT=<i>port-number</i>; Specifies the port number on which the database runs.
PWD	PWD=<i>password</i>; Specifies the password associated with the UID. Enclose the password in single quotation marks if it contains spaces or non-alphanumeric characters.
SSLLIBNAME	SSLLibName=<i>SSL/TLS-library-location</i>; Specifies the absolute path to the file that contains the SSL library to be used by the data source or connection when SSL is enabled.
STRIP_BLANKS	STRIP_BLANKS=YES NO; Specifies whether to remove trailing blanks from fixed-length CHARACTER buffers. By default, CHAR buffers are blank padded to the column size. Using STRIP_BLANKS=YES removes those trailing blanks from the buffers and update the returned length accordingly.
TRUSTSTORE	TrustStore=<i>location-or-contents-of-truststore-file</i>; Specifies either the path and file name of the truststore file or the contents of the TLS/SSL certificates to be used when TLS/SSL is enabled and server authentication is used.
UID	UID=<i>user-ID</i>; Specifies the user name (also called the user ID) that you use to connect to your database. If the user name contains spaces or nonalphanumeric characters, you must enclose it in quotation marks.

Option	Description
VALIDATESERVERCERTIFICATE	<code>ValidateServerCertificate=1 0;</code> Specifies whether the driver validates the certificate that is sent by the database server when SSL encryption is enabled. If set to 1 (enabled), the driver validates the certificate that is sent by the database server. If set to 0 (disabled), the driver does not validate the certificate that is sent by the database server.

For more information, see [“SAS/ACCESS Interface to Microsoft SQL Server”](#) in [SAS/ACCESS for Relational Databases: Reference](#).

Disabling Transaction Autocommit

By default, in a DS2 SQLSTMT package, any outstanding changes to the database are committed at the end of each SQL statement execute method. This feature is known as *autocommit*.

To potentially improve performance, you might want to execute multiple SQL statements within a single transaction. To support this behavior, SAS Micro Analytic Service includes a property that enables you to disable autocommit. For more information, see [“Committing Transactions”](#) on page 254.

If you disable autocommit and use a Microsoft SQL database, you must set `ENABLE_MARS=YES` in your connection string. This setting enables SAS Micro Analytic Service to allow transactions under a given Microsoft SQL Server connection.

Microsoft SQL Server Driver Reference

Overview

A SAS Micro Analytic Service deployment includes drivers that enable you to connect to a Microsoft SQL Server. To establish a connection, you must specify a valid connection string. You can use a standard connection string or a DSN-based connection string.

You specify the connection string in SAS Environment Manager using the `connectionstring` property. For information, see [“sas.microanalyticservice.properties: core”](#) on page 271.

Standard Connection String

A standard connection string uses the drivers that are configured in the `odbcinst.ini` file that is installed during the SAS Viya platform deployment.

Here is a sample Microsoft SQL Server standard connection string:

```
DRIVER=SQL; CONOPTS=(DRIVER=MSSQLSVR; CONOPTS=(DRIVER=SAS ACCESS to MS SQL Server;  
DB=MyDB; UID=MyUID; PWD=MyPW; HOST=MyHost.sas.com; PORT=1433; ENABLE_MARS=Yes))
```

Note the following information:

- The DS2 SQLSTMT package supports the FedSQL dialect. Therefore, the connection string must begin with `DRIVER=SQL` with the FedSQL language driver as the managing driver. For more information, see [“About the Connection String Configuration” on page 318](#).
- Although this sample connection string might appear on multiple lines, you should always enter a connection string as a single line.
- You specify the connection string in SAS Environment Manager using the `connectionstring` property. For information, see [“sas.microanalyticservice.properties: core” on page 271](#).

DSN-Based Connection String

A DSN-based connection string uses a DSN (data source name) that is configured in the `odbc.ini` file. The `odbc.ini` file is installed during the SAS Viya platform deployment.

Here is a sample Microsoft SQL Server DSN-based connection string:

```
DRIVER=SQL; CONOPTS=(DRIVER=MSSQLSVR; ODBC_DSN=SQL Server Wire Protocol; UID=MyUID;  
PWD=MyPW; CONOPTS=(HOST=MyHost.sas.com; PORT=1433; ENABLE_MARS=Yes))
```

Note the following information:

- The DS2 SQLSTMT package supports the FedSQL dialect. Therefore, the connection string must begin with `DRIVER=SQL` with the FedSQL language driver as the managing driver. For more information, see [“About the Connection String Configuration” on page 318](#).
- Although this sample connection string might appear on multiple lines, you should always enter a connection string as a single line.
- You specify the connection string in SAS Environment Manager using the `connectionstring` property. For information, see [“sas.microanalyticservice.properties: core” on page 271](#).

Connection Options

Here are the Microsoft SQL Server connection options:

Option	Description
CATALOG	CATALOG= * <i>catalog-name-map</i>; For a multi-catalog DBMS, the CATALOG option is needed only in a heterogeneous FedSQL connection string to resolve catalog collisions. For more information, see “About Using Database Catalogs” on page 319. Default: *
CLIENT_ENCODING	CLIENT_ENCODING=<i>encoding-value</i>; Specifies the character set encoding value for the client data, if it is different from the session encoding.
CONOPTS	CONOPTS= (<i>ODBC-compliant connection-string</i>); Specifies ODBC-compliant connection options using ODBC-style syntax. The options are passed to an underlying driver for processing.
DB	DB=<i>database-name</i>; Specifies the name of the Microsoft SQL Server database. If the database name contains spaces or nonalphanumeric characters, you must enclose it in quotation marks.
DEFAULT_CURSOR_TYPE	DEFAULT_CURSOR_TYPE=FORWARD_ONLY KEYSET DYNAMIC STATIC; Specifies the cursor type to use for statements. By default, the driver default cursor type is used. Specify a cursor type with this option if you know that a particular cursor type works more efficiently.
DM_UNICODE	DM_UNICODE= YES NO; Specifies which Unicode setting (UTF-8, UTF-16) the ODBC Driver Manager uses. By default, the driver assumes that the driver manager uses UTF-8 for Unicode ODBC API calls. If you use a different driver manager (such as unixODBC) that uses a different Unicode encoding, then specify it with this option.
DRIVER	DRIVER=SAS ACCESS to MS SQL Server MSSQLSRVR; Specifies the name of the data source driver. You must specify the driver.
ENABLE_MARS	ENABLE_MARS=YES; Specifies support for Multiple Active Result Sets (MARS). For more information, see “Disabling Transaction Autocommit” on page 338.

Option	Description
HOST	HOST=<i>host-name</i>; Specifies the host name of the database server.
INVALID_DATE	INVALID_DATE=WARN ERROR SILENT DEFAULT; By default, the TKTS ODBC driver does not validate the year portion of DATE or TIMESTAMP data to ensure they fall within the SAS date ranges. If INVALID_DATE is set to any other value than DEFAULT, then the driver scans all DATE and TIMESTAMP column data to validate the year values. If an invalid date is found, then the driver first sets the invalid DATE or TIMESTAMP value to NULL, and then either returns no message (SILENT), returns a warning (WARN) for each invalid value, or returns an error (ERROR) for the entire row. This is useful for data sources that are byte-based and have columns that contain variable-length encodings, such as UTF-8. If those UTF-8 columns contained only single-byte UTF-8 characters, then using NUMBYTESPERCHAR=1 can reduce the overall memory needed to read a particular table.
NUMBYTESPERCHAR	NUMBYTESPERCHAR=value; Specifies a nonzero positive integer to use as the number of bytes per character for character data. The underlying driver returns a specific column size value for each character column. If a NUMBYTESPERCHAR value is specified, then this column size is divided by the NUMBYTESPERCHAR value and the result is used as the column size.
ODBC_DSN	ODBC_DSN=SQL Server Wire Protocol; Specifies a configured ODBC Data Source Name (DSN) to which you want to connect. Use this option if you have existing Microsoft SQL Server ODBC data sources that are configured on your client. This method requires additional setup. This extra setup is done through the odbc.ini file.
PWD	PWD=<i>password</i>; Specifies the password associated with the user ID. Enclose password in single quotation marks if it contains spaces or non-alphanumeric characters.
PORT	PORT=<i>port-number</i>; Specifies the port number on which the database runs.
STRIP_BLANKS	STRIP_BLANKS=YES NO; Specifies whether to remove trailing blanks from fixed-length CHARACTER buffers. By default, CHAR buffers are blank padded to the column size. Using STRIP_BLANKS=YES removes those trailing blanks from the buffers and updates the returned length accordingly.
UID	UID=<i>user-id</i>;

Option	Description
	Specifies the user name (also called the user ID) that you use to connect to your database. If the user name contains spaces or nonalphanumeric characters, you must enclose it in quotation marks.

For more information, see [“SAS/ACCESS Interface to Microsoft SQL Server” in SAS/ACCESS for Relational Databases: Reference](#).

Disabling Transaction Autocommit

By default, in a DS2 SQLSTMT package, any outstanding changes to the database are committed at the end of each SQL statement execute method. This feature is known as *autocommit*.

To potentially improve performance, you might want to execute multiple SQL statements within a single transaction. To support this behavior, SAS Micro Analytic Service includes a property that enables you to disable autocommit. For more information, see [“Committing Transactions” on page 254](#).

If you disable autocommit and use a Microsoft SQL database, you must set `ENABLE_MARS=YES` in your connection string. This setting enables SAS Micro Analytic Service to allow transactions under a given Microsoft SQL Server connection.

Oracle Driver Reference

Connection String

A SAS Micro Analytic Service deployment includes the Oracle Instant Client driver. To connect, you must specify a valid connection string.

Here is a sample Oracle connection string:

```
DRIVER=SQL; CONOPTS=(DRIVER=ORACLE; CATALOG=oracle1; UID=MyUID; PWD=MyPW;
PATH=server.example.com:1521/myoradatabase)
```

Note the following information:

- The DS2 SQLSTMT package supports the FedSQL dialect. Therefore, the connection string must begin with `DRIVER=SQL` with the FedSQL language driver as the managing driver. For more information, see [“About the Connection String Configuration” on page 318](#).

- Although this sample connection string might appear on multiple lines, you should always enter a connection string as a single line.
- You specify the connection string in SAS Environment Manager using the `connectionstring` property. For information, see [“sas.microanalyticservice.properties: core” on page 271](#).

Connection Options

Here are the Oracle connection options:

Note: The [“Additional Database Connection Options”](#) also apply for this database type.

Option	Description
CATALOG	<code>CATALOG=catalog-identifier;</code> Specifies an arbitrary identifier for an SQL catalog, which groups logically related schemas. You must specify a catalog. For more information, see “About Using Database Catalogs” on page 319.
DRIVER	<code>DRIVER=ORACLE;</code> Specifies the name of the data source driver. You must specify the driver.
FORCE_NCHAR_PARAMETERS	<code>FORCE_NCHAR_PARAMETERS=YES NO;</code> Specifies whether the <code>OCI_ATTR_CHARSET_FORM</code> bind handle attribute is set to <code>SQLCS_NCHAR</code> for character parameter bindings. This setting can significantly impact query performance when using mixed data types. Oracle recommends that the <code>OCI_ATTR_CHARSET_FORM</code> attribute is set to <code>SQLCS_NCHAR</code> when binding character data types. This attribute allows NLS data that is passed in the query to be converted to the server national character set. Setting the <code>OCI_ATTR_CHARSET_FORM</code> attribute for character parameter bindings can prevent the use of indexes on character columns when the column type is not <code>NCHAR</code> or <code>NVARCHAR2</code>. The column indexes are not used when the columns are specified as a parameter in the <code>WHERE</code> clause of a FedSQL parameterized query. Query performance degrades when available indexes are not used. <code>FORCE_NCHAR_PARAMETERS=NO</code> can be set to work around the problem. However, there is a tradeoff. Support for national characters in parameter values depends on the database encoding. When the <code>OCI_ATTR_CHARSET_FORM</code> attribute is not set, insertion of NLS values into a table with <code>NVARCHAR2</code> columns using a parameterized <code>INSERT</code> statement can result in replacement characters in strings that cannot be transcoded to the database encoding. ■ YES: Sets the <code>OCI_ATTR_CHARSET_FORM</code> attribute for character parameter bindings to <code>SQLCS_NCHAR</code>.

Option	Description
	■ NO: Does not set the OCI_ATTR_CHARSET_FORM attribute for character parameter bindings. Default: YES
ORA_ENCODING	ORA_ENCODING=UNICODE; Specifies that strings returned from Oracle are in UNICODE format.
ORNUMERIC	ORANUMERIC=YES NO; Specifies how numbers that are read from or inserted into the Oracle NUMBER column are treated. ■ YES Indicates that non-integer values with explicit precision are treated as NUMERIC values. This option is sufficient for most operations. ■ NO Indicates that the numbers are treated as DOUBLE values. They might not have precision beyond 14 digits. DEFAULT: Yes
ORINTEGER	ORINTEGER=YES NO; Specifies how numbers that are read from or inserted into the Oracle NUMBER column are treated. ■ YES: Oracle NUMBER columns that can safely be converted to integer are reported as INTEGER. ■ NO: Specifies that the numbers are treated as DOUBLE values. This value is sufficient for most operations. Default: NO
PATH	PATH=connect-identifier; Specifies the Oracle connect identifier. A connect identifier can be a net service name, a database service name, or a net service alias. Connect identifiers used in a connection string cannot contain spaces, unless enclosed within single quotation marks or double quotation marks. Here is an example: <pre>path=server.example.com:1521/myoradatabase</pre>
PWD	PWD=password; Specifies an optional Oracle database password that is associated with the Oracle user ID. PWD= is always used with UID= and the associated password is case-sensitive.
SCHEMA	SCHEMA=value; Specifies an Oracle connection schema. If not specified, the schema is a zero-length string.

Option	Description
UID	UID="user-id"; Specifies an Oracle user ID. You must specify the user ID value in uppercase characters and enclose it in double quotation marks. Here is an example: USER="SCOTT".
USE_CACHED_CATALOG	USE_CACHED_CATALOG=YES NO; Specifies whether Oracle should use a cached catalog or recompile catalog on every run. ■ YES: Specifies that Oracle uses a cached catalog. Note: For more information, see the next section “Creating a Materialized View (USE_CACHED_CATALOG)”. ■ NO: Specifies that Oracle recompile the catalog on every run. Default: YES

For more information, see [“SAS/ACCESS Interface to Oracle”](#) in *SAS/ACCESS for Relational Databases: Reference*.

Creating a Materialized View (USE_CACHED_CATALOG)

The following example shows you how to create a materialized view. Use this script if USE_CACHED_CATALOG is set to YES above.

```

/*-----SAS_CACHED_CATALOG.SQL-----*/
/* This script is used to create the materialized and the synonym needed to
   get the ForeignKey metadata. Work with your DBA to set this up.
   Materialized views can be complex and so thorough understanding will help us
   use them effectively. Especially deciding how to do the refreshes.
   Here we provide the simplest possible steps to create the required materialized
   view and the command to refresh it manually. The materialized view below can
   be created in any schema with any name. Feel free to add whatever REFRESH
   options suits your purpose. Note that you might need additional steps based
   on the REFRESH option setting. Here we provide the simplest possible way to
   do this. The PUBLIC synonym pointing to this Materialized view must be
   named "SAS_CACHED_FK_CATALOG_PSYN". This synonym must be visible to
   PUBLIC (or the set of users who will be needing ForeignKey metadata) so that
   it is accessible from any schema.
*/

Create materialized view SAS_CACHED_FK_CATALOG_MATVIEW REFRESH ON DEMAND as SELECT
PKAC.OWNER as PKTABLE_SCHEM,
PKAC.TABLE_NAME as PKTABLE_NAME,
PKACC.COLUMN_NAME as PKCOLUMN_NAME,
FKAC.OWNER as FKTABLE_SCHEM,

```

```

FKAC.TABLE_NAME as FKTABLE_NAME,
FKACC.COLUMN_NAME as FKCOLUMN_NAME,
FKACC.POSITION as KEY_SEQ,
FKAC.CONSTRAINT_NAME as FK_NAME,
PKAC.CONSTRAINT_NAME as PK_NAME
from
sys.all_constraints PKAC, sys.all_constraints FKAC,
sys.all_cons_columns PKACC, sys.all_cons_columns FKACC

where

FKAC.r_constraint_name=PKAC.constraint_name and
FKAC.constraint_name=FKACC.constraint_name and
PKAC.constraint_name=FKACC.constraint_name and PKAC.constraint_type='P' and
FKAC.constraint_type='R' and FKAC.owner=FKACC.owner and PKAC.owner=FKACC.owner
and PKAC.table_name=FKACC.table_name and FKAC.table_name=FKACC.table_name and
FKACC.position = PKACC.position ;

/* The synonym name *must* be SAS_CACHED_FK_CATALOG_PUBLIC_SYNONYM */
create public synonym SAS_CACHED_FK_CATALOG_PSYN for SAS_CACHED_FK_CATALOG_MATVIEW;
grant all on SAS_CACHED_FK_CATALOG_PSYN to PUBLIC;

/*-----Manual REFRESH of the Materialized View-----*/
/* Note there are several ways to do this, consult with your DBA.
   Here are a couple of ways:
*/
execute DBMS_MVIEW.REFRESH('SAS_CACHED_FK_CATALOG_MATVIEW');
execute DBMS_SNAPSHOT.REFRESH('SAS_CACHED_FK_CATALOG_MATVIEW', '?');

```

PostgreSQL and Microsoft Azure PostgreSQL Driver Reference

Connection String

A SAS Micro Analytic Service deployment includes a driver that you can use to establish a connection to a PostgreSQL database. To connect, you must specify a valid connection string.

Here is a sample PostgreSQL connection string:

```
DRIVER=FEDSQL; CONOPTS=(DRIVER=POSTGRES; UID=MyUID; PWD=MyPWD; SSLMODE=prefer;
SERVER=MyServer.sas.com; PORT=5432; DB=myDB; CATALOG=Catalog1)
```

Note the following information:

- The DS2 SQLSTMT package supports the FedSQL dialect. Therefore, the connection string must begin with DRIVER=SQL with the FedSQL language

driver as the managing driver. For more information, see [“About the Connection String Configuration” on page 318](#).

- Although this sample connection string might appear on multiple lines, you should always enter a connection string as a single line.
- You specify the connection string in SAS Environment Manager using the `connectionstring` property. For information, see [“sas.microanalyticsservice.properties: core” on page 271](#).

Connection Options

Here are the PostgreSQL connection options:

Note: The [“Additional Database Connection Options”](#) also apply for this database type.

Option	Description
ALLOW_UNQUOTED_NAMES	<code>ALLOW_UNQUOTED_NAMES=YES NO;</code> Specifies whether to enclose table and column names in quotation marks. ■ YES: Specifies that table and column names are not quoted. This allows PostgreSQL tables and columns to be created in the default lowercase. ■ NO: Specifies that tables and columns are quoted. Default: NO
CATALOG	<code>CATALOG=catalog-identifier;</code> Specifies an arbitrary identifier for an SQL catalog, which groups schemas that are logically related. You must specify a catalog. For more information, see “About Using Database Catalogs” on page 319.
CLIENT_ENCODING	<code>CLIENT_ENCODING=encoding-value;</code> Specifies the character set encoding value for the client data, if it is different from the session encoding.
DB	<code>DB=database-name;</code> Specifies the name of the PostgreSQL database. Enclose the database name in single quotation marks if it contains spaces or non-alphanumeric characters.
DRIVER	<code>DRIVER=POSTGRES;</code> Specifies the name of the data source driver. You must specify a driver.
MAX_ACTIVE_STATEMENTS	<code>MAX_ACTIVE_STATEMENTS=integer-value;</code> Specifies the default number of active statements in a connection handle.

Option	Description
	Default: 1
MAX_BINARY_LEN	<code>MAX_BINARY_LEN=value;</code> Specifies a value, in bytes, that limits the length of long binary fields (LONG VARBINARY). Unlike other databases, PostgreSQL does not have a size limit for long binary fields. Default: 134200000
MAX_CHAR_LEN	<code>MAX_CHAR_LEN=value;</code> Specifies a value, in characters, that limits the length of character fields (CHAR and VARCHAR). Default: 65535
MAX_TEXT_LEN	<code>MAX_TEXT_LEN=value;</code> Specifies a value, in characters, that limits the length of long character fields (LONG VARCHAR). Default: 65535
max_t	<code>MAXVARCHARSIZE=value;</code> Specifies a value, in bytes, that limits the length of long binary fields (LONG VARBINARY). Unlike other databases, PostgreSQL does not have a size limit for long binary fields. Default: 134200000
NUMBYTESPERCHAR	<code>NUMBYTESPERCHAR=maximum-client-bytes;</code> Specifies the maximum number of bytes per single character in the database client encoding, which matches the SAS encoding. Maximum-client-bytes is the multiplying factor to use to adjust column lengths for STRING columns. When a SAS UTF-8 encoding is used, the length of STRING variables is four times the length of the default character length, or four times the length that is specified in the <code>MAX_CHAR_LEN=</code> connection option. Therefore, when <code>MAX_CHAR_LEN=8</code> is specified, a length of 32 bytes of storage is used. To actually get 8 bytes of storage, specify <code>MAX_CHAR_LEN=8</code> and <code>NUMBYTESPERCHAR=1</code>.
POSTGRES_DSN	<code>DSN=data-source-identifier;</code> Specifies a configured PostgreSQL Data Source Name (DSN) to which you want to connect. Use this option if you have existing PostgreSQL ODBC data sources that are configured on your client. This method requires additional setup. This extra setup is done through the <code>odbc.ini</code> file or a similarly named configuration file. Therefore, it is recommended that you use this connection method only if you have existing, functioning data sources that have been specified.
PORT	<code>PORT=port_number;</code>

Option	Description
	Specifies the port number that is used to connect to the specified PostgreSQL Server. Default: 5432
PWD	<code>PWD=password;</code> Specifies the password associated with the user ID. Enclose password in single quotation marks if it contains spaces or non-alphanumeric characters. You can also specify <code>PASSWORD=</code> with the <code>PWD=</code>, <code>PASS=</code>, and <code>PW=</code> aliases.
SCHEMA	<code>SCHEMA=value;</code> Specifies the default schema for the connection. If not specified, the schema, or list of schemas, is determined based on the value of the schema search path that is defined on the database server.
SERVER	<code>SERVER='server-name';</code> Specifies the server name or IP address of the PostgreSQL server to which you want to connect. Enclose the server name in single quotation marks if the name contains spaces or non-alphanumeric characters.
SSLMODE	<code>SSLMODE=allow disable prefer require verify-ca verify-full;</code> Determines whether or with what priority a secure SSL TCP/IP connection is negotiated with the PostgreSQL server. ■ allow: Specifies that encryption is not required, but encryption is accepted if the DBMS server requires it. ■ disable: Specifies that encryption is not used when connecting to the database. ■ prefer: Specifies that encryption is not required, but encryption should be used if it is available. ■ require: Specifies that encryption is required when connecting to the database ■ verify-ca: Specifies that encryption is required, and an authentication certificate might be required. Whether an authentication certificate is required depends on your database policy. ■ verify-full: Specifies that encryption and an authentication certificate are required. Default: prefer Note: Values are case sensitive. The first letter for each <code>SSLMODE</code> value must be specified in lowercase.
STRIP_BLANKS	<code>STRIP_BLANKS=YES NO;</code> Specifies whether to strip blanks from character fields.
USE_DECLARE_FETCH	<code>USE_DECLARE_FETCH=YES NO;</code> Specifies whether the PostgreSQL attribute <code>UseDeclareFetch=1</code> is enabled when connecting to the PostgreSQL database.

Option	Description
	■ YES: Correlates to <code>UseDeclareFetch=1</code>. Specifies that the result rows from the command cursor are returned to the application one at a time, as they are received from the server. ■ NO: Correlates to <code>UseDeclareFetch=0</code>. Specifies that the entire result set from the cursor is pushed to client memory before the first fetch is returned. This can be unworkable for commands that return a large number of rows. DEFAULT: YES
UID	<code>USER=user-ID;</code> Specifies the PostgreSQL user name (also called the <code>userID</code>) that you use to connect to your database.

For more information, see [“SAS/ACCESS Interface to PostgreSQL” in SAS/ACCESS for Relational Databases: Reference](#).

SingleStore Driver Reference

Connection String

A SAS Micro Analytic Service deployment includes a driver that you can use to connect to a SingleStore database. To connect, you must specify a valid connection string.

Here is a sample SingleStore connection string:

```
DRIVER=SQL; CONOPTS=(DRIVER=SSTORE; UID=MyUID; PWD=MyPW; HOST=MyHost;
PORT=3306; DB=MyDB; CATALOG=MyCatalog)
```

Note the following information:

- The DS2 SQLSTMT package supports the FedSQL dialect. Therefore, the connection string must begin with `DRIVER=SQL` with the FedSQL language driver as the managing driver. For more information, see [“About the Connection String Configuration” on page 318](#).
- Although this sample connection string might appear on multiple lines, you should always enter a connection string as a single line.
- You specify the connection string in SAS Environment Manager using the `connectionstring` property. For information, see [“sas.microanalyticservice.properties: core” on page 271](#).

Connection Options

Here are the SingleStore connection options:

Option	Description
AUTHSCOPE	<code>AUTHSCOPE="scope";</code> Specifies the resource for authenticating and authorizing access to Microsoft Azure when using SSO setup with Azure AD SSO and SAS Credential services. Enclose the scope value in single or double quotation marks. Default: <code>https://storage.azure.com/user_impersonation</code>
BULKLOAD	<code>BULKLOAD=YES NO;</code> When set to YES, inserts row data using the SingleStore bulk-loading interfaces. Default: NO
CATALOG	<code>CATALOG=* catalog-name-map;</code> For a multi-catalog DBMS, the CATALOG option is needed only in a heterogeneous FedSQL connection string to resolve catalog collisions. For more information, see “About Using Database Catalogs” on page 319.
DATABASE	<code>DATABASE=database-name;</code> Specifies the name of the SingleStore database. Enclose the database name in single quotation marks if it contains spaces or non-alphanumeric characters.
DRIVER	<code>DRIVER=SSTORE;</code> Specifies the name of the data source driver. You must specify a driver.
HOST	<code>HOST=host-name;</code> Specifies the host name or IP address where the SingleStore database is running. If the server name contains spaces or nonalphanumeric characters, you must enclose it in quotation marks.
PORT	<code>PORT=port-number;</code> Specifies the port number that is used to connect to the specified SingleStore database. DEFAULT: 3306
PWD	<code>PWD=password;</code> Specifies the password for the specified SingleStore UID. If the password contains spaces or nonalphanumeric characters, you must enclose it in quotation marks. Note: If a password is not provided, OAUTH2 authentication is attempted.

Option	Description
SSL_CA	<code>SSL_CA=path-and-file-name;</code> Specifies the full path to the certificate authority (CA) file. Certificate files enable your data source to support secure connections using encryption and validation. A CA file contains a list of trusted certificate authorities. A CA file is configured as part of configuring your data source server to use secure connections. This file must be specified in order to establish a secure connection to the server. For more information about configuring your data source and creating CA files, see the security information in your data source documentation.
SSL_CERT	<code>SSL_CERT=pathname;</code> Specifies the path to the client public key certificate file. Certificate files enable your data source to support secure connections using encryption and validation. A client public key certificate file identifies the server public key certificate. This file is configured as part of configuring the server for your data source to use secure connections. This certificate file must be specified in order to establish a secure connection to the server. For more information about configuring your data source and creating certificates, see the security information in your data source documentation.
SSL_CIPHER	<code>SSL_CIPHER=cipher-list;</code> Specifies a list of ciphers that are permitted to be used for encryption. Each secure connection to your data source uses an encryption cipher. The <code>SSL_CIPHER=</code> option provides a list of ciphers that are permitted on any particular secure connection. For more information about using encryption ciphers, see the security information in your data source documentation.
SSL_KEY	<code>SSL_KEY=path;</code> Specifies the path to the client private key file. A client private key file is configured as part of configuring your data source server. This file must be specified in order to establish a secure connection to the server. For more information about creating a client private key file, see your data source documentation.
UID	<code>UID=user-name;</code> Specifies the SingleStore user name (also called the user ID) that you use to connect to your database. If the user name contains spaces or nonalphanumeric characters, you must enclose it in quotation marks.

Snowflake Driver Reference

Connection String

A SAS Micro Analytic Service deployment includes drivers that you can use to connect to a Snowflake database. To connect, you must specify a valid connection string.

Here are some sample Snowflake connection strings:

Snowflake database on Amazon Web Services (AWS):

```
DRIVER=SQL; CONOPTS=(DRIVER=snowflake; UID=MyUID; PWD=MyPWD;
SERVER='MyServer.snowflakecomputing.com'; SCHEMA=MYSHEMA)
```

Snowflake database on Microsoft Azure:

```
DRIVER=SQL; CONOPTS=(DRIVER=snowflake; UID=MyUID; PWD=MyPWD;
CONOPTS=(DRIVER=SnowflakeDSIIDriver; SERVER='MyServer.snowflakecomputing.com';
DATABASE=MyDatabase; PORT=443; SCHEMA=MYSHEMA))
```

```
DRIVER=SQL; CONOPTS=(DRIVER=snowflake; UID=MyUID; PWD=MyPWD;
SERVER='MyServer.snowflakecomputing.com';
CONOPTS=(DATABASE=MyDatabase; SCHEMA=MYSHEMA); CATALOG=*)
```

```
DRIVER=SQL; CONOPTS=(DRIVER=snowflake; UID=MyUID; PWD=MyPWD;
SERVER='MyServer.snowflakecomputing.com'; CONOPTS=(DATABASE=MyDatabase;
SCHEMA=MYSHEMA); CATALOG=(A=DB1; B=DB2))
```

Note the following information:

- The DS2 SQLSTMT package supports the FedSQL dialect. Therefore, the connection string must begin with DRIVER=SQL with the FedSQL language driver as the managing driver. For more information, see [“About the Connection String Configuration” on page 318](#).
- Although this sample connection string might appear on multiple lines, you should always enter a connection string as a single line.
- You specify the connection string in SAS Environment Manager using the connectionstring property. For information, see [“sas.microanalyticservice.properties: core” on page 271](#).

Connection Options

Here are the Snowflake connection options:

Option	Description
AUTH	<code>AUTH=authentication-method;</code> Specifies authentication method, currently only valid value is OAUTH2.
AUTHSCOPE	<code>AUTHSCOPE="scope";</code> Specifies the resource for authenticating and authorizing access to Microsoft Azure when using SSO setup with Azure AD SSO and SAS Credential services. Enclose the scope value in single or double quotation marks.
CATALOG	<code>CATALOG=* catalog-name-map;</code> For a multi-catalog DBMS, the CATALOG option is needed only in a heterogeneous FedSQL connection string to resolve catalog collisions. For more information, see “About Using Database Catalogs” on page 319. By default, Snowflake connects with <code>catalog=*</code>, which includes the list of catalogs that are available for the user.
CLIENT_ENCODING	<code>CLIENT_ENCODING=encoding-value;</code> Specifies the character set encoding value for the client data, if it is different from the session encoding.
CONOPTS	<code>CONOPTS=(ODBC-compliant connection-string);</code> Specifies ODBC-compliant connection options using ODBC-style syntax. The options are passed to the driver for processing.
DATABASE	<code>DATABASE=database-name;</code> Specifies the name of the Snowflake database. Enclose the database name in single quotation marks if it contains spaces or non-alphanumeric characters.
DEFAULT_CURSOR_TYPE	<code>DEFAULT_CURSOR_TYPE=FORWARD_ONLY KEYSET DYNAMIC STATIC;</code> Override the default cursor. A default is not set.
DRIVER	<code>DRIVER=SnowflakeDSIIDriver snowflake;</code> Specifies the name of the data source driver. You must specify a driver.
PORT	<code>PORT=port-number;</code> Specifies the port number that is used to connect to the specified Snowflake database. DEFAULT: 443
PWD	<code>PWD=password;</code> Specifies the password for the specified Snowflake UID. If the password contains spaces or nonalphanumeric characters, you must enclose it in quotation marks.
SERVER	<code>SERVER=server-name;</code>

Option	Description
	Specifies the host name or IP address where the Snowflake database is running. If the server name contains spaces or nonalphanumeric characters, you must enclose it in quotation marks.
SNOW_DSN	SNOW_DSN=DSN-name; Specifies a valid ODBC-compliant database DSN that contains connection information for connecting to the ODBC-compliant database. Use this option if you have existing Snowflake ODBC data sources that are configured on your client. This method requires additional setup. This extra setup is done through the <code>odbc.ini</code> file or a similarly named configuration file. Therefore, it is recommended that you use this connection method only if you have existing, functioning data sources that have been specified.
STRIP_BLANKS	STRIP_BLANKS=YES NO; Specifies whether to strip blanks from character fields.
UID	UID=user-name; Specifies the Snowflake user name (also called the user ID) that you use to connect to your database. If the user name contains spaces or nonalphanumeric characters, you must enclose it in quotation marks.

For more information, see [“SAS/ACCESS Interface to Snowflake” in SAS/ACCESS for Relational Databases: Reference](#).

Additional Database Connection Options

The following database connection options apply to Google BigQuery, Oracle, and PostgreSQL databases.

Option	Description
CT_PRESERVE	CT_PRESERVE=STRICT SAFE FORCE FORCE_COL_SIZE; Enables users to control how data types are mapped. Note that data type mapping is disabled when CT_PRESERVE is set to STRICT. If the requested type does not exist on the target database, an error is returned. Here are the options: ■ STRICT: The requested type must exist in the target database. No type promotion occurs. If the type does not exist, an error is returned. ■ SAFE: Target data types are upscaled only if they do not result in a loss of precision or scale. When character encodings are changed, the new column size is recalculated to ensure all characters can be stored in the new encoding.

Option	Description
	■ FORCE: This is the default for all drivers. The best corresponding target data type is chosen, even if it could potentially result in a loss of precision or scale. When character encodings are changed, the new column size is recalculated to ensure all characters can be stored in the new encoding. ■ FORCE_COL_SIZE: This option is the same as FORCE, except that the column size for the new encoding is the same as the original encoding. This option can be used to avoid column size creep. However, the resulting column might be too large or too small for the target data.
DEFAULT_ATTR	<code>DEFAULT_ATTR=(attr=value;...);</code> Used to specify connection handle or statement handle attributes that are supported for initial connect-time configuration, where <code>attr=value</code> corresponds to any of the following options: ■ CURSORS=<i>n</i> - Connection handle option. This option controls the driver's use of client-side, result set cursors. The possible values are 0, 1, or 2. □ 0 : Causes the driver to use client-side static cursor emulation if a scrollable cursor is requested but the database server cannot provide one. □ 1 : Causes the driver to always use client-side static cursor emulation if a scrollable cursor is requested. The database server's native cursor is not used. □ 2 : (Default) Causes the driver to never use client-side static cursor emulation if a scrollable cursor is requested. The database server's native cursor is used if available. Otherwise, the cursor is forward-only. Example: <code>DEFAULT_ATTR=(CURSORS=2)</code> ■ USE_EVP=<i>n</i> - Statement handle option. This option optimizes the driver for large result sets. The possible values are 0 (OFF) or 1 (ON), which is the default. Example: <code>DEFAULT_ATTR=(USE_EVP=0)</code> ■ XCODE_WARN=<i>n</i> - Statement handle option. Used to warn about possible character transcoding errors that occur during row input or output operations. Possible values are 0 (returns an error), 1 (returns a warning), or 2 (ignore transaction errors). 0 is the default. Example: <code>DEFAULT_ATTR=(XCODE_WARN=1)</code>
DRIVER_TRACE	<code>DRIVER_TRACE=API SQL ALL;</code> Requests tracing information, which logs transaction records to an external file that can be used for debugging purposes. The driver writes a record of each command that is sent to the database to the trace log based on the specified tracing level, which determines the type of tracing information. Here are the tracing levels: ■ ALL : Activates all trace levels. ■ API : Specifies that API method calls be sent to the trace log. This option is most useful if you are having a problem and need to send a trace log to SAS Technical Support for troubleshooting. ■ DRIVER : Specifies that driver-specific information be sent to the trace log.

Option	Description
	■ SQL : Specifies that SQL statements that are sent to the database management system (DBMS) be sent to the trace log. Tracing information is DBMS specific, but most table services drivers log SQL statements such as SELECT and COMMIT. Default: Tracing is not activated. Note: If you activate tracing, you must also specify the location of the trace log with DRIVER_TRACEFILE=. Note that DRIVER_TRACEFILE= is resolved against the TRACEFILEPATH set in ALTER SERVER. TRACEFILEPATH is relative to the server's content root location. (Optional) You can control trace log formatting with DRIVER_TRACEOPTIONS=. Interaction: You can specify one trace level, or you can concatenate more than one by including the (OR) symbol. For example, <code>driver_trace=api sql</code> generates tracing information for API calls and SQL statements.
DRIVER_TRACEFILE	<code>DRIVER_TRACEFILE='filename';</code> Used to specify the name of the text file for the trace log. Include the file name and extension in single or double quotation marks (for example, <code>driver_tracefile='mytrace.log'</code>). Default: The default TRACEFILE location applies to a relative file name, and it is placed relative to TRACEFILEPATH. Requirement: DRIVER_TRACEFILE is required when activating tracing using DRIVER_TRACE. Interaction: (Optional) You can control trace log formatting with DRIVER_TRACEOPTIONS=.
DRIVER_TRACEOPTIONS	<code>DRIVER_TRACEOPTIONS=APPEND THREADSTAMP TIMESTAMP;</code> Specifies options in order to control formatting and other properties for the trace log: ■ APPEND Adds trace information to the end of an existing trace log. The contents of the file are not overwritten. ■ THREADSTAMP Prepends each line of the trace log with a thread identification. ■ TIMESTAMP Prepends each line of the trace log with a time stamp. Default: The trace log is overwritten with no thread identification or time stamp.

Appendix 4

SAS Data Quality Function Support

About SAS Data Quality Functions 359

About SAS Data Quality Functions

SAS Data Quality provides SAS language elements that perform data quality operations such as matching and standardization. The data quality operations analyze and transform data to improve accuracy and consistency. The result is higher quality data, which yields higher-value analytical results.

For information about the functions that are supported for use by SAS Micro Analytic Service, see [“Functions Supported in CAS” in SAS Data Quality: Language Reference](#).

Appendix 5

SAS Micro Analytic Service Metrics

<i>SAS Micro Analytic Service Grafana Dashboard</i>	361
---	-----

SAS Micro Analytic Service Grafana Dashboard

SAS provides several Grafana dashboards that provide metrics for SAS Viya platform components. This includes a SAS Micro Analytic Service dashboard.

The SAS Micro Analytic Service dashboard displays metrics about the performance and usage of SAS Micro Analytic Service. The dashboard displays information about throughput, latency, and scores organized by module, instance, and client identifier. Additional metrics about the health and operation of SAS Micro Analytic Service pods are also reported.

For more information, see [SAS Viya Platform: Deploying SAS Viya Monitoring for Kubernetes](#).

Appendix 6

SAS Micro Analytic Service Tuning Guidelines

<i>SAS Micro Analytic Service Tuning Guidelines</i>	363
Overview	363
SAS Micro Analytic Service Configuration	364
Apache Tomcat Tuning	365
Large Modules and Virtual Memory Resources	366

SAS Micro Analytic Service Tuning Guidelines

Overview

Proper tuning can significantly improve the performance of SAS Micro Analytic Service. However, there is no single set of configuration recommendations that is ideal for all the models that SAS Micro Analytic Service can execute. This appendix explains the principles that you can apply to tune for optimal performance.

SAS Micro Analytic Service Configuration

Eventing and Authorization

The performance of SAS Micro Analytic Service can potentially be improved by disabling certain SAS Viya features that might not apply to your implementation. These features are controlled by the following system properties in the Micro Analytic Score service **jvm** configuration definition in SAS Environment Manager.

- **MVC eventing:** SAS Viya generates an event when a transaction is executed in SAS Micro Analytic Service, even when there are no watchers for that event. You might improve throughput by disabling this functionality. To do so, set the following in SAS Environment Manager:
 - **java_option_event_configuration_webmvc_enabled:** -
`Dsas.event.configuration.webmvc.enabled=false`
- **Authorization:** Transactions are sent to SAS Micro Analytic Service via a POST request to a REST endpoint. In most cases, transactions are sent from an external system. These transactions are not in the context of a specific user identity that is defined in SAS Viya. If this is the case for your environment, the authorization feature is not useful and you can disable authorization events from SAS Micro Analytic Service. To do so, set the following in SAS Environment Manager:
 - **java_option_auth:** -`Dsas.authorization=false`
 - **java_option_auth_remote:** -`Dsas.authorization.remote=false`
- **General eventing:** You can disable all eventing to improve throughput. However, doing so disables the contact and response history feature. To disable all eventing, set the following in SAS Environment Manager:
 - **java_option_eventing:** -`Dsas.event.enabled=false`
 - **java_option_event_configuration_enabled:** -
`Dsas.event.configuration.enabled=false`

IMPORTANT Do not disable the `sas.event.enabled` and `sas.event.configuration.enabled` properties if your decisions use contact and response history.

Note: You must restart SAS Micro Analytic Service for any property changes to take effect. For more information, see [“Applying Configuration Changes” on page 30](#).

Core Threads

In general, adding core threads increases throughput for a given load. However, for each core thread that is added, the marginal gains in throughput diminish sharply.

For optimal performance, it is recommended that you set the number of core threads equal to the number of cores on the server.

SAS Micro Analytic Service instances with a greater number of core threads tend to be more resilient to sub-optimal Apache Tomcat tuning conditions. Because of this, tuning is particularly important for SAS Micro Analytic Service instances with fewer than 40 threads.

For more information, see [“sas.microanalyticsservice.system: asynchronousexecution”](#) on page 269 and [“sas.microanalyticsservice.system: core”](#) on page 272.

Apache Tomcat Tuning

Most of the default Apache Tomcat application server settings are sufficient to achieve high throughput with SAS Micro Analytic Service. However, there is one setting that you can adjust to help increase performance—the Spring Boot `server.tomcat.threads.max` server application property. For more information, see the [Spring Boot Reference Documentation](#).

The assigned `server.tomcat.threads.max` value is influenced by both the number of SAS Micro Analytic Service core threads and the nature of the workload.

SAS Micro Analytic Service instances with a low number of core threads (fewer than 20) typically perform best with a low `server.tomcat.threads.max` value. In this case, an appropriate value could be between 20 and 100.

Alternatively, the `server.tomcat.threads.max` value is not as important when executing complex models with many SAS Micro Analytic Service core threads (more than 20).

Note: This is the case for the BigDecision2 model shown in the table below.

Here are some examples of optimal `maxThread` tunings:

Model	Model Complexity	Core Threads	Optimal MaxThreads Value
BigDecision1	complex	5	30
BigDecision2	complex	80	100
SimpleEcho1	trivial	5	50

Model	Model Complexity	Core Threads	Optimal MaxThreads Value
SimpleEcho2	trivial	80	75

Large Modules and Virtual Memory Resources

Large modules or large numbers of modules often require a large number of mapped memory requests. This might lead to a situation in which the virtual memory is exceeded, resulting in a memory allocation failure.

If you encounter unexpected memory allocation failures and your system has plenty of free memory, you should increase the value that is assigned to the `vm.max_map_count` property. For more information, see [“Configure Default Virtual Memory Resources” in SAS Viya Platform: Deployment Guide](#).

Appendix 7

SAS Quality Knowledge Base (QKB)

Configuring a SAS Quality Knowledge Base (QKB) 367

Configuring a SAS Quality Knowledge Base (QKB)

The SAS Quality Knowledge Base (QKB) is a collection of files that store data and logic that define data management operations such as parsing, standardization, and matching. SAS software products refer to the QKB when performing data management operations, also referred to as data cleansing, on your data. Each SAS QKB is defined by a locale that specifies the language or character set that is used for managing different types of data.

To configure SAS Micro Analytic Service to use a specific QKB, use the `kubectl` command to set two environment variables when the SAS Micro Analytic Service pod starts. One environment variable specifies the QKB and the other environment variable assigns the locale. Here is an example that specifies `QKB CI 33` QKB and the `ENUSA` locale.

```
kubectl -n $viya set env deployment/sas-microanalytic-score
DQSETUPLOC="QKB CI 33"
kubectl -n $viya set env deployment/sas-microanalytic-score
DQLOCALE="(ENUSA) "
```

IMPORTANT The `DQLOCALE` value must be enclosed in parentheses.

Note: This example assumes that the QKB assets exist in the sas-quality-knowledge-base PVC.

For more information about the SAS Quality Knowledge Base, see [SAS Quality Knowledge Base: User's Guide](#).

Appendix 8

REST Server Error Messages and Resolutions

REST Server Error Messages and Resolutions 369

REST Server Error Messages and Resolutions

The following table contains SAS Micro Analytic Service REST server error messages, as well as possible causes and remedies.

HTTP Code	Error Message	Explanation	Remedy
Errors related to missing resources			
404	The module <code>module_ID</code> not found.	A module with the specified ID does not exist. This typically occurs when attempting to do one of the following for the specified module: ■ view, update, delete the module ■ execute a step of the module	Specify a module ID that exists in the system.

HTTP Code	Error Message	Explanation	Remedy
		■ validate data input for the execution of a step of the module	
404	The step step_ID of module module_ID was not found.	The specified module exists, but the referenced step was not found. This typically occurs when attempting to do one of the following: ■ access the specified module to execute a step ■ validate data input for the execution of a step	Specify a step ID that exists in the specified module ID.
404	The submodule submodule_ID of module module_ID was not found.	The specified module exists, but the referenced submodule was not found. This can occur when attempting to access the specified submodule to view its properties or source code.	Specify a submodule ID that exists in the specified module ID.
Errors related to create and update operations			
400	Cannot create or update the module. The specified media type type_name is not supported. The only supported media type is type_name .	The media type must be text/vnd.sas.source.ds2 or text/application.source.ds2.	Specify the correct media type.
400	Cannot create or update the module. DS2 and COMPOSITE are the only valid values for language.	The programming language is not DS2 or COMPOSITE.	Supply DS2 or COMPOSITE modules only. Other languages might be supported in the future.
400	Cannot create or update module. The specified module ID must not be empty or missing.	Under certain circumstances, it is possible to specify the module ID as part of the module definition. However, when doing so the module specification must not be an empty string or missing.	Specify a valid module ID string.

HTTP Code	Error Message	Explanation	Remedy
400	Cannot create or update the module. Unable to parse the package name.	The DS2 package must be properly formed so that the package name can be parsed.	Supply a properly formed DS2 package.
400	Cannot create or update module. The value of field field_name must not be empty or missing.	When creating or updating a module, the payload contains certain mandatory fields, for example, scope, type, and code. Mandatory fields cannot be missing and the associated content must not be null or an empty string.	Assign a valid value for the fields that are referenced in the message.
400	Cannot create or update the module. At least one of the fields field_name_1 or field_name_2 must be present.	Either field_name_1 or field_name_2 is required input to create or update the module.	Supply either field_name_1 or field_name_2 .
400	Cannot create or update the module. Only one of the fields field_name or field_name can be present.	Only one field name (either field_name_1 or field_name_2) can be present to create or update the module.	Supply either field_name_1 or field_name_2 .
400	Cannot create or update module. The field field_name must not be repeated.	When creating or updating a module, the payload can contain multiple name value pairs called properties. Property names must be unique.	Ensure that property names are unique.
400	Cannot create or update module. An unexpected end of source was encountered while parsing a comment.	DS2 code that contains comments must be properly delimited.	Refer to SAS DS2 Language Reference for information about comment syntax rules.
400	Cannot create or update module due to unmatched single quotation marks.	If strings are defined in the DS2 program, they must be enclosed in single quotation marks.	Refer to SAS DS2 Language Reference for information about character constant syntax rules.
400	Cannot create or update module due to unmatched double quotation marks.	If delimited variables names are defined in the DS2 program, they must be enclosed in matching double quotation marks.	Refer to SAS DS2 Language Reference for information about delimited variable reference syntax rules.

HTTP Code	Error Message	Explanation	Remedy
400	Cannot create or update module. No DS2 package was found in the source code.	DS2 source was not found.	Refer to SAS DS2 Language Reference for information about DS2 package syntax rules.
400	Cannot create or update module. More than one DS2 package found in the source code.	Only a single DS2 package is accepted to create a module.	Ensure that only a single DS2 package is contained in the source code. Multiple packages can be separated and created as separate modules.
400	Cannot create or update module <code>module_ID</code> due to the following compilation errors: <code>error_messages</code> .	The DS2 source code contains compilation errors.	Resolve the issues, and then resubmit the source code. For more information, see SAS DS2 Language Reference .
400	Cannot create module. The specified module ID <code>module_ID</code> is already in use.	When creating a module, the specified module ID must not already exist in the system.	Depending on the configuration, the module ID is either supplied or derived from the package name. To avoid this error, specify the appropriate module ID or DS2 package.
400	The module <code>module_ID</code> is not a COMPOSITE module. Only COMPOSITE modules can have submodules.	This is typically returned when the submodules of a module that does not support submodules are accessed.	Select a COMPOSITE module.
400	Cannot create or update the module. The file <code>file_name</code> is not accessible from this server.	This is typically returned when the file containing the analytic store module is referenced and the server cannot access the file.	Ensure that the file <code>file_name</code> exists and is accessible by the web service.
400	Cannot create or update the module. The submodule name <code>name</code> must not be repeated.	The supplied module definition contains more than one submodule with the same name.	Ensure that the submodule names in the module definition are unique.

HTTP Code	Error Message	Explanation	Remedy
400	Cannot update the module module_ID . The language cannot be changed.	The module definition used for updating a module contains a different language value than the original module. After a module is created, the language cannot be changed.	To preserve the same language, supply the same language value or remove the module language field from the module definition. To create a module with a different language, delete the original module and create a new one.
Errors related to accessing or executing the steps of a module, or validating the data needed before execution.			
400	Cannot execute step step_ID of module module_ID .	The input data supplied to execute a module step is incorrect. Subsequent error messages will provide further details.	Refer to the subsequent messages related to this issue for more information.
400	Expected integer_value , input parameters, but received integer_value .	An incorrect number of parameters were supplied to execute a module step.	Supply the correct number of parameters.
400	The parameter parameter_name is not defined.	The named parameter is not defined for the module step.	Select the correct parameter name or execute a different module step.
400	Cannot assign the value specified_value to the parameter parameter_name of type type .	An inappropriate value was received for the parameter, for example, supplying a string value to an integer parameter.	Specify appropriate matching values.
400	Cannot assign an array of size integer_value to the array parameter parameter_name . A maximum of integer_value elements can be accepted.	The supplied array is larger than the maximum array size specified by the module. SAS Micro Analytic Service rejects arrays larger than this size.	Supply an array with a size equal to or smaller than the maximum size allowed.
400	Cannot assign a string size of integer_value to the string parameter parameter_name . A maximum of integer_value elements can be accepted.	The supplied string is larger than the maximum string size specified by the module. SAS Container Runtime rejects strings larger than this size.	Supply a string with a size equal to or smaller than the maximum size allowed.

HTTP Code	Error Message	Explanation	Remedy
400	Cannot assign a string size of integer_value to the string parameter parameter_name . A maximum of integer_value elements can be accepted.	The supplied string is larger than the maximum string size specified by the module. SAS Micro Analytic Service rejects strings larger than this size.	Supply a string with a size equal to or smaller than the maximum size allowed.
400	Cannot assign the value specified_value to the array element parameter_name [element_number] of type type .	An inappropriate value was received for the array parameter, for example, supplying a string value to an integer array parameter.	Specify appropriate matching values.
400	The value of field field_name must not be empty or missing.	The name and value parts of the parameter must be available to execute the module step.	Specify appropriate matching values.
400	The field field_name must not be repeated.	You cannot specify more than one value for a parameter.	Specify a single value.
400	Cannot parse the value of the data grid parameter parameter_name .	The value of a data grid parameter does not match the structure of a data grid.	Ensure that the supplied value of the parameter is appropriate for the data grid.
400	Cannot process the value of the data grid parameter parameter_name because it exceeds integer_value characters.	There is a size limit for a data grid. This is returned when the data grid is larger than the internal size of the data grid parameter.	Restructure the request data into multiple data grid objects.
403	Cannot access the steps of the PRIVATE module module_ID .	The steps of private modules are not accessible.	Re-create the module as a public module.
403	Source code is only available for modules or submodules of type DS2.	Returned when trying to access the source code of a module that does not have source code.	Access the source code of a different module or submodule.
500	Error error_message received when executing the step step_ID of the module module_ID .	An error was returned by the SAS Micro Analytic Service core during execution of the modules.	Search this appendix for information about the error message.
Transfer Errors			
400	The transfer object content is invalid. The content should not be modified after export.	The exported object appears to be corrupted or modified following the export.	Export the object from the source system

HTTP Code	Error Message	Explanation	Remedy
			again, and then retry the import.
400	The transfer object content is invalid for module <code>module_ID</code> . The content should not be modified after export.	The exported object for the given module appears to be corrupted or modified following the export.	Export the object from the source system again, and then retry the import.
Internal Errors			
500	Internal Error: The DS2 compiler encountered an unrecoverable error while compiling the module. Please check the log files for further diagnosis.	In most cases, this error is caused by insufficient stack space for the SAS Micro Analytic Service core TK subsystem.	In SAS Environment Manager, increase the value of the configuration parameter <code>core.tktstacksizekbytes</code> , as appropriate.
500	An internal error occurred. Please check the log files for further diagnosis.	This is a generic error that occurs during execution. It indicates that an unrecoverable situation has occurred.	Please contact SAS Technical Support.
500	An internal error occurred. Cannot load the file <code>file_name</code> . Please ensure that the file exists and the server is able to access it.	This is typically returned when the file containing the analytic store model is not accessible during execution of the module.	Please contact SAS Technical Support.

Appendix 9

SAS Micro Analytic Service Return Codes

SAS Micro Analytic Service Return Codes 377

SAS Micro Analytic Service Return Codes

The SAS Micro Analytic Service core component, tkmas, supports the following return codes. Depending on logging settings, an associated message might be logged. When a message is logged, any substitution parameters (indicated by %s for string and %d for number) are filled in. The other SAS Micro Analytic Service interface layers, such as the Java interface and the REST interface, might log additional messages that are not listed below.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958744063	0x8b3ff001	MASBadArgs	Invalid arguments.
-1958744062	0x8b3ff002	MASInternalError	Internal error.
-1958744061	0x8b3ff003	MASFailure	SAS Micro Analytic Service encountered a failure.
-1958744060	0x8b3ff004	MASFail	%s encountered a failure.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958744059	0x8b3ff005	MASUnexFail	%s encountered an unexpected failure.
-1958744058	0x8b3ff006	MASUnexInternal	%s encountered an unexpected internal failure.
-1958744057	0x8b3ff007	MASUnexFailIn	%s encountered an unexpected failure in %s.
-1958744056	0x8b3ff008	MASFailIn	%s encountered a failure in %s.
-1958744055	0x8b3ff009	MASFailWithText	%s encountered a failure in %s: %s.
-1958744054	0x8b3ff00a	MASSFGCBLock	Failed to obtain the SFGCB lock.
-1958744053	0x8b3ff00b	MASExeLock	Failed to obtain the .exe lock.
-1958744052	0x8b3ff00c	MASLockCreate	Failed to create the %s lock.
-1958744051	0x8b3ff00d	MASEventCreate	Failed to create the %s event for thread %d.
-1958744050	0x8b3ff00e	MASThreadCreate	Failed to create SAS Micro Analytic Service worker thread %d of %d.
-1958744049	0x8b3ff00f	MASCPUCount	Failed to determine the number of CPUs. Setting the number of worker threads to %d.
-1958744048	0x8b3ff010	MASThreadCount	The number of threads requested, %d, exceeds the limit. The maximum allowable threads = %d times the number of CPUs = %d.
-1958744047	0x8b3ff011	MASThreadPoolSize	Worker thread pool size is set to: %d.
-1958744046	0x8b3ff012	MASInitAlready	SAS Micro Analytic Service was already initialized.
-1958744045	0x8b3ff013	MASInitFailed	SAS Micro Analytic Service failed to initialize.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958744044	0x8b3ff014	MASNotLicensed	SAS Micro Analytic Service is not licensed.
-1958744043	0x8b3ff015	MASLicSvcInitFailed	License service failed to initialize.
-1958744042	0x8b3ff016	MASNotInitialized	SAS Micro Analytic Service is not initialized.
-1958744041	0x8b3ff017	MASTermFailed	SAS Micro Analytic Service failed to terminate successfully.
-1958744040	0x8b3ff018	MASArgTrunc	The maximum size of parameter %d in the %s call is not large enough, and the value has been truncated at %d characters.
-1958744039	0x8b3ff019	MASCompStatus	Compiler encountered status 0x %X.
-1958744038	0x8b3ff01a	MASUnsupportedType	Unsupported type.
-1958744037	0x8b3ff01b	MASUnknownType	Unknown type.
-1958744036	0x8b3ff01c	MASNoSuchPackage	Package not found.
-1958744035	0x8b3ff01d	MASNoSuchMethod	Method not found.
-1958744034	0x8b3ff01e	MASNoSuchRevision	Revision not found.
-1958744033	0x8b3ff01f	MASRevisionGet	Failed to get revision.
-1958744032	0x8b3ff020	MASNoSuchModule	Module not found.
-1958744031	0x8b3ff021	MASNoSuchUserContext	User context not found.
-1958744030	0x8b3ff022	MASModuleCtxtCreate	Failed to create module context.
-1958744029	0x8b3ff023	MASUserCtxtCreate	Failed to create user context.
-1958744028	0x8b3ff024	MASArgTypeMismatch	Argument type mismatch.
-1958744027	0x8b3ff025	MASArgCoutMismatch	Argument count mismatch.
-1958744026	0x8b3ff026	MASClientCodegenError	Code generation error.
-1958744025	0x8b3ff027	MASDS2CompileError	DS2 compilation error.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958744024	0x8b3ff028	MASDS2RuntimeError	DS2 run-time error.
-1958744023	0x8b3ff029	MASTKGNoEntryPoint	Code generation did not find an entry point.
-1958744022	0x8b3ff02a	MASTKGGenericError	Code generation generic error.
-1958744021	0x8b3ff02b	MASInvalidRequest	Invalid request.
-1958744020	0x8b3ff02c	MASMissingEntryPoints	Missing entry points.
-1958744019	0x8b3ff02d	MASUnassignedInput	Unassigned input.
-1958744018	0x8b3ff02e	MASInternalOnly	Internal only.
-1958744017	0x8b3ff02f	MASOnlyValidForDS2	Valid only for DS2 code.
-1958744016	0x8b3ff030	MASOnlyValidForC	Valid only for C code.
-1958744015	0x8b3ff031	MASExecutionException	Exception occurred during execution.
-1958744014	0x8b3ff032	MASCompilationException	Exception occurred during compilation.
-1958744013	0x8b3ff033	MASDS2ThreadUnsupported	DS2 thread unsupported.
-1958744012	0x8b3ff034	MASTKEDSError	DS2 error.
-1958744011	0x8b3ff035	MASUnrecognizedLanguage	Unrecognized language.
-1958744010	0x8b3ff036	MASUnspecifiedDataType	Unspecified data type.
-1958744009	0x8b3ff037	MASTKThreadingError	Threading error.
-1958744008	0x8b3ff038	MASFatalProgRepoLost	Program repository lost.
-1958744007	0x8b3ff039	MASSaveToRepo	Failed to save to repository.
-1958744006	0x8b3ff03a	MASLog4SASCfgFailed	Logging configuration failed.
-1958744005	0x8b3ff03b	MASDS2CompileStart	User context '%s' compiling module '%s' on thread %d.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958744004	0x8b3ff03c	MASDS2CompileFinish	User context '%s' module '%s' thread %d compilation succeeded.
-1958744003	0x8b3ff03d	MASDS2CompileFailed	User context '%s' module '%s' thread %d new revision failed, RC = %d.
-1958744002	0x8b3ff03e	MASStartup	SAS Micro Analytic Service Started
-1958744001	0x8b3ff03f	MASShutdown	service Micro Analytic Score Service shutdown
-1958744000	0x8b3ff040	MASAsyncException	SAS Micro Analytic Service received async exception code %d.
-1958743999	0x8b3ff041	MASAsyncInitFailed	SAS Micro Analytic Service failed to install async exception handler.
-1958743998	0x8b3ff042	MASShutdownJNI	SAS Micro Analytic Service calling JVM System.exit(0).
-1958743997	0x8b3ff043	MASExecDeletePending	Attempt to execute method %s while deletion pending for module context %s revision %d.
-1958743996	0x8b3ff044	MASMTXDeletePending	Attempt to add module context &s while deletion pending for user context %s.
-1958743995	0x8b3ff045	MASRevDeletePending	Attempt to create revision while deletion pending for module context %s.
-1958743994	0x8b3ff046	MASRevDelDeletePending	Attempt to delete revision while deletion pending for module context %s.
-1958743993	0x8b3ff047	MASRevDelRefCount	Pending delete called for module context %s with ref count %d.
-1958743992	0x8b3ff048	MASRevDelRefCountError	Delete called for module context %s with ref count %d.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958743991	0x8b3ff049	MASMTXDelete	Garbage collection is deleting module context %s.
-1958743990	0x8b3ff04a	MASCTXDeletePending	Attempt to delete user context %s while being deleted by another thread.
-1958743989	0x8b3ff04b	MASCTXGetCDTDelPending	Attempt to retrieve creation time from user context %s while deletion pending.
-1958743988	0x8b3ff04c	MASCTXGetMDTDelPending	Attempt to retrieve modified time from user context %s while deletion pending.
-1958743987	0x8b3ff04d	MASMTXGetCDTDelPending	Attempt to retrieve creation time from module context %s while deletion pending.
-1958743986	0x8b3ff04e	MASMTXGetMDTDelPending	Attempt to retrieve modified time from module context %s while deletion pending.
-1958743985	0x8b3ff04f	MASMTXGetRevDelPending	Attempt to retrieve highest revision from module context %s while deletion pending.
-1958743984	0x8b3ff050	MASMTXGetIUODelPending	Attempt to retrieve internal use flag from module context %s while deletion pending.
-1958743983	0x8b3ff051	MASRevGetCDTDelPending	Attempt to retrieve revision %d creation time from module context %s while deletion pending.
-1958743982	0x8b3ff052	MASMTXGetMsgDelPending	Attempt to retrieve compilation messages from module context %s while deletion pending.
-1958743981	0x8b3ff053	MASMTXRegDeletePending	Attempt to register name while deletion pending for module context %s.
-1958743980	0x8b3ff054	MASMTXLangDelPending	Attempt to retrieve language of module context %s while deletion pending.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958743979	0x8b3ff055	MASMTXGetDispDelPending	Attempt to retrieve display name from module context %s while deletion pending.
-1958743978	0x8b3ff056	MASMTXGetCSrcDelPending	Attempt to retrieve C source code from module context %s revision %d while deletion pending.
-1958743977	0x8b3ff057	MASCTXGetPkgsDelPending	Attempt to retrieve packages from user context %s while deletion pending.
-1958743976	0x8b3ff058	MASMTXGetMthsDelPending	Attempt to retrieve methods from module context %s while deletion pending.
-1958743975	0x8b3ff059	MASNoSuchEntryPoint	Entry point not found.
-1958743974	0x8b3ff05a	MASMTXGetSigDelPending	Attempt to retrieve method %s signature from module context %s while deletion pending.
-1958743973	0x8b3ff05b	MASCTXLdOOTBDelPending	Private load out-of-the-box packages for user context %s while deletion pending.
-1958743972	0x8b3ff05c	MASCTXRegIntDelPending	Attempt to publish internal package %s to user context %s while deletion pending.
-1958743971	0x8b3ff05d	MASCTXRemIntDelPending	Attempt to remove internal package %s from user context %s while deletion pending.
-1958743970	0x8b3ff05e	MASCreateGCAFailed	Attempt to create garbage collection control structures failed.
-1958743969	0x8b3ff05f	MASGarbageCollection	Garbage collection interval.
-1958743968	0x8b3ff060	MASGarbageCollectionDel	Garbage collection found assets ready to delete.
-1958743967	0x8b3ff061	MASGCEException	Exception occurred during garbage collection run.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958743966	0x8b3ff062	MASProgRepoUpdateError	Error obtaining exclusive lock to update DS2 program repository.
-1958743965	0x8b3ff063	MASCTXDelete	Garbage collection is deleting user context %s.
-1958743964	0x8b3ff064	MASRevDelete	Garbage collection is deleting module context %s revision %d.
-1958743963	0x8b3ff065	MASDS2Fatal	Module context %s revision %d generated fatal run-time exception. Deleting revision.
-1958743962	0x8b3ff066	MASGarbageCollectionTerm	Garbage collection is freeing control assets during shut down.
-1958743961	0x8b3ff067	MASShutdownHang	Worker thread did not interrupt after %d seconds during shutdown.
-1958743960	0x8b3ff068	MASGCInvalidIntervalHigh	Specifies that the garbage collection interval is above the maximum. Setting to default value.
-1958743959	0x8b3ff069	MASGCInvalidIntervalLow	Specifies that the garbage collection interval is below the minimum. Setting to default value.
-1958743958	0x8b3ff06a	MASGCInvalidGraceHigh	Specifies that the grace period is above the maximum. Setting to default value.
-1958743957	0x8b3ff06b	MASGCInvalidGraceLow	Specifies that the grace period is below the minimum. Setting to default value.
-1958743956	0x8b3ff06c	MASGCMissingInterval	Garbage collection interval is not specified. Setting to default value.
-1958743955	0x8b3ff06d	MASGCMissingGracePeriod	Grace period is not specified. Setting to default value.
-1958743954	0x8b3ff06e	MASModuleStats	Check the log for module statistics.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958743953	0x8b3ff06f	MASInvalidDS2Connection	Attempt to create TKTS driver connection failed.
-1958743952	0x8b3ff070	MASDS2FatalRecompiled	DS2 package fatal error. Auto-recompile succeeded.
-1958743951	0x8b3ff071	MASDS2FatalRecompFailed	DS2 package fatal error. Transaction failed. Recompile failed. Ejecting revision.
-1958743950	0x8b3ff072	MASDS2RevisionEjected	DS2 package fatal error. Max retry exceeded. Ejecting revision. Correct and republish.
-1958743949	0x8b3ff073	MASDBConnLost	Connection to the database lost. Check the log for details.
-1958743948	0x8b3ff074	MASDBConnReestablished	Lost connection reestablished for user context.
-1958743947	0x8b3ff075	MASDBConnRetryLimit	Maximum connection retry attempts exceeded for user context.
-1958743946	0x8b3ff076	MASDBConnDoesNotExist	Attempt to execute SQLSTMT, when no connection exists.
-1958743945	0x8b3ff077	MASDBConnRetryThreadErr	Error while creating database connection retry thread.
-1958743944	0x8b3ff078	MASDBConnRetryAttempt	Connection retry attempt unsuccessful.
-1958743943	0x8b3ff079	MASNameRegisterFailed	Unable to register tkmas in the threaded kernel named registry. DS2 programs that call Python scripts will not function.
-1958743942	0x8b3ff07a	MASDS2PythonNameRequired	AS DS2 Python constructor missing Python module name.
-1958743941	0x8b3ff07b	MASDS2PythonCreateError	Unable to create SAS Micro Analytic Service DS2 Python package.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958743940	0x8b3ff07c	MASDS2PythonInitError	Unable to initialize support for SAS Micro Analytic Service DS2 Python package.
-1958743939	0x8b3ff07d	MASUnsupportedFunction	Unsupported function.
-1958743938	0x8b3ff07e	MASDS2NotInitialized	Attempt to perform action on uninitialized SAS Micro Analytic Service DS2 Python package.
-1958743937	0x8b3ff07f	MASDS2PythonParmError	SAS Micro Analytic Service DS2 Python package parameter mismatch.
-1958743936	0x8b3ff080	MASDS2PythonArgNameReqd	SAS Micro Analytic Service DS2 Python missing argument name.
-1958743935	0x8b3ff081	MASDS2PythonArgValueReqd	AS DS2 Python missing argument value.
-1958743934	0x8b3ff082	MASDS2PythonArgInvalid	SAS Micro Analytic Service DS2 Python invalid argument value.
-1958743933	0x8b3ff083	MASDS2PythonThreadError	Invalid operation: DS2 callback into SAS Micro Analytic Service received an unrecognized thread.
-1958743932	0x8b3ff084	MASPythonCompileEx	Exception thrown while initializing Python or compiling Python script.
-1958743931	0x8b3ff085	MASDS2InvalidMaxRecomp	Invalid maximum DS2 recompile count given. Setting to default value.
-1958743930	0x8b3ff086	MASDBInvalidIntervalHigh	Specified DBMS connection retry interval is above the maximum. Setting to default value.
-1958743929	0x8b3ff087	MASDBInvalidIntervalLow	Specified DBMS connection retry interval is below the minimum. Setting to default value.
-1958743928	0x8b3ff088	MASDBInvalidMaxRetry	Invalid setting for maximum DBMS reconnection attempts. Setting to default value.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958743927	0x8b3ff089	MASDBCCreateConnErr	SAS Micro Analytic Service failed to create a connection.
-1958743926	0x8b3ff08a	MASDBCCreateConn	SAS Micro Analytic Service created a connection.
-1958743925	0x8b3ff08b	MASGCCanBeDeleted	Garbage collection is checking module context for deletion pending.
-1958743924	0x8b3ff08c	MASRepoLockRemovePriv	Locking program repository to remove internal package.
-1958743923	0x8b3ff08d	MASRepoUnlockRemovePriv	Released program repository lock after removing internal package.
-1958743922	0x8b3ff08e	MASRepoLockRemoveRev	Locking program repository to remove module context.
-1958743921	0x8b3ff08f	MASRepoUnlockRemoveRev	Released program repository lock, after removing module context.
-1958743920	0x8b3ff090	MASRepoLockCreate	Creating a lock for user context.
-1958743919	0x8b3ff091	MASRepoLockDestroy	Destroying a lock for user context.
-1958743918	0x8b3ff092	MASRepoLockPackageComp	Locking program repository during compilation of package.
-1958743917	0x8b3ff093	MASRepoUnlockPackageComp	Released program repository lock after compilation of package.
-1958743916	0x8b3ff094	MASRepoUnlockCompCrash	Released program repository lock due to DS2 compiler crash while compiling package.
-1958743915	0x8b3ff095	MASRepoLockPackageSave	Locking program repository to save package after successful compilation.
-1958743914	0x8b3ff096	MASRepoUnlockPackageSave	Released program repository after saving package.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958743913	0x8b3ff097	MASRepoLockPackagePriv	Locking program repository to save internal package.
-1958743912	0x8b3ff098	MASRepoUnlockPackagePriv	Released program repository after saving internal package.
-1958743911	0x8b3ff099	MASPythonNotLoaded	Python extension not loaded. Python must be installed in order to execute Python within SAS Micro Analytic Service.
-1958743910	0x8b3ff09a	MASTKTSConnHndlFail	Failed to create a table services connection handle.
-1958743909	0x8b3ff09b	MASDBDDisconnected	SAS Micro Analytic Service disconnected database from user context.
-1958743908	0x8b3ff09c	MASDBDDisconnect	SAS Micro Analytic Service encountered a failure when attempting to disconnect the database from the user context.
-1958743907	0x8b3ff09d	MASPercentS	Internal error. Check the SAS Micro Analytic Service Core log.
-1958743906	0x8b3ff09e	MASPythonCompileErr	Error compiling the Python script for the module.
-1958743905	0x8b3ff09f	MASDS2MissingArray	A missing array argument is not supported with DS2.
-1958743904	0x8b3ff0a0	MASDS2EmptyArray	An empty array argument is not supported with DS2.
-1958743903	0x8b3ff0a1	MASDS2ArrayReplaced	Missing or insufficiently sized DS2 array argument has been replaced with new array of size %d.
-1958743902	0x8b3ff0a2	MASDS2OutputTransError	Error %d when converting CHAR string of length %d to TKChar string.
-1958743901	0x8b3ff0a3	MASDS2InputTransError	Error %d when converting TKChar string of length %d to CHAR string.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958743900	0x8b3ff0a4	MASDS2PythonOutputTrans	Error %d when converting Python CHAR string of length %d to TKChar string.
-1958743899	0x8b3ff0a5	MASDS2PythonInputTrans	Error %d when converting TKChar string of length %d to CHAR string for Python.
-1958743898	0x8b3ff0a6	MASDBCr8ConnNoSub	SAS Micro Analytic Service created a default data source connection.
-1958743897	0x8b3ff0a7	MASDBCr8ConnErrNoSub	SAS Micro Analytic Service failed to create a default data source connection.
-1958743896	0x8b3ff0a8	MASDBDisconnNoSub	SAS Micro Analytic Service disconnected from the default data source.
-1958743895	0x8b3ff0a9	MASDBDisconnErrNoSub	SAS Micro Analytic Service encountered a failure when attempting to disconnect from the default data source.
-1958743894	0x8b3ff0aa	MASDS2ScanError	Out of memory or malformed DS2 encountered while scanning the package %s source code prior to dictionary generation.
-1958743893	0x8b3ff0ab	MASDS2ParseError	Out of memory or malformed DS2 encountered while parsing the package %s method %s during dictionary generation.
-1958743892	0x8b3ff0ac	MASMTXGetDictDelPending	Attempt to retrieve the dictionary from module context %s revision %d while deletion pending.
-1958743892	0x8b3ff0ad	MASCFuncProtoNotSupp	Part of the C function prototype is not supported.
-1958743890	0x8b3ff0ae	MASDupModuleName	Module name %s already exists. Module name must be unique within the user context.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958743889	0x8b3ff0af	MASDupDS2Package	The DS2 package name %s is already bound to module %s. Separate modules cannot represent the same DS2 package.
-1958743888	0x8b3ff0b0	MASIndexOutOfRangeSet	The index is out of range while setting an argument. Argument %d specified when number of arguments is %d.
-1958743887	0x8b3ff0b1	MASIndexOutOfRangeGet	The index is out of range while retrieving an argument. Argument %d specified when number of arguments is %d.
-1958743886	0x8b3ff0b2	MASIntTypeExpected	The argument %d in method %Us should be an integral type used to specify the length of the previous argument, which is an array.
-1958743885	0x8b3ff0b3	MASOutArgExpected	The argument %d in method %Us should be an output argument. All input arguments must precede output arguments.
-1958743884	0x8b3ff0b4	MASDS2pymas	DS2 PyMAS package encountered a failure.
-1958743883	0x8b3ff0b5	MASDS2pymasFailIn	DS2 PyMAS package encountered a failure in %Us.
-1958743882	0x8b3ff0b6	MASDS2pymasPubUTF8	DS2 PyMAS package failed to publish module %Us.
-1958743881	0x8b3ff0b7	MASDS2pymasPubTK	DS2 PyMAS package failed to publish module %s.
-1958743880	0x8b3ff0b8	MASDS2pymasUsed	The DS2 PyMAS package's use method has already been called on this package instance. Create a separate PyMAS instances for each method that is used.
-1958743879	0x8b3ff0b9	MASThrdPoolSizeDiff	SAS Micro Analytic Service has already been initialized with a worker thread pool size of %d.

Return Code	Hexadecimal Code	#define Symbol	Message or Description
-1958743878	0x8b3ff0ba	MASSymbolTableCreateFailed	SAS Micro Analytic Service failed to create a symbol table.
-1958743877	0x8b3ff0bb	MASMethodExecutionFailed	SAS Micro Analytic Service failed to execute a method.
-1958743725	0x8b3ff153	MASRevNumMiss	The specified revision number is missing.
-1958743747	0x8b3ff13d	MASDS2masmcNotInMas	DS2 "mascall" package cannot be used outside Micro Analytic Service.
-1958743749	0x8b3ff13b	MASDS2masmc	DS2 "mascall" package encountered a failure.
-1958743672	0x8b3ff188	MASAllocParmsNeeded	A successful call to allocParms or allocRevParms must precede setter/getter method calls.
-2143305726	0x803fc002	TK_NOMEM	Insufficient memory.

